

COMPLETE COURSE NOTES · MODULES 1-8

Data Communication Networks

Every topic on the mid-sem and quiz syllabus, explained from zero in plain English — with the lecture-deck diagrams, every formula, and every example worked out step by step.

EC321

B.Tech III EC · Semester V

8 modules

All 37 mid-sem topics

Mid-sem topic checklist

All **37 topics** from the mid-sem and quiz syllabus, with the exact section that covers each one. Use this as your revision tick-list.

MID-SEM SYLLABUS → WHERE IT IS COVERED			
#	TOPIC	COVERED IN	GO TO
1	Five Components of Data Communication	Module 1 — message, sender, receiver, medium, protocol	§1.2
2	Data Flow Types	Simplex, half-duplex, full-duplex	§1.3
3	TCP/IP and OSI Models	Seven layers, layer duties, the two models compared	§2.2–2.4
4	Encapsulation and Decapsulation	Headers added going down, stripped going up	§2.2
5	Network Performance Parameters	PDR, throughput, latency (4 delays), jitter	§6.2
6	Optical Fiber Cable	TIR, structure, modes, step vs graded index	§3.4
7	Coaxial Cable	Four layers, RG categories, BNC connectors	§3.3
8	Twisted Pair Cable	Why twisting works, UTP vs STP, categories 1–7	§3.2
9	Bus Topology	All five topologies with a full comparison table	§1.5
10	Ring Topology		§1.5
11	Star Topology		§1.5
12	Mesh Topology		§1.5
13	Hybrid Topology		§1.5
14	Network Devices	Repeater, hub, bridge, switch, router, gateway	§8.1, 8.6
15	MAC Address Classification	Unicast, multicast, broadcast; OUI and NIC parts	§8.7
16	Learning Bridge and Loop Problem	The learning rule, the loop, spanning tree	§8.4–8.5
17	IP Address and Port Address	The four address levels and what changes per hop	§2.5
18	Bit Stuffing	The five-1s rule, worked both ways	§5.1
19	Byte Stuffing	ESC before FLAG or ESC, worked both ways	§5.1
20	CRC for Error Detection	Modulo-2 division, fully worked division	§4.2
21	Hamming Code for Error Correction	Position method and matrix method, both worked	§4.3
22	Simplest Protocol	The two noiseless-channel protocols	§5.3
23	Stop and Wait		§5.3

#	TOPIC	COVERED IN	GO TO
24	Stop and Wait ARQ	Timers, sequence numbers, utilisation numericals	§5.4
25	Go-Back-N	Windows, why size $\leq 2^n$, walkthroughs	§5.5
26	Selective Repeat ARQ	Buffering, NAK rule, side-by-side vs Go-Back-N	§5.6
27	Polling MAC Protocol	All three controlled-access methods compared	§6.5
28	Reservation MAC Protocol		§6.5
29	Token Passing MAC Protocol		§6.5
30	Pure Aloha	Vulnerable time, throughput curves, all examples	§6.3
31	Slotted Aloha		§6.3
32	CSMA	Persistence methods, vulnerable time	§6.4
33	CSMA/CD	Collision detection, minimum frame size	§6.4
34	CSMA/CA	IFS, contention window, acknowledgments	§6.4
35	FDMA	All three channelization methods compared	§6.6
36	TDMA		§6.6
37	CDMA		§6.6

HOW TO READ THIS

Read a module top to bottom, in order — each one uses ideas from the one before it. Wherever you see a **Plain words** box, that is the same idea said again in everyday language; if the main text felt heavy, that box is your safety net. Every **Example** is fully solved — cover the answer and try it yourself first. The **Check yourself** questions at the end of each module are the fastest way to find out what did not stick.

On a phone: tap *Contents* at the top to jump to any section. Wide diagrams and tables can be **slid sideways** with your finger to see the whole thing.

MODULE 01

Introduction to Data Communication and Networks

The vocabulary everything else is built on: what data communication actually is, the five parts every communication needs, the three directions data can flow, the five shapes a network can be wired in, and why rules and standards exist at all.

Lecture slides 4–21 · Chapter 1

1.1

What data communication actually means

Three words come first, and they are easy to mix up.

Telecommunication

Communication *at a distance*. The prefix *tele* means “far”. Telephone, television and telegraph are all telecommunication.

Data

Information presented in whatever form the two sides have **agreed upon**. This agreement part matters. If I write a number as 101 and you read it as one-zero-one in binary (five) while I meant one hundred and one, then we never agreed, so it was never usable data.

Data communication

The **exchange of data between two devices** through some form of transmission medium, such as a wire cable.

Notice the word **exchange**. It is not enough for one machine to throw bits into a wire. Data communication means the bits arrive, and arrive in a form the other side can understand.

PLAIN WORDS

Data communication is two machines successfully passing agreed-upon information to each other over some path. Three things must all be true: something is sent, something arrives, and both sides understand it the same way.

1.2

The five components of a data communication system

Every single act of data communication — from a WhatsApp message to a satellite downlink — is made of exactly these five parts. If any one is missing, communication fails.

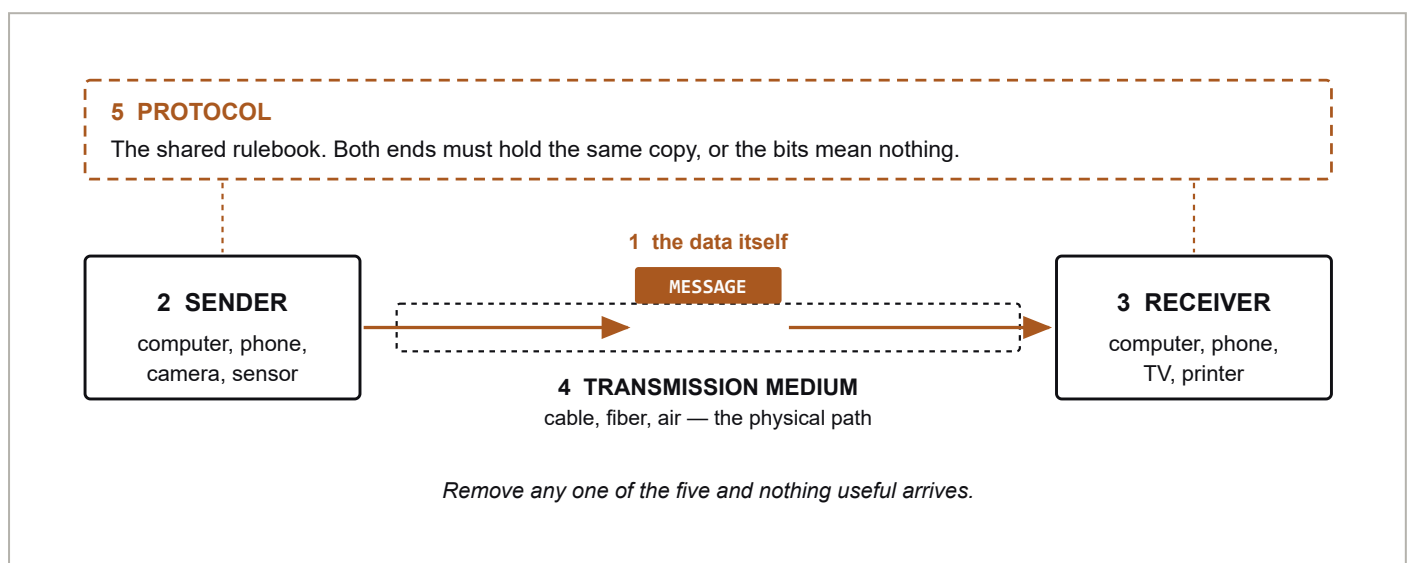
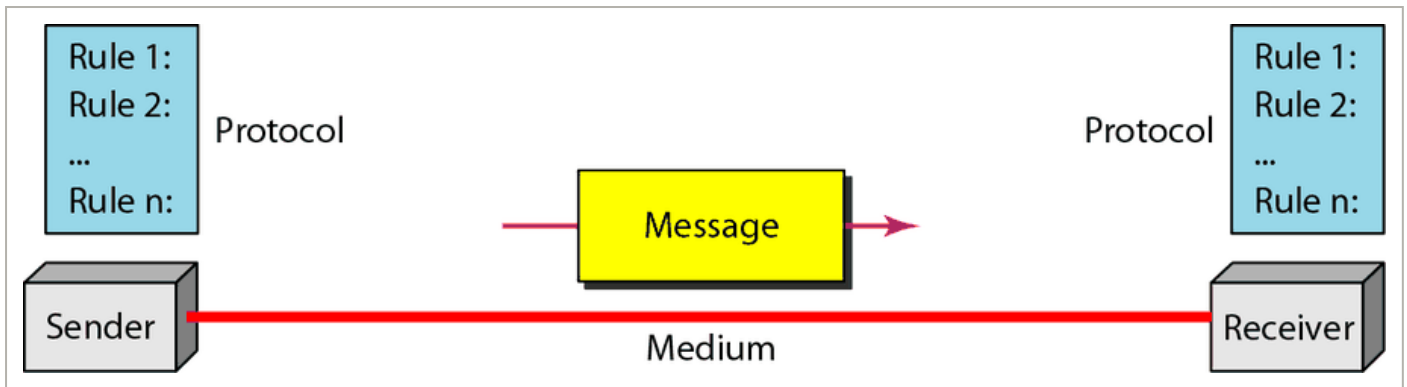


Fig 1.1 The five components. Four of them sit in the physical path; the fifth, the protocol, is an agreement held at *both* ends at once — which is why it is drawn as a band across the top and not as a box in the middle.



Slide 6 The five components as drawn in the lecture deck.

THE FIVE COMPONENTS IN ONE LINE EACH			
#	COMPONENT	WHAT IT IS	EXAMPLE
1	Message	The information to be communicated — the actual content	Text, numbers, pictures, audio, video
2	Sender	The device that sends the message	Computer, mobile phone, video camera, sensor
3	Receiver	The device that receives the message	Computer, phone, television, printer
4	Transmission medium	The physical path the message travels along	Twisted-pair cable, coaxial cable, fiber optic, radio waves
5	Protocol	The set of rules that governs the communication — an agreement between the two devices	TCP/IP, HTTP, HDLC

WHY THE PROTOCOL MATTERS MOST

Two devices can be perfectly connected by a perfect cable and still not communicate. A person who speaks only Hindi and a person who speaks only Japanese can stand in the same room with clear air between them — message, sender, receiver and medium are all present — and still exchange nothing. The missing piece is the protocol.

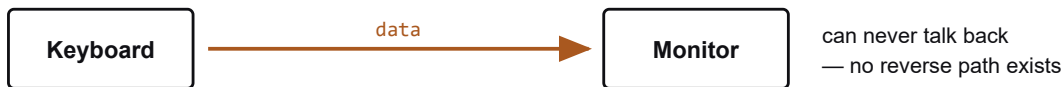
1.3

Data flow types: simplex, half-duplex, full-duplex

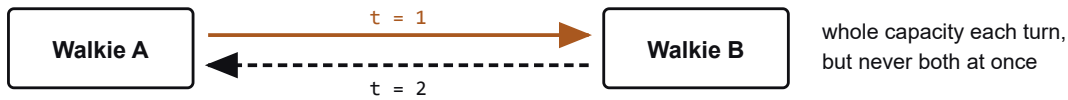
Once two devices are linked, the next question is: **in which directions can data move, and when?** There are exactly three answers.

SIMPLEX

one direction only — the whole channel capacity goes one way

**HALF-DUPLEX**

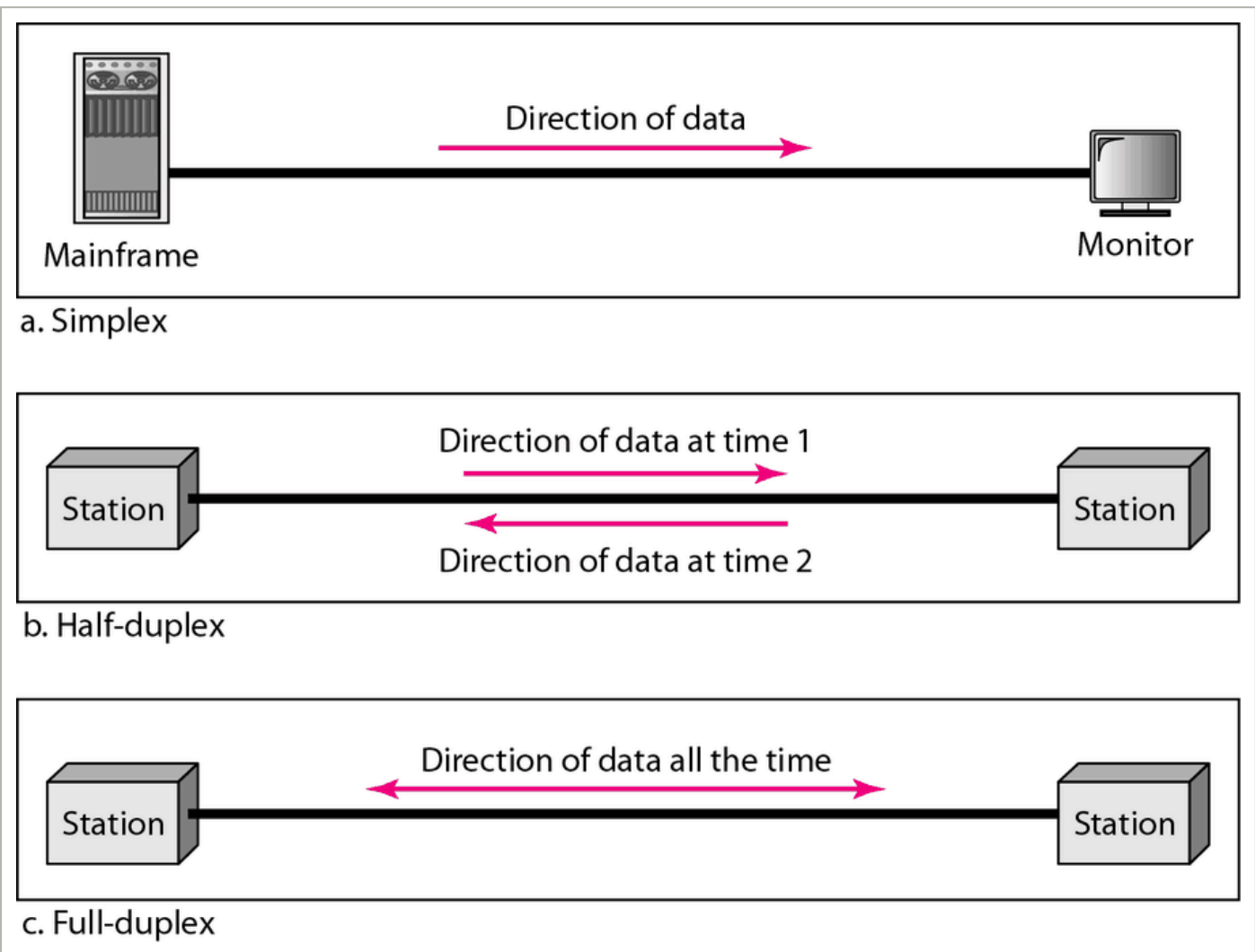
both directions, but only one at a time — the channel is shared, taking turns

**FULL-DUPLEX**

both directions at the same time — capacity is split, or two separate paths are used



Fig 1.2 The difference between the three modes is not *how many* arrows exist but *when* they may be used. Half-duplex has two arrows that must take turns; full-duplex has two arrows that may run together.



Slide 7 Data flow types from the lecture deck: simplex, half-duplex and full-duplex.

COMPARISON OF THE THREE DATA FLOW MODES			
MODE	DIRECTION	CHANNEL CAPACITY	EVERYDAY EXAMPLE
Simplex	One-way only. One device only sends, the other only receives.	Entire capacity used in the one direction	Keyboard → computer; computer → monitor; radio broadcast; TV broadcast
Half-duplex	Both ways, but only one direction at any one moment	Whole capacity is given to whichever side is currently sending	Walkie-talkies; CB radio; a single-lane bridge with traffic taking turns
Full-duplex (duplex)	Both ways at the same time	Capacity is shared between the two directions, or two physically separate paths carry them	Telephone call; modern Ethernet; a two-lane road

PLAIN WORDS

Think of a road. **Simplex** is a one-way street. **Half-duplex** is a narrow bridge where cars from one side go first, then cars from the other. **Full-duplex** is a normal two-lane road where both directions move together.

1.4

Networks, nodes, links and types of connection

What a network is

A **network** is a set of devices — often called **nodes** — connected by **communication links**.

A node can be a computer, a printer, or *any* other device capable of sending or receiving data generated by other nodes on the network. A link is the path between two nodes.

Two types of connection

A link joining nodes can be arranged in only two ways.

Point-to-point

A **dedicated** link between exactly two devices. The entire capacity of the link is reserved for those two.

Example: your TV remote and the TV — the infrared beam is used only by that pair.

Multipoint (multidrop)

More than two devices **share** a single link.

Capacity is shared — either spatially (several send at once) or in time (they take turns).

Example: several computers tapped onto one bus cable.

REMEMBER

Point-to-point = **dedicated** capacity for two devices. Multipoint = **shared** capacity for many devices. The whole of Module 6 exists because multipoint links have to be shared fairly.

1.5

Categories of topology, and how they compare

Topology means the way the links and nodes are arranged — the geometric shape of the network. It is about the *layout of the connections*, not the physical placement of the machines in the room.

There are five basic topologies.

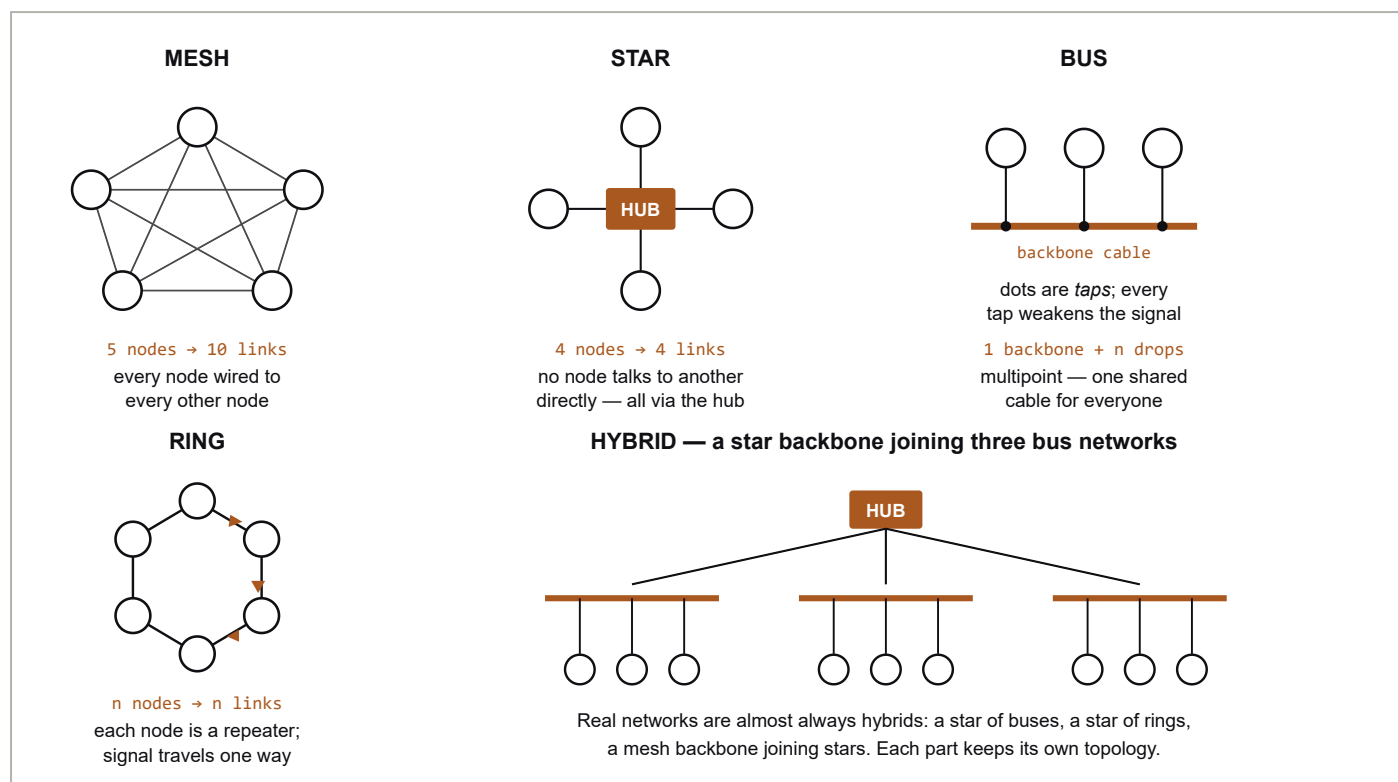


Fig 1.3 The five topologies. The important difference between them is **how many links each node needs** and **what breaks when one link or one device fails** — which is exactly what the comparison table below sets out.

Mesh topology

Every device has a **dedicated point-to-point link to every other device**. Nothing is shared.

MESH: LINKS AND PORTS NEEDED

$$\text{Number of duplex links} = \frac{n(n-1)}{2}$$

$$\text{Ports (NICs) per device} = n - 1$$

where n is the number of devices. Each link is counted once because one duplex link serves both directions.

EXAMPLE 1.1

A fully connected mesh has 5 devices. How many links are needed, and how many ports does each device need? What if we grow to 20 devices?

For $n = 5$: links = $5 \times (5 - 1) / 2 = 5 \times 4 / 2 = \mathbf{10 \text{ links}}$, and each device needs $5 - 1 = \mathbf{4 \text{ ports}}$.

For $n = 20$: links = $20 \times 19 / 2 = \mathbf{190 \text{ links}}$, and each device needs $\mathbf{19 \text{ ports}}$.

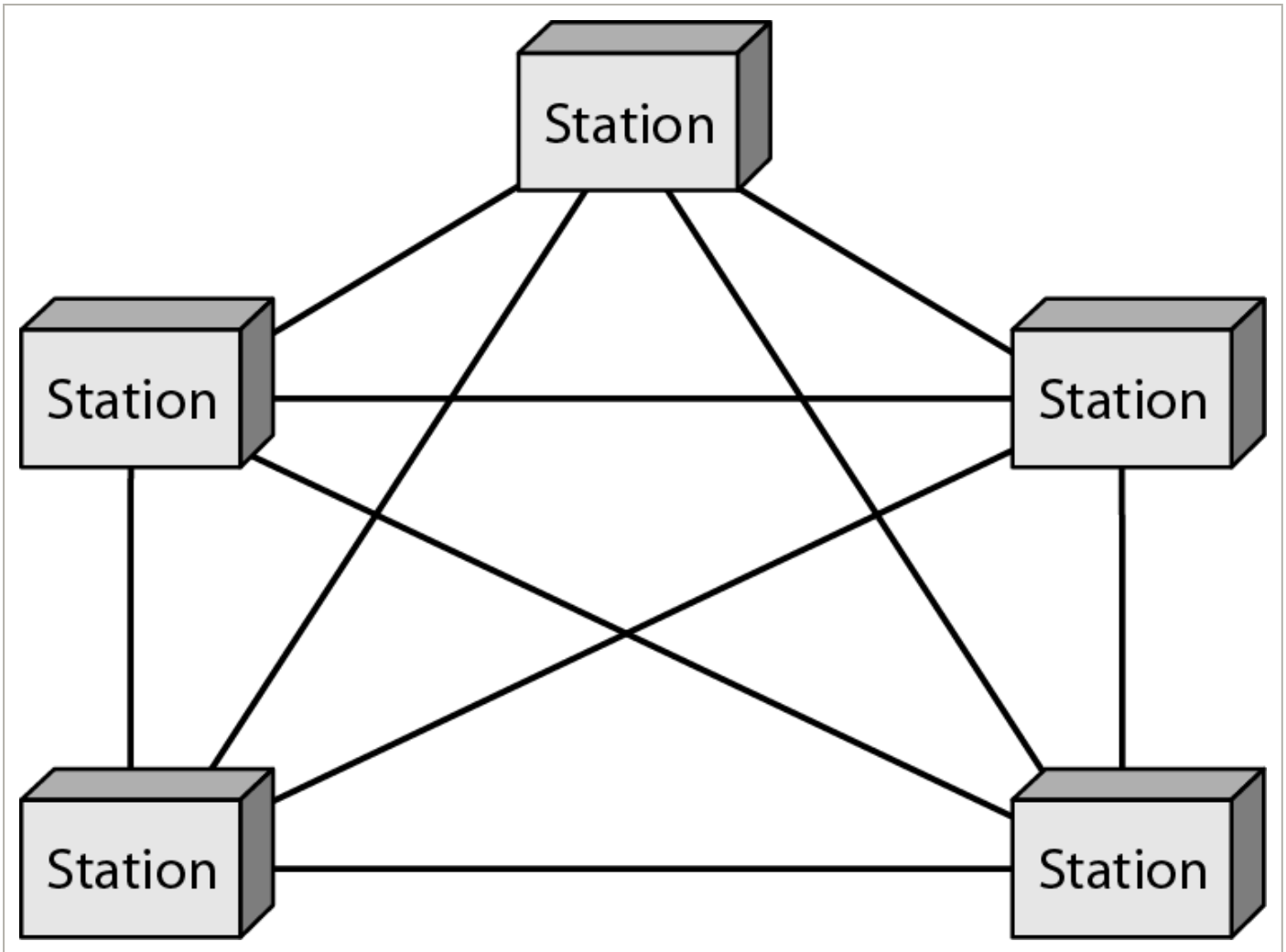
Growing from 5 to 20 devices — four times as many — multiplied the cabling by **nineteen**. This is why mesh is never used for a whole office.

Answer: 5 devices → 10 links, 4 ports each. 20 devices → 190 links, 19 ports each. Cost grows roughly as n^2 .

Advantages of mesh:

- **No traffic problems.** Every link is dedicated, so two devices never compete for the same wire.
- **Robust.** If one link goes down, it does not take the rest of the system with it.
- **Privacy and security.** A message travels only on the dedicated link to its destination, so no other device can see it.
- **Easy fault identification and isolation.** Traffic can be rerouted to avoid a suspect link, and the faulty link is easy to point at.

Disadvantages of mesh: the amount of cabling and the number of ports per device. Installation and reconnection are difficult, the sheer bulk of wiring can be more than the available space (walls, ceilings, floors) allows, and the hardware cost of all those ports is high. Because of this, mesh is normally used only in a limited way — for example, as a **backbone connecting the main computers** of a bigger hybrid network.



Slide 11 A fully connected mesh topology with five devices — every node wired to every other node.

Star topology

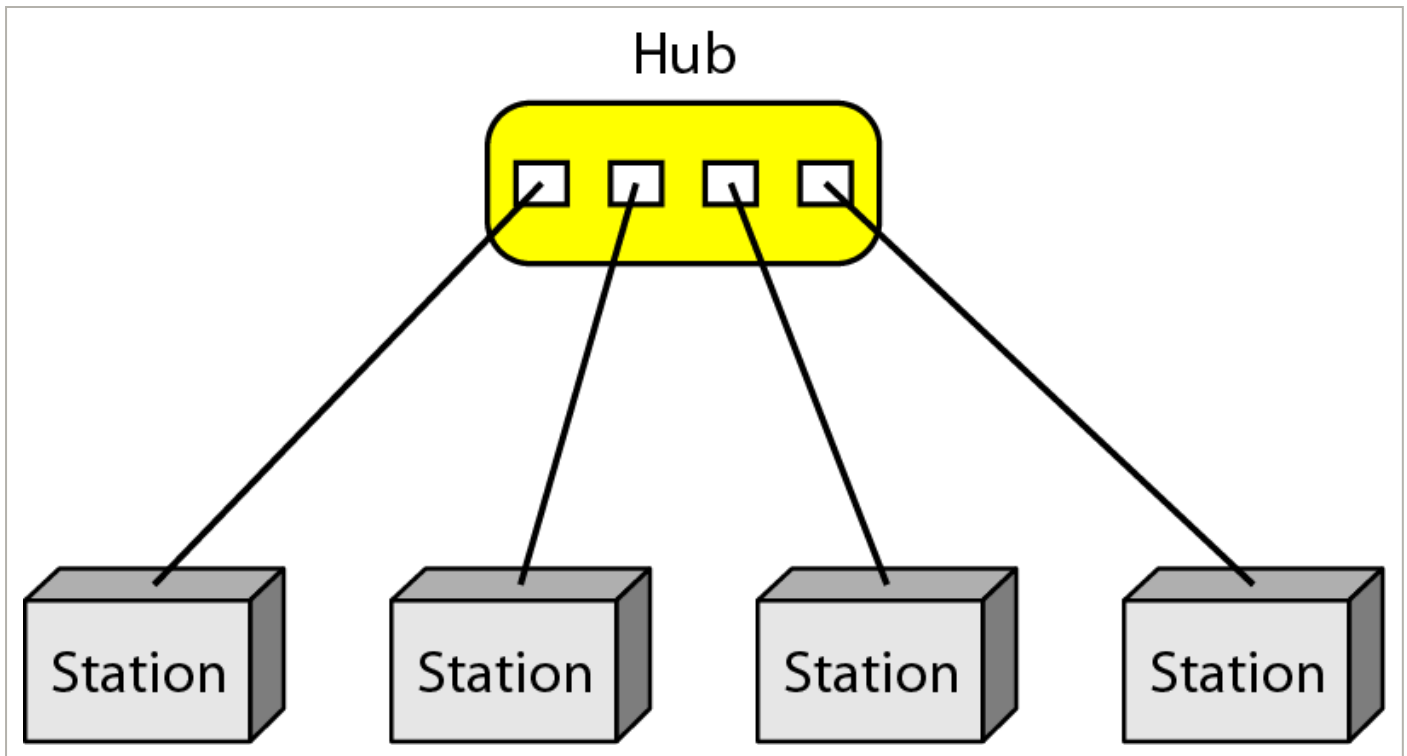
Each device has a dedicated point-to-point link to a **central controller, usually called a hub**. Devices are *not* directly linked to one another. To send data, a device sends it to the hub, and the hub relays it to the intended receiver.

Advantages: much cheaper than mesh — each device needs only **one link and one port**. Installation and reconfiguration are easy: adding, moving or deleting a device means touching only one connection. It is also robust in a useful way — if one link fails, only that one device is affected. Fault identification is easy because the hub can watch all links.

Disadvantage: the hub is a **single point of failure**. If the hub goes down, the whole network is dead. It also needs more cable than ring or bus, because every device gets its own run back to the hub.

REMEMBER

Star is the topology of almost every modern office LAN. When you plug an Ethernet cable into a wall socket that runs back to a switch in a cupboard, you are in a star.



Slide 12 A star topology connecting four stations through a central hub.

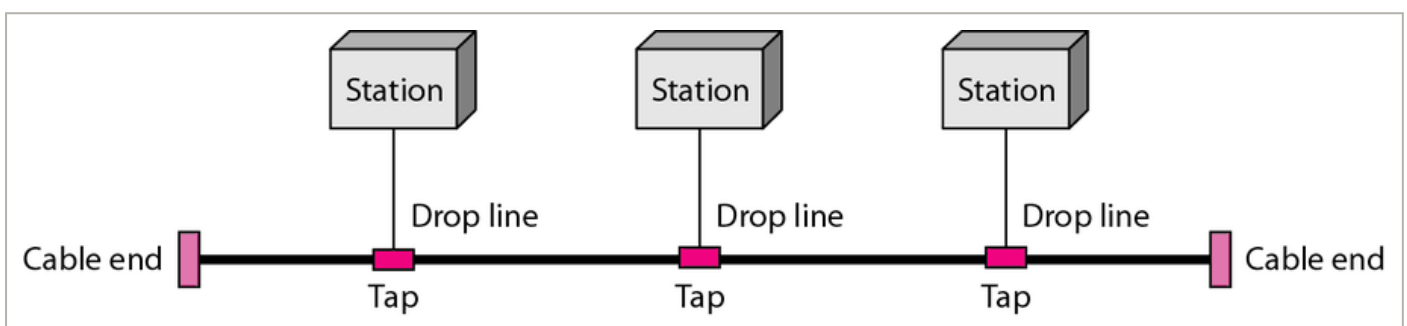
Bus topology

A bus is **multipoint**. One long cable — the **backbone** — links all the devices. Nodes connect to it by **drop lines** and **taps**. A drop line is the connection running from the device to the backbone; a tap is the connector that pierces the cable to touch the metal core.

As a signal travels along the backbone it gets **weaker**, both with distance and because each tap leaks a little energy. This is the physical fact behind every limitation of bus:

Advantages: easy installation and less cabling than mesh, star or ring — you run one backbone through the building and spur off it.

Disadvantages: difficult reconnection and fault isolation. The number of taps and the distance between them is **limited** by signal loss. Adding new devices later may require modifying or replacing the backbone. A fault or break in the backbone **stops all transmission**, even between devices on the same side of the break, because the damaged spot creates reflected noise.



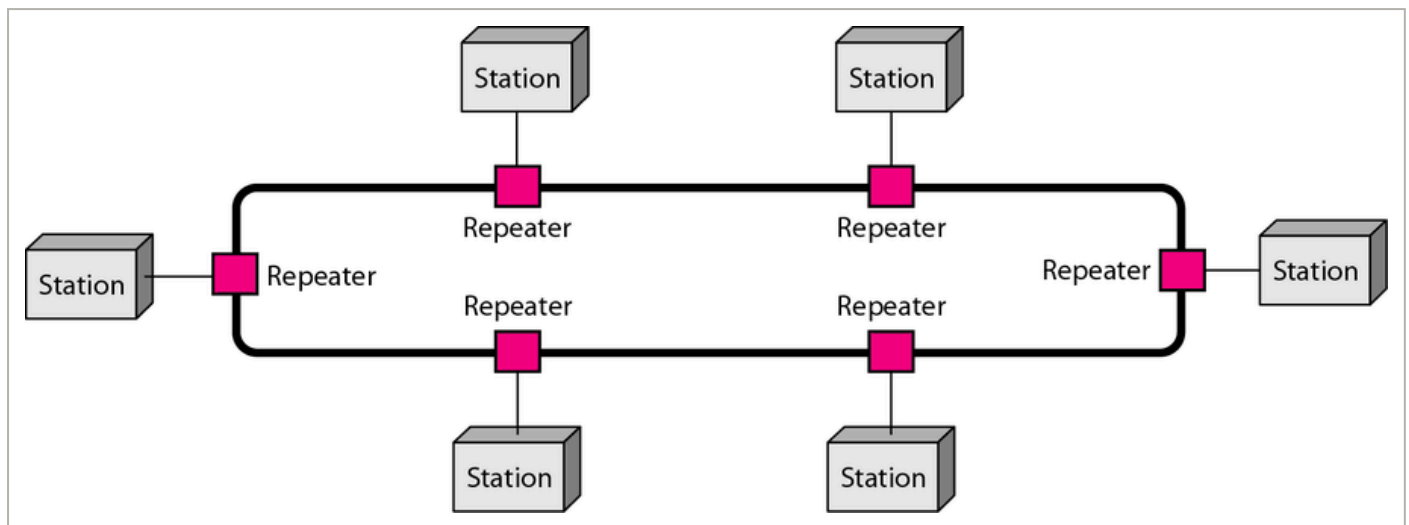
Slide 13 A bus topology connecting three stations by drop lines and taps off one backbone cable.

Ring topology

Each device has a dedicated point-to-point connection with only the **two devices on either side of it**. A signal passes along the ring in one direction, from device to device, until it reaches its destination. Each device contains a **repeater**: when it receives a signal not meant for it, it regenerates the bits and passes them on.

Advantages: easy to install and reconfigure — to add or remove a device you change only two connections. Fault isolation is simple: normally a signal circulates all the time, so if a device does not receive a signal within a set period, it can raise an alarm that alerts the operator to the problem and its location.

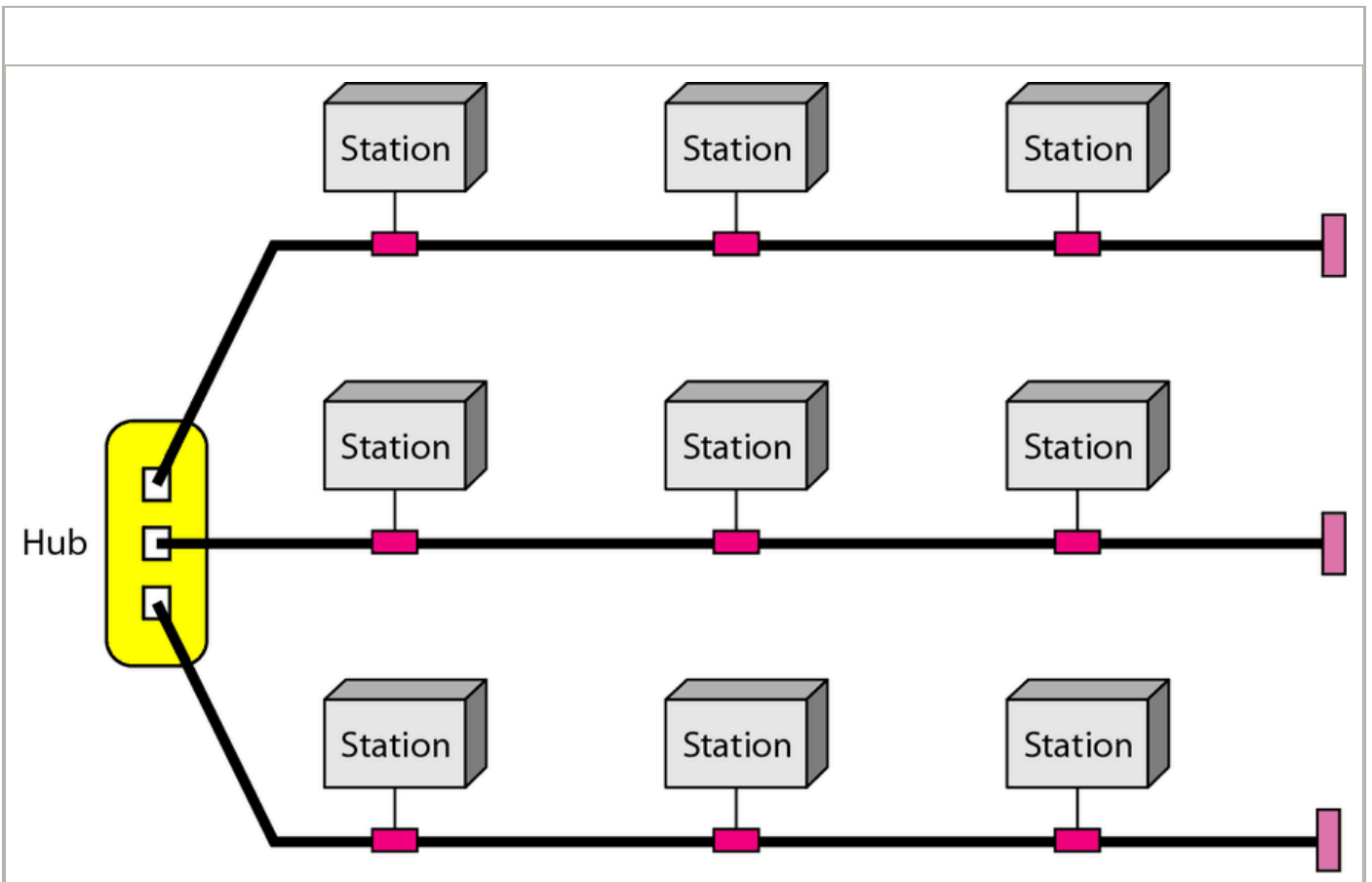
Disadvantages: the traffic is **unidirectional**, and a break anywhere in the ring — one dead station or one cut cable — disables the *entire* network. (This can be fixed with a **dual ring**, or a switch that can close the break.)



Slide 14 A ring topology connecting six stations, each acting as a repeater.

Hybrid topology

A network built out of the others — for instance a **star backbone with three bus networks** hanging off it, as in the last panel of Fig 1.3. Real installations are nearly always hybrids, because each part of a building has different needs.



Slide 15 A hybrid topology — a star backbone with three bus networks hanging off it.

COMPARISON OF THE FIVE TOPOLOGIES					
TOPOLOGY	LINKS FOR N NODES	PORTS PER NODE	MAIN ADVANTAGE	MAIN WEAKNESS	IF ONE LINK/DEVICE DIES
Mesh	$n(n-1)/2$	$n-1$	Dedicated links: no traffic contention, private, easy fault isolation	Enormous cabling and port cost; hard to install	Rest of network keeps working
Star	n	1	Cheap, easy to install and reconfigure, easy fault detection	Hub is a single point of failure; more cable than bus/ring	One link dies → only that node offline. Hub dies → whole network offline
Bus	1 backbone + n drop lines	1 tap	Least cabling; simple to lay out	Signal weakens per tap, so taps and length are limited; hard to isolate faults	Backbone break → whole network stops
Ring	n	2	Easy to add/remove nodes; built-in fault alarm	Traffic is one-way only; needs every node working	Any break → whole ring stops (unless dual ring)
Hybrid	depends on mix	depends	Each area gets the topology that suits it; scales well	More complex to design and manage	Damage is usually contained to one sub-network

1.6

Categories of networks: LAN, WAN and MAN

Networks are also classified by **how much ground they cover and who owns them**.

LAN — Local Area Network

A LAN is **privately owned** and links the devices in a single office, building or campus. It may be as small as two PCs and a printer, or spread through a whole factory. LANs are usually a few kilometres across at most.

In the simplest arrangement — an **isolated LAN** — a set of computers is wired to a single hub sitting in a wiring closet. Each device on a LAN is identified by a unique **address**, and a packet carries the address of both the sender and the receiver.

WAN — Wide Area Network

A WAN covers a **large geographical area**: a state, a country, or the world. It carries data, voice, image and video over long distances. Two forms appear in the slides:

- **Point-to-point WAN** — a single line, leased from a telephone or cable company, connecting one computer or LAN to another computer or LAN.
- **Switched WAN** — a network of many interconnected switching nodes, so traffic can take different routes. The backbone of the Internet is a switched WAN.

MAN — Metropolitan Area Network

A MAN sits **between** a LAN and a WAN in size: it spans a town or city. A cable TV network across a city is a MAN.

Internetwork

When two or more networks are connected together, the result is an **internetwork** (or *internet*, small “i”). A **heterogeneous network** is one made of different kinds of network joined together — for example four WANs and two LANs wired into one system, which is exactly the case shown in the slides. Routers are the devices that join these different networks.

LAN VS MAN VS WAN			
FEATURE	LAN	MAN	WAN
Area covered	Room, building, campus (up to a few km)	A city or town	Country, continent, worldwide
Ownership	Private — one organisation	Often a single service provider	Usually shared / leased from carriers
Data rate	Highest	Medium	Lower than LAN
Error rate	Lowest	Medium	Highest (long distances, many hops)
Typical example	Office network, computer lab	Cable TV network in a city	The Internet backbone

1.7

The Internet, and how it is organised

The Internet (capital “I”) is **one specific, particular internetwork** — the global one. It is a communication system that has brought a huge amount of information within reach and organised it for use.

It is not one network owned by one company. It is **hierarchical**, built in tiers:

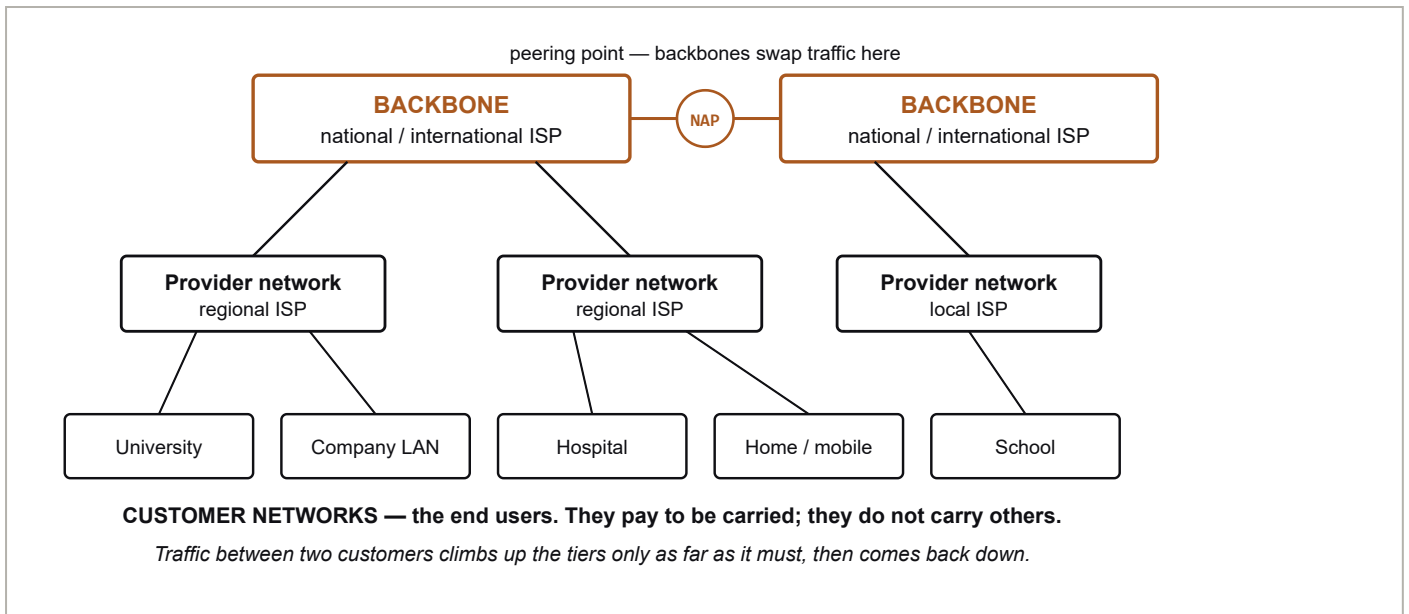
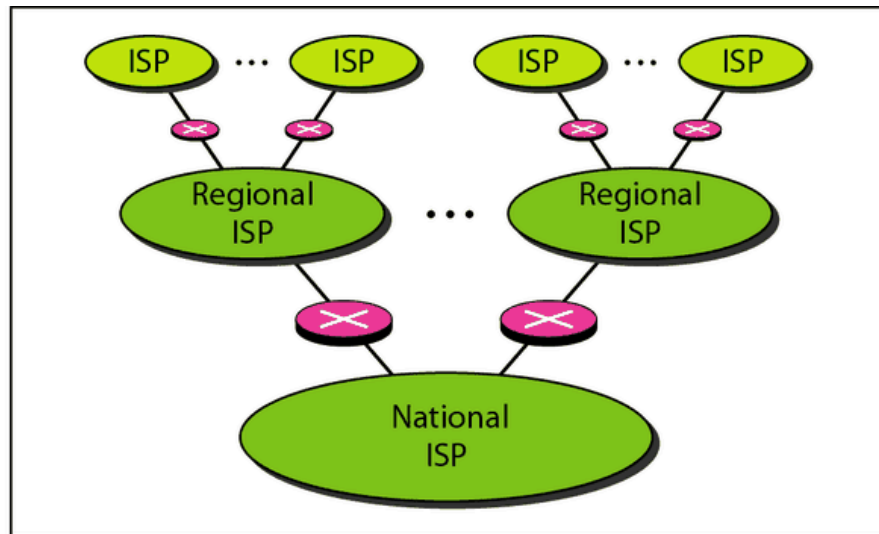
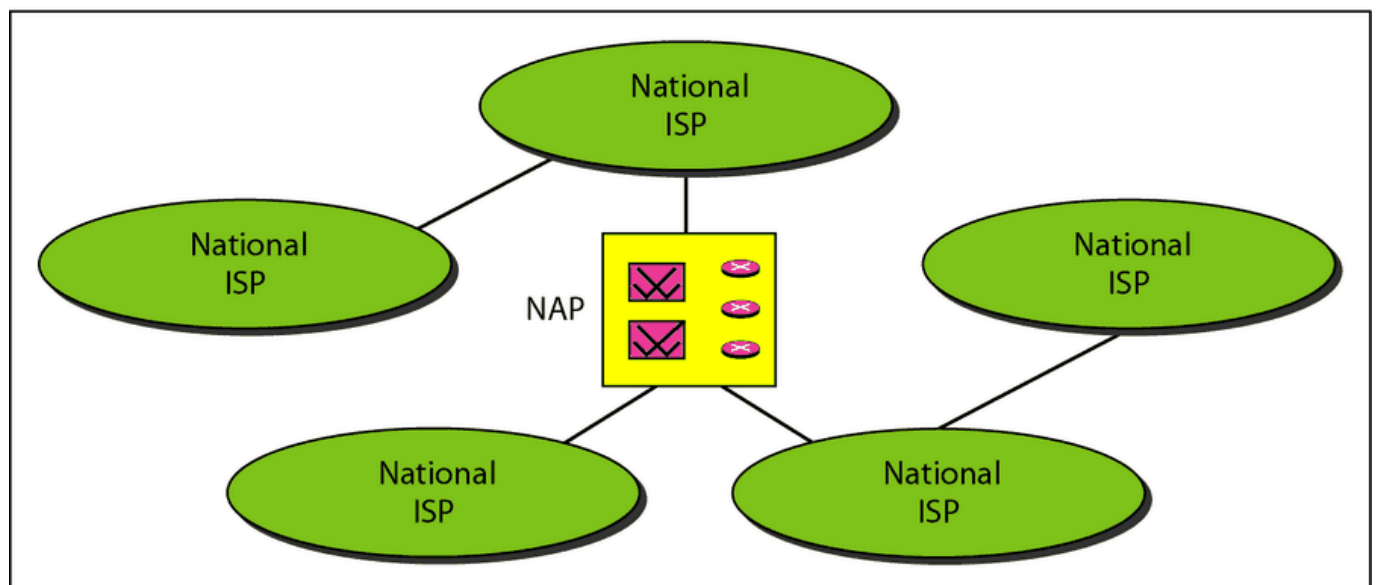


Fig 1.4 The Internet's tiers. Backbones carry traffic between each other at peering points; provider networks buy capacity from backbones and sell it on; customer networks sit at the leaves.



a. Structure of a national ISP



b. Interconnection of national ISPs

Slide 20 Hierarchical organization of the Internet, as shown in the lecture deck.

- **Backbones** — large networks run by international and national ISPs, joined to one another at **peering points** so traffic can cross between them.
- **Provider networks** — regional and local ISPs. They buy capacity from the backbones and resell it.
- **Customer networks** — universities, companies, homes. They pay for access and are the actual users of the data.

1.8

Protocols and standards, and why we need them

What a protocol is

A **protocol** is a set of rules that governs data communication. The word is a synonym for *rule*. It defines *what* is communicated, *how* it is communicated, and *when* it is communicated. A protocol has three key elements:

Syntax

The **structure or format** of the data — the order in which the bits are presented. For example: the first 8 bits are the sender's address, the next 8 bits are the receiver's address, and the rest is the message.

Semantics

The **meaning** of each section of bits. Given a pattern of bits, semantics answers: what does this pattern mean, and what action should be taken because of it?

Timing

When data may be sent and **how fast**. If a sender produces 100 Mbps but the receiver can only absorb 1 Mbps, the transmission will overload the receiver and data will be lost.

Why protocols and standards are needed

Devices from different manufacturers, running different software, made in different decades, all have to work together. Without an agreed protocol:

- the receiver cannot tell where one field ends and the next begins (no shared syntax);
- the receiver cannot tell what a bit pattern is asking it to do (no shared semantics);
- the sender can flood the receiver (no shared timing).

Standards turn a protocol from a private arrangement into a public one, so that any two devices that follow the standard will interoperate. This is why an Indian-made router talks happily to a Korean-made laptop.

Two categories of standard

De facto — “by fact”

Standards that were never formally approved but became standard **because everybody used them**. They may be *proprietary* (created by one company for its own products) or *non-proprietary* (developed by a group, later widely adopted).

De jure — “by law”

Standards that have been **legislated by an officially recognised body**. These have gone through a formal approval process.

Standards organisations

BODIES THAT CREATE AND ENFORCE STANDARDS		
BODY	FULL NAME	ROLE
ISO	International Organization for Standardization	A multinational body, established 1947, dedicated to worldwide agreement on international standards. It produced the OSI model.
ITU-T	International Telecommunication Union — Telecommunication Standardization Sector	Standards for telephony and telecommunication (formerly CCITT).
ANSI	American National Standards Institute	A private non-profit that coordinates US standards.
IEEE	Institute of Electrical and Electronics Engineers	The largest professional engineering society. Its Project 802 produced the LAN standards — 802.3 for Ethernet, 802.11 for Wi-Fi.
EIA	Electronic Industries Alliance	Standards for physical connection interfaces and signalling, such as the RS-232 serial standard.
FCC	Federal Communications Commission	A <i>regulatory agency</i> — it controls radio, television and wire/cable communication in the public interest.

REMEMBER THIS DISTINCTION

ISO is the organisation. OSI is the model. They are not the same word rearranged, and it is a favourite exam trap.

Internet standards

On the Internet, a specification goes through a defined life. Anyone may write an **Internet draft** — a working document with no official status and a six-month lifetime. If it is recommended for publication it becomes an **RFC** (Request for Comment), which is given a number and a maturity level:

- 1 Proposed standard** — stable, well understood, and of enough interest to the community. It must be tested and implemented by different groups.
- 2 Draft standard** — reached after at least two successful independent implementations.
- 3 Internet standard** — reached after demonstrated success; it becomes a full standard.

Three other levels exist off to the side: **historic** (superseded or never went anywhere), **experimental** (work not meant for operational use), and **informational** (general information about the Internet).

Each RFC also carries a **requirement level**: *required* (every Internet system must implement it — IP and ICMP, for example), *recommended*, *elective*, *limited use*, or *not recommended*.

Module 1 at a glance

- Data communication needs **five** things: message, sender, receiver, transmission medium, protocol. The protocol is the only one not physically in the path.
- Data flow is **simplex** (one way), **half-duplex** (both ways, taking turns) or **full-duplex** (both ways at once).
- Connections are **point-to-point** or **multipoint**; topologies are **mesh**, **star**, **bus**, **ring** and **hybrid**. A mesh of n nodes needs $n(n-1)/2$ links and $n-1$ ports per node.
- Networks by size: **LAN**, **MAN**, **WAN**. A protocol fixes **syntax** (format), **semantics** (meaning) and **timing** (when and how fast).

CHECK YOURSELF — MODULE 1

Q Name the five components of data communication, and say which one is not physically located between sender and receiver.

Message, sender, receiver, transmission medium, protocol. The **protocol** is not in the path — it is an agreement held at both ends simultaneously.

Q Your TV remote and TV: which data flow mode? Your phone call: which mode? Why are they different?

Remote → TV is **simplex**: the remote only sends, the TV only receives, there is no reverse path at all. A phone call is **full-duplex**: both people can speak at the same time and be heard.

Q A fully connected mesh has 8 devices. How many links, and how many ports per device?

Links = $n(n-1)/2 = 8 \times 7 / 2 = 28$ links. Ports = $n - 1 = 7$ per device.

Q In a star topology, one cable is cut. In a bus topology, the backbone is cut. What is the difference in effect?

In the **star**, only the single device on that cable goes offline — everyone else carries on through the hub. In the **bus**, the break stops *all* transmission, because the damaged point reflects signals back and creates noise across the whole backbone.

Q Why is a bus topology limited in the number of devices it can carry?

Because the signal weakens as it travels, and **every tap leaks some energy**. Past a certain number of taps, or a certain distance between them, the signal is too weak to be read correctly.

Q What are the three key elements of a protocol?

Syntax (format and order of the bits), **semantics** (what each part means and what to do about it), and **timing** (when to send and how fast).

Q What is the difference between a de facto and a de jure standard?

De facto means “by fact” — it became standard through widespread use, without formal approval. **De jure** means “by law” — it was formally approved by an officially recognised standards body.

Q Is the Internet a LAN, a MAN, a WAN, or something else?

It is an **internetwork** — many networks joined together. Its backbone is a **switched WAN**, and hanging off it are provider networks and customer networks, many of which are LANs.

MODULE 02

Network Models: OSI and TCP/IP

A network is far too complicated to design as one lump. The answer is to cut the job into layers, give each layer one clear duty, and let each layer trust the one below it. This module explains why that works, walks through all seven OSI layers, and then sorts out the four completely different kinds of address a message carries.

Lecture slides 22–61 · Chapter 2

2.1

Why we use layers at all

We already use layers in daily life without noticing. Take the example the lectures use: **two friends who communicate by postal mail**.

Sending a letter has three natural stages at your end, three at the post office, and three at your friend's end.

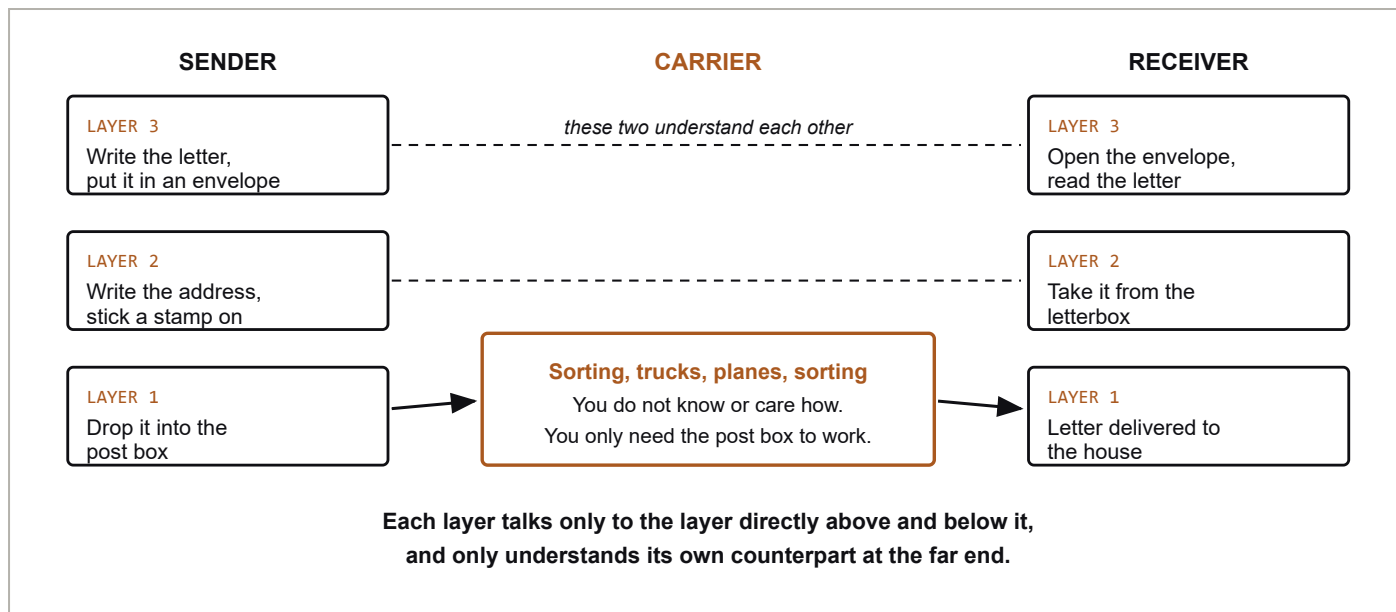


Fig 2.1 Sending a letter, drawn as layers. The dashed lines are the pairs that *understand* each other; the solid arrows are the only places where real handover happens. Notice you never speak to the truck driver.

PLAIN WORDS

Layering means: **break a big job into small jobs, stack them, and let each level trust the level under it.** You write the letter. You do not also have to drive the van. And if the post office swaps its vans for trains tomorrow, your letter-writing does not change at all.

That last sentence is the whole point of layering in networks. **You can change one layer without rewriting the others.** Wi-Fi replaced cables at the bottom two layers, and every web browser kept working unchanged.

2.2

The OSI model

The **ISO** (International Organization for Standardization), established in **1947**, is a multinational body devoted to worldwide agreement on international standards. Its standard covering all aspects of network communication is the **Open Systems Interconnection (OSI) model**, first introduced in the late 1970s.

DO NOT CONFUSE THESE

ISO is the organisation. OSI is the model.

An *open system* is a set of protocols that allows two different systems to communicate **regardless of their underlying architecture**. The purpose of OSI is to allow communication between different systems without requiring changes to their hardware or software.

Layered architecture

OSI has **seven layers**. Three ideas govern how they fit together:

- 1 Each layer has a defined set of duties.** No layer does another layer's job.

- 2 Each layer uses the services of the layer just below it, and provides services to the layer just above it.** The boundary between two layers is called an **interface**.
- 3 Peer-to-peer processes.** Layer n at the sender communicates with layer n at the receiver, and with nothing else. Layer 4 at your machine is having a conversation with layer 4 at the far machine, even though the actual bits go all the way down to layer 1, across the wire, and all the way back up.

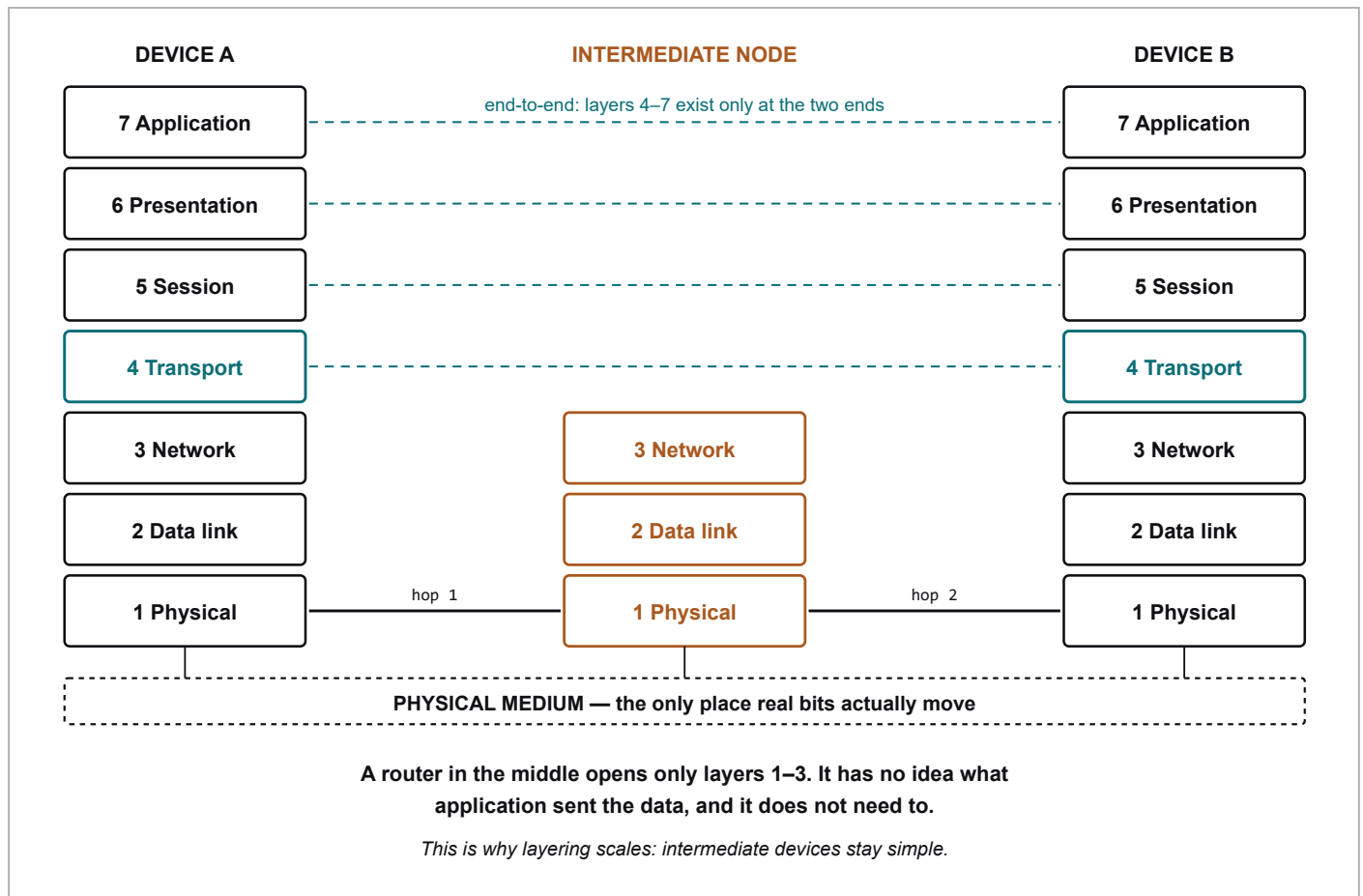
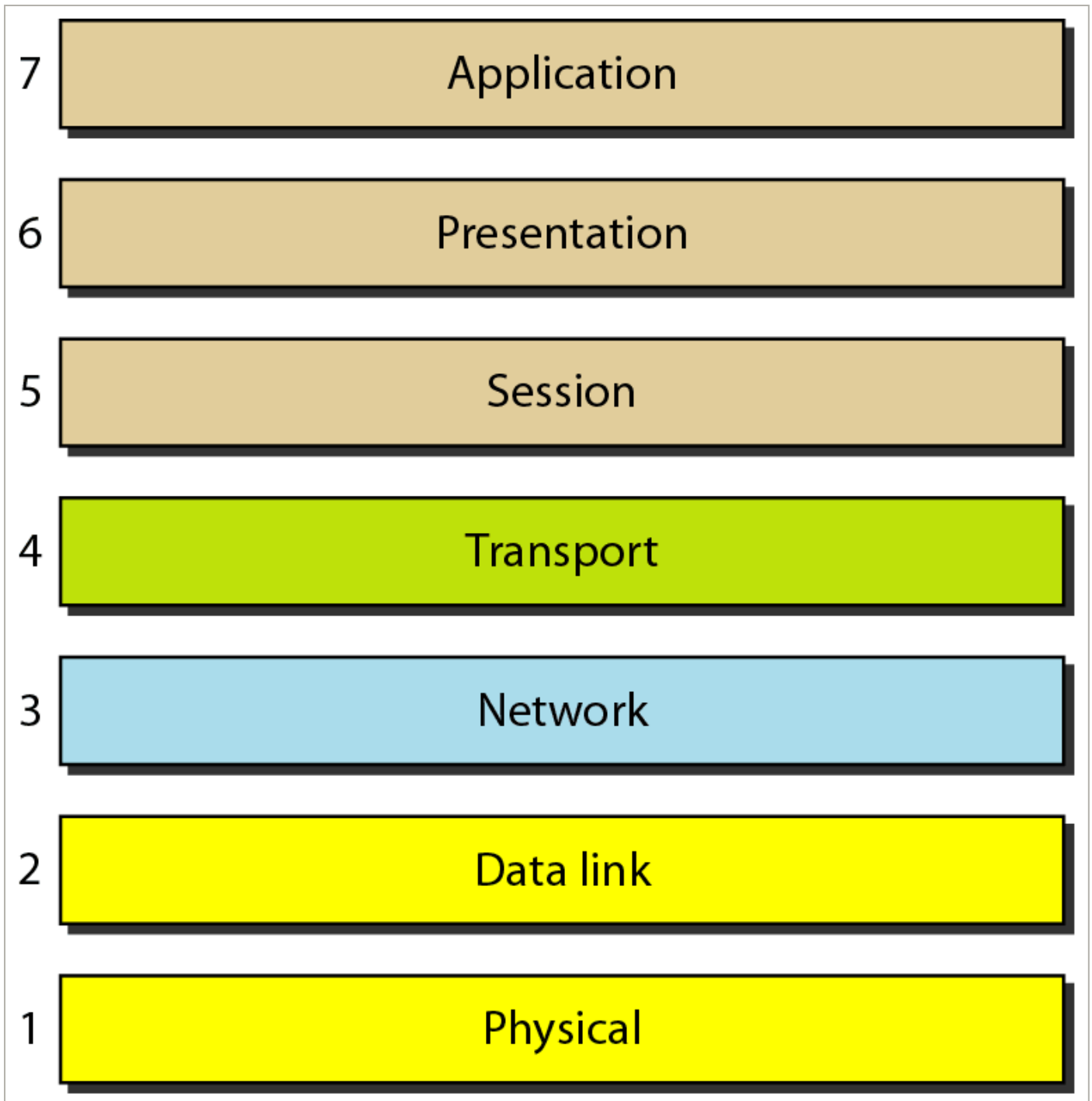


Fig 2.2 Peer-to-peer processes across the OSI stack. The teal dashed lines show which layers hold a conversation; the solid lines at the bottom show where bits are actually handed over. The intermediate node is missing layers 4-7 entirely — that absence is the mechanism, not an omission in the drawing.



Slide 27 The seven layers of the OSI model, from the lecture deck.

Encapsulation

When data travels down the stack, each layer wraps what it received from above inside its own envelope. This wrapping is called **encapsulation**.

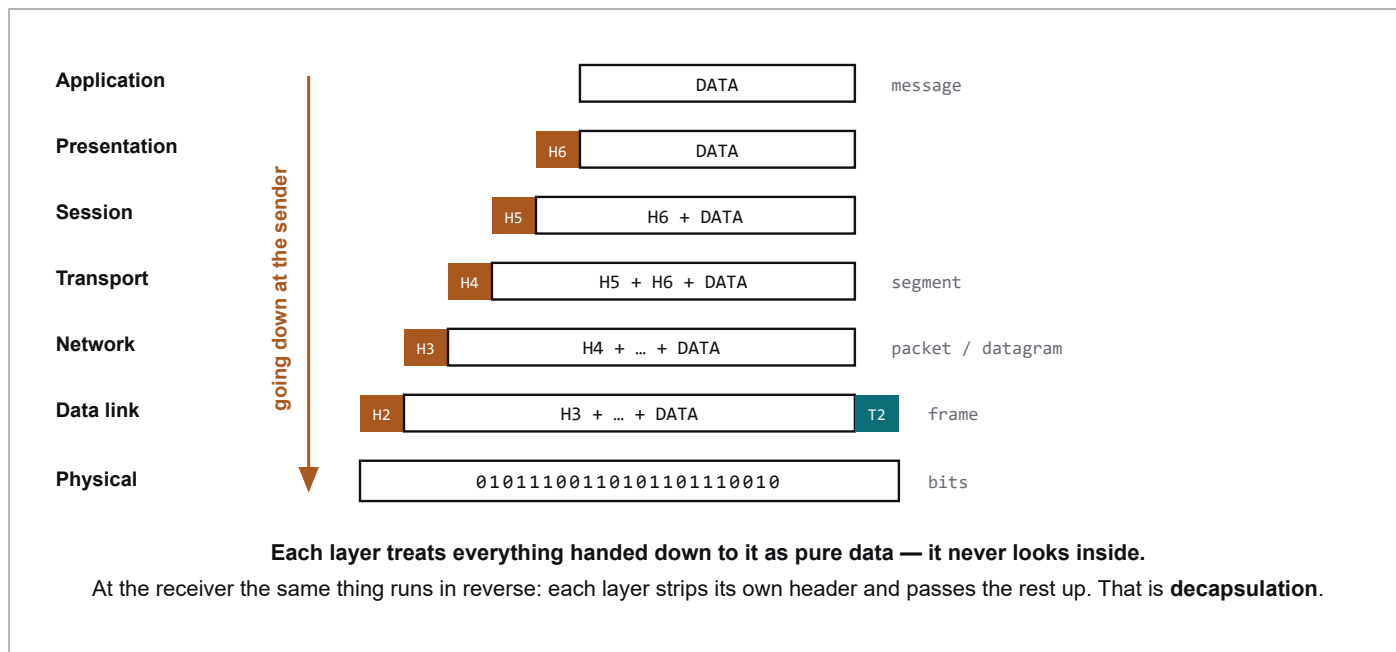
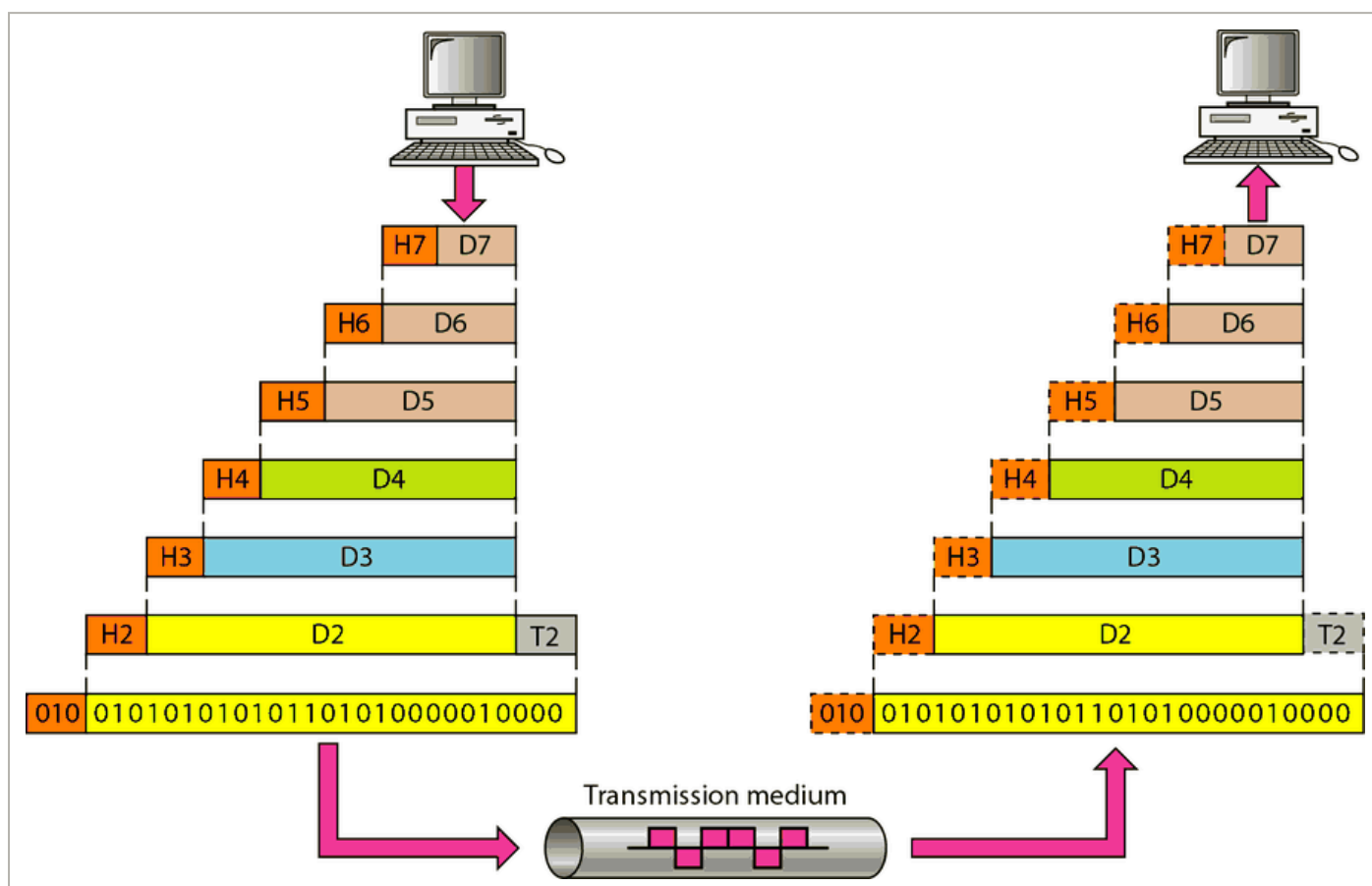


Fig 2.3 Encapsulation. Note that only the **data link layer** adds a trailer (T2) as well as a header — that trailer carries the error-check bits, which is why it must come last, after everything else is in place.



Slide 29 An exchange using the OSI model — headers added on the way down at the sender and stripped on the way up at the receiver.

PLAIN WORDS

Encapsulation is like posting a gift. You put the gift in a box, the box in a bag, the bag in a courier packet, and the courier puts a barcode sticker on the outside. Every wrapper is added by someone who does not care what is inside

— they only care about their own sticker. At the far end, each wrapper is removed by the matching person.

2.3

The seven layers, one at a time

Going from the bottom up. For each layer, notice **the unit of data it handles** and **how far its responsibility reaches** — those two facts answer most exam questions.

Layer 1 — Physical layer

The physical layer is responsible for **moving individual bits from one hop (node) to the next**. It coordinates the functions needed to carry a bit stream over a physical medium, and it deals with the mechanical and electrical specifications of the interface and the medium.

Its duties:

- **Physical characteristics of interfaces and medium** — the connector, the cable, the pins.
- **Representation of bits** — how a 0 and a 1 are turned into a signal (voltage levels, light pulses). This is encoding.
- **Data rate** — how many bits are sent per second, which fixes the duration of one bit.
- **Synchronization of bits** — the sender's clock and the receiver's clock must be synchronised, or the receiver will sample at the wrong moments.
- **Line configuration** — point-to-point or multipoint.
- **Physical topology** — mesh, star, bus, ring or hybrid.
- **Transmission mode** — simplex, half-duplex or full-duplex.

Layer 2 — Data link layer

The data link layer is responsible for **moving frames from one hop (node) to the next**. It takes a raw, unreliable bit pipe from the physical layer and turns it into a link that looks reliable to the layer above.

Its duties:

- **Framing** — divide the bit stream into manageable units called frames. (Module 5, §5.1.)
- **Physical addressing** — add a header with the sender's and receiver's physical (MAC) addresses. If the frame is going outside the network, the receiver address is that of the router that connects to the next network.
- **Flow control** — stop a fast sender from drowning a slow receiver. (§5.2.)
- **Error control** — detect damaged or lost frames and get them re-sent; usually done with a trailer. (§5.2.)
- **Access control** — when several devices share one link, decide which one may transmit. (Module 6.)

HOP-TO-HOP DELIVERY

“Hop” means one link, from one node to the next node. The data link layer's job ends at the far end of *one* wire. It has no idea there is a wider journey going on.

Layer 3 — Network layer

The network layer is responsible for the **delivery of a packet from the original source device to the final destination device** — possibly across many networks and many hops. This is called **source-to-destination delivery**.

Its duties:

- **Logical addressing** — add a header with the source and destination *logical* (IP) addresses, which work across network boundaries.
- **Routing** — when independent networks are connected into an internetwork, choose the route a packet should take.

Layer 4 — Transport layer

The transport layer is responsible for the **delivery of a message from one process to another**. A process is a running program. This is **process-to-process delivery**, and it must be *reliable*.

Its duties:

- **Service-point (port) addressing** — the network layer gets a packet to the right computer; the transport layer gets it to the right *program* on that computer.
- **Segmentation and reassembly** — break a message into numbered segments, and put them back together in order at the far end, replacing any that got lost.
- **Connection control** — it may be *connectionless* (each segment sent independently) or *connection-oriented* (set up a connection first, use it, then release it).
- **Flow control** — but end-to-end, across the whole path, not just one link.
- **Error control** — again end-to-end, and process-to-process.

EASY TO CONFUSE

The **data link** layer does flow and error control on **one hop**. The **transport** layer does flow and error control **end to end**, over the entire journey. Same words, completely different scope.

Layer 5 — Session layer

The session layer is the **dialog controller** of the network. It is responsible for **dialog control and synchronization**.

- **Dialog control** — it lets two systems enter into a dialogue, in half-duplex or full-duplex mode.
- **Synchronization** — it adds *checkpoints* to a data stream. If you are sending a 2000-page file and you place a checkpoint every 100 pages, then a failure at page 523 means resending from page 501, not page 1.

Layer 6 — Presentation layer

The presentation layer is concerned with the **syntax and meaning** of the information exchanged. It is responsible for **translation, compression, and encryption**.

- **Translation** — two computers may use different character encodings or number formats. The presentation layer converts the sender's format into a common format, and then into the receiver's format.
- **Encryption** — transform the data so it is unreadable to anyone intercepting it, and decrypt it at the other end. Needed for privacy.

- **Compression** — reduce the number of bits to be sent. Particularly important for text, audio and video.

Layer 7 — Application layer

The application layer is responsible for **providing services to the user**. It is the window through which the user (a person or a piece of software) reaches the network.

The services it provides include:

- **Network virtual terminal** — lets a user log on to a remote host as if sitting at it.
- **File transfer, access, and management (FTAM)** — access, retrieve, and manage files on a remote computer.
- **Mail services** — e-mail forwarding and storage.
- **Directory services** — distributed databases and lookup of global information.

SUMMARY OF ALL SEVEN OSI LAYERS				
#	LAYER	RESPONSIBILITY IN ONE LINE	DATA UNIT	SCOPE
7	Application	To allow access to network resources	Message	End to end
6	Presentation	To translate, encrypt, and compress data	Message	End to end
5	Session	To establish, manage, and terminate sessions	Message	End to end
4	Transport	To provide reliable process-to-process message delivery and error recovery	Segment	Process to process
3	Network	To move packets from source to destination; to provide internetworking	Packet / datagram	Source to destination
2	Data link	To organise bits into frames; to provide hop-to-hop delivery	Frame	Hop to hop
1	Physical	To transmit bits over a medium; to provide mechanical and electrical specifications	Bit	Hop to hop

MEMORY AID

From layer 7 down to 1: **All People Seem To Need Data Processing.**

From layer 1 up to 7: **Please Do Not Throw Sausage Pizza Away.**

2.4

The TCP/IP protocol suite

OSI is a *model* — a clean theoretical framework. **TCP/IP is what the Internet actually runs on.** The layers of TCP/IP do not exactly match those of OSI.

The original TCP/IP suite was defined as having **four layers**: host-to-network, internet, transport, and application. However, when TCP/IP is compared with OSI, we can say the suite is made of **five layers**: physical, data link, network, transport, and application.

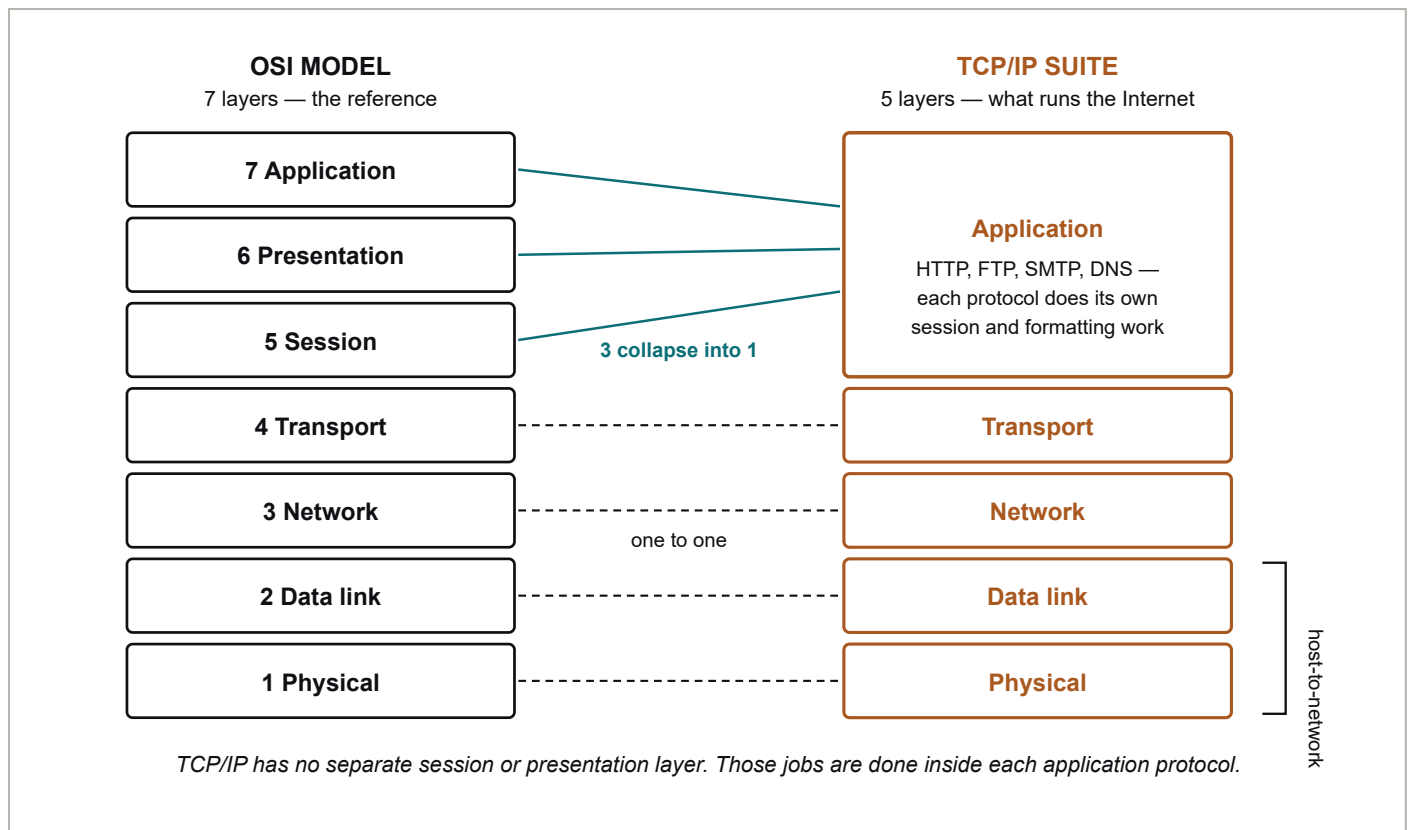
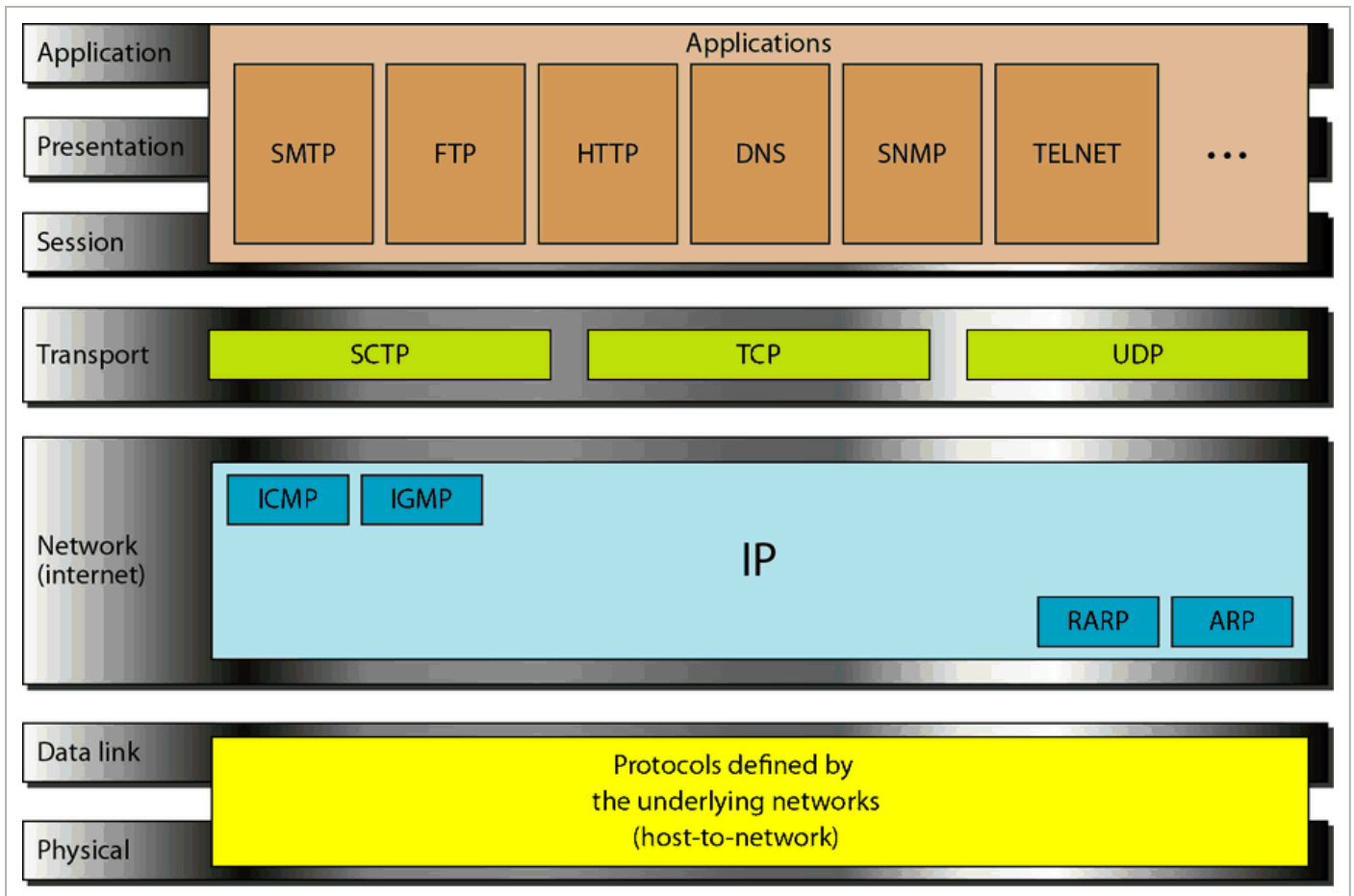


Fig 2.4 OSI against TCP/IP. The only real structural difference is at the top: the three upper OSI layers are folded into one TCP/IP application layer, and the original TCP/IP “host-to-network” layer is what we now split into data link and physical.



Slide 50 TCP/IP and the OSI model side by side, from the lecture deck.

OSI MODEL VS TCP/IP SUITE		
POINT	OSI MODEL	TCP/IP SUITE
Number of layers	7	4 originally (host-to-network, internet, transport, application); 5 when compared with OSI
Developed by	ISO	Grew out of research on the ARPANET; protocols came before the model
Session & presentation	Two separate layers	No separate layers — their duties sit inside the application protocols
Approach	Model first, then protocols. A reference framework.	Protocols first, then the model was written to describe them.
Use today	Mainly a teaching and reference model	Actually implemented — the Internet runs on it

2.5

Four levels of address

An internet using the TCP/IP protocols uses **four different levels of address**: physical, logical, port and specific. Each belongs to a different layer, and each answers a different question.

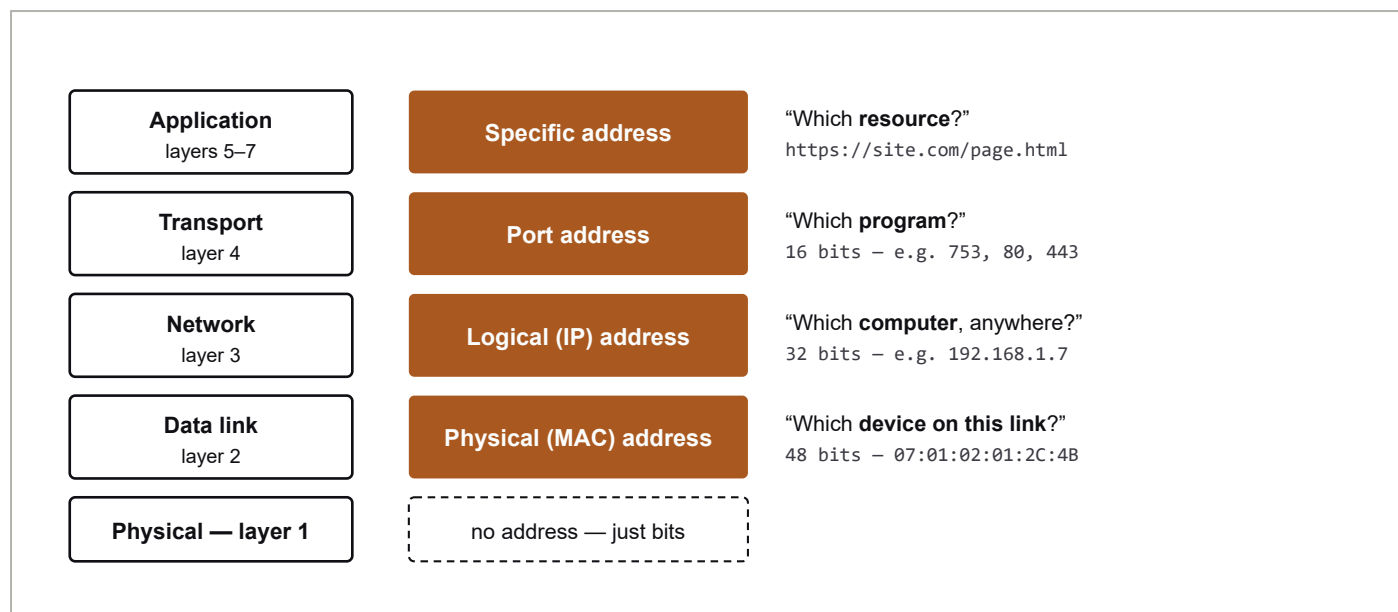
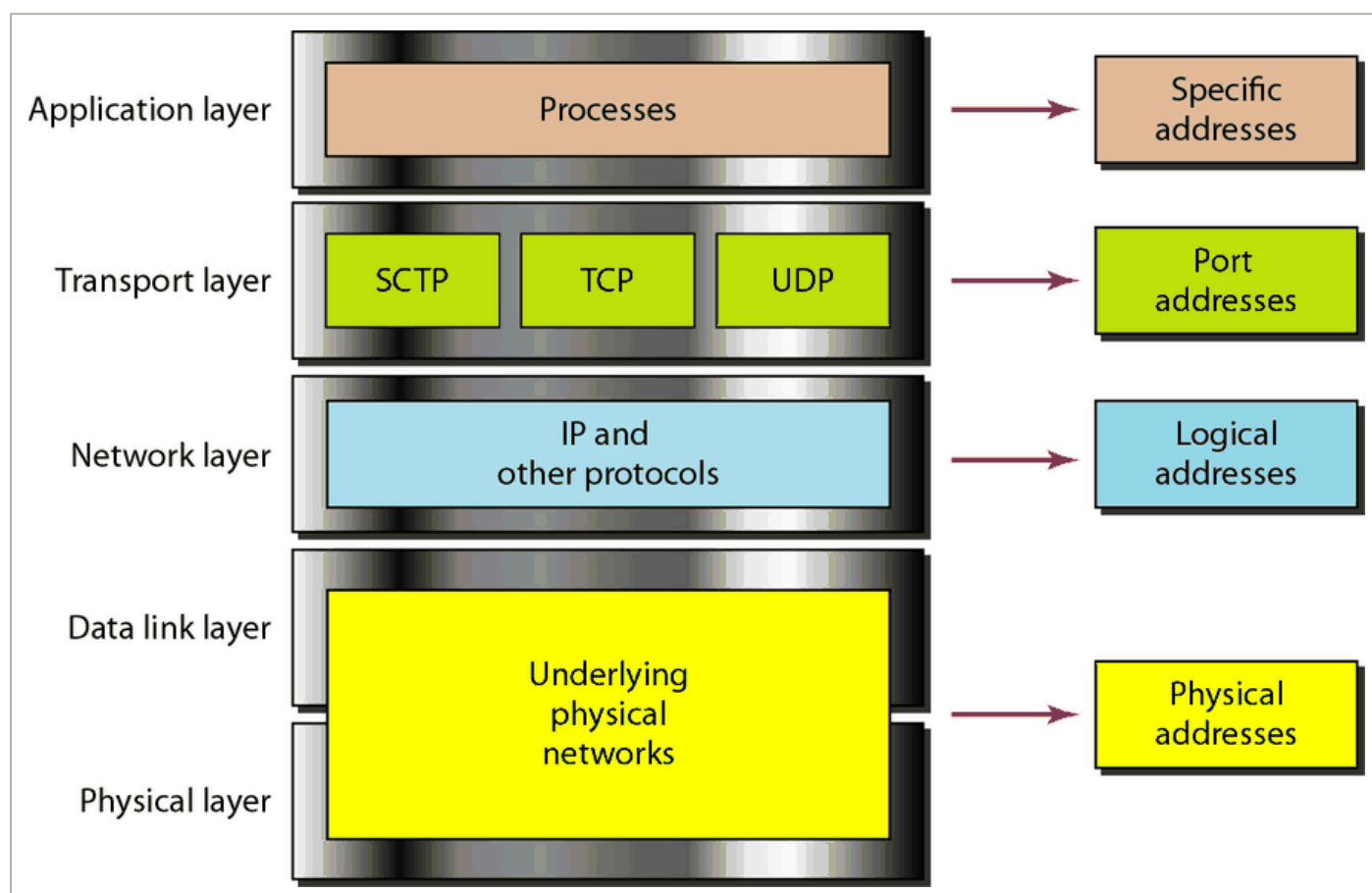


Fig 2.5 The four addresses and the layer each one lives at. Read the right-hand column as four different questions; each address exists because a different layer needs a different question answered.



Slide 53 Relationship of layers and addresses in TCP/IP.

Physical address (MAC address)

The physical address, also called the **link address**, is the address of a node as defined by its LAN or WAN. It is the **lowest-level address**, included in the frame used by the data link layer, and it is the *local* address — meaning it is only unique locally, and it does not work outside the network.

Most local-area networks use a **48-bit (6-byte) physical address**, written as **12 hexadecimal digits**, with every byte (2 hex digits) separated by a colon:

A 6-BYTE PHYSICAL ADDRESS	
	07: 01: 02: 01: 2C: 4B
6 bytes × 8 bits = 48 bits. Each byte is written as 2 hexadecimal digits, so 6 bytes = 12 hex digits.	

EXAMPLE 2.1 — FROM THE SLIDES

A node with physical address 10 sends a frame to a node with physical address 87. The two nodes are connected by a link in a bus-topology LAN. What happens?

The computer with physical address 10 is the **sender**; the computer with physical address 87 is the **receiver**. Because it is a bus, the frame physically reaches every node on the cable.

Each node reads the destination address field in the frame header. Every node except 87 sees an address that is not its own, and **drops** the frame. Only node 87 keeps it and passes the data up to its network layer.

Answer: All nodes receive the electrical signal; only the node whose physical address matches the destination field accepts the frame. The rest discard it.

Logical address (IP address)

Physical addresses are not enough for an internetwork, because different networks can use completely different physical addressing schemes. We need a **universal addressing system** in which each host can be identified uniquely, regardless of the underlying physical network. That is the **logical address** — in IPv4, a **32-bit** address.

In an internet with routers connecting several LANs, **each device has a pair of addresses — logical and physical — for each connection it has**. A computer connected to only one link has just one pair. A **router connected to three networks has three pairs of addresses**, one pair for each connection.

Port address

Getting the data to the right computer is not the end of the job. A computer runs many programs at once — a browser, an e-mail client, a file transfer. The transport layer needs to know **which process** the data belongs to. That label is the **port address**, and in TCP/IP it is **16 bits**, written as one decimal number:

A 16-BIT PORT ADDRESS
753

16 bits means port numbers run from 0 to 65535.

Specific address

Some applications have their own addressing that is friendlier for people. These are **application-specific addresses** — for instance, a **Uniform Resource Locator (URL)** or **Uniform Resource Identifier (URI)** that pinpoints a specific resource on the web, or an e-mail address. These get converted by the application into the port and logical addresses underneath.

The single most important rule about addresses

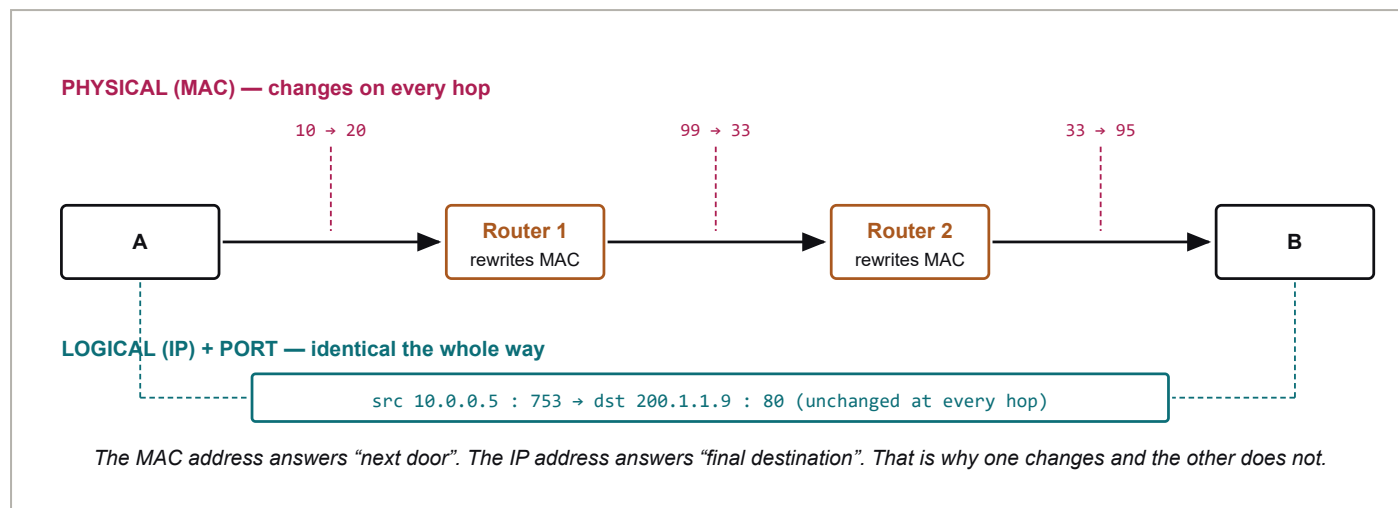


Fig 2.6 What changes and what does not, across a three-hop journey. Each router strips the old frame and builds a new one with new physical addresses, but never touches the IP addresses or the port numbers inside.

REMEMBER — THIS IS EXAMINED CONSTANTLY

The **physical addresses change from hop to hop**, but the **logical and port addresses usually remain the same** for the whole journey.

PLAIN WORDS

Imagine posting a parcel from Delhi to Chennai. The **final address on the label** (Chennai) never changes — that is the IP address. But at each sorting centre the parcel is put into a **different sack going to the next centre**, and each sack has its own tag — that is the MAC address. The **flat number** at the Chennai address, which tells you which family in the building it is for, is the port number.

Module 2 at a glance

- Layering exists so each layer does one job and trusts the layer below it; a layer can be replaced without touching the others.
- **OSI** has seven layers: physical, data link, network, transport, session, presentation, application. **TCP/IP** has the practical five.

- Going down the stack is **encapsulation** — each layer adds its own header. Going up is **decapsulation**.
- Four addresses, one per level: **physical (MAC)** for hop to hop, **logical (IP)** for host to host, **port** for process to process, **specific** for the application.

CHECK YOURSELF — MODULE 2

Q ISO or OSI — which is the organisation and which is the model? When was ISO established?

ISO (International Organization for Standardization) is the **organisation**, established in **1947**. **OSI** (Open Systems Interconnection) is the **model** it produced, introduced in the late 1970s.

Q Which layer is responsible for hop-to-hop delivery of frames? Which is responsible for source-to-destination delivery? Which for process-to-process?

Data link layer → hop-to-hop (frames). **Network** layer → source-to-destination (packets). **Transport** layer → process-to-process (segments).

Q Both the data link layer and the transport layer do “flow control” and “error control”. What is the difference?

The **scope**. The data link layer does it across **one link, one hop**. The transport layer does it **end to end**, across the whole path, between two processes.

Q Which layer does translation, compression and encryption?

The **presentation** layer (layer 6).

Q Which layer adds a trailer as well as a header, and why must the trailer come last?

The **data link** layer. Its trailer holds the error-detection bits (such as a CRC), which are calculated over everything else in the frame — so it can only be worked out and appended once the rest of the frame is complete.

Q How many bits are in a MAC address, an IPv4 address, and a port address? Write a MAC address in the standard form.

MAC = **48 bits** (6 bytes, 12 hex digits), e.g. `07:01:02:01:2C:4B`. IPv4 = **32 bits**. Port = **16 bits**, e.g. `753`.

Q A router is connected to three networks. How many address pairs does it have, and why?

Three pairs — one logical and one physical address for *each* connection. A device gets a pair per connection, so a computer with a single link has only one pair.

Q A packet goes from A through two routers to B. Which addresses change and which stay the same?

The **physical (MAC) addresses change at every hop** — each router builds a fresh frame for the next link. The **logical (IP) and port addresses stay the same** the entire way.

Q How many layers does TCP/IP have, and which OSI layers have no direct counterpart in it?

TCP/IP was defined with **four** layers (host-to-network, internet, transport, application), and is described as **five** when lined up against OSI. OSI's **session** and **presentation** layers have no separate counterpart — their jobs are handled inside the individual application protocols.

MODULE 03

Transmission Media

The actual physical stuff the bits travel through. Three kinds of cable, and three kinds of invisible wave. For each one: what it is made of, the physics that makes it work, what it costs you, and where it is used.

Lecture slides 62–100 · Chapter 3

3.1

Where transmission media sit, and the two classes

Strictly speaking, transmission media sit **below the physical layer** and are *controlled* by it. They are not a layer themselves — they are the road the physical layer drives on.

A transmission medium can be broadly defined as anything that can carry information from a source to a destination. There are two classes.

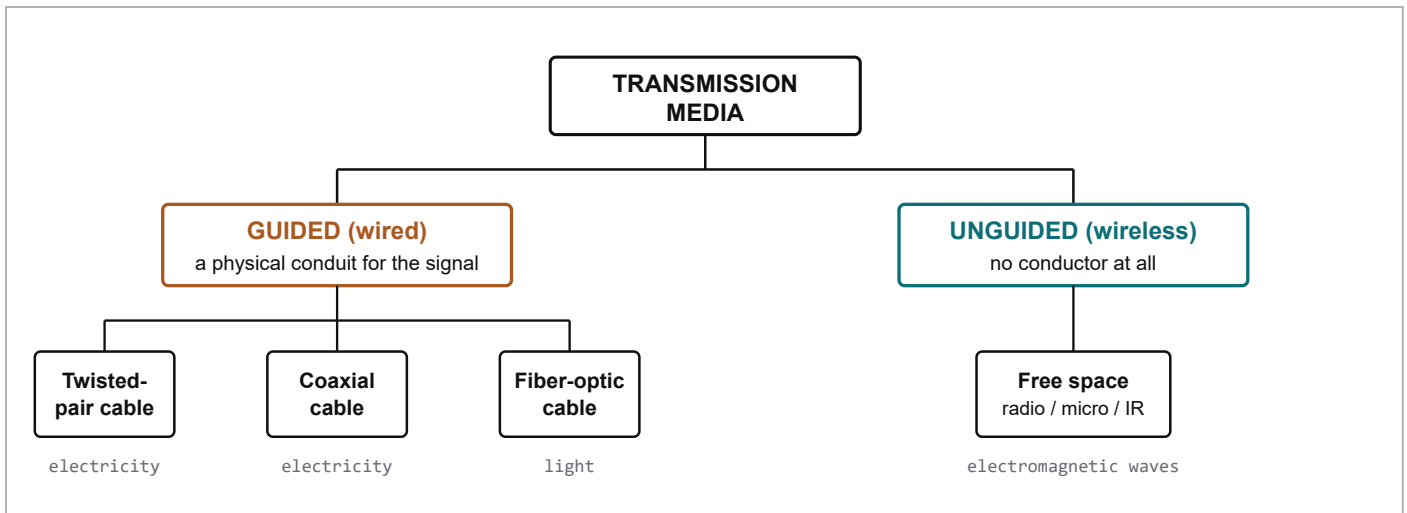
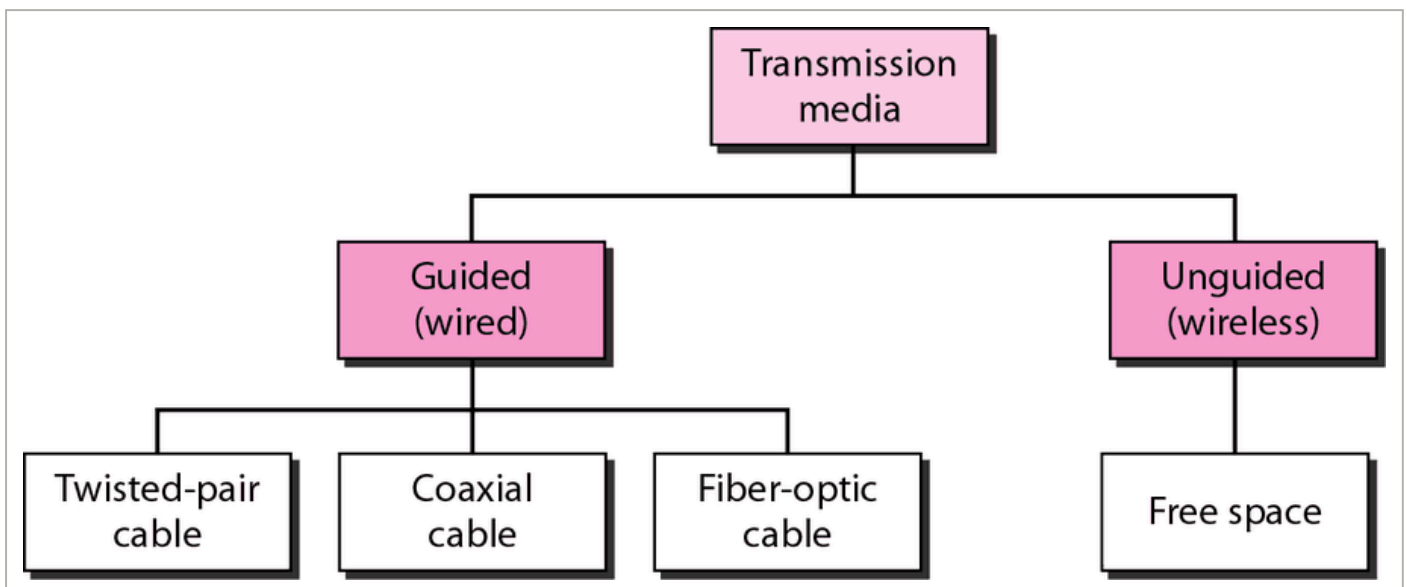


Fig 3.1 The two classes of medium. The bottom row shows *what physically carries the signal* in each — and that single fact explains most of the differences that follow.



Slide 64 Classes of transmission media, from the lecture deck.

- **Guided media** provide a **conduit from one device to another**. The signal is confined inside something physical. These are twisted-pair cable, coaxial cable, and fiber-optic cable.
- **Unguided media** transport electromagnetic waves **without using a physical conductor**. This is normally called *wireless* communication.

3.2

Twisted-pair cable

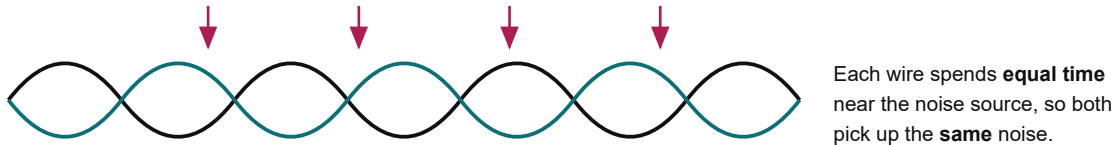
A twisted-pair cable is wiring in which **two conductors of a single circuit are twisted together**. Each wire is a copper conductor with its own plastic insulation.

Why the twisting

The twisting is not decoration — it is the entire reason the cable works, and it defeats two different problems.

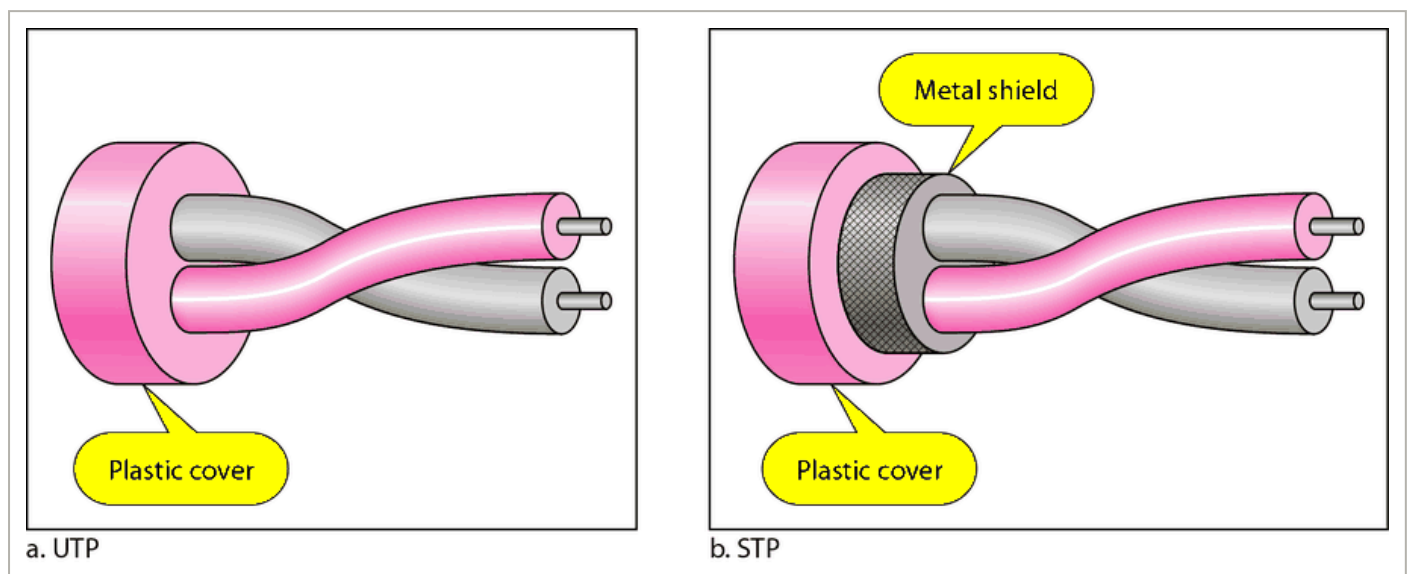
PARALLEL WIRES — the problem

Wire 1 gets a big dose of noise, wire 2 gets a small one. The difference is **unequal** — so subtracting the two signals leaves leftover noise. The data is corrupted.

TWISTED PAIR — the fix

The receiver subtracts wire 2 from wire 1. The wanted signal (opposite on the two wires) doubles; the identical noise on both wires **cancels to zero**. That is differential signalling.

Fig 3.2 The mechanism of twisting. Twisting does not block noise — it makes the noise *equal on both wires*, so that the receiver's subtraction removes it. This is why the fix is twisting *plus* differential signalling, and why one without the other does not work.



Slide 68 UTP and STP cables — note the extra foil shield on the STP.

The two problems twisting solves:

- **Electromagnetic interference (EMI)** from external sources — motors, fluorescent lights, power cables.
- **Crosstalk** from adjacent pairs inside the same cable — one pair's signal leaking into another.

Working principle, in three parts

- 1 **Twisting.** When two wires are twisted, any external EMI affects both wires **equally**. Because the wanted signals on the two wires are out of phase (opposite), the interference cancels out when they are compared.
- 2 **Differential signalling.** Two wires carry the same signal but with **opposite polarity**. The receiver looks at the *difference* between them, which further reduces noise and improves signal integrity.

- 3 Connectors.** Twisted-pair cables typically use **RJ45** connectors in networking applications — the familiar 8-pin clip-in plug.

UTP and STP

UTP — Unshielded Twisted Pair

The **most common** type. There is no shielding layer, which makes the cable more **flexible and easier to install**, and cheaper.

Widely used in telecommunication and networking.

Examples: **Cat 5e, Cat 6, Cat 6a**.

STP — Shielded Twisted Pair

Has a **shielding layer** — a metal foil or braided mesh — that protects the twisted pairs from external EMI.

Used in environments with **high interference** and for high-speed networks. Example: **Cat 7**.

NOTE ON NAMING

The term “twisted-pair cable” usually refers to a cable containing **several** twisted pairs. A Cat 5e, Cat 6 or Cat 7 cable typically contains **four twisted pairs — 8 wires in total**. The single most important use of twisted pair is the **standard Ethernet cable**.

Categories of UTP

CATEGORIES OF UNSHIELDED TWISTED-PAIR CABLE			
CAT	SPECIFICATION	DATA RATE	USE
1	Unshielded twisted-pair used in telephone	< 0.1 Mbps	Telephone
2	Unshielded twisted-pair originally used in T-lines	2 Mbps	T-1 lines
3	Improved Cat 2, used in LANs	10 Mbps	LANs
4	Improved Cat 3, used in Token Ring networks	20 Mbps	LANs
5	Cable wire is normally 24 AWG with a jacket and outside sheath	100 Mbps	LANs
5E	An extension to Cat 5 with extra features to minimise crosstalk and EMI	125 Mbps	LANs
6	Matched components from the same manufacturer; must be tested at 200 Mbps	200 Mbps	LANs
7	Sometimes called SSTP (shielded screen twisted-pair). Each pair individually wrapped in helical metallic foil, plus an overall foil shield and outside sheath. The shield cuts crosstalk and raises the data rate.	600 Mbps	LANs

Performance

Twisted-pair performance is measured by comparing **attenuation against frequency**. Attenuation is how much the signal weakens. The key result: **attenuation increases sharply at frequencies above 100 kHz**, which is why repeaters are needed at intervals, and why higher categories of cable (with better twisting and shielding) are required for higher data rates.

3.3

Coaxial cable

Coaxial cable — usually shortened to **coax** — carries signals of **higher frequency ranges** than twisted-pair. It does this by using a completely different geometry: instead of two side-by-side wires, one conductor sits *inside* the other, sharing the same axis. That is what “co-axial” means.

Structure — four layers, from the inside out

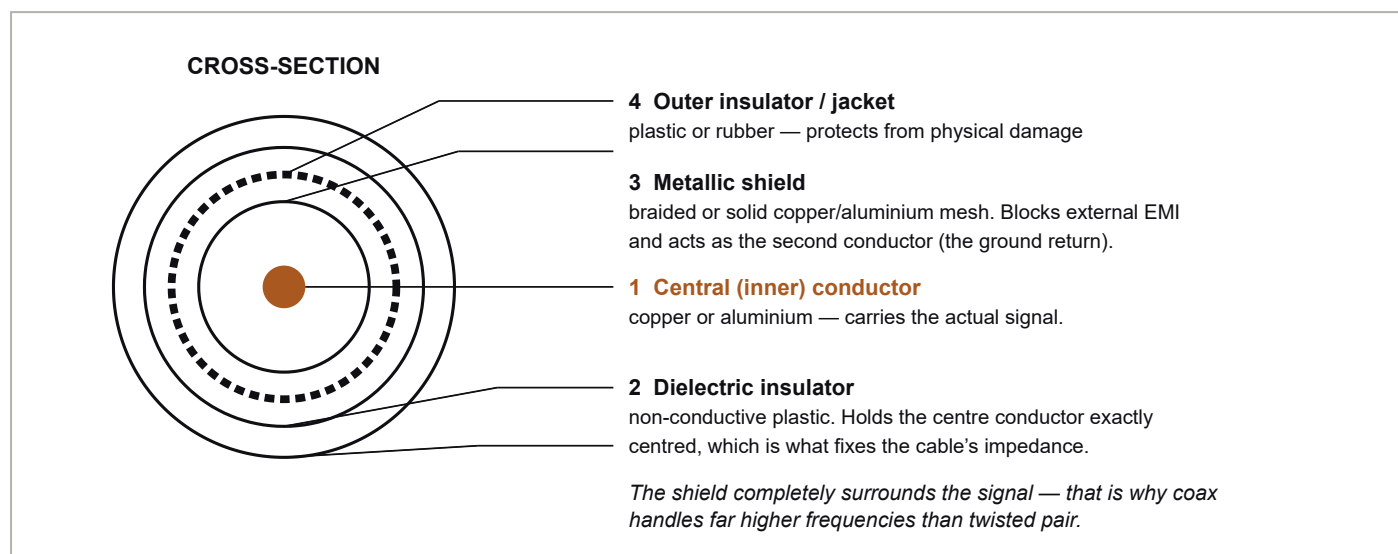
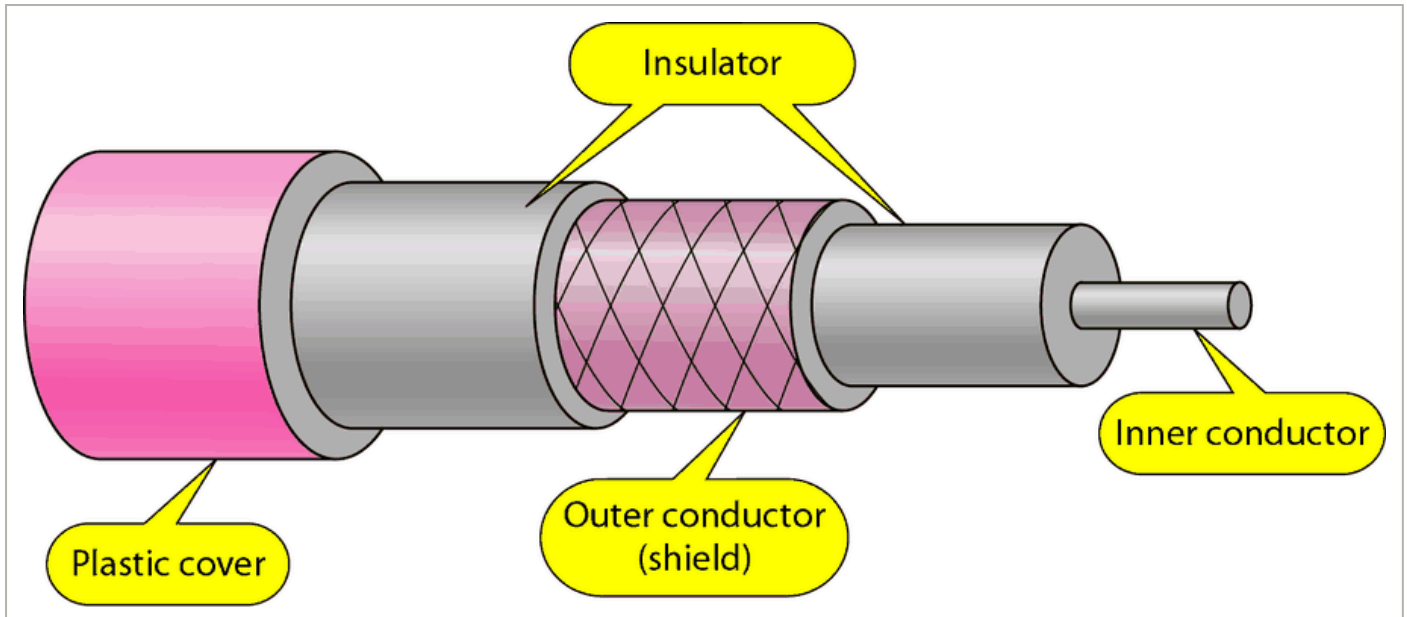


Fig 3.3 Coaxial cable in cross-section. The signal travels on the inner conductor and is *enclosed on all sides* by the shield, so almost no energy escapes and almost no interference gets in. Twisted pair has no such enclosure.



Slide 74 Coaxial cable as shown in the lecture deck.

- 1 Central (inner) conductor** — usually copper or aluminium. It carries the electrical signal.
- 2 Dielectric insulator** — surrounds the central conductor. Made of a non-conductive material such as plastic; it insulates the conductor.
- 3 Metallic shield** — a braided or solid metallic layer, often copper or aluminium, that protects the signal from external EMI.
- 4 Outer insulator** — the outermost layer, plastic or rubber, which protects the cable from physical damage.

Categories of coaxial cable

Coax is categorised by **RG (radio government) ratings**. Each RG number denotes a unique set of physical specifications.

CATEGORIES OF COAXIAL CABLE		
CATEGORY	IMPEDANCE	USE
RG-59	75 Ω	Cable TV
RG-58	50 Ω	Thin Ethernet
RG-11	50 Ω	Thick Ethernet

BNC connectors

A **BNC (Bayonet Neill-Concelman)** connector is a coaxial RF connector used for terminating coaxial cables. It is named after its inventors, Paul Neill and Carl Concelman, and is widely used in video and audio transmission, RF and microwave systems, and test equipment.

Its structure and features:

- **Bayonet locking mechanism** — a rotating ring with two small lugs that fit into matching slots on the mating connector. You align the lugs with the slots, push in, and twist the ring to lock. This gives a secure and reliable connection.
- **Centre pin** — makes contact with the central conductor of the cable and carries the signal.
- **Outer shield** — the outer metal shell contacts the cable's outer shield, providing grounding and EMI shielding.
- **Insulating dielectric** — separates the centre pin from the outer shield inside the connector, maintaining the characteristic impedance of the cable.

WHY IMPEDANCE KEEPS BEING MENTIONED

If the impedance changes anywhere along the path — at a badly made joint, for example — part of the signal **reflects back** instead of going forward. That reflection is noise. Keeping the impedance constant end to end is why the dielectric spacing and the connector design matter so much.

Performance and uses

Coax performance is also measured as **attenuation versus frequency**. Coax has a **much higher bandwidth** than twisted pair, but its attenuation is *much higher* than twisted pair's in absolute terms — the signal weakens fast, so repeaters are needed more frequently.

Uses: transmitting **cable television signals**, **internet connections**, and other data communications; also thin and thick Ethernet in older LANs.

3.4

Fiber-optic cable

Optical fiber is a flexible, transparent fiber made of high-quality glass (silica) or plastic, slightly thicker than a human hair, that acts as a **waveguide or light pipe** to transmit light between the two ends of the fiber.

It is used primarily in telecommunications and networking, where it enables long-distance, high-speed data transmission. It works on the principle of **total internal reflection**, guiding light through the core with minimal loss — which makes it very effective for moving large amounts of data over long distances with high fidelity and low attenuation.

The physics: bending of light and total internal reflection

Light travels in a straight line as long as it moves through a single uniform substance. If it passes from one substance into another, its direction **changes**. How it changes depends on the angle at which it hits the boundary, and on the **refractive index** (the optical density) of the two substances.

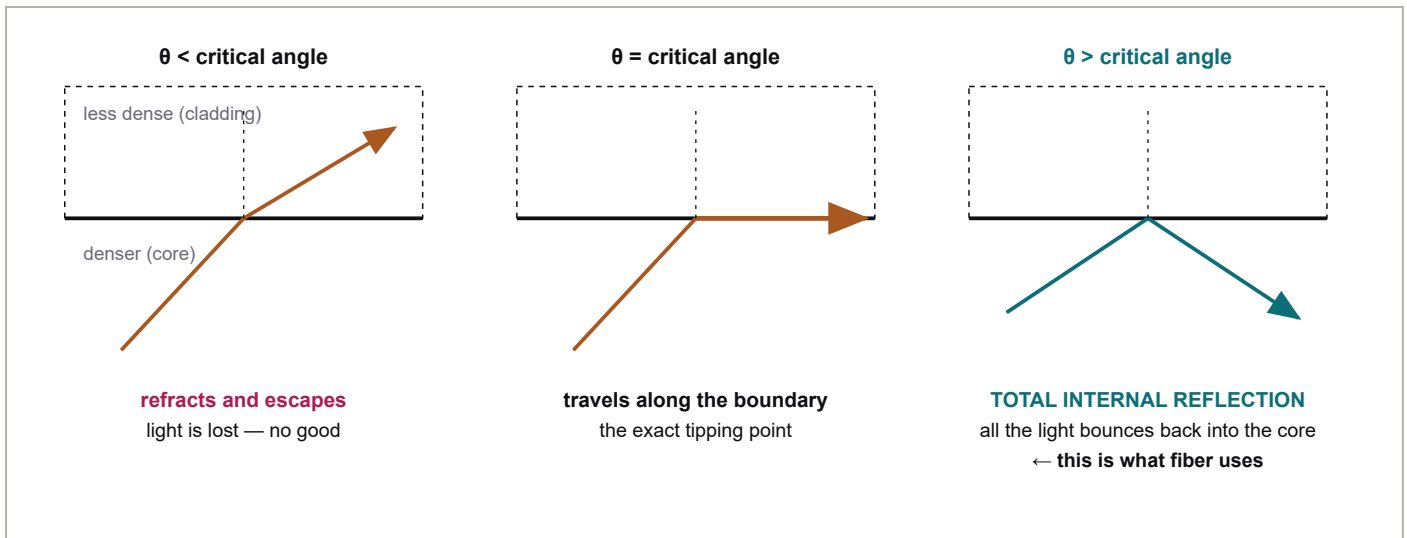
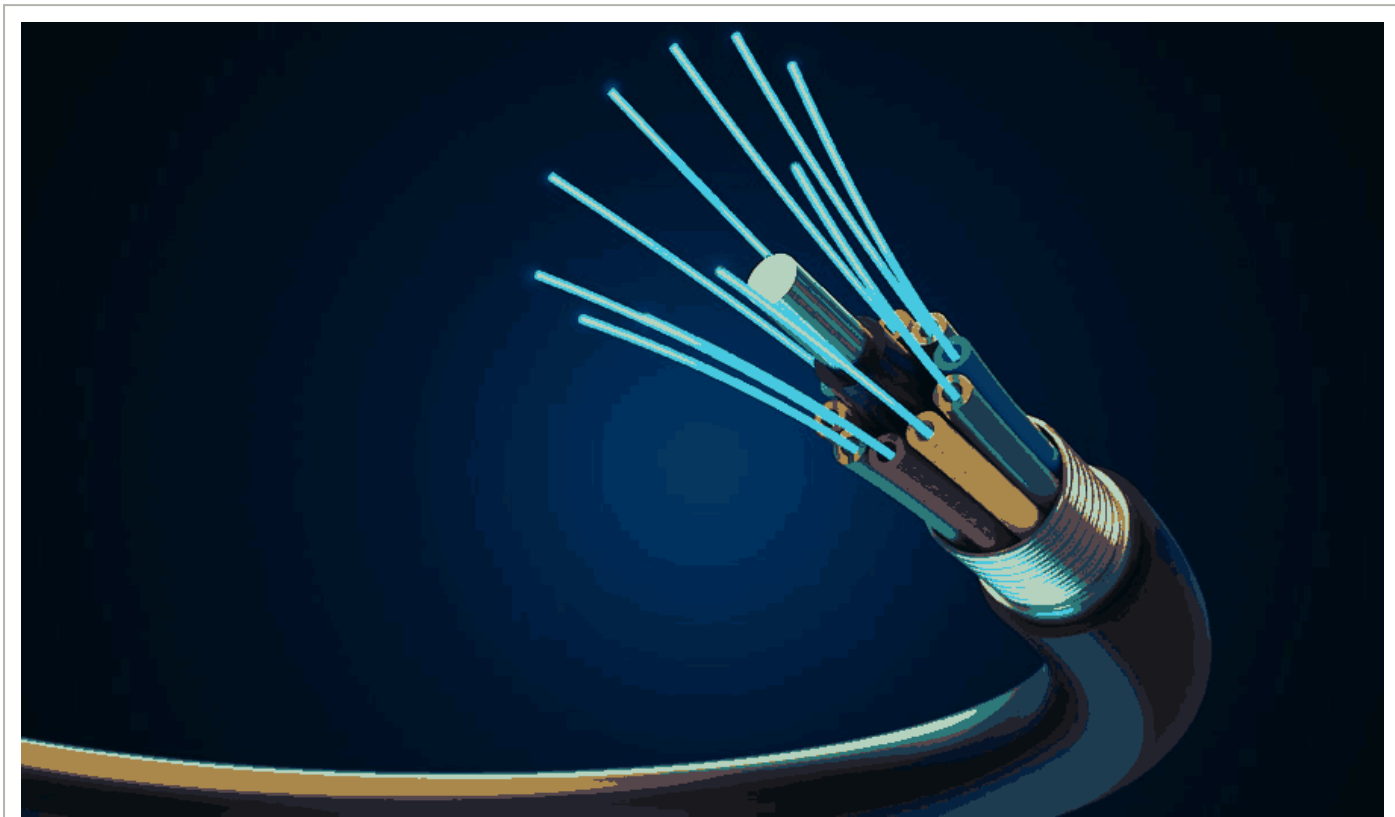


Fig 3.4 The three cases at a core–cladding boundary. Fiber optics works by making sure light *always* arrives at more than the critical angle, so case C happens every time and no light ever leaves the core.



Slide 81 Optical fiber — core, cladding and jacket.

The rule to remember: when light goes from a **denser** medium into a **less dense** one, and the angle of incidence is **greater than the critical angle**, the light does not pass through at all — it is **reflected entirely back** into the denser medium. This is **total internal reflection**. The cladding has a **lower refractive index** than the core precisely so that this happens.

Structure of an optical fiber cable

LAYERS OF A FIBER-OPTIC CABLE, INSIDE OUT			
#	LAYER	MADE OF	FUNCTION
1	Core	High-purity glass or plastic	The central part where the light actually travels. Its diameter decides the mode of transmission — single-mode or multi-mode.
2	Cladding	Glass or plastic with a lower refractive index than the core	Reflects light back into the core by total internal reflection, keeping the signal inside so it can travel long distances.
3	Coating / buffer coating	Plastic	Protects core and cladding from physical damage and moisture; adds mechanical strength.
4	Strengthening fibers	Aramid yarn, e.g. Kevlar	Adds strength so the cable does not break during handling, installation and operation.
5	Outer jacket	PVC or polyurethane	Protects the whole assembly from moisture, chemicals and abrasion.

Propagation modes

There are two modes for propagating light along an optical channel, and the multimode one has two sub-types — giving three arrangements in total.

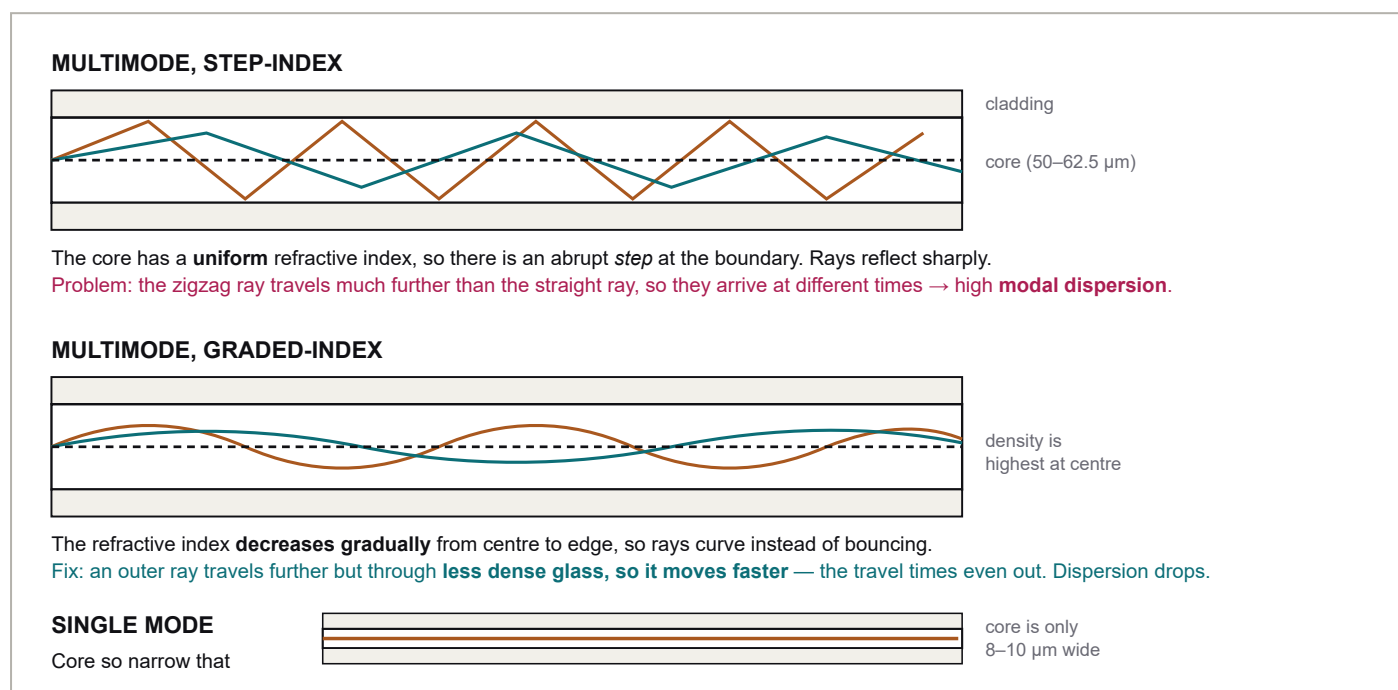
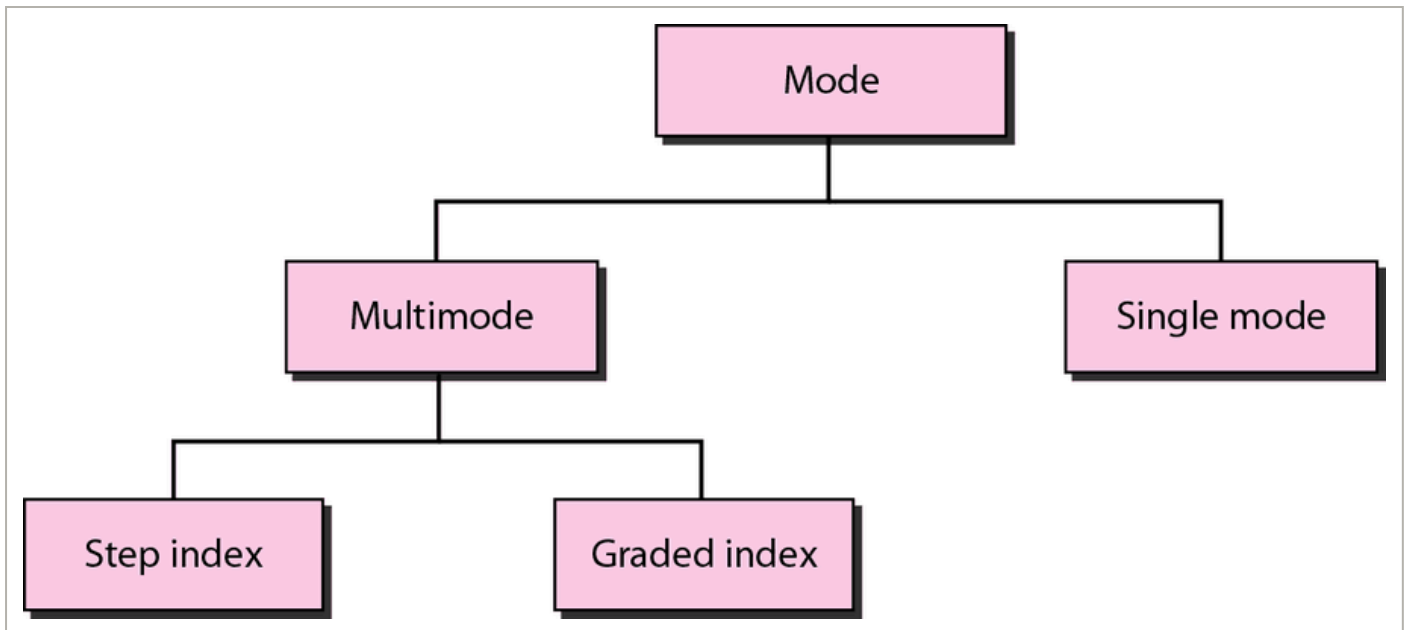
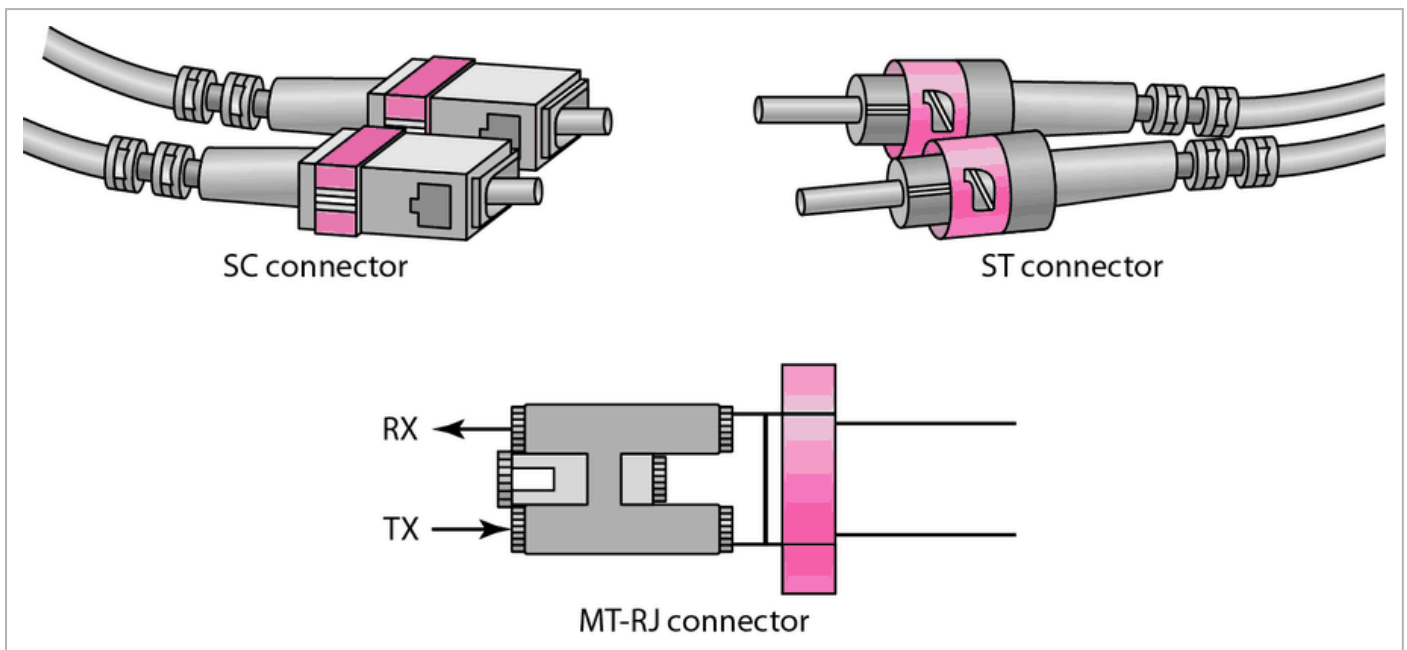


Fig 3.5 The three propagation modes. Notice what actually changes: step-index rays *bounce*, graded-index rays *curve*, and single-mode has a core too narrow for more than one path. Each step reduces modal dispersion — the spreading out of a pulse because its rays arrive at different times.



Slide 86 Propagation modes: multimode step-index, multimode graded-index and single mode.



Slide 91 Fiber-optic cable connectors — SC, ST and MT-RJ.

SINGLE-MODE FIBER VS MULTI-MODE FIBER		
PARAMETER	SINGLE-MODE FIBER (SMF)	MULTI-MODE FIBER (MMF)
Definition	Carries light directly down the fiber with minimal loss	Can carry multiple light rays (modes) simultaneously
Core diameter	8–10 μm	50–62.5 μm
Cladding diameter	125 μm	125 μm
Light source	Laser	LED or laser
Wavelengths used	1310 nm, 1550 nm	850 nm, 1300 nm
Bandwidth	Higher — virtually unlimited	Lower — up to several hundred MHz/km
Modal dispersion	Minimal	Higher
Attenuation	Lower (0.4 dB/km at 1310 nm)	Higher (3.5 dB/km at 850 nm)
Cost	Higher	Lower
Installation	More complex	Easier

STEP-INDEX FIBER VS GRADED-INDEX FIBER		
PARAMETER	STEP-INDEX FIBER	GRADED-INDEX FIBER
Core refractive index	Uniform	Gradually decreasing from centre to edge
Cladding refractive index	Uniform, lower than core	Uniform, lower than the outer core
Index profile	Abrupt step at the core–cladding boundary	Gradual, parabolic-like profile
Light path	Sharp reflections within the core	Curved paths, from continuous refraction
Modal dispersion	Higher in multi-mode fibers	Lower — travel times are more uniform
Bandwidth	Lower in multi-mode fibers	Higher, because dispersion is reduced
Manufacturing	Simpler and cheaper	More complex and expensive
Applications	Single-mode for long-distance, high-bandwidth	Multi-mode for medium distance, high-bandwidth

Fiber types

Fiber sizes are written as **core diameter / cladding diameter**, both in micrometres.

STANDARD FIBER TYPES			
TYPE	CORE (MM)	CLADDING (MM)	MODE
50/125	50.0	125	Multimode, graded index
62.5/125	62.5	125	Multimode, graded index
100/125	100.0	125	Multimode, graded index
7/125	7.0	125	Single mode

Connectors and performance

Three connector types are used for fiber:

- **SC** (subscriber channel) — a push/pull locking connector, used for cable TV.
- **ST** (straight-tip) — a bayonet-locking connector, more reliable than SC, used for networking devices.
- **MT-RJ** — the same size as RJ45, so it is convenient where space is tight.

Performance: attenuation in fiber is **flatter and far lower** than in either copper medium. A repeater is needed only every 50 km or so, against about 5 km for coax and 2 km for twisted pair.

Advantages of fiber

Higher bandwidth than any copper cable. **Less signal attenuation**, so far fewer repeaters. **Immunity to electromagnetic interference** — light is not affected by electrical noise at all. **Resistance to corrosive materials**, since glass is more inert than copper. **Light weight**. **Greater immunity to tapping**, so more secure.

Disadvantages of fiber

Installation and maintenance need expertise — splicing glass is a skilled job. **Unidirectional light propagation:** if you need two-way traffic you need two fibers. And the **cost** of the cable and the interface equipment is higher than copper.

3.5

Unguided media: wireless

Unguided media transport **electromagnetic waves without using a physical conductor**. The signals are broadcast through air (or vacuum) and so are available to anyone with a device able to receive them.

Wireless transmission uses the band from about **3 kHz to 900 THz**. Within it, three groups matter for communication.

THE THREE WIRELESS WAVE GROUPS			
WAVES	FREQUENCY RANGE	PROPAGATION	USED FOR
Radio waves	3 kHz - 1 GHz (broadly 3 Hz - 300 GHz)	Omnidirectional — travel in all directions, so sender and receiver antennas need not be aligned. Can penetrate walls. Low-band waves can pass through the ionosphere.	Multicast communication: AM/FM radio, television, paging systems, maritime radio
Microwaves	1 GHz - 300 GHz (a subset of radio, from 300 MHz up)	Unidirectional — can be narrowly focused, so sending and receiving antennas must be aligned. Line-of-sight only; cannot penetrate walls. Very high bandwidth available.	Unicast communication: cellular telephones, satellite networks, wireless LANs
Infrared	300 GHz - 430 THz (700 nm - 1 mm wavelength)	Cannot penetrate walls at all. That containment is an advantage: an infrared system in one room does not interfere with one in the next room.	Short-range communication in a closed area using line-of-sight: TV remotes, IrDA device-to-device links

Infrared waves are electromagnetic radiation with wavelengths **longer than visible light but shorter than microwaves**. They divide into three regions:

- **Near-infrared (NIR):** 700 nm to 1,400 nm
- **Mid-infrared (MIR):** 1,400 nm to 3,000 nm
- **Far-infrared (FIR):** 3,000 nm to 1 mm

Propagation methods

Wireless signals reach their destination in one of three ways, and which one applies depends entirely on frequency.

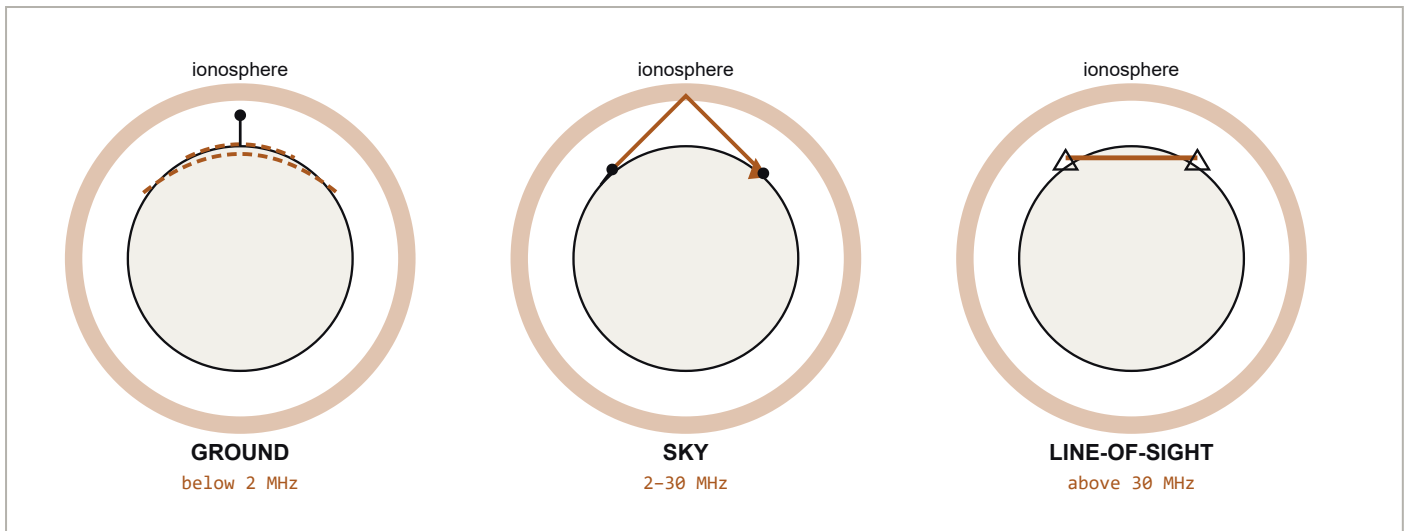


Fig 3.6 The three propagation methods, with the frequency range that produces each. The mechanism differs: ground waves follow the Earth's curvature, sky waves are *reflected by the ionosphere* back down, and line-of-sight waves are not bent at all so both antennas must see each other.

- **Ground propagation** (below 2 MHz) — low-frequency signals travel in all directions from the antenna and **follow the curvature of the planet**, hugging the Earth. Distance depends on the power of the signal: more power, more distance.
- **Sky propagation** (2–30 MHz) — higher-frequency signals radiate upward into the ionosphere, where they are **reflected back to Earth**. This allows greater distances with lower power.
- **Line-of-sight propagation** (above 30 MHz) — very high frequency signals travel in **straight lines**, so the antennas must be directional, facing each other, and either tall enough or close enough not to be blocked by the Earth's curvature.

Bands

FREQUENCY BANDS AND THEIR APPLICATIONS			
BAND	RANGE	PROPAGATION	APPLICATION
VLF (very low frequency)	3–30 kHz	Ground	Long-range radio navigation
LF (low frequency)	30–300 kHz	Ground	Radio beacons and navigational locators
MF (middle frequency)	300 kHz–3 MHz	Sky	AM radio
HF (high frequency)	3–30 MHz	Sky	Citizens band (CB), ship and aircraft communication
VHF (very high frequency)	30–300 MHz	Sky and line-of-sight	VHF TV, FM radio
UHF (ultrahigh frequency)	300 MHz–3 GHz	Line-of-sight	UHF TV, cellular phones, paging, satellite
SHF (superhigh frequency)	3–30 GHz	Line-of-sight	Satellite communication
EHF (extremely high frequency)	30–300 GHz	Line-of-sight	Radar, satellite

SPOT THE PATTERN

Read the propagation column top to bottom: **ground** → **sky** → **line-of-sight**. As frequency rises, the signal stops bending — first it stops following the ground, then it stops bouncing off the ionosphere, and finally it goes dead straight. Everything else in the table follows from that.

Antennas

Omnidirectional antenna

Sends out waves in **all directions**. Used for radio waves, where sender and receiver do not need to be aligned. Cheap, and good for broadcast (one-to-many).

Unidirectional antenna

Focuses the wave into a **narrow beam**. Used for microwaves. The two types are the **parabolic dish** (which gathers all incoming waves to a single focal point) and the **horn antenna** (which scoops the signal and directs it). Both antennas must be carefully aligned.

Putting all the media side by side

MASTER COMPARISON OF TRANSMISSION MEDIA						
MEDIUM	SIGNAL CARRIED	BANDWIDTH	ATTENUATION	EMI IMMUNITY	COST	TYPICAL USE
Twisted pair	Electrical	Low–moderate	High above 100 kHz	Poor (UTP) to fair (STP)	Lowest	Telephone, LAN cabling
Coaxial	Electrical	Higher than twisted pair	Higher than twisted pair	Good — full shield	Medium	Cable TV, older Ethernet
Fiber optic	Light	Highest — virtually unlimited	Lowest by far	Total — immune	Highest	Backbones, long distance
Radio waves	EM waves	Low–moderate	Varies with weather and distance	Poor — open to interference	Low	Broadcast radio, TV, paging
Microwaves	EM waves	Very high	Affected by rain	Poor, but beam is narrow	Medium	Cellular, satellite, Wi-Fi
Infrared	EM waves	High	Blocked by any obstacle	Good indoors — walls contain it	Very low	Remote controls, short links

Module 3 at a glance

- Media are **guided** (twisted pair, coaxial, fibre) or **unguided** (radio, microwave, infrared).
- Twisted pair**: the twist is what cancels interference. UTP is cheap and everywhere; STP adds a shield. **Coaxial** carries more bandwidth than twisted pair.
- Fibre** works by **total internal reflection** — the ray must hit the boundary at more than the critical angle. Modes: **single mode** (one narrow path, longest reach), **multimode step index** (many paths, most spreading), **multimode graded index** (refractive index falls away from the centre, so the long paths travel faster and the spreading is reduced).
- Fibre wins on bandwidth, attenuation, weight and immunity to electrical noise; it loses on cost, fragility and the skill needed to splice it.

CHECK YOURSELF — MODULE 3

Q What is the difference between guided and unguided media? Give the members of each class.

Guided media provide a physical conduit for the signal: twisted-pair cable, coaxial cable, fiber-optic cable. **Unguided** media carry electromagnetic waves with no physical conductor — free space, using radio waves, microwaves and infrared.

Q Why are the wires in a twisted pair twisted? Would twisting alone be enough?

Twisting makes any external interference affect **both wires equally**. On its own that would not help — it works because the receiver uses **differential signalling**: it takes the *difference* between the two wires, so the identical noise cancels while the wanted signal (opposite on the two wires) survives. Twisting also reduces **crosstalk** between neighbouring pairs.

Q UTP or STP — which is more flexible and cheaper, and which handles a noisy factory better?

UTP is more flexible, easier to install and cheaper, because it has no shield. **STP** handles high-interference environments better, because its metal foil or braided mesh shield blocks external EMI.

Q How many twisted pairs and how many wires are in a standard Cat 6 cable? What connector does it use?

Four twisted pairs = 8 wires in total, terminated in an **RJ45** connector.

Q Name the four layers of a coaxial cable from inside out, and say which one sets the impedance.

Central (inner) conductor, dielectric insulator, metallic shield, outer insulator/jacket. The **dielectric insulator** sets the spacing between inner conductor and shield, and that spacing is what fixes the characteristic impedance.

Q RG-59, RG-58 and RG-11 — what impedance and what use for each?

RG-59: 75 Ω , cable TV. **RG-58**: 50 Ω , thin Ethernet. **RG-11**: 50 Ω , thick Ethernet.

Q State the condition for total internal reflection, and explain why the cladding must have a lower refractive index than the core.

Light must travel from a **denser to a less dense** medium, and hit the boundary at an angle of incidence **greater than the critical angle**. The cladding must be optically *less* dense than the core, otherwise the core would not be the denser medium and total internal reflection could never happen — the light would simply refract out and be lost.

Q Why does graded-index fiber have less modal dispersion than step-index fiber?

In graded-index fiber the refractive index falls gradually from the centre outward. An outer ray travels a **longer path but through less dense glass, so it goes faster**. The extra distance and the extra speed roughly cancel, so all rays arrive at nearly the same time and the pulse spreads much less.

Q A 7/125 fiber: what do the numbers mean, and is it single or multi mode?

Core diameter **7 μm** , cladding diameter **125 μm** . A 7 μm core is far too narrow for multiple modes, so it is **single mode**.

Q Give two clear advantages and two clear disadvantages of fiber over copper.

Advantages: far higher bandwidth; much lower attenuation (repeaters every ~50 km instead of ~2–5 km); complete immunity to EMI; harder to tap. **Disadvantages:** installation and maintenance need specialist skill; light travels one way only, so two-way traffic needs two fibers; higher cost of cable and interfaces.

Q Radio waves or microwaves — which is used for multicast and which for unicast, and why?

Radio waves are *omnidirectional*, so one transmission reaches everybody — that suits **multicast** (broadcast radio, TV, paging). **Microwaves** can be focused into a narrow beam, so a transmission goes to one specific receiver — that suits **unicast** (cellular, satellite links, wireless LANs).

Q Name the three propagation methods with their frequency ranges.

Ground propagation, below 2 MHz — follows the Earth's curvature. **Sky** propagation, 2–30 MHz — reflected off the ionosphere. **Line-of-sight** propagation, above 30 MHz — travels in straight lines between aligned antennas.

Q Which band is used for AM radio, which for FM radio, and which for satellite communication?

AM radio → **MF** (300 kHz–3 MHz), sky propagation. FM radio and VHF TV → **VHF** (30–300 MHz). Satellite communication → **SHF** (3–30 GHz), line-of-sight; UHF and EHF also carry satellite traffic.

MODULE 04

Error Detection and Correction

Wires are noisy, and bits flip. This module covers the two techniques your syllabus names: **CRC**, which tells you a frame is broken, and **Hamming code**, which tells you *which bit* is broken and fixes it. Both are worked out digit by digit.

Mid-sem topics 20 & 21 · also Practicals 3 and 4

WHERE THIS FITS

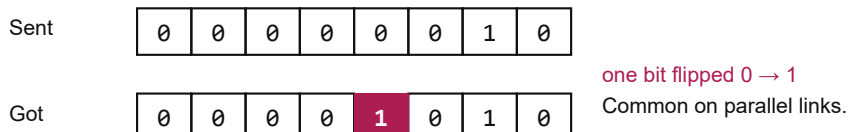
These two topics are on your mid-sem list but are **not in the lecture slide deck** — they were covered through the SCILAB practicals instead. This module fills that gap, using the same data values as your lab experiments so the two match.

4.1

Types of error, and the idea of redundancy

When a frame travels over a link, noise can change some bits. There are two shapes this damage takes.

SINGLE-BIT ERROR — exactly one bit changes



BURST ERROR — two or more bits within a stretch change



Fig 4.1 The two error types. **Burst length** is counted from the first corrupted bit to the last, inclusive — bits in between may well be correct.

PLAIN WORDS

A **single-bit error** is one bit flipping. A **burst error** is a run of damage — noise usually lasts longer than one bit time, so on a real serial link burst errors are far more common than single-bit errors.

The one idea behind all error control: redundancy

You cannot detect an error by looking at the data alone — any pattern of bits is a legal pattern of bits. So the sender must **send extra bits that carry no new information**, computed from the data by a rule both sides know. The receiver applies the same rule and checks whether the extra bits still agree with the data. This is **redundancy**.

Error detection

Only asks: **is this frame damaged, yes or no?** It does not say where or how badly. Cheaper. **CRC** does this.

Error correction

Asks the harder question: **which bit is wrong?**
Needs more redundant bits, but the receiver can repair the frame without asking for a retransmission. **Hamming code** does this.

4.2

Cyclic Redundancy Check (CRC)

CRC is an **error-detection technique** used in networking and digital communication to detect whether data has been corrupted during transmission. It is the check used by Ethernet and by HDLC's FCS field.

The whole method rests on one operation: **binary division using XOR instead of subtraction**. This is called **modulo-2 arithmetic**, and it has no carries and no borrows.

MODULO-2 ARITHMETIC — THE ONLY RULE YOU NEED		
$0 + 0 = 0$	$0 - 0 = 0$	
$0 + 1 = 1$	$0 - 1 = 1$	Addition and subtraction are the same operation: XOR
$1 + 0 = 1$	$1 - 0 = 1$	
$1 + 1 = 0$	$1 - 1 = 0$	

How CRC works

Sender side

The sender treats the data as a binary number. It divides the data by a predefined binary number called the **generator polynomial**. The **remainder** from this division is the CRC value. The sender appends this CRC to the data and transmits it.

Receiver side

The receiver performs the **same division** on the received data, *including* the CRC. If the remainder is **0**, the data is assumed error-free. If the remainder is **non-zero**, an error is detected and the frame is discarded or retransmitted.

THE PROCEDURE, STEP BY STEP
Let the generator have n bits.
SENDER
1. Append n - 1 zeros to the end of the dataword.
2. Divide that by the generator, using XOR at each step.
3. The remainder (n - 1 bits) is the CRC .
4. Transmit codeword = dataword + CRC (append, do not add)
RECEIVER
5. Divide the whole received codeword by the same generator.
6. Remainder = 0 → accept (no error detected)
Remainder ≠ 0 → reject (error detected)
Note the CRC is always one bit shorter than the generator. A 4-bit generator gives a 3-bit CRC.

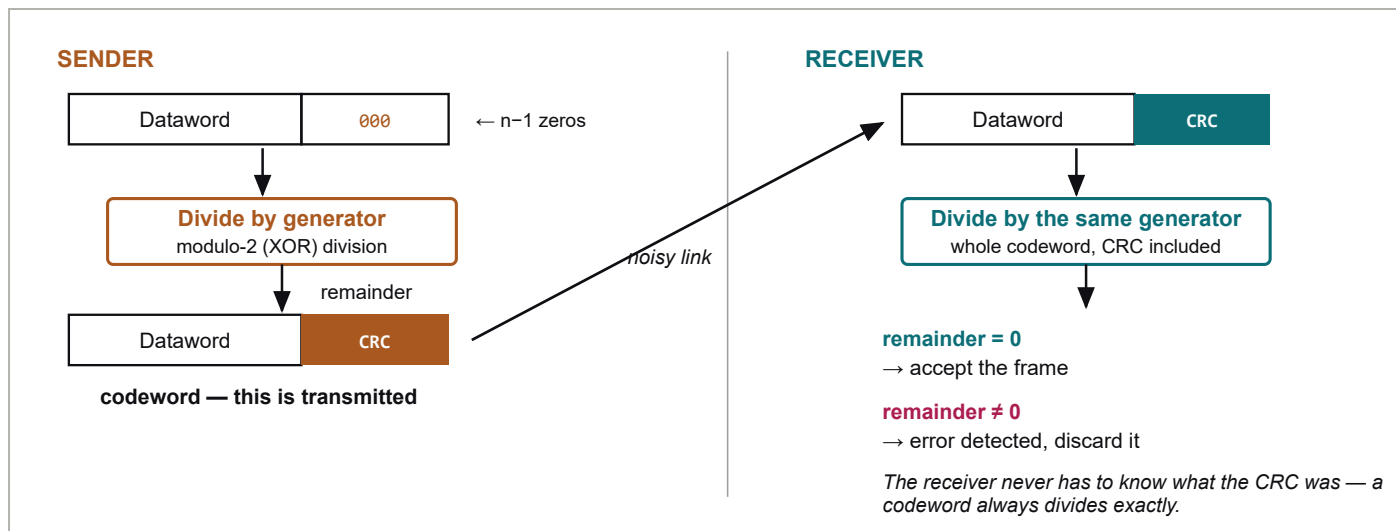


Fig 4.2 The CRC mechanism. The trick that makes it work: appending the remainder makes the codeword **exactly divisible** by the generator, so the receiver only has to check for a zero remainder — it never needs to compare CRC values.

EXAMPLE 4.1 — CRC, FULLY WORKED

Dataword = 1001, generator = 1011. Find the CRC and the transmitted codeword, then verify at the receiver.

Step 1 — append zeros. The generator has 4 bits, so append $4 - 1 = 3$ zeros:

1001 → 1001000

Step 2 — divide by 1011 using XOR. At each step, if the leading bit of the current window is 1 we XOR with the generator; if it is 0 we XOR with 0000 (that is, do nothing) and slide on.

THE DIVISION		
1 0 0 1 0 0 0	← dividend (dataword + 3 zeros)	
1 0 1 1	← leading bit is 1, so XOR	
0 0 1 0 0 0 0		
0 0 0 0	← leading bit is 0, XOR with zeros	
0 1 0 0 0 0		
1 0 1 1	← leading bit is 1, so XOR	
0 0 1 1 0		
0 0 0 0	← leading bit is 0	
1 1 0	← REMAINDER = CRC	

Step 3 — build the codeword. Append the 3-bit CRC to the original dataword:

codeword = 1001 + 110 = 1001110

Step 4 — receiver checks. Divide 1001110 by 1011 . Working through the same way gives remainder 000 , so the frame is accepted.

Step 5 — what if a bit flips? Suppose bit 3 is corrupted, so 1011110 arrives instead. Dividing that by 1011 gives remainder 110 , which is not zero — **the error is detected** and the frame is discarded.

CRC = 110

Transmitted codeword = 1001110

Clean receive: remainder 000 → accept

Corrupted receive (1011110): remainder 110 → reject

EXAMPLE 4.2 – THE VALUES FROM YOUR EXPERIMENT 3

Dataword = 11011110 , generator = 1011 . Find the CRC and the codeword.

Generator is 4 bits, so append 3 zeros: $11011110 \rightarrow 11011110000$

Dividing 11011110000 by 1011 in modulo-2 arithmetic leaves the 3-bit remainder 010 .

Codeword = $11011110 + 010 = 11011110010$

Receiver check: $11011110010 \div 1011$ leaves remainder $000 \rightarrow$ accepted.

CRC = 010 · Codeword = 11011110010 · Receiver remainder = 000 (accept)

Polynomial form

The generator is often written as a **polynomial** instead of a bit string. Each bit position is a power of x , starting from the right at x^0 . A 1 means that term is present.

BIT STRING TO POLYNOMIAL			
1	0	1	1
x^3	x^2	x^1	x^0
✓	X	✓	✓
$1011 = x^3 + x + 1$			
The degree of the polynomial (here 3) equals the number of CRC bits. Standard generators include CRC-8, CRC-16 and CRC-32 (used by Ethernet).			

What CRC can and cannot catch

DETECTION POWER OF A WELL-CHOSEN CRC GENERATOR		
ERROR TYPE	DETECTED?	CONDITION ON THE GENERATOR
All single-bit errors	Yes	The generator must have at least two terms (more than one 1 bit)
All odd numbers of bit errors	Yes	The generator must have the factor (x + 1)
Burst errors shorter than the CRC	Yes — all of them	Burst length < degree of the generator
Longer bursts	Almost all	A CRC of r bits misses only about 1 in 2^r of them — for CRC-32 that is 1 in 4.3 billion

IMPORTANT LIMITATION

CRC only **detects**. It cannot say which bit is wrong, so it cannot repair anything. The only cure is retransmission — which is exactly what the ARQ protocols in Module 5 are for.

4.3

Hamming code

Hamming code is an error detecting **and error correcting** code. It adds redundant parity bits to the data so that a single-bit error can be **located and repaired at the receiver**, with no retransmission at all.

How many redundant bits are needed?

THE REDUNDANCY CONDITION

$$2^r \geq m + r + 1$$

m = number of data bits

r = number of redundant (parity) bits

Why this works: the syndrome of r bits must be able to name **every possible error position** (there are $m + r$ of them) **plus** the “no error” case — that is $m + r + 1$ outcomes, and r bits give 2^r patterns.

EXAMPLE 4.3 — SIZING THE CODE

How many redundant bits are needed for 4 data bits?

Try $r = 2$: $2^2 = 4$, $m + r + 1 = 4 + 2 + 1 = 7$. Is $4 \geq 7$? **No**.

Try $r = 3$: $2^3 = 8$, $m + r + 1 = 4 + 3 + 1 = 8$. Is $8 \geq 8$? Yes.

$r = 3$. So 4 data bits + 3 parity bits = 7 bits total — this is the (7,4) Hamming code, the one your syllabus fixes on.

Where the parity bits go

The 7 positions are numbered 1 to 7. Parity bits sit at the positions that are powers of 2 — 1, 2 and 4. Everything else holds data.

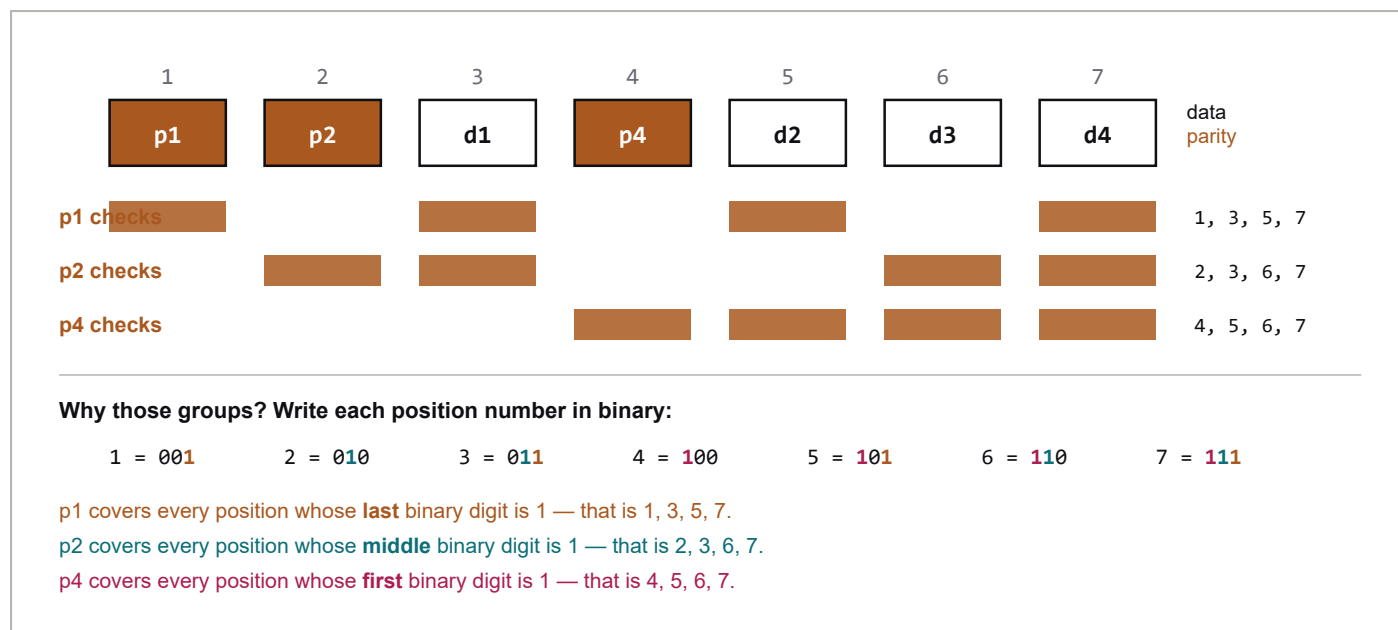


Fig 4.3 The (7,4) layout and why the check groups are what they are. This binary-digit pattern is the entire cleverness of Hamming code: it makes the three parity results *spell out the error position in binary*.

EXAMPLE 4.4 — ENCODING, FULLY WORKED

Encode the dataword **1011** using the (7,4) Hamming code with even parity.

Step 1 — place the data bits at positions 3, 5, 6, 7:

position 3 = 1, position 5 = 0, position 6 = 1, position 7 = 1

Step 2 — compute each parity bit so its group has an even number of 1s.

p1 covers positions 1, 3, 5, 7. The data there is 1, 0, 1 → that is two 1s, already even → **p1 = 0**

p2 covers positions 2, 3, 6, 7. The data there is 1, 1, 1 → three 1s, odd → **p2 = 1** to make it even

p4 covers positions 4, 5, 6, 7. The data there is 0, 1, 1 → two 1s, even → **p4 = 0**

Step 3 — write out the codeword (positions 1 to 7):

Codeword = 0 1 1 0 0 1 1
 p1 p2 d1 p4 d2 d3 d4

Quick check – p1 group {0,1,0,1} has two 1s ✓ even; p2 group {1,1,1,1} has four ✓ even; p4 group {0,0,1,1} has two ✓ even.

EXAMPLE 4.5 – DETECTING AND CORRECTING AN ERROR

The codeword **0110011** was sent, but **0110111** arrives — position 5 has flipped from 0 to 1. Show that the receiver finds and fixes it, without being told where the error is.

The receiver recomputes each parity check over the *received* bits. Received: pos1=0, pos2=1, pos3=1, pos4=0, pos5=1, pos6=1, pos7=1.

c1 — check positions 1, 3, 5, 7 = 0, 1, 1, 1 → three 1s, **odd** → parity fails → **c1 = 1**

c2 — check positions 2, 3, 6, 7 = 1, 1, 1, 1 → four 1s, **even** → parity holds → **c2 = 0**

c4 — check positions 4, 5, 6, 7 = 0, 1, 1, 1 → three 1s, **odd** → parity fails → **c4 = 1**

Now read the syndrome as a binary number, most significant first: c4 c2 c1

1 0 1 in binary = **5** in decimal.

The error is at position 5. The receiver flips bit 5 back, recovering **0110011**, then reads positions 3, 5, 6, 7 to get the data **1011**.

Syndrome **c4c2c1 = 101 = 5** → bit 5 is wrong.

Flip it: **0110111** → **0110011**

Recovered data = **1011** — the original, repaired with no retransmission.

If the syndrome had been **000**, it would mean no error was found.

THE WHOLE POINT, IN ONE LINE

The three parity checks are arranged so that their pass/fail pattern, read as a binary number, is the position of the broken bit. Syndrome 000 means no error; any other value points straight at the culprit.

The matrix method — the same code, written with G and H

This is the form used in your SCILAB practical, and it is the form that scales to bigger codes. Instead of reasoning about positions, we multiply by two matrices.

GENERATOR AND PARITY CHECK MATRICES

$$G = [I_4 \mid P] \text{ is } 4 \times 7 \quad H = [P^T \mid I_3] \text{ is } 3 \times 7$$

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \quad H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Encode: $c = d \cdot G \pmod{2}$

Syndrome: $s = H \cdot r^T \pmod{2}$

Here the codeword is *systematic*: the first 4 bits are the data unchanged and the last 3 are parity. That is why G starts with the identity matrix.

EXAMPLE 4.6 – THE MATRIX METHOD, WORKED

Take $d = [1 \ 0 \ 1 \ 1]$. Encode it with G , corrupt bit 3, then locate the error with H .

Encode. $c = d \cdot G \pmod{2}$. The first four bits come straight through as the data. The three parity bits are:

$$p_1 = 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 1 = 3 \rightarrow 1 \quad p_2 = 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 1 \cdot 0 = 2 \rightarrow 0 \quad p_3 = 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 2 \rightarrow 0$$

$$c = [1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0]$$

Corrupt bit 3: $r = [1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0]$

Syndrome. $s = H \cdot r^T \pmod{2}$, one row of H at a time:

$$\text{row 1} \cdot r = 1 + 0 + 0 + 1 + 1 + 0 + 0 = 3 \rightarrow 1$$

$$\text{row 2} \cdot r = 1 + 0 + 0 + 0 + 0 + 0 + 0 = 1 \rightarrow 1$$

$$\text{row 3} \cdot r = 0 + 0 + 0 + 1 + 0 + 0 + 0 = 1 \rightarrow 1$$

$$s = [1 \ 1 \ 1]. \text{ Now compare this against the columns of } H. \text{ Column 3 of } H \text{ is } [1 \ 1 \ 1] \text{ — a match.}$$

Syndrome $[1 \ 1 \ 1]$ = column 3 of H → error at bit 3.

Flip bit 3: $[1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0]$. First four bits = **1011** = the original data.

If there were no error the syndrome would be $[0 \ 0 \ 0]$.

WHY THE COLUMNS OF H ARE THE ANSWER

Every column of H is different and none is all-zero. A single-bit error at position i always produces a syndrome equal to **column i of H** — so matching the syndrome against the columns names the broken bit. That is the same fact as the position method, just written as matrices.

CRC VS HAMMING CODE		
FEATURE	CRC	HAMMING CODE
Purpose	Detection only	Detection and correction
Method	Modulo-2 division by a generator polynomial	Parity bits at power-of-2 positions; syndrome names the bad bit
Redundant bits	degree of the generator (e.g. 32 for CRC-32)	r , where $2^r \geq m + r + 1$
Handles bursts	Yes — excellent at burst detection	No — corrects only one bit per codeword
On finding an error	Discard and ask for retransmission (ARQ)	Repair it locally, no retransmission needed
Overhead	Low — a few bytes for a whole frame	High — 3 parity bits for every 4 data bits
Used in	Ethernet FCS, HDLC FCS, ZIP files	ECC memory, satellite links, places where retransmission is costly

Module 4 at a glance

- Errors are **single-bit** or **burst**. The cure for both is **redundancy** — extra bits added by the sender and checked by the receiver.
- **CRC**: append r zeros (r = degree of the generator), divide by the generator using **XOR**, and replace the zeros with the remainder. The receiver divides the whole codeword; remainder zero means no error detected.
- **Hamming code**: parity bits sit at the power-of-two positions 1, 2, 4, 8... Each checks the positions whose number contains its own bit. Adding up the failing checks gives the position of the wrong bit.
- Detection tells you something broke; correction tells you what it should have been.

CHECK YOURSELF — MODULE 4

Q What is a burst error, and how is burst length measured?

Two or more bits in the data unit have changed. The **burst length** is counted from the **first corrupted bit to the last corrupted bit**, inclusive — bits in between may be correct.

Q Why can't you detect errors without sending extra bits?

Because any pattern of data bits is a legal pattern — there is nothing about the data alone that says it is wrong. You need **redundancy**: extra bits computed from the data by a rule both sides know, so the receiver can check whether data and check bits still agree.

Q In CRC, how many zeros do you append, and how long is the CRC?

If the generator has n bits, you append $n - 1$ zeros, and the CRC (the remainder) is $n - 1$ bits long. So a 4-bit generator gives a 3-bit CRC.

Q Find the CRC for dataword 1101 with generator 1011.

Append 3 zeros: 1101000. Divide by 1011 using XOR:

$$1101 \text{ XOR } 1011 = 0110 \rightarrow \text{bring down: } 1100$$

$$1100 \text{ XOR } 1011 = 0111 \rightarrow \text{bring down: } 1110$$

$$1110 \text{ XOR } 1011 = 0101$$

CRC = 101, codeword = 1101101.

Q The receiver divides and gets a remainder of 000. What does it conclude, and could it still be wrong?

It concludes **no error was detected** and accepts the frame. It could still be wrong — a burst longer than the CRC can, very occasionally, produce a zero remainder. With r CRC bits roughly 1 in 2^r such bursts slip through (1 in 4.3 billion for CRC-32).

Q Write the generator 1011 as a polynomial. What is its degree, and what does the degree tell you?

$1011 = x^3 + x + 1$. Degree 3, which is the **number of CRC bits** produced.

Q How many redundant bits are needed for 8 data bits?

Need $2^r \geq m + r + 1$ with $m = 8$.

$r = 3$: $8 \geq 12$? No. $r = 4$: $16 \geq 13$? **Yes**.

$r = 4$, giving a 12-bit codeword — a (12,8) Hamming code.

Q In the (7,4) code, which positions hold parity bits and which hold data? Why those positions?

Parity at positions **1, 2, 4** (the powers of 2); data at **3, 5, 6, 7**. Powers of 2 are used because each parity bit then appears in **exactly one** of the three check groups, which keeps the syndrome unambiguous.

Q Encode the dataword 1101 with the (7,4) Hamming code (even parity).

Place data at 3,5,6,7: pos3=1, pos5=1, pos6=0, pos7=1.

p1 (1,3,5,7) sees 1,1,1 → three 1s, odd → **p1 = 1**

p2 (2,3,6,7) sees 1,0,1 → two 1s, even → **p2 = 0**

p4 (4,5,6,7) sees 1,0,1 → two 1s, even → **p4 = 0**

Codeword = 1 0 1 0 1 0 1

Q A receiver computes c1 = 1, c2 = 1, c4 = 0. Where is the error?

Read the syndrome as c4 c2 c1 = **0 1 1** = 3 in decimal. The error is at **position 3**. Flip that bit to correct it.

Q What does a syndrome of 000 mean? And why can Hamming code not fix two simultaneous errors?

000 means no error was detected.

It cannot fix two errors because the syndrome has only enough room to name *one* position. Two flipped bits produce a syndrome that points at some *third*, innocent position — so the code would “correct” the wrong bit and make things worse.

Q In the matrix method, how do you find the error position from the syndrome?

Compute $s = H \cdot r^T \bmod 2$, then compare s against the **columns of H** . The column it matches gives the position of the corrupted bit. A single-bit error at position i always yields exactly column i .

Q Which would you choose for an Ethernet frame, and which for a deep-space link? Why?

Ethernet → **CRC**. Retransmission is cheap and fast on a LAN, so you only need cheap detection, and CRC is excellent against bursts.

Deep-space link → **Hamming (or a stronger correcting code)**. A round trip could take hours, so asking for a retransmission is far too costly — the receiver must repair the data itself.

MODULE 05

Data Link Control

The physical layer hands up a stream of bits that may be damaged, lost or arriving too fast. This module is how the data link layer turns that mess into something dependable:

cut the stream into frames, stop the sender from flooding the receiver, and get anything broken sent again. Five protocols, each fixing a weakness in the one before.

Lecture slides 101-161 · Chapter 4

5.1

Framing

Framing is the process of breaking data down into manageable units called **frames**, so that data can be transmitted efficiently and reliably over a network.

A frame contains not only the raw data but also **control information** — headers, trailers, and checksums — to ensure proper delivery. The data link layer needs to pack bits into frames so that **each frame is distinguishable from another**.

PLAIN WORDS

Our postal system already does framing. Putting a letter into an envelope **separates one piece of information from another** — the envelope is the delimiter. Without envelopes, a sack of loose pages would be one unreadable pile. Framing puts envelopes around chunks of bits.

Why frames rather than one long stream?

- **Error control becomes affordable.** If a single bit is damaged, you only resend that one frame, not the whole file.
- **Addressing becomes possible.** Each frame can carry its own source and destination address.
- **Sharing becomes fair.** On a shared link, other stations get a turn between frames.

Two types of framing

Character-oriented (byte-oriented)

Uses special **characters** to mark the beginning and end of frames. Data is a multiple of 8 bits.

Bit-oriented

Uses a special **bit pattern** to frame the data. Data can be any number of bits.

Character-oriented framing, and byte stuffing

The frame looks like this: a **FLAG** byte, then a header, then the data, then a trailer, then a closing **FLAG** byte. The flag is a specific 8-bit pattern that means “frame boundary”.

This creates an immediate problem. What if the *data* happens to contain a byte identical to the flag — for example if you are sending an image or a compiled program, where any byte value can appear? The receiver would see that byte and wrongly conclude the frame ended early.

The fix is **byte stuffing**, also called **character stuffing**: a technique used to differentiate between data bytes and control bytes, so that special control characters do not appear in the actual data being transmitted.

THE BYTE STUFFING RULE

Byte stuffing is the process of adding 1 extra byte whenever there is a FLAG or ESCAPE character in the text.

The extra byte is the **ESC** (escape) character. It means: “the next byte is data, not a control byte — do not interpret it.”

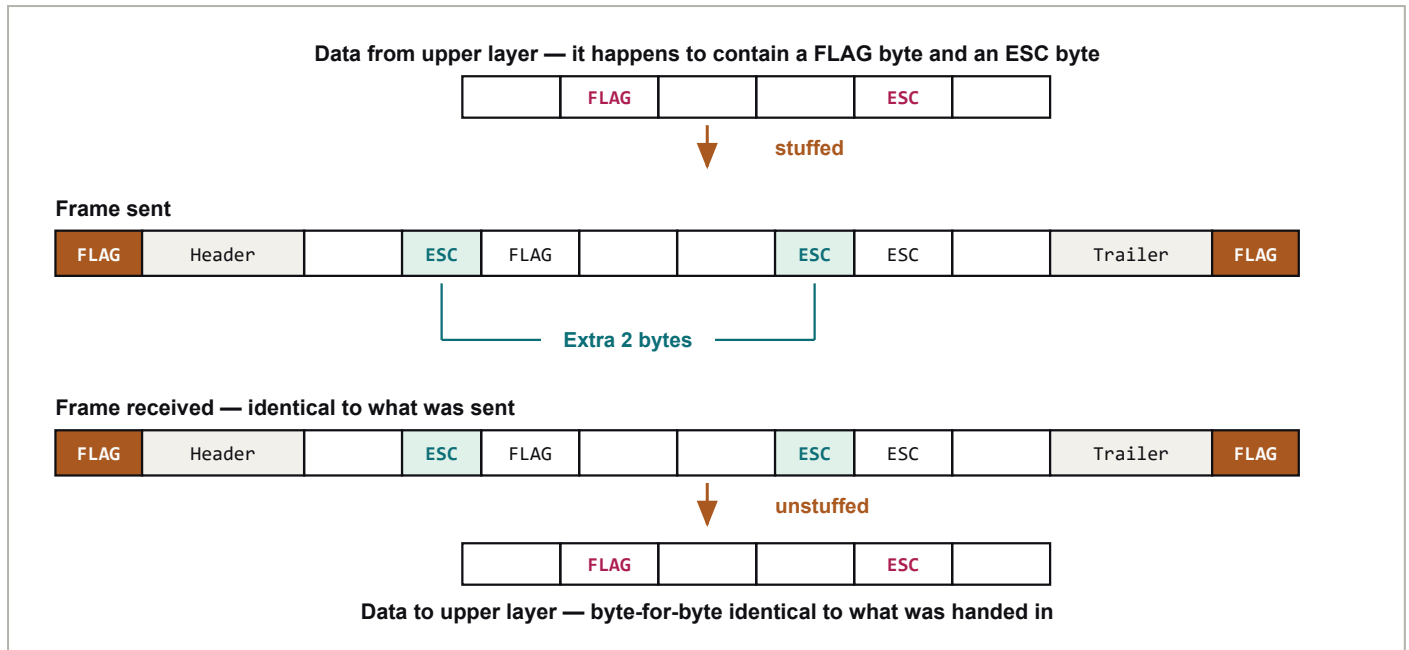
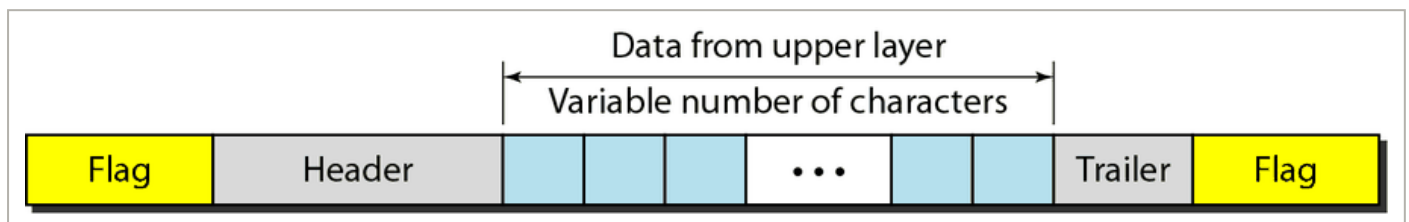
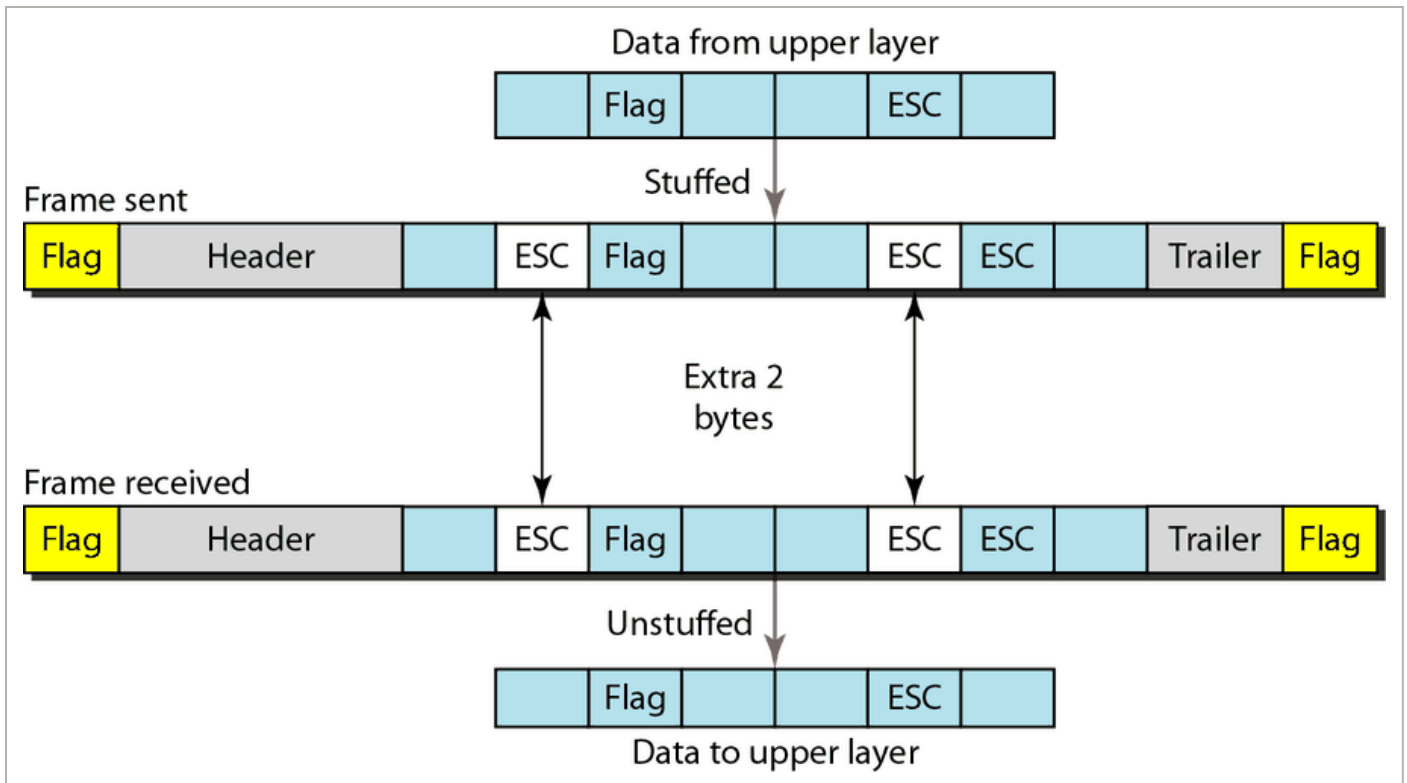


Fig 5.1 Byte stuffing and unstuffing. The receiver’s rule is simple: on seeing an ESC, **throw the ESC away and take the next byte literally as data**. Because ESC itself must be escapable, an ESC in the data becomes ESC ESC — which is why two extra bytes appear here.



Slide 105 A frame in a character-oriented protocol.



Slide 106 Byte stuffing and unstuffing, from the lecture deck.

EXAMPLE 5.1 – BYTE STUFFING

The data to be sent is: A FLAG B ESC C FLAG . Write the stuffed data, and then show that unstuffing recovers the original.

Stuffing. Walk through the data. Put an ESC in front of every FLAG and every ESC you meet:

A → plain data, leave it. FLAG → insert ESC before it. B → leave it. ESC → insert ESC before it. C → leave it. FLAG → insert ESC before it.

Unstuffing. Walk through the received data. Each time you see an ESC, delete it and accept the next byte as plain data.

Stuffed: A ESC FLAG B ESC ESC C ESC FLAG

Length: 6 bytes in → 9 bytes out (3 extra bytes added)

Unstuffed: A FLAG B ESC C FLAG – the original, exactly

Bit-oriented framing, and bit stuffing

In a bit-oriented protocol, the frame boundary is marked by an 8-bit **flag pattern**:

THE FLAG

0 1 1 1 1 1 1 0 (six 1s between two 0s)

In hexadecimal this is 0x7E . It is the flag used by HDLC (Module 7).

Again the same danger: what if this pattern turns up inside the data? **Bit stuffing** is the technique that prevents special bit patterns, which might be mistaken for control information, from appearing in the transmitted data.

THE BIT STUFFING RULE

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

1 Original data from upper layer — 13 bits

0 0 0 1 1 1 1 1 1 1 1 0 0
 five consecutive 1s — danger

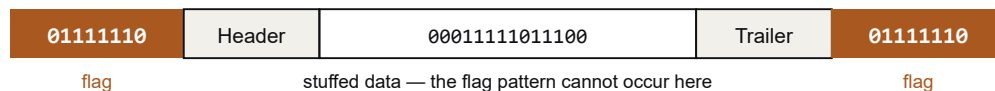
A run of eight 1s. The receiver would read 0111110 inside it and stop early.

2 Sender stuffs a 0 after the fifth 1 — now 14 bits

0 0 0 1 1 1 1 1 0 1 1 1 0 0
 stuffed 0

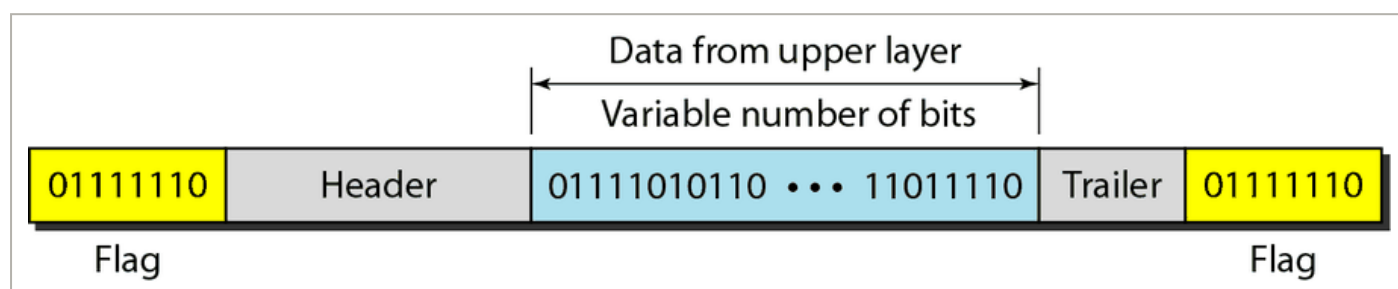
The count resets. Only three 1s follow, so no second stuffing is needed.

3 What actually goes on the wire

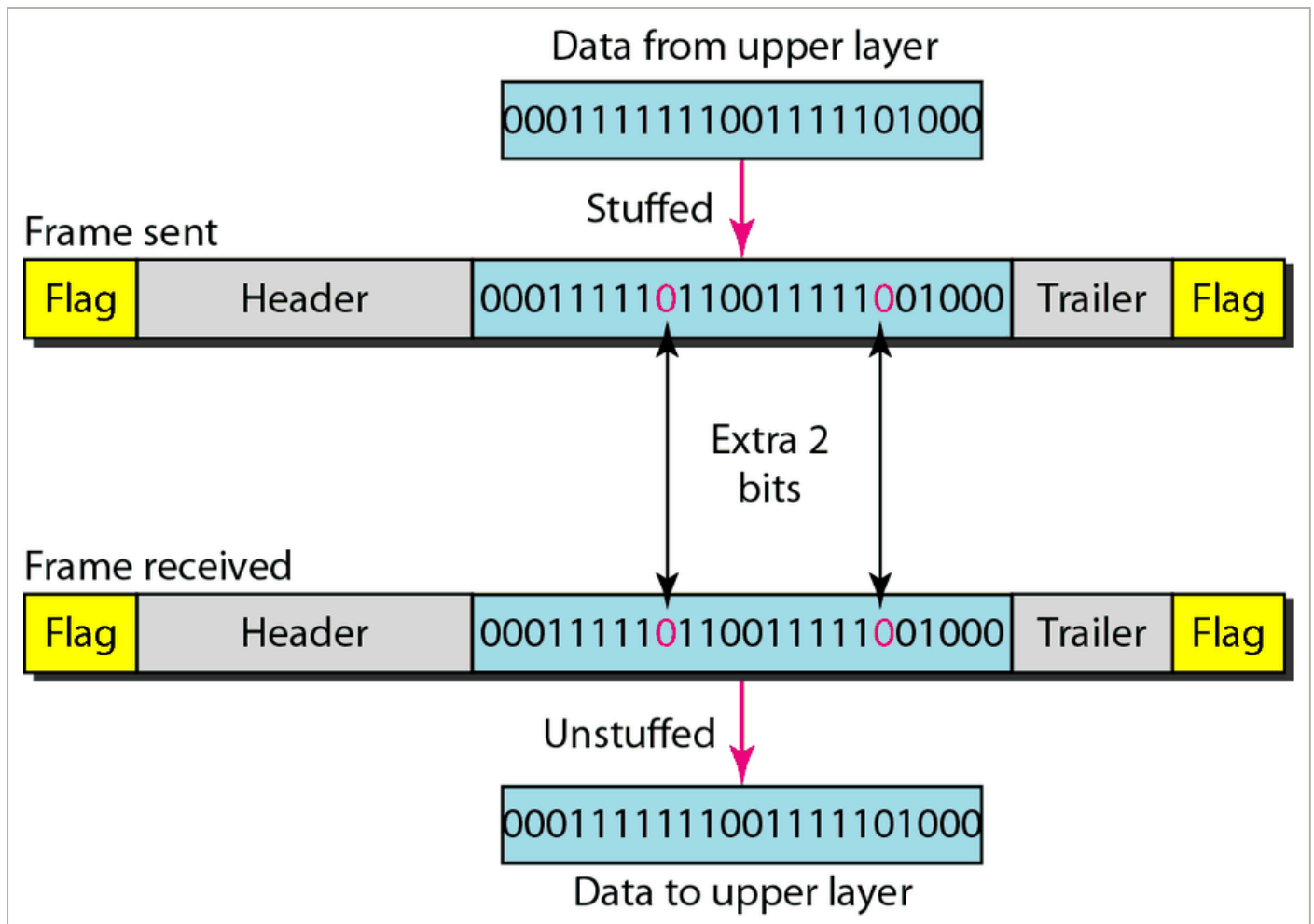


4 Receiver: after five 1s, if the next bit is a 0, delete it → recovers 0001111111100 exactly.

Fig 5.2 Bit stuffing, step by step. The guarantee it buys: a run of six 1s can never appear in the stuffed data, so the only place the flag pattern exists is at the real frame boundaries.



Slide 108 A frame in a bit-oriented protocol.



Slide 110 Bit stuffing and unstuffing, from the lecture deck.

EXAMPLE 5.2 – BIT STUFFING AND DE-STUFFING

Apply bit stuffing to the data 0 1 1 1 1 1 1 0 1 1 1 1 1 0. Then de-stuff your answer to check.

Stuffing. Read left to right, counting consecutive 1s. As soon as the count hits 5, write a 0 and reset the count.

0 → write 0, count = 0

1 1 1 1 1 → write them; count reaches 5 → **write a stuffed 0**, reset count

1 1 → write them, count = 2

0 → write it, count = 0

1 1 1 1 1 → write them; count reaches 5 → **write a stuffed 0**, reset

1 → write it, count = 1

0 → write it

De-stuffing. Read the received bits, counting consecutive 1s. Each time the count hits 5, look at the next bit: if it is a 0, it was stuffed — delete it and reset.

Original (16 bits): 0111111101111110

Stuffed (18 bits): 011111011011111010

2 bits added. De-stuffing removes exactly those two 0s and returns 0111111101111110.

Note: the original ended in 01111110 — the flag pattern itself. Stuffing broke it up, which is the whole point.

A second, separate use of bit stuffing: clock synchronisation

In synchronous communication systems, long sequences of the **same bit** — all 0s or all 1s — cause a different problem. A receiver recovers its clock from the **transitions** (the edges) in the signal. A long flat run has no edges, so the receiver's clock drifts and it starts sampling at the wrong moments.

Bit stuffing breaks up long sequences, providing transitions that help the receiver stay synchronised with the transmitter's clock.

WHY DOES BIT STUFFING ONLY WATCH FOR CONSECUTIVE 1S?

Both long runs of 0s and long runs of 1s cause clock trouble, so this looks inconsistent. The reason is that framing comes first. Bit-oriented protocols like HDLC use a frame delimiter **made of 1s** (01111110), so stuffing must target runs of **1s** to protect that delimiter. Inserting a 0 after five 1s guarantees the delimiter cannot appear in the data.

Long runs of 0s are dealt with elsewhere — by **physical-layer encoding** (Manchester encoding, 4B/5B encoding) and by **scrambling**. Different layer, different tool.

5.2

Flow control and error control

These are the **two most important responsibilities of the data link layer**. Together they are known as **data link control**.

Flow control

A set of procedures used to **restrict the amount of data the sender can send before waiting for acknowledgment**.

It exists because the receiver has a finite buffer and a finite processing speed. Without flow control, a fast sender overruns a slow receiver and data is lost even though nothing was damaged.

Error control

In the data link layer, error control is based on **automatic repeat request (ARQ)** — that is, the **retransmission of data**.

The receiver detects that something is wrong (or the sender notices nothing came back) and the frame is simply sent again.

PLAIN WORDS

Flow control answers: “how much may I send before I pause?” — it protects the receiver from being flooded. **Error control** answers: “what do I do when something goes wrong?” — and in the data link layer the answer is always “send it again”.

Two pieces of vocabulary you will meet constantly from here on:

ACK (acknowledgment)

A small control frame from the receiver meaning “I got that one correctly.”

NAK (negative acknowledgment)

A small control frame from the receiver meaning “that one arrived damaged — send it again.”

5.3

The five protocols, and the two for noiseless channels

The data link layer combines framing, flow control and error control to deliver data from one node to another. Five protocols do this, and they split into two families according to **what kind of channel they assume**.

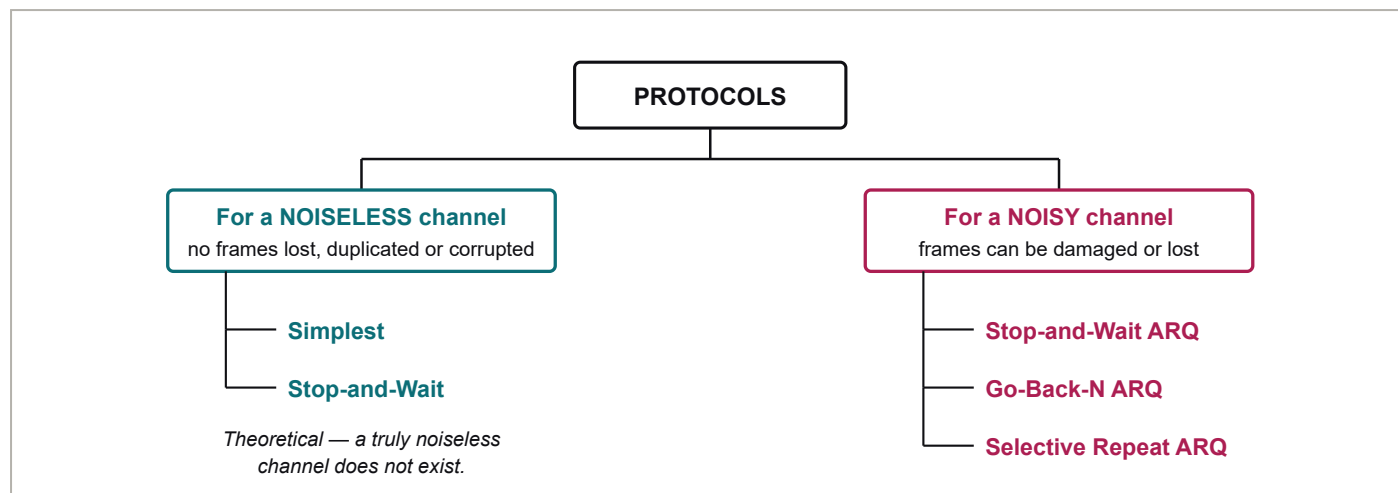


Fig 5.3 The five protocols. The left branch is a teaching ladder — noiseless channels are non-existent — but you must understand it, because each noisy protocol is its noiseless cousin plus a timer and a sequence number.

Protocol 1 — the Simplest protocol

This one has **no flow control and no error control**. It assumes both sender and receiver are so fast they can handle any frame instantly.

How it runs: the sender’s data link layer takes data from its network layer, makes a frame, and sends it. The receiver’s data link layer receives a frame, extracts the data, and delivers it upward. That is all.

How it fails: the sender sends a sequence of frames *without even thinking about the receiver*. If the receiver is slower, frames pile up and are lost. To send three frames, three events happen at the sender and three at the receiver.

Protocol 2 — Stop-and-Wait

This adds **flow control, and nothing else**. The rule is simply: **send one frame, then stop and wait for an ACK before sending the next**.

The sender sends one frame and waits for feedback. When the ACK arrives, it sends the next frame. Sending two frames involves the sender in **four events** (send, receive ACK, send, receive ACK) and the receiver in **two**.

How it fails: the channel is assumed perfect. If a frame is lost on the way, the sender waits forever for an ACK that will never come, and the whole conversation freezes.

5.4

Stop-and-Wait ARQ

Noiseless channels do not exist. Stop-and-Wait ARQ is Stop-and-Wait made to survive a real channel. It adds exactly **three** things.

THE THREE ADDITIONS

1. **Keep a copy** of every frame sent, until its ACK arrives.
2. **A timer**. Start it when a frame is sent. If it expires with no ACK, resend the kept copy.
3. **Sequence numbers** on frames and ACKs, modulo 2 (so 0, 1, 0, 1 ...)

Point by point:

- **Error correction is done by keeping a copy of the sent frame and retransmitting the frame when the timer expires.** This one mechanism handles both a damaged frame and a lost frame — in both cases no ACK comes back, the timer runs out, and the copy is resent.
- **Sequence numbers are based on modulo-2 arithmetic**, so they alternate 0, 1, 0, 1. Only two values are needed. (Why two? Because only one frame is ever outstanding, so the receiver only ever needs to tell “this one” apart from “the next one”.)
- **The acknowledgment number always announces, in modulo-2 arithmetic, the sequence number of the *next* frame expected.** So ACK 1 means “I have frame 0, now send me frame 1.”

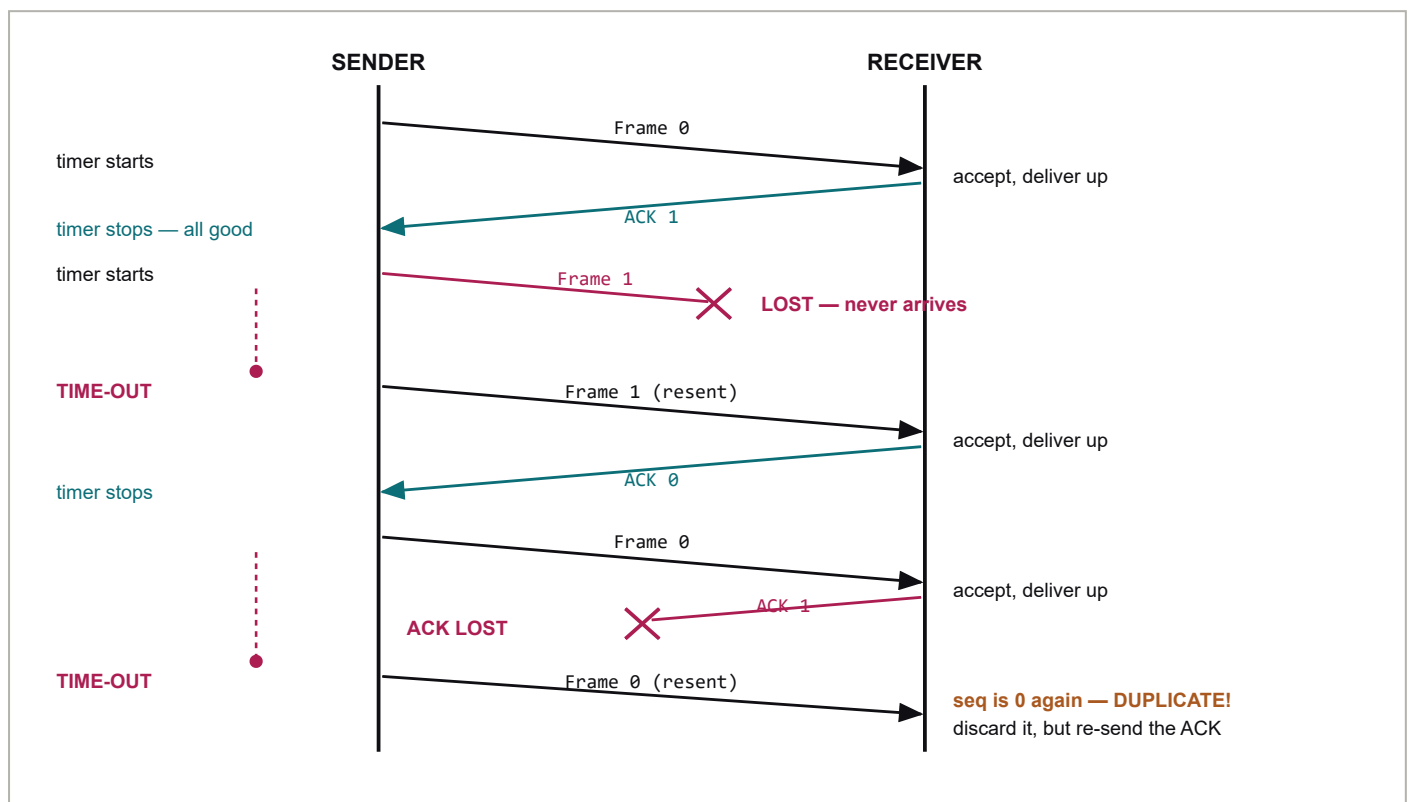


Fig 5.4 Stop-and-Wait ARQ across three situations. The bottom case is why sequence numbers are necessary at all: **the sender cannot tell a lost frame from a lost ACK** — both look like silence. So it resends, and only the sequence number lets the receiver spot the duplicate and throw it away instead of delivering the same data twice.

THE KEY INSIGHT

A time-out means only “*no ACK came back*”. The sender has **no idea** whether the frame was lost or the acknowledgment was lost. It resends either way. The sequence number is what protects the receiver from that guesswork.

The cost of Stop-and-Wait: bandwidth-delay product

Stop-and-Wait ARQ is correct, but it can be extremely wasteful. To see how badly, we need one quantity.

BANDWIDTH-DELAY PRODUCT

Bandwidth-delay product = bandwidth $\times$ round-trip delay

This is the **number of bits that could be in flight on the link at one time** — the “capacity of the pipe”. Compare it against how much you actually send per round trip, and you get the link utilisation.

EXAMPLE 5.3 — FROM THE SLIDES

In a Stop-and-Wait ARQ system the bandwidth of the line is 1 Mbps, and 1 bit takes 20 ms to make a round trip. What is the bandwidth-delay product? If the data frames are 1000 bits long, what is the utilisation percentage of the link?

Step 1 — bandwidth-delay product.

$$(1 \times 10^6) \times (20 \times 10^{-3}) = 20,000 \text{ bits}$$

Step 2 — what that means. The system *can* send 20,000 bits during the time it takes for data to go from sender to receiver and back again.

Step 3 — what it actually sends. Stop-and-Wait sends only **one frame** per round trip — that is 1000 bits.

$$\text{Utilisation} = 1000 / 20,000$$

Bandwidth-delay product = 20,000 bits

Utilisation = $1000 / 20,000 = 0.05 = 5\%$

95% of the link’s capacity is wasted. For a link with high bandwidth or long delay, Stop-and-Wait ARQ throws away the capacity of the link.

EXAMPLE 5.4 — THE FIX, PREVIEWED

Same link as Example 5.3. What is the utilisation if we use a protocol that can send up to 15 frames before stopping to worry about acknowledgments?

The bandwidth-delay product is unchanged: still **20,000 bits**.

The system can now send up to 15 frames $\times$ 1000 bits = **15,000 bits** during one round trip.

Utilisation = 15,000 / 20,000

$$\text{Utilisation} = 15,000 / 20,000 = 0.75 = 75\%$$

Fifteen times the throughput on the same physical link, with no hardware change at all. (If frames are damaged the real figure is lower, because they must be resent.)

PLAIN WORDS

Stop-and-Wait is like posting one letter and refusing to write the next until the reply arrives. The post is perfectly capable of carrying twenty letters at once — you are just choosing not to use it. **Sliding windows**, coming next, let you keep several letters in the post at the same time.

Efficiency formula (useful shortcut)

Examiners often want the same result written as a ratio. Define:

EFFICIENCY IN TERMS OF A

$$a = T_p / T_t \quad (\text{propagation time} \div \text{transmission time})$$

$$\text{Stop-and-Wait ARQ: } \eta = 1 / (1 + 2a)$$

$$\text{Go-Back-N / Sel. Rep: } \eta = N / (1 + 2a), \text{ capped at } 1$$

where N = send window size

T_t = frame size $\div$ bandwidth (time to push one frame onto the wire). T_p = one-way propagation time. The two forms agree: with the numbers of Example 5.3, $T_t = 1000/10^6 = 1$ ms and $2T_p = 20$ ms, so $a = 10$ and $\eta = 1/21 \approx 4.8\%$, matching the 5% found above.

5.5

Go-Back-N ARQ

Go-Back-N ARQ is a **sliding window protocol** for reliable data transmission. It increases efficiency by **allowing multiple frames to be sent before needing an acknowledgment**. It is an improvement over Stop-and-Wait ARQ.

SEQUENCE NUMBERS IN GO-BACK-N

Sequence numbers are **modulo 2^m** ,
where m is the size of the sequence number field in bits.

Modulo 2^m means: divide by 2^m and keep the remainder. So with $m = 3$, sequence numbers run 0,1,2,3,4,5,6,7,0,1,2,... — they wrap around.

The two windows

The whole protocol is controlled by two “windows” — imaginary boxes sliding along the list of sequence numbers.

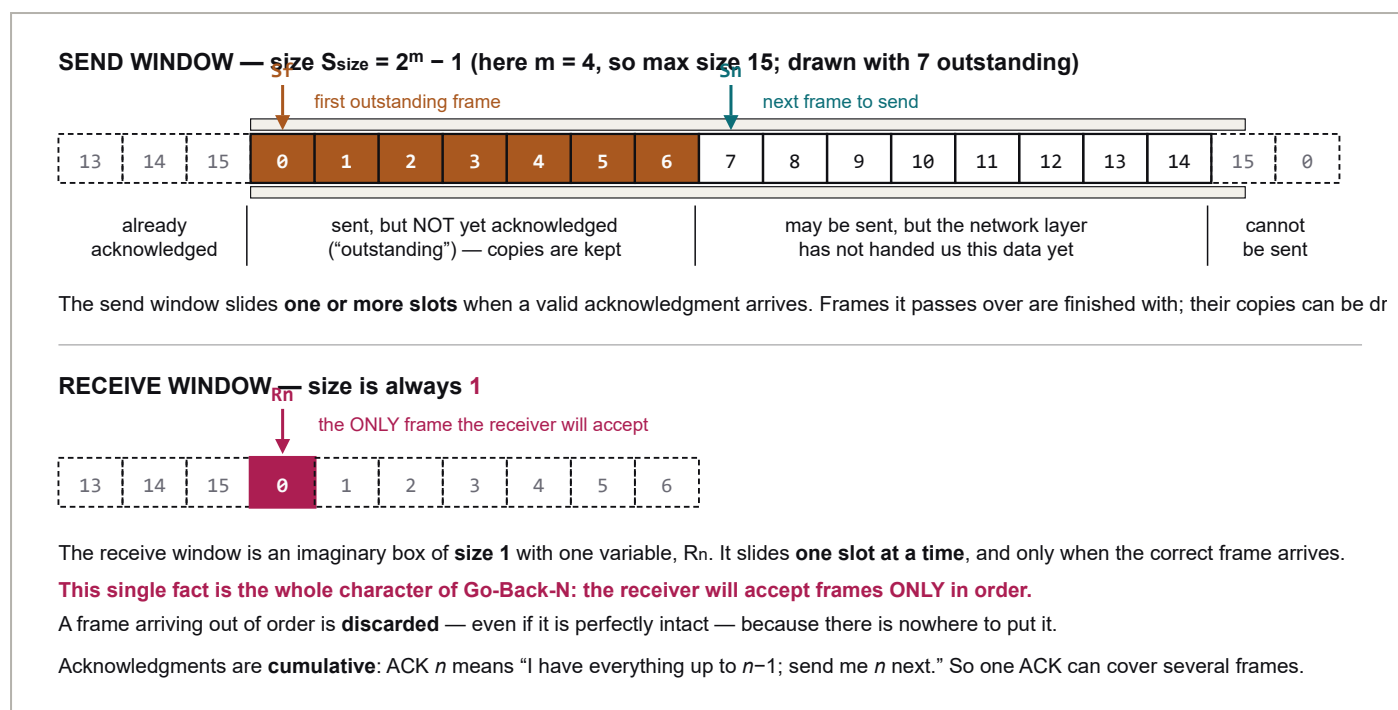
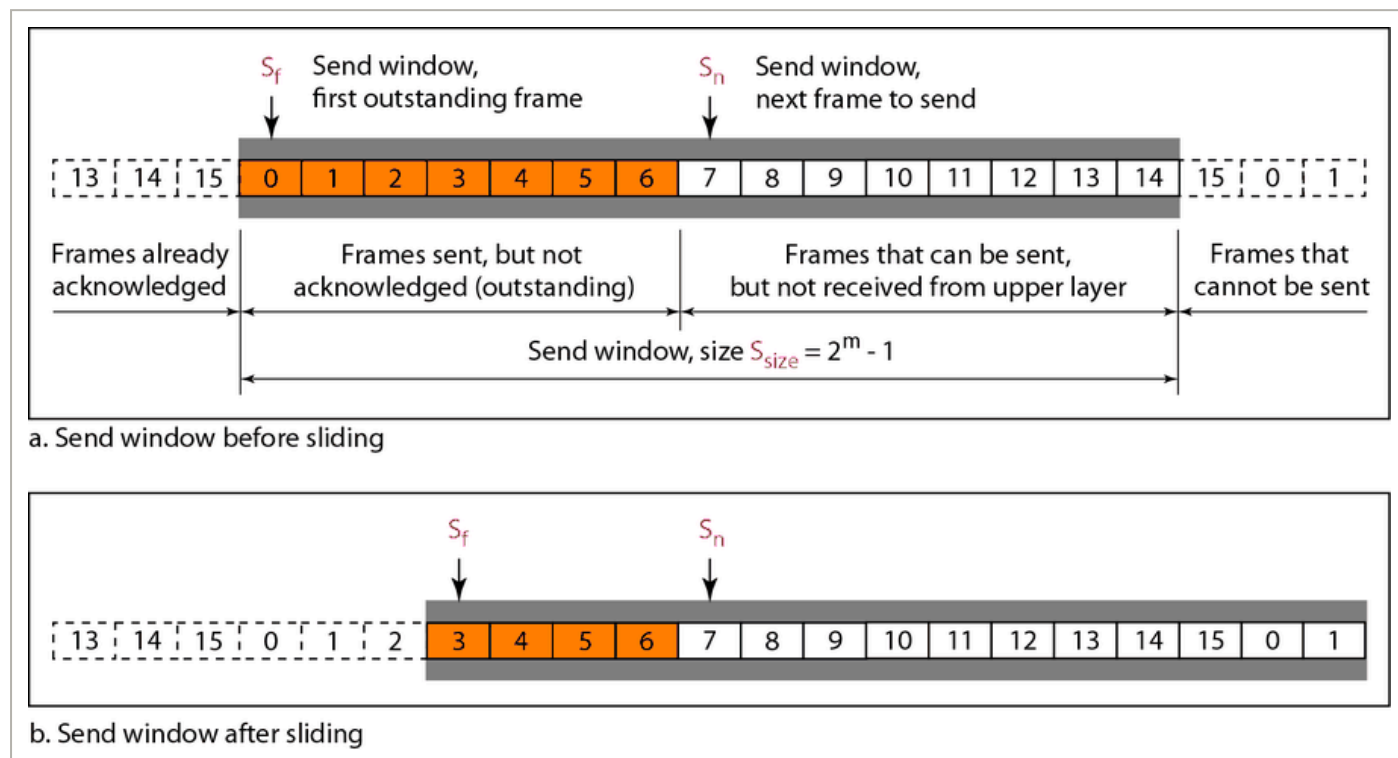
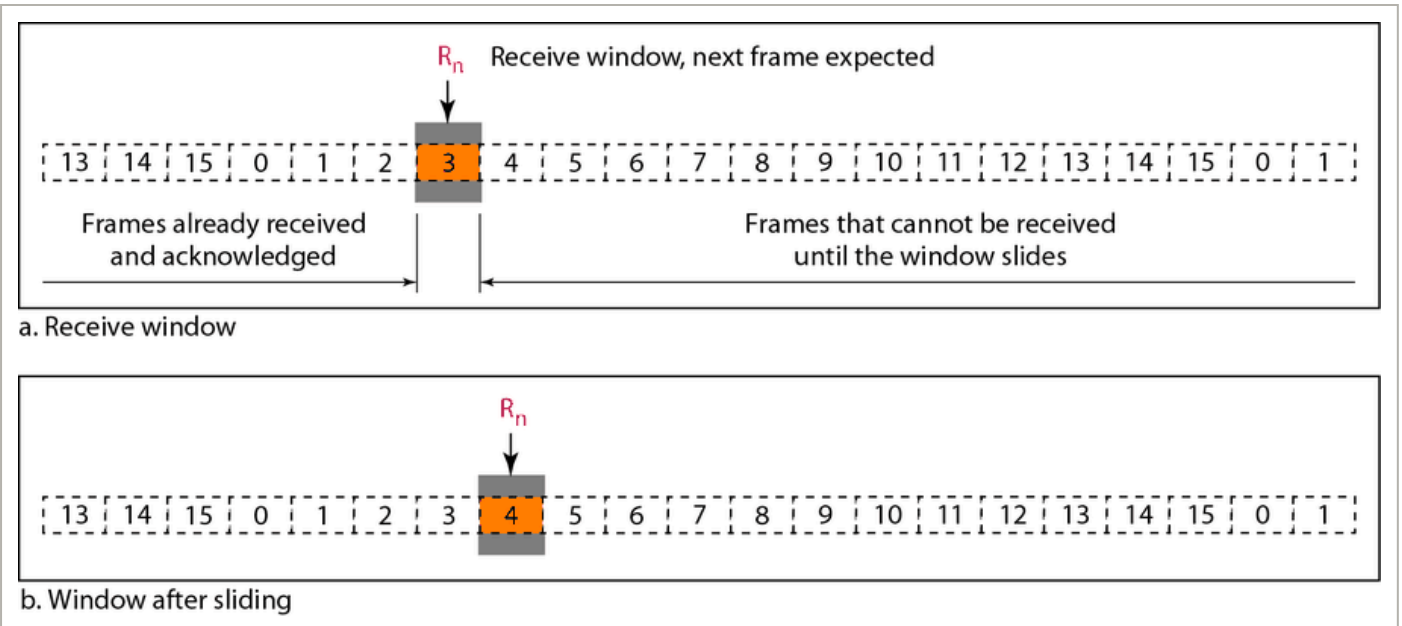


Fig 5.5 The Go-Back-N windows. The send window has *three* variables (S_f , S_n , S_{size}) and the receive window has *one* (R_n) — and that asymmetry is exactly why out-of-order frames must be thrown away.



Slide 135 Send window for Go-Back-N ARQ, from the lecture deck.



Slide 137 Receive window for Go-Back-N ARQ — size 1, one variable R_n .

REMEMBER

In Go-Back-N ARQ, the size of the **send window must be less than 2^m** (so at most $2^m - 1$). The size of the **receive window is always 1**.

Why must the send window be less than 2^m ?

Because if the window were the full 2^m , the receiver could not tell a **retransmission** apart from a **brand-new frame**.

Take $m = 2$, so sequence numbers are 0, 1, 2, 3 and $2^m = 4$.

- **Window size 3 (correct, < 4):** the sender sends frames 0, 1, 2. All three arrive but every ACK is lost. The timer expires, so the sender resends frame 0. The receiver is now waiting for frame 3, sees sequence number 0, recognises it as an old duplicate and **correctly discards** it.
- **Window size 4 (wrong, $= 4$):** the sender sends 0, 1, 2, 3. All arrive, all ACKs are lost. The timer expires and it resends frame 0. But the receiver has now wrapped around and is waiting for frame 0 — the new one. It sees sequence number 0 and **erroneously accepts** the duplicate as fresh data.

Keeping the window one slot smaller than the sequence space leaves a permanent gap that makes the two cases distinguishable.

Two worked walkthroughs

WALKTHROUGH A — DELAYED AND LOST ACKS

The forward channel is reliable but the reverse channel is not. No data frames are lost, but some ACKs are delayed and one is lost. What happens?

After initialisation there are seven sender events. *Request* events are triggered by data arriving from the network layer; *arrival* events are triggered by acknowledgments coming up from the physical layer.

There is **no time-out event** here, because all outstanding frames get acknowledged before the timer expires.

The important detail: although **ACK 2 is lost**, **ACK 3 serves as both ACK 2 and ACK 3**. This is **cumulative acknowledgment** doing its job — ACK 3 says “send me frame 3 next”, which implicitly confirms frames 0, 1 and 2 all arrived.

Lesson: Go-Back-N tolerates lost acknowledgments easily. A later cumulative ACK covers for an earlier lost one, so nothing needs resending.

WALKTHROUGH B – A LOST DATA FRAME (THE “GO BACK” CASE)

Frames 0, 1, 2 and 3 are sent, but frame 1 is lost. Trace what happens.

1. Frame 0 arrives correctly and is accepted.
2. Frame 1 is lost in transit.
3. Frames 2 and 3 arrive at the receiver — *undamaged* — but they are **discarded, because they are received out of order**. The receive window is still sitting on 1, and there is no buffer for anything else.
4. The sender receives no acknowledgment for frames 1, 2 or 3. Its timer finally expires.
5. On time-out the sender **resends all outstanding frames — 1, 2 and 3** — because it does not know what went wrong. It only knows nothing has been confirmed since frame 0.

Note that resending frames 1, 2 and 3 is the response to **one single event**. While the sender is busy responding to it, it cannot handle other events. So when ACK 2 arrives, the sender is still transmitting frame 3, and the physical layer must wait until the current event is finished and the data link layer returns to its sleeping state. The same happens with ACK 3, which arrives while the sender is busy with ACK 2, and again with ACK 4. Before the second timer expires, all outstanding frames have been sent and the timer is stopped.

Result: one lost frame caused **three** frames to be retransmitted. Frames 2 and 3 arrived perfectly and were thrown away anyway. That is the “go back N” behaviour, and it is the protocol’s central weakness.

A NEAT RELATIONSHIP

Stop-and-Wait ARQ is a special case of Go-Back-N ARQ in which the size of the send window is 1. You do not have two unrelated protocols — you have one protocol with a dial on it.

Drawbacks of Go-Back-N

- **Inefficient use of bandwidth.** If a single frame is lost or corrupted, *all* subsequent frames must be retransmitted, even the ones that were received correctly. This wastes significant bandwidth, especially where packet loss is infrequent. The retransmission of multiple frames increases overhead and reduces efficiency, particularly on networks with high bandwidth and low error rates.

- **Increased latency.** Because multiple frames must be resent after a single error, extra delay is introduced. The sender must wait for the acknowledgments of the retransmitted frames, which lengthens the round-trip time. This is especially bad on high-latency networks.
- **Unnecessary retransmissions.** All frames after the lost one within the sender's window get resent regardless of whether they were received correctly. This can cause network congestion and inefficient use of resources.

5.6

Selective Repeat ARQ

Selective Repeat solves all three of those drawbacks with a single change: **give the receiver a buffer, so it can hold out-of-order frames instead of discarding them.**

Efficient bandwidth use

The receiver acknowledges each frame **individually** and **buffers out-of-order frames** until the missing one is retransmitted. So only the specific lost or corrupted frames are resent — not the entire set of following frames.

Reduced latency

Since correctly received frames are buffered rather than thrown away, the protocol avoids the delay of Go-Back-N's bulk retransmissions. Better suited to networks with high latency or high bandwidth.

Minimised retransmissions

Only the frames that were not correctly received are resent. This reduces the load on the network and improves overall efficiency.

WINDOW SIZES IN SELECTIVE REPEAT

In Selective Repeat ARQ, the size of the sender and receiver window must be at most one-half of 2^m

$$S_{\text{size}} = R_{\text{size}} \leq 2^{m-1}$$

Note the two differences from Go-Back-N: the receive window is now the *same size* as the send window (not 1), and the limit is *half* the sequence space (not one less than it).

Why only half?

With $m = 2$ (sequence numbers 0,1,2,3) and a window of $2 = 2^{m-1}$, the receiver's window covers {0,1}. If both ACKs are lost and frame 0 is resent, the receiver's window has not moved past 0, so it recognises the duplicate.

But if the window were 3, the sender could send 0, 1, 2; all arrive, all ACKs are lost; it resends frame 0. The receiver's window is now {3,0,1} — which *includes* 0 — so it **erroneously accepts** the retransmitted frame 0 as a new frame. Halving the window guarantees the old and new meanings of a sequence number can never both be inside the window at once.

The comparison that matters: Go-Back-N against Selective Repeat

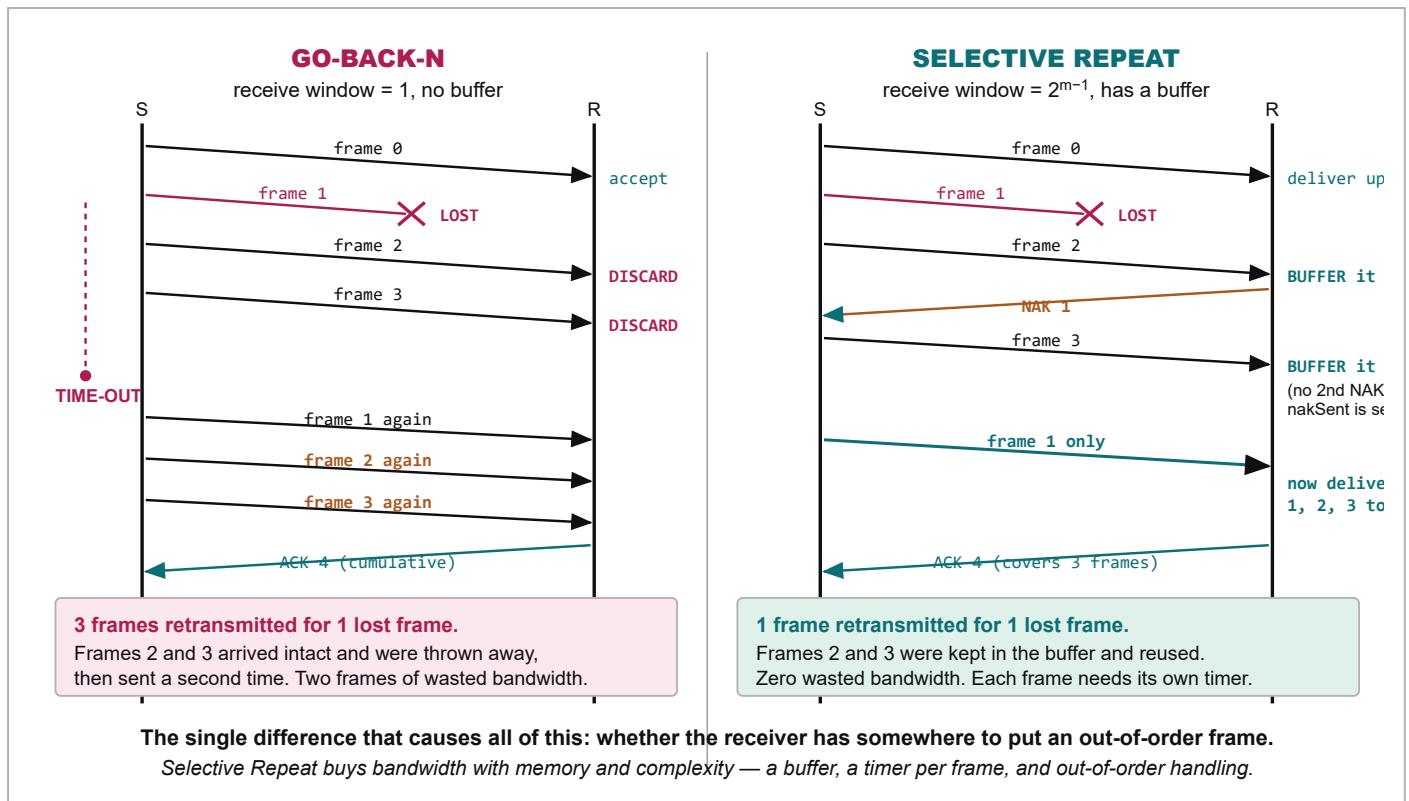
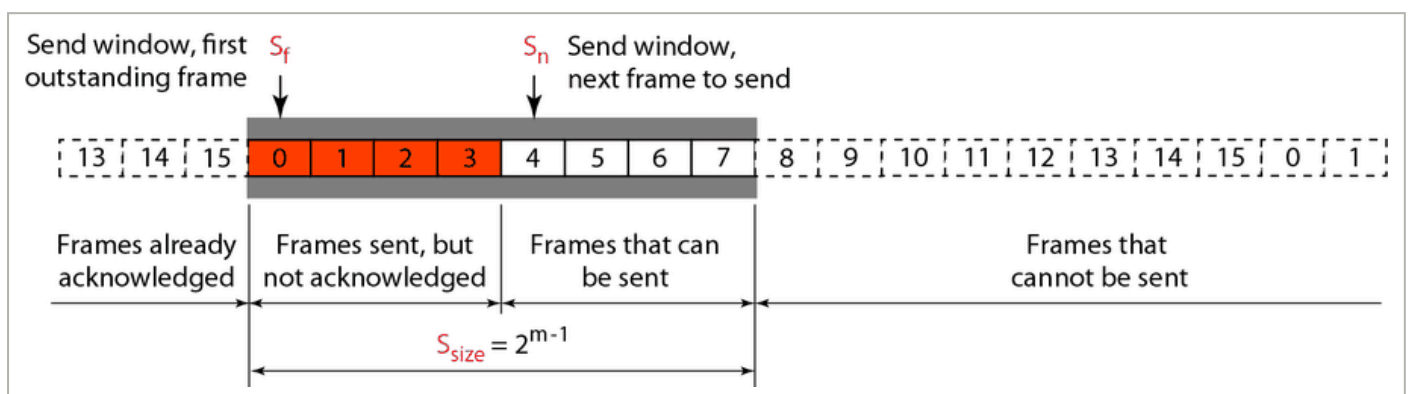
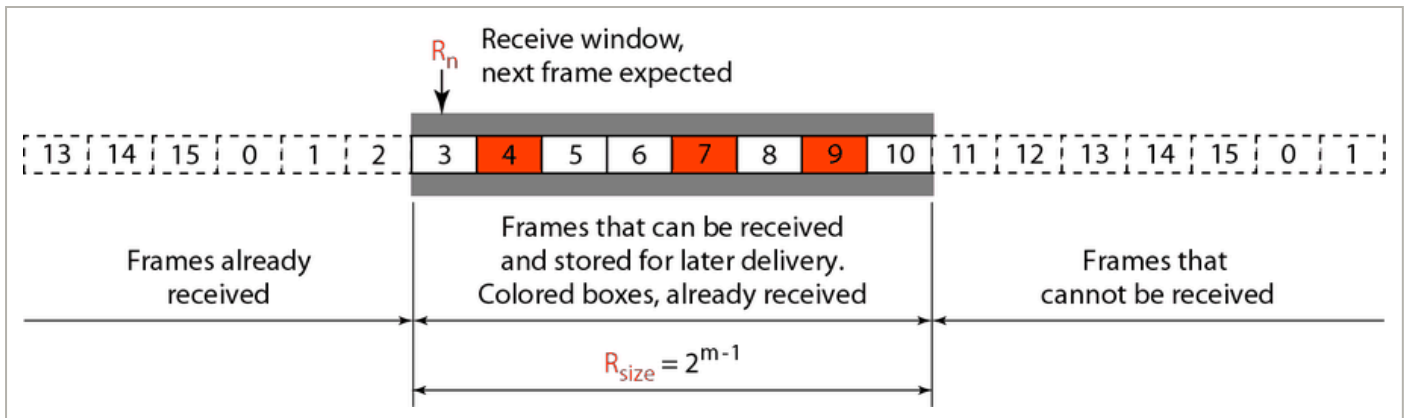


Fig 5.6 The same failure — frame 1 lost — handled by both protocols. Follow the two boxes at the bottom: **three retransmissions versus one**. Everything else about the two protocols is a consequence of the receiver's buffer.



Slide 150 Send window for Selective Repeat ARQ.



Slide 151 Receive window for Selective Repeat ARQ — the same size as the send window, which is what allows buffering.

The details of Selective Repeat, spelled out

Timers. The most obvious difference in practice is the **number of timers**. Here **each frame sent or resent needs its own timer**, so the timers have to be numbered (0, 1, 2, 3). In the walkthrough above: the timer for frame 0 starts at the first request and stops when its ACK arrives; the timer for frame 1 starts at the second request, restarts when a NAK arrives, and finally stops when the last ACK arrives; the other two timers start when their frames are sent and stop at the last arrival event.

Acceptance versus delivery. At the receiver we must now distinguish between *accepting* a frame and *delivering* it to the network layer. When frame 2 arrives it is **stored and marked**, but it cannot be delivered, because frame 1 is missing. When frame 3 arrives it too is marked and stored, and still nothing can be delivered. Only when a copy of frame 1 finally arrives can frames 1, 2 and 3 all be delivered to the network layer.

THE TWO CONDITIONS FOR DELIVERING FRAMES UPWARD

1. A set of **consecutive** frames must have arrived.
2. The set must start from the **beginning of the window**.

The NAK rule. A NAK is sent after the second arrival, but *not* after the third, although the two situations look identical. The reason: the protocol does not want to crowd the network with unnecessary NAKs and unnecessary resent frames. A second NAK would still be NAK 1, telling the sender to resend frame 1 again — which has already been done. So the first NAK sent is **remembered** (using a variable called `nakSent`) and is not sent again until the window slides.

REMEMBER

A NAK is sent **once for each window position**, and it always names the **first slot in the window**.

The ACK rule. In the walkthrough only **two ACKs** are sent. The first acknowledges only frame 0; the second acknowledges three frames at once. This is because in Selective Repeat, **ACKs are sent when data are delivered to the network layer**. If the data belonging to n frames are delivered in one shot, only **one ACK** is sent for all of them.

All three ARQ protocols side by side

STOP-AND-WAIT ARQ VS GO-BACK-N ARQ VS SELECTIVE REPEAT ARQ			
FEATURE	STOP-AND-WAIT ARQ	GO-BACK-N ARQ	SELECTIVE REPEAT ARQ
Send window size	1	$\leq 2^m - 1$	$\leq 2^{m-1}$
Receive window size	1	1	= send window ($\leq 2^{m-1}$)
Sequence numbers	Modulo 2 (0, 1)	Modulo 2^m	Modulo 2^m
Out-of-order frames	Cannot arise	Discarded — no buffer	Buffered and marked
Acknowledgment	One per frame	Cumulative — one ACK can cover many frames	Individual ; one ACK sent when a batch is delivered upward
NAK used?	No	No (time-out does the work)	Yes — once per window position
Timers needed	1	1 (for the oldest outstanding frame)	One per outstanding frame
On one lost frame	Resend that frame	Resend all outstanding frames	Resend only the lost frame
Efficiency	Lowest — $1/(1+2\alpha)$	Good — $N/(1+2\alpha)$	Best — $N/(1+2\alpha)$ with fewer resends
Receiver complexity	Simplest	Simple	Most complex — buffer, sorting, many timers
Best suited to	Very short, slow links	Links with low error rates	Links with high error rates or long delay

5.7

Piggybacking

So far the diagrams have shown data flowing one way and acknowledgments coming back the other way as **separate frames**. In real life communication is usually **bidirectional** — both sides have data to send.

Piggybacking means: instead of sending a separate ACK frame, **carry the acknowledgment inside a data frame that you were going to send anyway**. The ACK rides on the back of the data — hence the name.

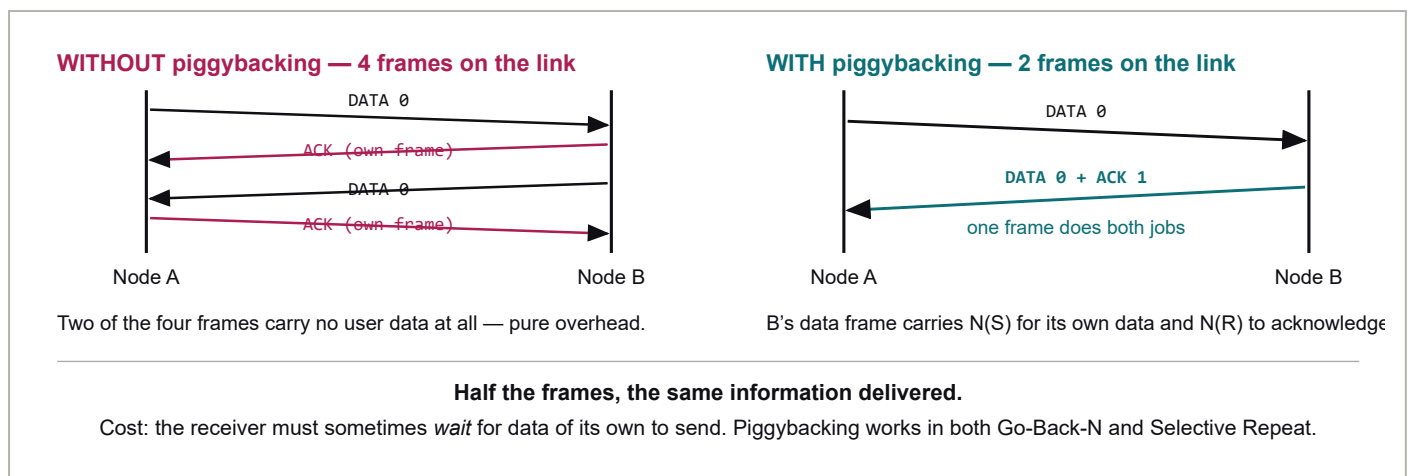


Fig 5.7 Piggybacking, drawn as the difference it makes. The mechanism is a data frame that carries *two* sequence numbers: N(S) for the data it holds, and N(R) to acknowledge the other direction — exactly the HDLC I-frame you will meet in Module 7.

PLAIN WORDS

Piggybacking is like replying to a letter: instead of sending one postcard saying “got your letter” and a second letter with your own news, you write “got your letter” at the top of the letter you were already writing. One envelope, one stamp, both jobs done.

Module 5 at a glance

- **Framing** turns a bit stream into frames. Character-oriented framing needs **byte stuffing**; bit-oriented framing uses the flag 01111110 and **bit stuffing** — insert a 0 after five consecutive 1s.
- **Flow control** stops a fast sender drowning a slow receiver. **Error control** is ARQ: detect, then retransmit.
- **Stop-and-Wait ARQ**: window 1, sequence numbers modulo 2. Simple, and wastes the link on long paths.
- **Go-Back-N**: sender window $2^m - 1$, receiver window 1. One loss means resending everything after it.
- **Selective Repeat**: both windows 2^{m-1} . Only the lost frame is resent, at the price of buffers and sorting at the receiver.

CHECK YOURSELF — MODULE 5

Q What is framing, and give the postal analogy for it.

Framing is breaking data into manageable units called **frames**, each carrying control information as well as data, so that each frame is distinguishable from another. The analogy: **putting a letter into an**

envelope separates one piece of information from another — the envelope is the delimiter.

Q State the byte stuffing rule and the bit stuffing rule.

Byte stuffing: add one extra byte (ESC) whenever a FLAG or ESC character appears in the text. **Bit stuffing:** add one extra 0 whenever five consecutive 1s follow a 0 in the data, so the receiver does not mistake the pattern for a flag.

Q Bit-stuff this data: 01111111111110

Count consecutive 1s and insert a 0 at every fifth:

0 · 11111 →stuff 0 · 11111 →stuff 0 · 111 · 0

Stuffed: 0 11111 0 11111 0 111 0 = 01111101111101110

15 bits in, 17 bits out — 2 stuffed bits added.

Q Why does bit stuffing target runs of 1s and not runs of 0s, when both cause clock problems?

Because bit-oriented protocols like HDLC use a frame delimiter **made of 1s** (01111110), so stuffing must break up runs of 1s to protect that delimiter. Long runs of 0s are handled at a different layer, by **physical-layer encoding** (Manchester, 4B/5B) and by **scrambling**.

Q Distinguish flow control from error control in one sentence each.

Flow control restricts how much data the sender may send before waiting for an acknowledgment, so a fast sender cannot flood a slow receiver. **Error control** in the data link layer is based on **ARQ** — the retransmission of data that was damaged or lost.

Q In Stop-and-Wait ARQ, what does ACK 1 mean? And why are sequence numbers needed at all?

The acknowledgment number always announces, modulo 2, the sequence number of the **next frame expected**. So ACK 1 means “I have frame 0 correctly; now send frame 1.”

Sequence numbers are needed because a time-out tells the sender only that *no ACK came* — it cannot tell a lost frame from a lost ACK. It must resend either way, and the sequence number is what lets the receiver detect and discard the **duplicate**.

Q A link has bandwidth 2 Mbps and a round-trip delay of 30 ms. Frames are 1500 bits. Find the bandwidth-delay product and the utilisation under Stop-and-Wait ARQ.

$BDP = (2 \times 10^6) \times (30 \times 10^{-3}) = 60,000 \text{ bits.}$

Stop-and-Wait sends one frame (1500 bits) per round trip.

Utilisation = $1500 / 60,000 = 0.025 = 2.5\%$. Over 97% of the link is wasted.

Q Same link as above. What window size would you need to reach roughly full utilisation?

You need to fill the pipe: $60,000 \text{ bits} \div 1500 \text{ bits per frame} = 40 \text{ frames}$. So a send window of 40 gives about 100% utilisation.

To get a window of 40 you need $2^m - 1 \geq 40$, so $2^m \geq 41$, giving $m = 6$ ($2^6 = 64$, allowing a window up to 63).

Q State the send and receive window sizes for Go-Back-N and for Selective Repeat.

Go-Back-N: send window $\leq 2^m - 1$; receive window **always 1**.

Selective Repeat: send window = receive window $\leq 2^{m-1}$ (one half of 2^m).

Q Why must the Go-Back-N send window be less than 2^m rather than equal to it?

If it equalled 2^m , the receiver could not distinguish a retransmitted frame from a new one. With $m = 2$ and window 4: send 0,1,2,3, all arrive, all ACKs lost, timer expires, resend frame 0 — but the receiver has wrapped round and is now expecting frame 0, so it **erroneously accepts the duplicate**. Keeping the window one slot smaller leaves a permanent gap that keeps the two cases distinguishable.

Q Frames 0,1,2,3 are sent under Go-Back-N and frame 1 is lost. Exactly which frames get retransmitted, and what happened to frames 2 and 3?

Frames **1, 2 and 3** are all retransmitted. Frames 2 and 3 *did* arrive undamaged, but the receive window is size 1 and still sitting on frame 1, so they were **discarded as out of order**. With no ACKs coming back, the sender's timer expires and it resends everything outstanding, because it does not know what went wrong.

Q Same situation under Selective Repeat. What is different?

Only **frame 1** is retransmitted. Frames 2 and 3 are **buffered and marked** at the receiver rather than discarded. A **NAK 1** tells the sender which frame to resend. Once frame 1 arrives, frames 1, 2 and 3 are delivered to the network layer together, and a single ACK covers all three.

Q State the two conditions that must both hold before Selective Repeat delivers frames to the network layer.

First, a set of **consecutive** frames must have arrived. Second, the set must **start from the beginning of the window**.

Q Why does Selective Repeat send NAK 1 after frame 2 arrives, but not again after frame 3 arrives?

To avoid crowding the network with useless NAKs and duplicate retransmissions. A second NAK would still be NAK 1, asking for a frame that has already been requested. So the first NAK is remembered in a variable called `nakSent` and is not repeated until the window slides. A NAK is sent **once per window position**, and always names the first slot of the window.

Q How is Stop-and-Wait ARQ related to Go-Back-N ARQ?

Stop-and-Wait ARQ is a special case of Go-Back-N ARQ with a send window size of 1.

Q What is piggybacking, and what does the receiver pay for it?

Carrying an acknowledgment **inside a data frame** travelling the other way, instead of sending a separate ACK frame. It halves the number of frames on the link. The cost: the acknowledging side may have to **wait until it has data of its own to send**, which delays the ACK.

MODULE 06

Multiple Access Control

When many stations share one link, somebody has to decide who talks when. There are only three philosophies: let everyone try and cope with the crashes, appoint a controller, or divide the channel up in advance. This module works through all three, with every numerical example from the lectures solved in full.

Lecture slides 162–220 · Chapter 7

6.1

The data link layer splits into two sublayers

Module 5 dealt with a link joining exactly two nodes, where no sharing question arises. But on a **multipoint** or **broadcast** link, one more job appears: deciding which station may use the channel. Because of this, the data link layer is divided into **two functionality-oriented sublayers**.

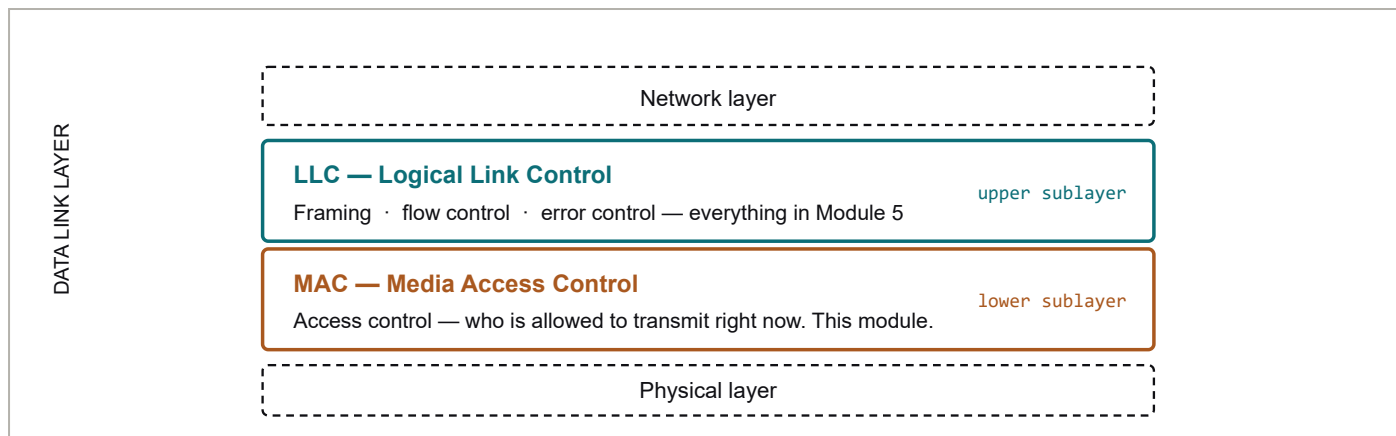


Fig 6.1 The two sublayers. LLC is the same on every kind of network; MAC changes completely depending on whether the link is Ethernet, Wi-Fi, or a satellite channel — which is exactly why it is split out.

6.2

Performance parameters for a network

Before comparing access methods, we need ways to **measure** how well a network is doing. Four numbers matter, and your syllabus names all four.

1. Packet Delivery Ratio (PDR)

PDR is the ratio of the number of packets successfully delivered to the destination to the total number of packets sent by the source, expressed as a percentage.

PACKET DELIVERY RATIO
$\text{PDR} = \left(\frac{\text{packets received by the destination}}{\text{packets sent by the source}} \right) \times 100\%$
Significance. PDR indicates the network's <i>reliability</i> . A high PDR means most packets are reaching their destination — good performance. A low PDR points to high packet loss, network congestion, or poor link quality.

2. Throughput

Throughput is the rate at which data is **successfully** transmitted over the network. Usually measured in bits per second (bps), but it can also be expressed in packets per second (pps).

THROUGHPUT
$\text{Throughput} = \frac{\text{total data successfully received}}{\text{total time taken for transmission}}$
Significance. Throughput measures how much <i>useful</i> data is getting through. High throughput means the network's resources are being used efficiently, and it reflects the actual capacity of the network to handle traffic.

BANDWIDTH IS NOT THROUGHPUT

Bandwidth is the theoretical maximum the link could carry. **Throughput** is what you actually get after collisions, retransmissions, headers and congestion have taken their cut. Throughput is always less than bandwidth, and often much less — Example 5.3 in Module 5 showed a 1 Mbps link delivering 5% of its bandwidth.

3. Latency (delay)

Latency is the time it takes for a data packet to travel from the source to the destination across a network, typically measured in milliseconds. It is made of **four separate components**, and knowing them apart is examined often.

THE FOUR COMPONENTS OF LATENCY		
COMPONENT	WHAT IT IS	WHAT IT DEPENDS ON
Propagation delay	The time taken for a signal to travel from the sender to the receiver	Distance ÷ propagation speed. Nothing else — not bandwidth, not frame size
Transmission delay	The time taken to push all of the packet's bits onto the link	Frame size ÷ bandwidth
Processing delay	The time taken by routers and switches to process the packet headers	Device speed, number of hops
Queuing delay	The time a packet spends waiting in queues before being transmitted	Traffic load — the only component that varies wildly

TOTAL LATENCY

$$\text{Latency} = \text{propagation} + \text{transmission} + \text{processing} + \text{queuing}$$

$$\text{Propagation delay} = \frac{\text{distance}}{\text{propagation speed}}$$

$$\text{Transmission delay} = \frac{\text{message size}}{\text{bandwidth}}$$

Significance. Low latency is crucial for real-time applications such as VoIP, online gaming and video conferencing, where delays badly affect the user's experience. High latency makes a network feel laggy and unresponsive.

4. Jitter

Jitter is the **variation** in the time delay of received packets — a measure of how much packet arrival times wobble. Also measured in milliseconds.

JITTER

$$\text{Jitter} = |D(i+1) - D(i)|$$

where $D(i)$ is the delay experienced by packet i . So jitter is the absolute difference between the delays of two consecutive packets.

Significance. High jitter causes poor quality in real-time voice and video: packets arrive out of order, giving choppy audio or video. Low jitter indicates a stable, predictable network — essential for applications needing a steady stream of data.

EXAMPLE 6.1 — ALL FOUR PARAMETERS TOGETHER

A source sends 5000 packets. 4750 arrive at the destination. The 4750 packets carry 6,000,000 bits of data in total, and the whole transfer took 4 seconds. Three consecutive packets experienced delays of 22 ms, 27 ms and 25 ms. Find the PDR, the throughput, and the jitter between each consecutive pair.

$$\text{PDR} = (4750 / 5000) \times 100\% = 0.95 \times 100\%$$

$$\text{Throughput} = 6,000,000 \text{ bits} / 4 \text{ s}$$

$$\text{Jitter between packets 1 and 2} = |27 - 22| ; \text{ between packets 2 and 3} = |25 - 27|$$

PDR = 95% (5% of packets were lost)

Throughput = 1,500,000 bps = 1.5 Mbps

Jitter (1→2) = 5 ms · Jitter (2→3) = 2 ms

Note jitter is an **absolute** value — the delay dropping from 27 to 25 still counts as 2 ms of jitter, not -2 ms.

5. The bottleneck link

One more idea that the four numbers above do not capture on their own. A path from source to destination is a chain of links, and each one has its own bandwidth. The path does *not* run at the average of those bandwidths, and it certainly does not run at the fastest one.

THE RULE

The throughput of an end-to-end path is set by its slowest link. That link is called the **bottleneck link**. A 1 Gbps fibre backbone feeding a 10 Mbps home router delivers 10 Mbps, and upgrading the fibre to 10 Gbps changes nothing at all.

This is why queues exist. Data arrives at a router faster than the outgoing link can send it, so the excess has to wait somewhere — in a buffer. That waiting is the **queuing delay** from the table above, and it is the only delay component that depends on how busy the network is rather than on physics or hardware.

WHAT HAPPENS WHEN THE QUEUE FILLS

A buffer is finite. While the arrival rate stays below the outgoing link rate, the queue drains as fast as it fills and the delay stays small. Push past that point and the queue grows until the buffer is full, after which every further packet is simply **dropped**. So congestion shows up first as rising latency and jitter, and only then as falling PDR — which is exactly the order in which a network “feels slow” before it starts failing.

6.3

Random access, and the ALOHA protocols

The three families of multiple-access protocol:

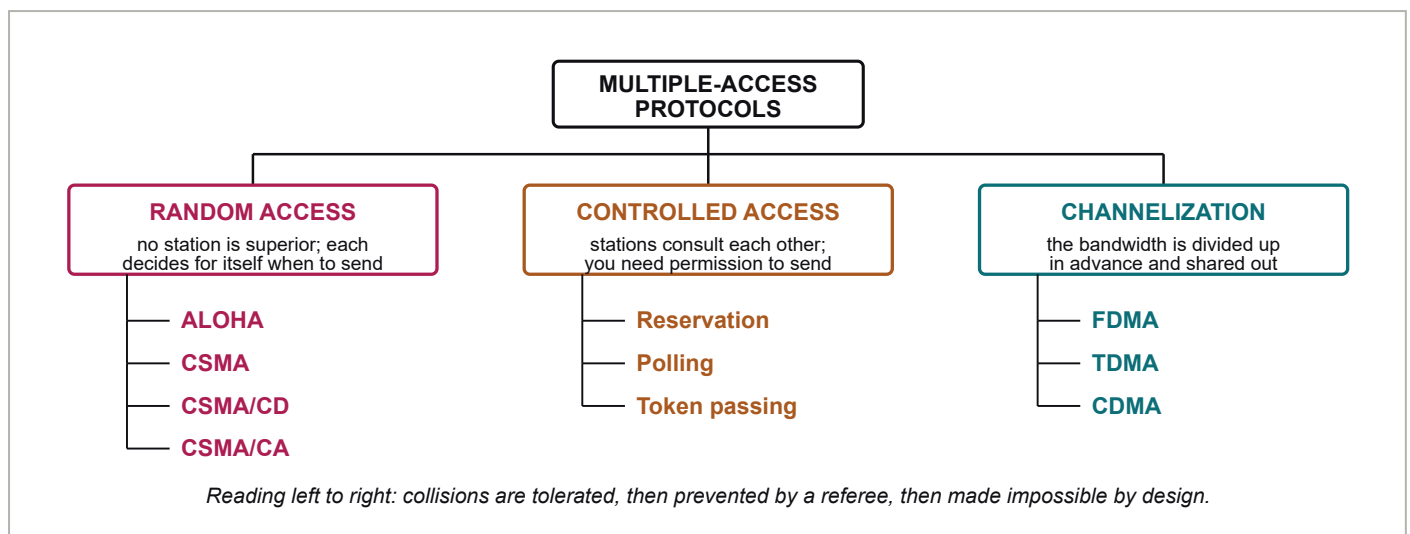


Fig 6.2 The three families. Each column trades something away: random access trades wasted bandwidth for simplicity, controlled access trades delay for order, channelization trades flexibility for guaranteed capacity.

In **random access** or **contention** methods, **no station is superior to another and none is assigned control over another**. No station permits, or refuses to permit, another station to send. At each instant, a station that has data to send uses a procedure defined by the protocol to decide whether or not to send.

Pure ALOHA

The original and simplest random access protocol. The rule is brutally simple:

- 1 Devices **transmit data whenever they have data to send**. No looking, no asking.
- 2 After transmission, the device **waits for an acknowledgment**.
- 3 If no acknowledgment arrives (because there was a collision), the device **waits for a random time** and then retransmits.

A **collision** occurs when two or more devices transmit at the same time, leading to garbled messages. Both frames are destroyed — not just one.

The back-off time

The random wait after a collision is called the **back-off time**, T_B . It is computed by a **binary exponential back-off** rule.

BINARY EXPONENTIAL BACK-OFF

$$T_B = R \times T_p$$

where R is a random number chosen from $\{0, 1, 2, \dots, 2^K - 1\}$

and K is the number of attempts made so far.

Each failed attempt increases K by 1, so the range of possible waits **doubles** every time. This spreads the colliding stations further and further apart until one gets through. If $K > 10$ it is **normally set to 10**, and after $K_{\max} = 15$ attempts the station gives up and reports failure. (Some texts use T_{fr} in place of T_p ; follow whichever your question states.)

EXAMPLE 7.1 – FROM THE SLIDES

The stations on a wireless ALOHA network are a maximum of 600 km apart. Assuming signals propagate at 3×10^8 m/s, find the value of T_B for different values of K .

Step 1 — find the propagation time T_p .

$$T_p = \text{distance} / \text{speed} = (600 \times 10^3) / (3 \times 10^8) = 2 \text{ ms}$$

Step 2 — work out the range of R for each K , then multiply by 2 ms.

(a) $K = 1$. The range is $\{0, 1\}$. The station needs to generate a random number with a value of 0 or 1. So T_B is either 0 ms (0×2) or 2 ms (1×2), depending on the outcome of the random variable.

(b) $K = 2$. The range is $\{0, 1, 2, 3\}$. So T_B can be 0, 2, 4, or 6 ms.

(c) $K = 3$. The range is $\{0, 1, 2, 3, 4, 5, 6, 7\}$. So T_B can be 0, 2, 4, ..., 14 ms.

(d) We need to mention that if $K > 10$, it is normally set to 10.

$$T_p = 2 \text{ ms}$$

$$K = 1 \rightarrow T_B \in \{0, 2\} \text{ ms}$$

$$K = 2 \rightarrow T_B \in \{0, 2, 4, 6\} \text{ ms}$$

$$K = 3 \rightarrow T_B \in \{0, 2, 4, 6, 8, 10, 12, 14\} \text{ ms}$$

$$K > 10 \rightarrow \text{capped at } K = 10$$

Vulnerable time

The **vulnerable time** is the length of time during which there is a **risk of collision**. This is the single most important quantity in ALOHA, and it is what Slotted ALOHA exists to cut in half.

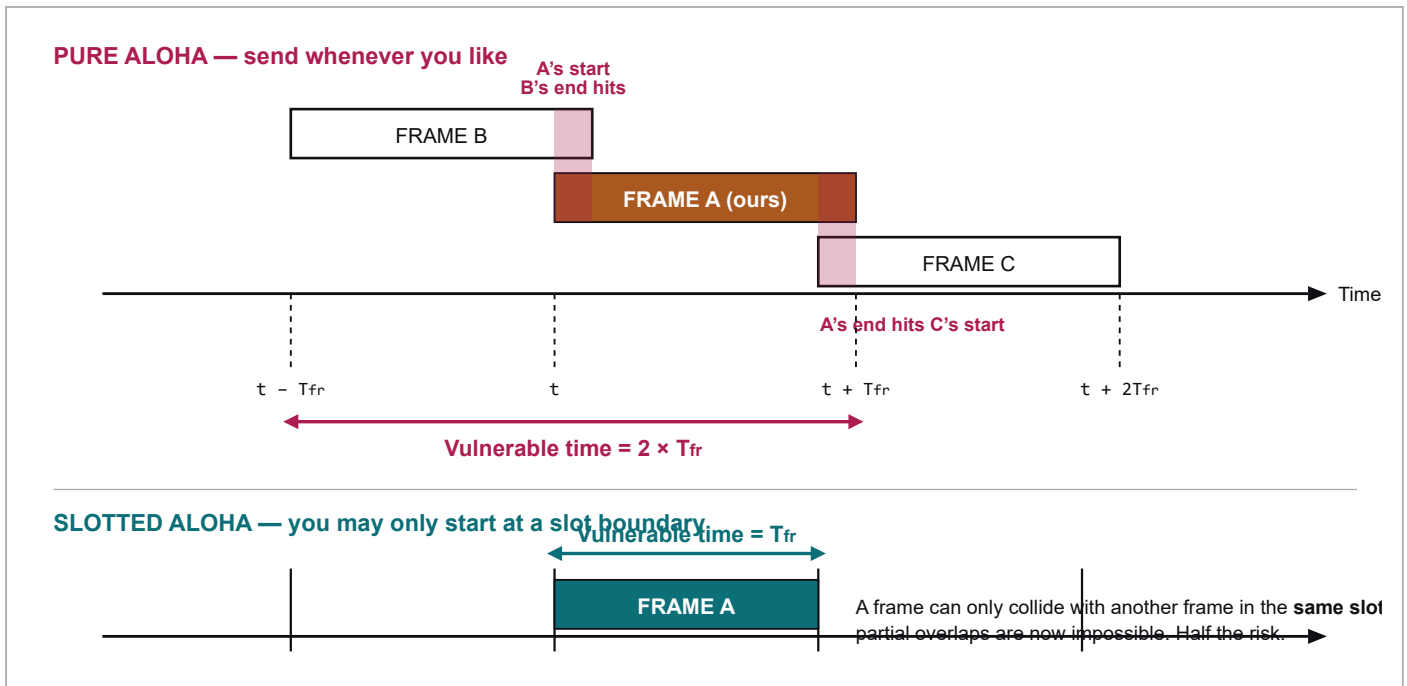
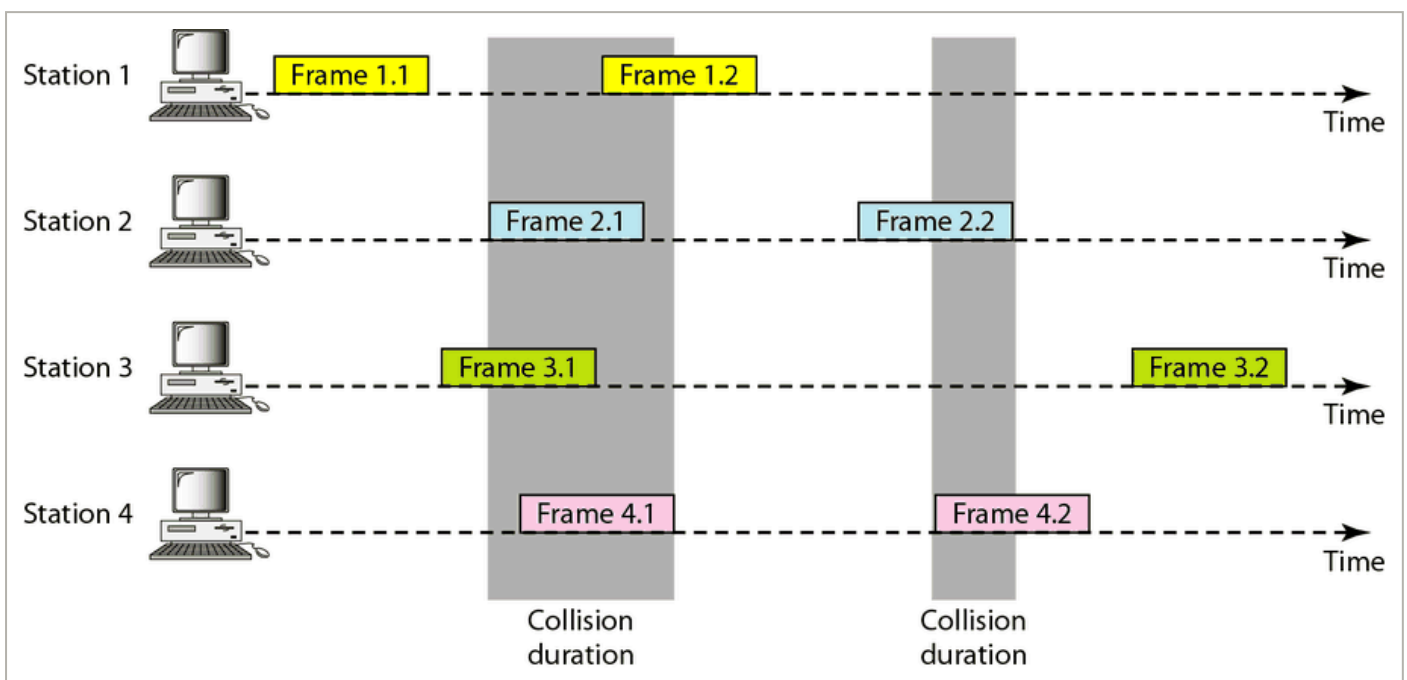
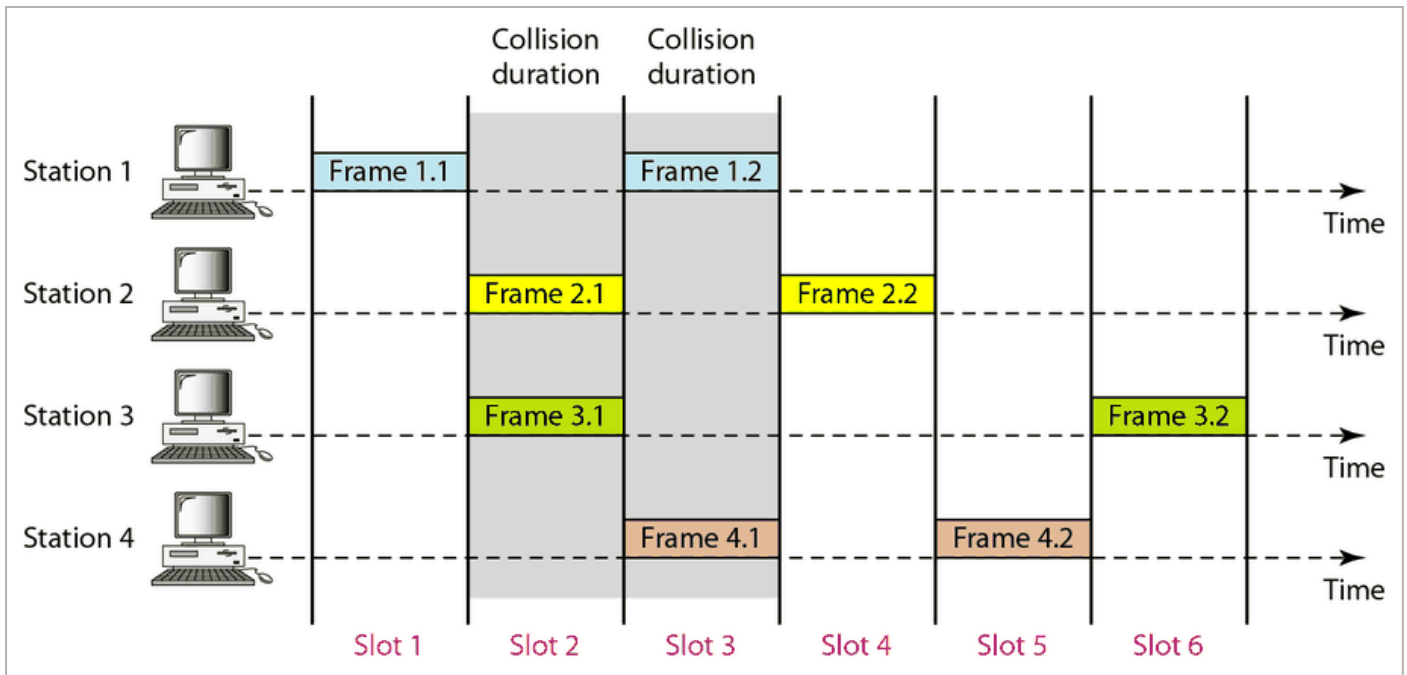


Fig 6.3 Why slotting halves the danger. In pure ALOHA a frame is exposed for $2T_{fr}$ because another station can start any time from one frame-time before to one frame-time after. Forcing every station to start only at slot boundaries removes the “before” half of that window, leaving T_{fr} .



Slide 172 Frames in a pure ALOHA network — collisions where transmissions overlap.



Slide 181 Frames in a slotted ALOHA network — every frame must start on a slot boundary.

VULNERABLE TIME

Pure ALOHA : vulnerable time = $2 \times T_{fr}$

Slotted ALOHA : vulnerable time = T_{fr}

where T_{fr} = average frame transmission time = $\frac{\text{frame size}}{\text{bandwidth}}$

EXAMPLE 7.2 – FROM THE SLIDES

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the requirement to make this frame collision-free?

Step 1 — frame transmission time.

$$T_{fr} = 200 \text{ bits} / 200 \text{ kbps} = 1 \text{ ms}$$

Step 2 — vulnerable time for pure ALOHA.

$$\text{Vulnerable time} = 2 \times T_{fr} = 2 \times 1 \text{ ms} = 2 \text{ ms}$$

Step 3 — state the requirement in words. This means **no station should send later than 1 ms before this station starts transmission, and no station should start sending during the 1 ms period that this station is sending.**

$$T_{fr} = 1 \text{ ms} \quad \cdot \quad \text{Vulnerable time} = 2 \text{ ms}$$

The channel must be free of any other transmission start for the 1 ms before *and* the 1 ms during our frame.

Throughput of ALOHA

THROUGHPUT FORMULAS — MEMORISE THESE

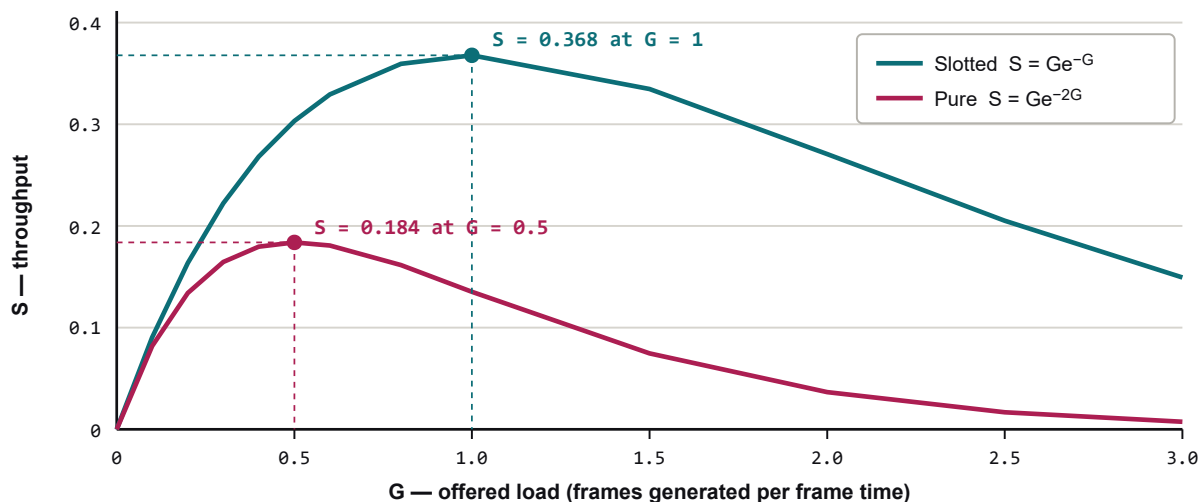
Pure ALOHA : $S = G \times e^{-2G}$

maximum $S_{\max} = 0.184$ when $G = \frac{1}{2}$

Slotted ALOHA : $S = G \times e^{-G}$

maximum $S_{\max} = 0.368$ when $G = 1$

G is the average number of frames generated by the system during one frame transmission time — the **offered load**. S is the fraction that survives. Note the pattern: slotted ALOHA has **exactly double** the maximum throughput, because it has exactly half the vulnerable time.



Both curves fall after their peak: pushing more traffic in produces more collisions, so **less** gets through.

Fig 6.4 ALOHA throughput against offered load. The shape is the lesson: throughput *peaks and then collapses*. Beyond the peak, every extra frame you offer destroys more than it delivers — the network chokes on its own retransmissions.

EXAMPLE 7.3 — PURE ALOHA THROUGHPUT

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the throughput if the system (all stations together) produces (a) 1000 frames/s, (b) 500 frames/s, (c) 250 frames/s?

The frame transmission time is $200 / 200 \text{ kbps} = 1 \text{ ms}$. So “frames per millisecond” is the same as G .

(a) 1000 frames per second = 1 frame per millisecond, so the load $G = 1$.

$$S = G e^{-2G} = 1 \times e^{-2} = 0.135 \text{ (13.5\%)}$$

Throughput = $1000 \times 0.135 = 135 \text{ frames}$. Only 135 frames out of 1000 will probably survive.

(b) 500 frames per second = $\frac{1}{2}$ frame per millisecond, so $G = 0.5$.

$$S = 0.5 \times e^{-1} = \mathbf{0.184} \text{ (18.4\%)}$$

Throughput = $500 \times 0.184 = \mathbf{92 \text{ frames}}$. Only 92 out of 500 survive. Note this is the **maximum throughput case**, percentage-wise.

(c) **250 frames per second** = $\frac{1}{4}$ frame per millisecond, so $G = \mathbf{0.25}$.

$$S = 0.25 \times e^{-0.5} = \mathbf{0.152} \text{ (15.2\%)}$$

Throughput = $250 \times 0.152 = \mathbf{38 \text{ frames}}$. Only 38 out of 250 survive.

- (a) $G = 1.0 \rightarrow S = 0.135 \rightarrow 135 \text{ frames/s}$
- (b) $G = 0.5 \rightarrow S = 0.184 \rightarrow 92 \text{ frames/s} \leftarrow \text{best percentage}$
- (c) $G = 0.25 \rightarrow S = 0.152 \rightarrow 38 \text{ frames/s}$

Notice (a) offers the most traffic but delivers a *worse percentage* than (b). More load is not more throughput.

Slotted ALOHA

Slotted ALOHA was invented to improve pure ALOHA's efficiency. The channel is **divided into time slots of T_{fr} seconds**, and a station is **forced to wait until the beginning of the next slot** before sending. Because a frame can now only collide with another frame in the same slot, partial overlaps are impossible — and the vulnerable time drops to T_{fr} .

EXAMPLE 7.4 – SLOTTED ALOHA THROUGHPUT

A slotted ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the throughput if the system produces (a) 1000, (b) 500, (c) 250 frames per second?

Frame transmission time = $200 / 200 \text{ kbps} = \mathbf{1 \text{ ms}}$, as before.

(a) $G = 1. S = G e^{-G} = 1 \times e^{-1} = \mathbf{0.368} \text{ (36.8\%)}$

Throughput = $1000 \times 0.368 = \mathbf{368 \text{ frames}}$. This is the maximum possible for slotted ALOHA.

(b) $G = 0.5. S = 0.5 \times e^{-0.5} = \mathbf{0.303} \text{ (30.3\%)}$

Throughput = $500 \times 0.303 = \mathbf{151 \text{ frames}}$.

(c) $G = 0.25. S = 0.25 \times e^{-0.25} = \mathbf{0.195} \text{ (19.5\%)}$

Throughput = $250 \times 0.195 = \mathbf{49 \text{ frames}}$.

- (a) $G = 1.0 \rightarrow S = 0.368 \rightarrow 368 \text{ frames/s} \leftarrow \text{maximum}$
- (b) $G = 0.5 \rightarrow S = 0.303 \rightarrow 151 \text{ frames/s}$
- (c) $G = 0.25 \rightarrow S = 0.195 \rightarrow 49 \text{ frames/s}$

Compare with Example 7.3: at $G = 1$, slotted delivers **368** against pure ALOHA's **135** — nearly three times as much, from one rule change.

6.4

The CSMA family

ALOHA's flaw is obvious in hindsight: **stations transmit blindly**. **Carrier Sense Multiple Access (CSMA)** fixes it with a rule that any human would think of first: **listen before you speak**.

A station **senses the medium** (checks whether there is a carrier — another station's signal) before trying to use it. This reduces the chance of collision but does not eliminate it.

Why CSMA still has collisions: propagation delay

The reason is **propagation delay**. When a station sends a frame, it takes time for the first bit to reach every other station. During that gap, another station may sense the medium, find it apparently idle, and start sending.

VULNERABLE TIME IN CSMA

vulnerable time = T_p
the propagation time

If station A starts at t_1 , then B senses at some point before A's signal reaches B, C senses before it reaches C, and D senses before it reaches D. Each of them sees an idle channel that is not really idle. Once A's signal has propagated to every station, the danger is over. So the window of risk is exactly one propagation time.

Persistence methods

What should a station do when it senses the medium? Three answers exist, and the choice is called the **persistence method**.

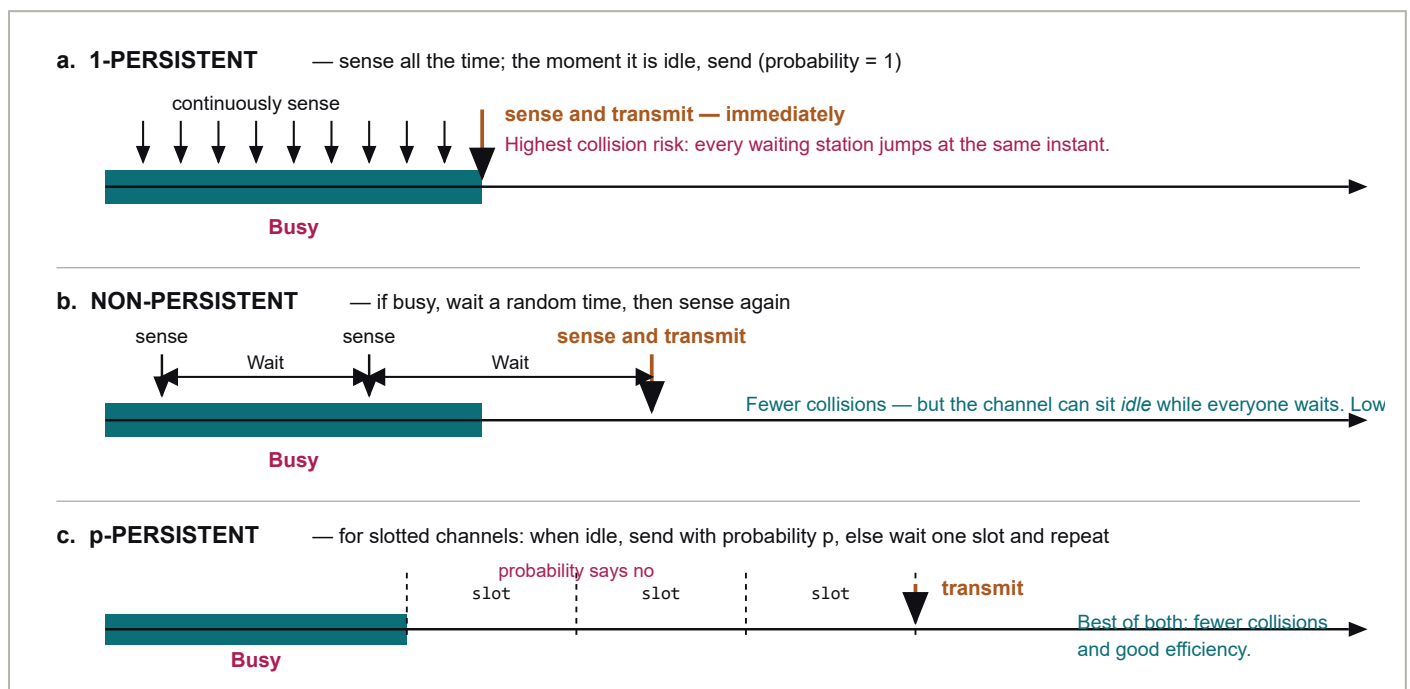
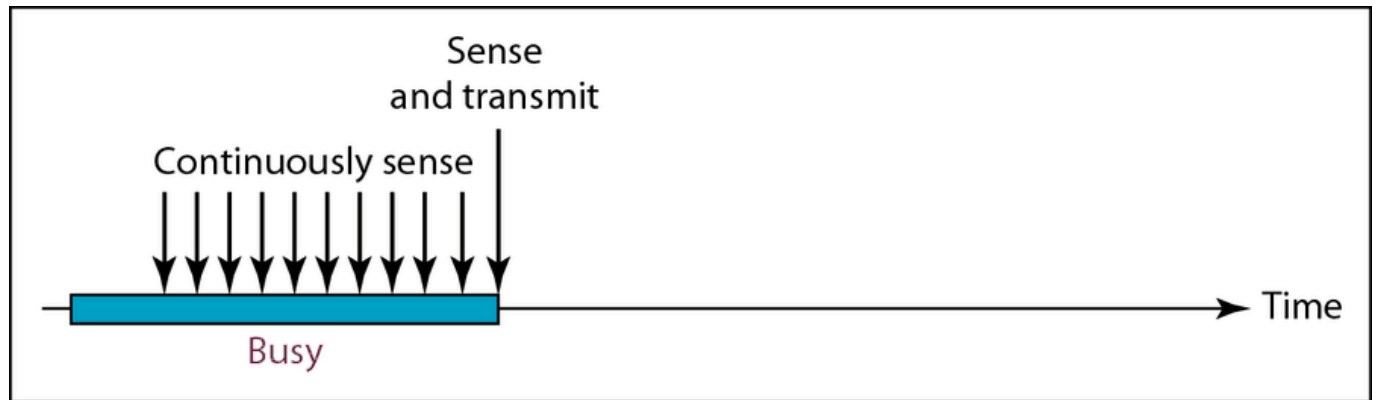
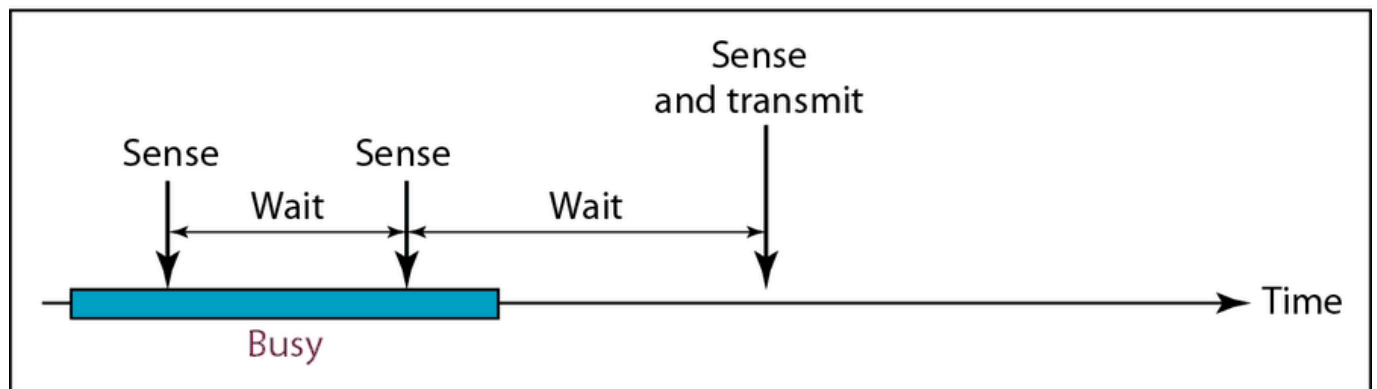


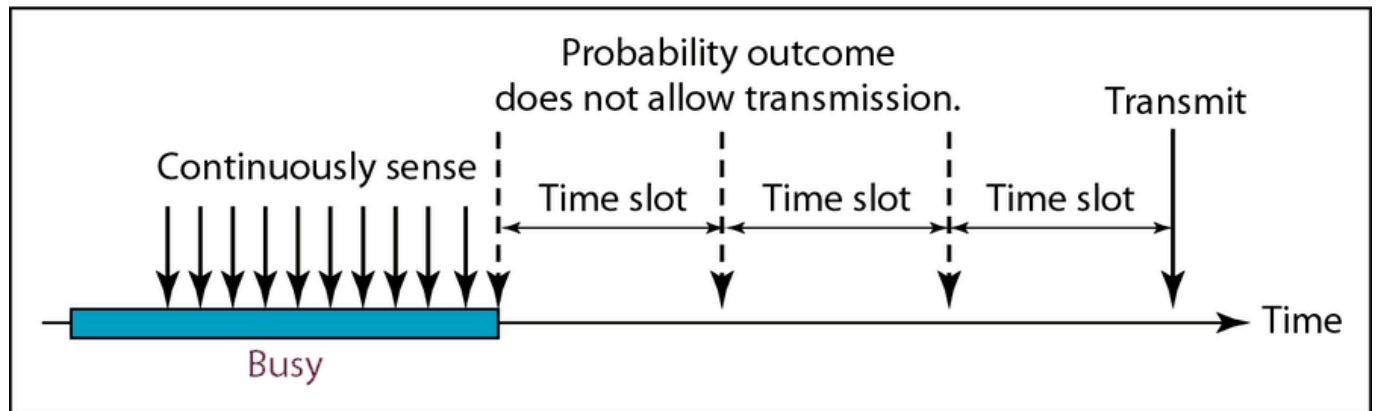
Fig 6.5 The three persistence methods. Read them as a trade-off between two costs: 1-persistent wastes bandwidth on collisions, non-persistent wastes it on idle time, and p-persistent tunes between the two with the value of p .



a. 1-persistent



b. Nonpersistent



c. p-persistent

Slide 188 Behavior of the three persistence methods, from the lecture deck.

COMPARISON OF THE THREE PERSISTENCE METHODS				
METHOD	WHEN CHANNEL IS BUSY	WHEN CHANNEL BECOMES IDLE	COLLISION RISK	CHANNEL EFFICIENCY
1-persistent	Keeps sensing continuously	Sends immediately (probability 1)	Highest — all waiting stations transmit at the same moment	High, when few stations
Non-persistent	Waits a random time, then senses again	Sends immediately if it happens to find it idle	Lowest — waits are staggered	Lower — the channel may be idle while stations are still waiting
p-persistent	Keeps sensing continuously (slotted channel)	Sends with probability p ; with probability $1-p$ waits one time slot and repeats	Medium — tunable	Best overall — combines the advantages of the other two

CSMA/CD — Carrier Sense Multiple Access with Collision Detection

CSMA tells a station how to *avoid* starting a collision, but says nothing about what to do if one happens anyway.

CSMA/CD adds that: a station **monitors the medium while it transmits**, and if it detects a collision, it **aborts immediately** and sends a short **jamming signal** so every other station knows the frame is dead.

Aborting early is the point — there is no sense in wasting the channel finishing a frame that is already ruined.

The minimum frame size requirement

For collision detection to work at all, a station must **still be transmitting when the collision news reaches it**. In the worst case, a collision happens at the far end of the cable just as our frame's first bit gets there, and the news has to travel all the way back. That takes $2 \times T_p$. So:

MINIMUM FRAME SIZE IN CSMA/CD
$T_{fr} \geq 2 \times T_p$ Minimum frame size = bandwidth $\times 2T_p$
If your frame is shorter than this, you will have finished sending and stopped listening before you could possibly learn that it collided — so collision detection silently fails.

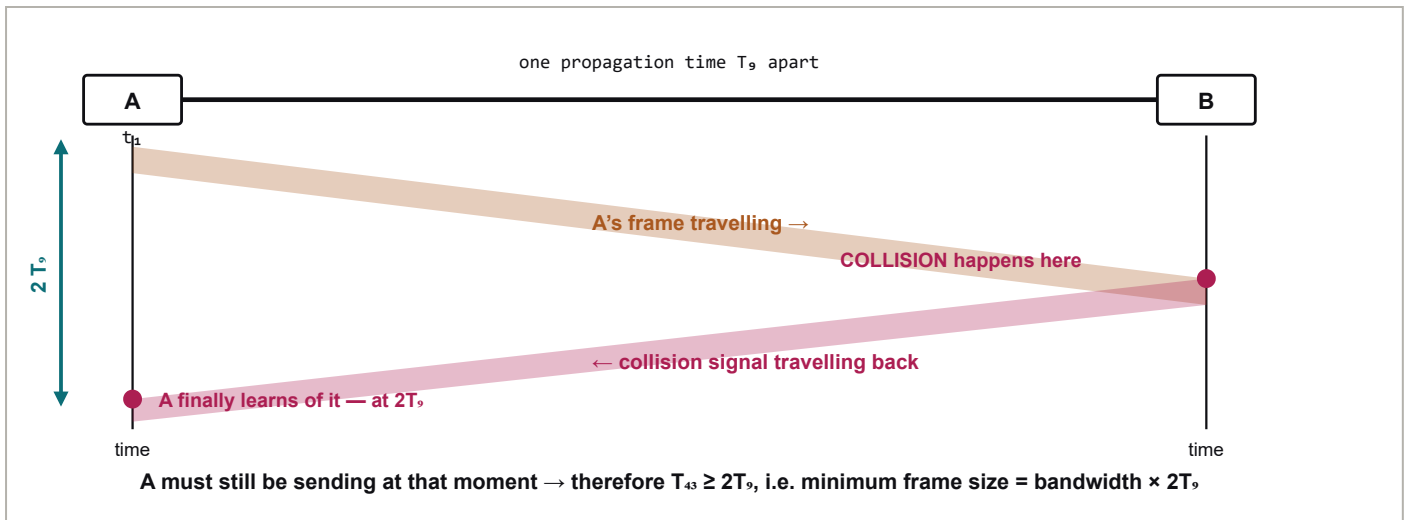
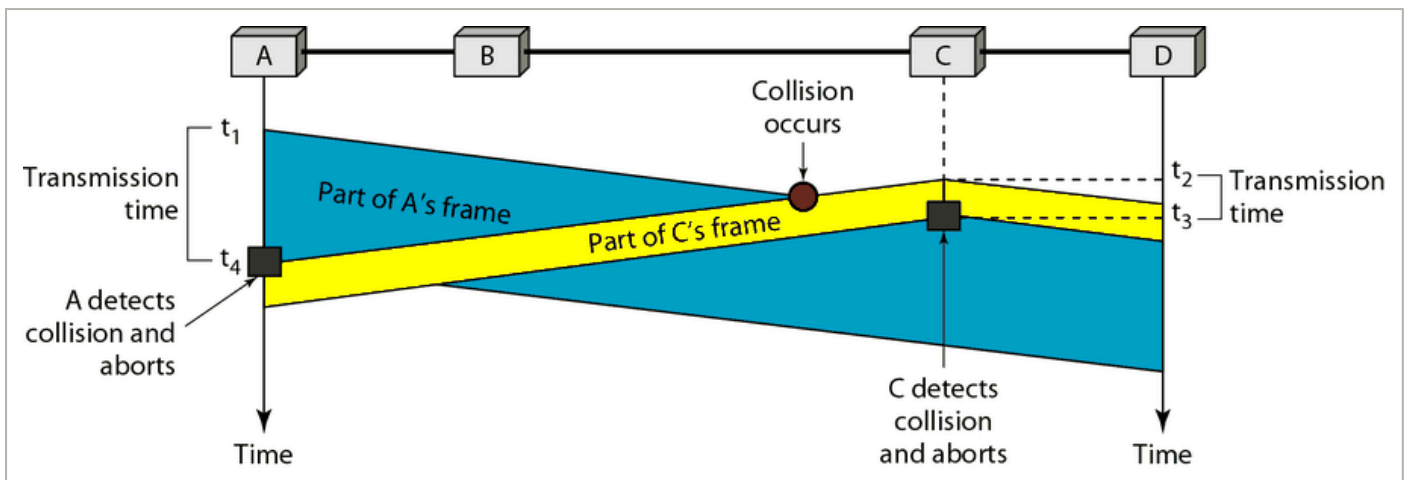


Fig 6.6 The worst case that sets the minimum frame size. The collision occurs at the furthest point at the last possible instant, so the bad news needs a full round trip — $2T_p$ — to get back to A.



Slide 191 Collision and abortion in CSMA/CD.

EXAMPLE 7.5 – FROM THE SLIDES

A network using CSMA/CD has a bandwidth of 10 Mbps. If the maximum propagation time (including delays in the devices, and ignoring the time needed to send a jamming signal) is $25.6 \mu\text{s}$, what is the minimum size of the frame?

Step 1 — find the required frame transmission time.

$$T_{fr} = 2 \times T_p = 2 \times 25.6 \mu\text{s} = 51.2 \mu\text{s}$$

This means that in the worst case, a station needs to transmit for a period of $51.2 \mu\text{s}$ to detect the collision.

Step 2 — convert that time into a number of bits.

$$\text{Minimum frame size} = \text{bandwidth} \times T_{fr} = 10 \text{ Mbps} \times 51.2 \mu\text{s}$$

$$= (10 \times 10^6) \times (51.2 \times 10^{-6}) = 512 \text{ bits}$$

Step 3 — convert to bytes. $512 / 8 = 64 \text{ bytes}$.

$T_{fr} = 51.2 \mu s$ · Minimum frame size = 512 bits = 64 bytes

This is actually the minimum frame size for Standard Ethernet – the 64-byte figure in the IEEE 802.3 standard comes directly from this calculation.

Energy levels: how a station tells idle from busy from collision

A station monitoring the channel sees three distinct **energy levels**:

- **Zero energy level** → the channel is **idle**.
- **Normal energy level** → a station is successfully transmitting; the channel is **busy**.
- **Abnormally high energy level** → **collision** — two signals are adding together.

CSMA/CA — Carrier Sense Multiple Access with Collision Avoidance

CSMA/CD works beautifully on a wire. On a **wireless** network it does not, for three reasons:

- 1 To detect a collision, a station must be able to **send and receive at the same time**, which doubles the cost and complexity of the radio.
- 2 Because of **fading**, a station may simply not hear a collision that is happening at the other end.
- 3 The **hidden station problem**: two stations may both be in range of the receiver but out of range of *each other*, so neither can sense the other at all.

So wireless networks use **collision avoidance** instead of detection. Three strategies do the work: **interframe space**, **contention window**, and **acknowledgments**.

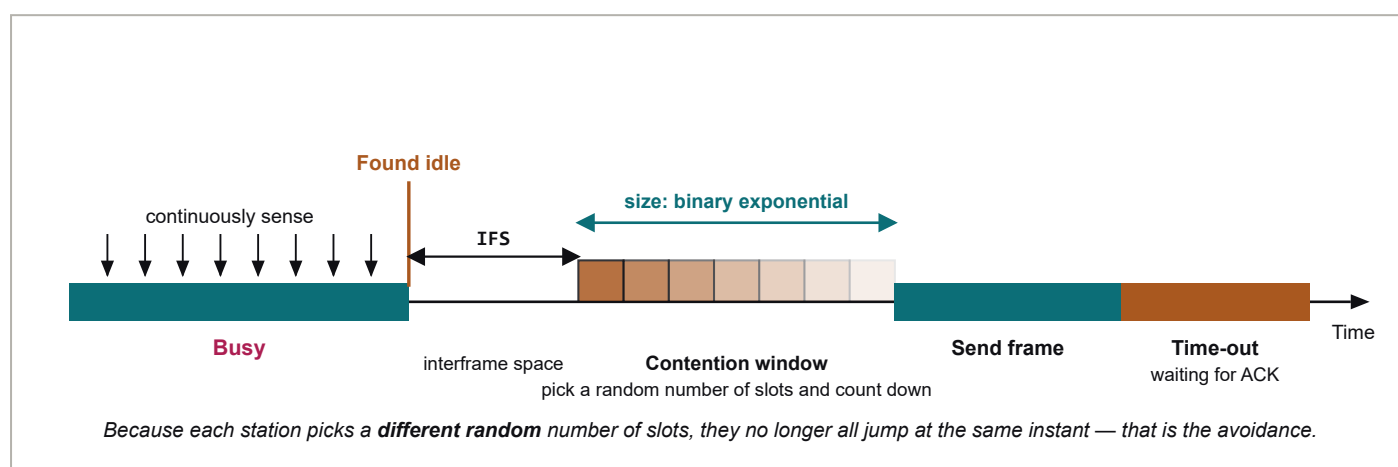
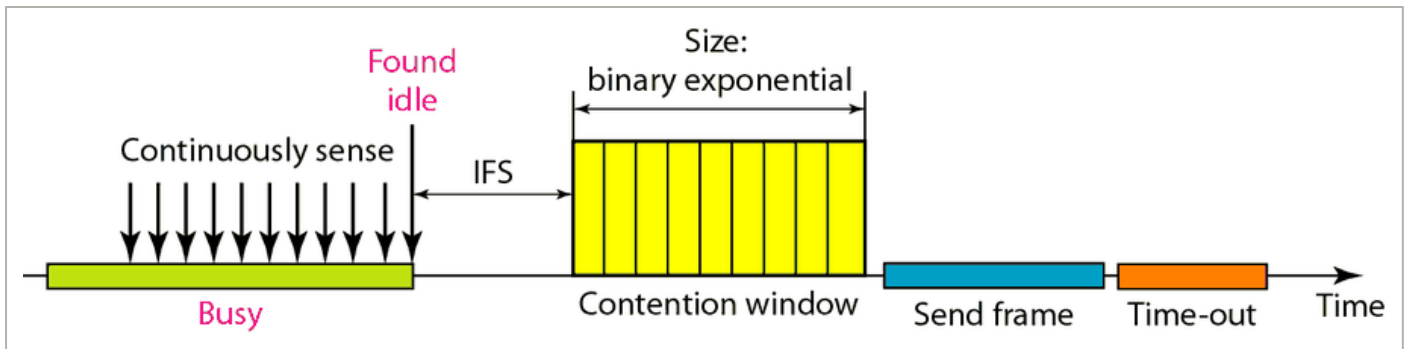


Fig 6.7 CSMA/CA timing. The contention window is the heart of it: 1-persistent CSMA fails because every waiting station transmits at the same moment, and a random per-station countdown is precisely the fix.



Slide 195 Timing in CSMA/CA — IFS, contention window, then the frame.

THE THREE COLLISION-AVOIDANCE STRATEGIES IN CSMA/CA		
STRATEGY	HOW IT WORKS	WHY IT IS NEEDED
Interframe space (IFS)	When a station finds the channel idle, it does not send immediately — it waits for a period called the IFS.	A distant station may <i>already</i> have started transmitting, and its signal simply has not reached us yet. Waiting the IFS gives that signal time to arrive. IFS can also be used to define the priority of a station or a frame — a shorter IFS means higher priority, because you get to jump in first.
Contention window	An amount of time divided into slots . A station ready to send chooses a random number of slots as its wait time. The window follows binary exponential back-off : it starts at one slot and doubles each time the station cannot find the channel idle after the IFS.	It staggers the waiting stations so they do not all transmit at the same moment. Doubling on failure means that under heavy load the stations spread out further, easing the congestion.
Acknowledgment	A positive ACK plus a time-out timer .	Since a collision cannot be detected while sending, the only proof of success is the receiver saying so. No ACK before the time-out means the frame must be resent.

A DETAIL THAT GETS EXAMINED

In CSMA/CA, if the station finds the channel busy during its countdown, it does **not restart** the timer of the contention window. It **stops** the timer and **restarts it from where it left off** when the channel becomes idle again.

This is deliberately fair. A station that has already waited through 8 slots keeps that credit, so it does not lose its place in the queue every time somebody else transmits.

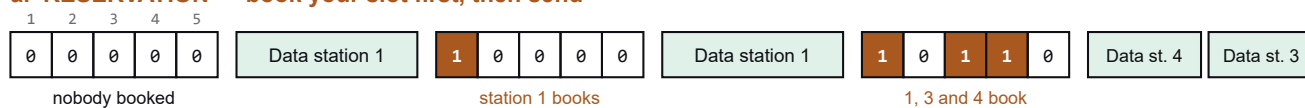
ALL THE RANDOM-ACCESS PROTOCOLS COMPARED					
PROTOCOL	SENSE BEFORE SENDING?	VULNERABLE TIME	MAX THROUGHPUT	HANDLES COLLISIONS BY	USED IN
Pure ALOHA	No	$2 T_{fr}$	18.4% at $G = 0.5$	Random back-off and resend	Early satellite / radio
Slotted ALOHA	No	T_{fr}	36.8% at $G = 1$	Random back-off; must start on a slot boundary	Satellite reservation channels
CSMA	Yes	T_p	Much higher	Persistence method (1-, non-, p-persistent)	Basis of the rest
CSMA/CD	Yes, and while sending	T_p	Highest of the four	Detect: abort, jam, back off, resend	Wired Ethernet (IEEE 802.3)
CSMA/CA	Yes, plus IFS and back-off <i>before</i> sending	T_p	Lower than CD (overhead of IFS + ACK)	Avoid: IFS, contention window, ACK	Wireless LAN (IEEE 802.11)

6.5

Controlled access

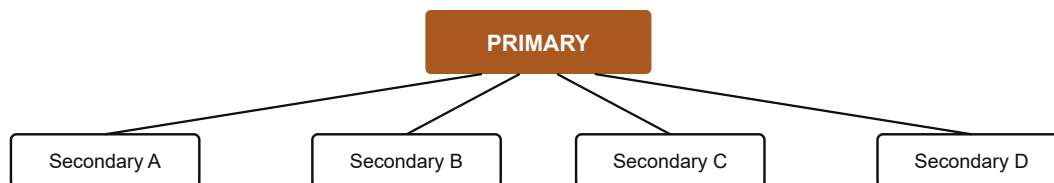
In **controlled access**, the stations **consult one another to find which station has the right to send**. A station **cannot send unless it has been authorised** by other stations. Three methods are used.

a. RESERVATION — book your slot first, then send



Each interval begins with a **reservation frame** holding one minislot per station. To send, a station puts a 1 in *its own* minislot. Everyone reads the frame, so everyone knows the order. Data then flows in that order — **collisions are impossible**.

b. POLLING — a primary station invites each secondary in turn

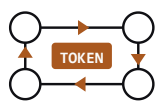


SELECT — the primary has data *for* a secondary, so it sends a select frame first to tell that station to get ready to receive.

POLL — the primary asks each secondary in turn “have you anything to send?” A secondary replies with data, or with a NAK meaning nothing to send.

Weakness: if the primary fails, the whole network stops. And polling stations with nothing to say wastes the channel.

c. TOKEN PASSING — only whoever holds the token may transmit

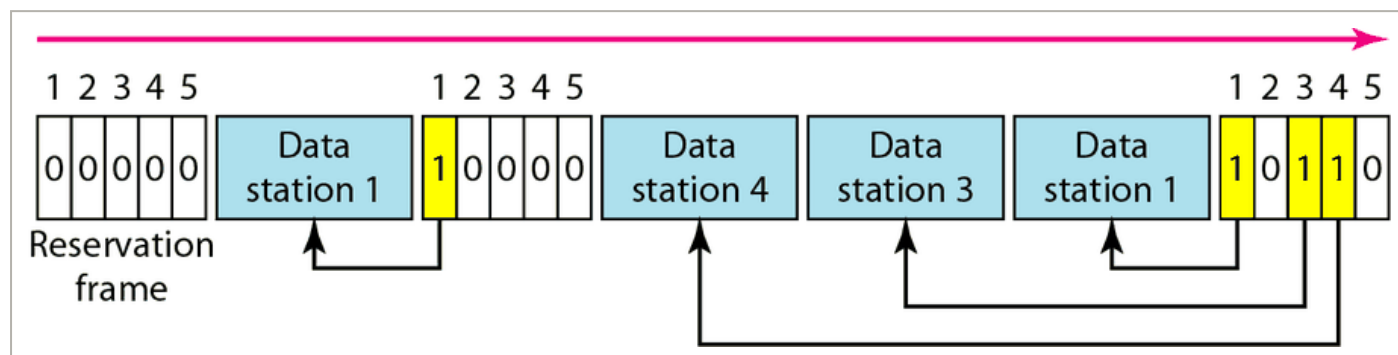


A special frame called the **token** circulates around a **logical ring**. Hold it, and you may send — for a limited time. Then you pass it to your *logical successor*. No token, no transmission, so collisions cannot occur.

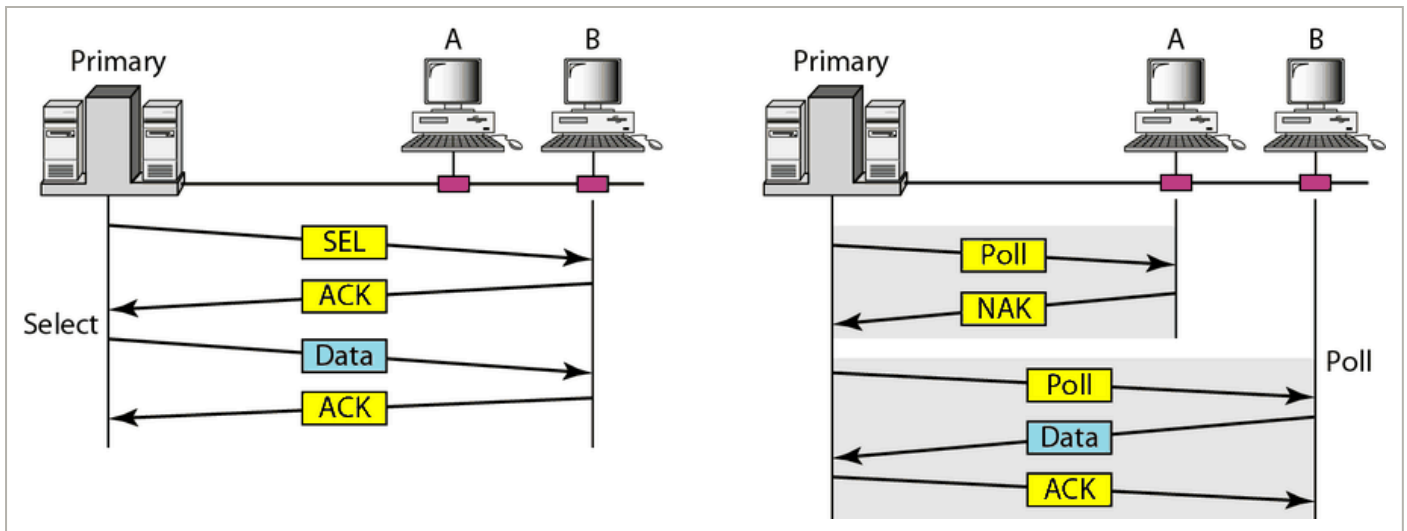
The **logical** ring need not be a physical ring: it can be built on a physical ring, dual ring, bus ring or star ring.

Duties: a token can be lost or duplicated, so token management (monitoring, regenerating, setting a holding time) is required.

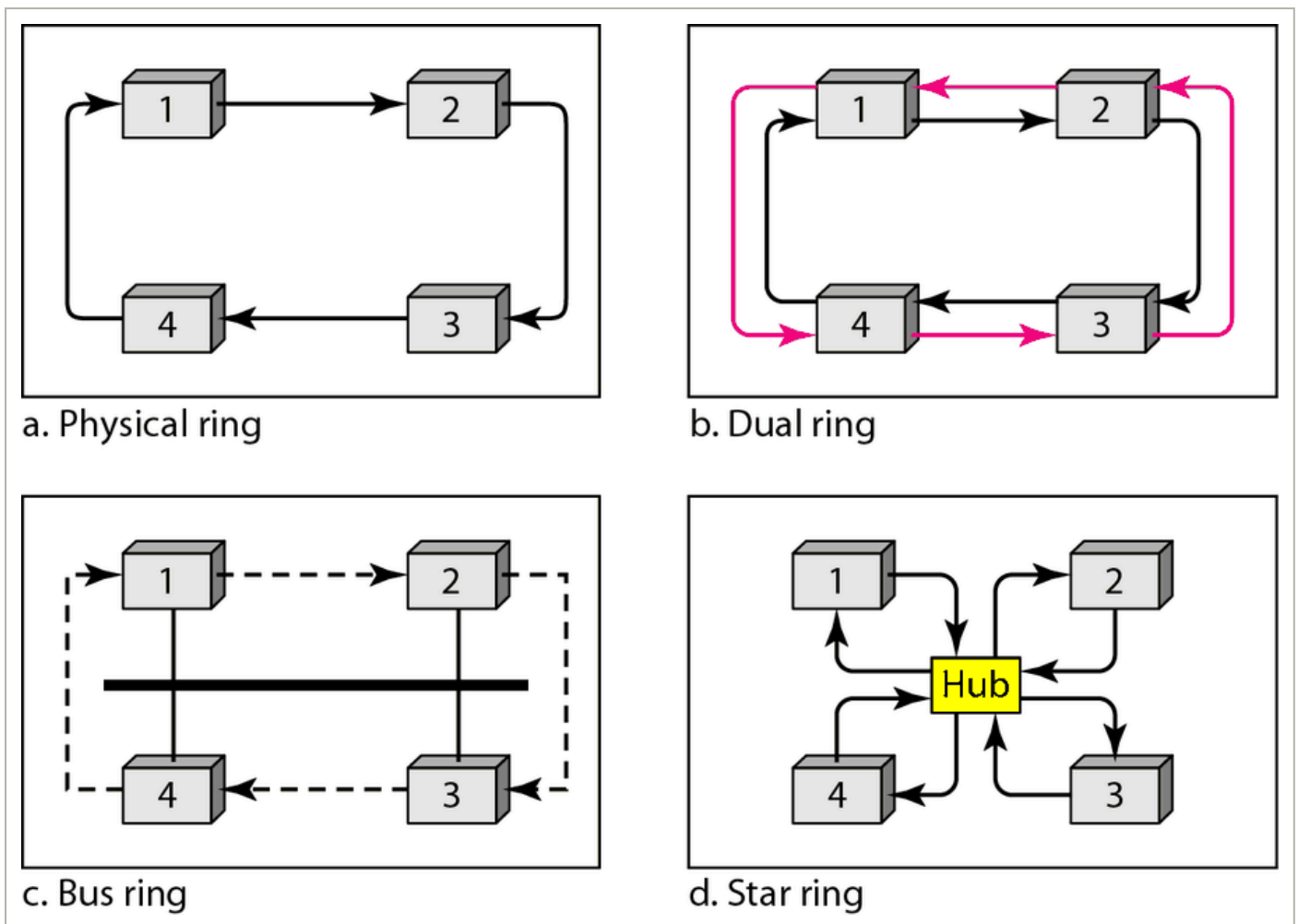
Fig 6.8 The three controlled-access methods. All three make collisions *structurally impossible* rather than merely unlikely — but each introduces a new single point of failure: the reservation frame, the primary station, or the token.



Slide 200 Reservation access method — a reservation frame with one minislot per station.



Slide 201 Select and poll functions in the polling access method.



Slide 202 Logical ring and physical topology in token passing — physical ring, dual ring, bus ring and star ring.

COMPARISON OF CONTROLLED-ACCESS METHODS				
METHOD	WHO GRANTS PERMISSION	HOW ORDER IS DECIDED	MAIN WEAKNESS	USED IN
Reservation	The stations collectively, via the reservation frame	By the minislot each station owns — a 1 means “I want to send”	Minislots are wasted when stations have nothing to send	Satellite links
Polling	A single primary station	The primary’s polling order	Primary is a single point of failure ; polling idle stations wastes time	Multipoint / mainframe terminals
Token passing	Whoever currently holds the token	The logical ring order (each station’s predecessor and successor)	The token can be lost or duplicated , so it needs management	Token Ring, Token Bus, FDDI

6.6

Channelization

Channelization is a multiple-access method in which the **available bandwidth of a link is shared in time, frequency, or through code**, between different stations. Three protocols do this.

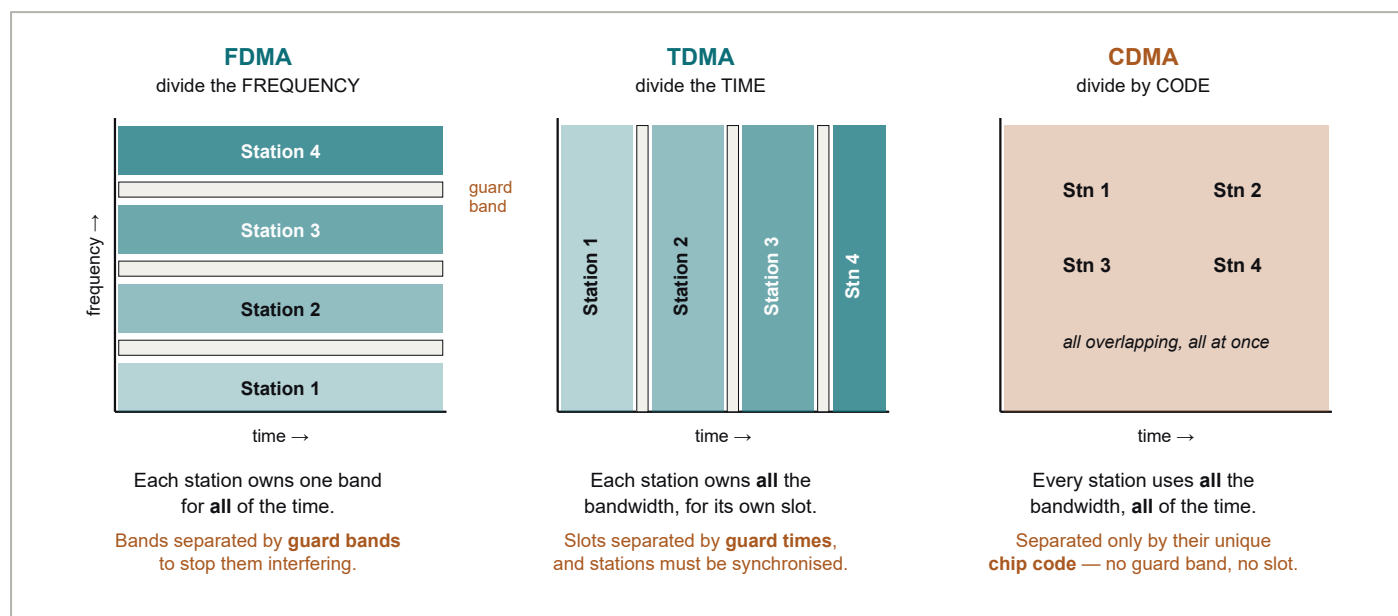
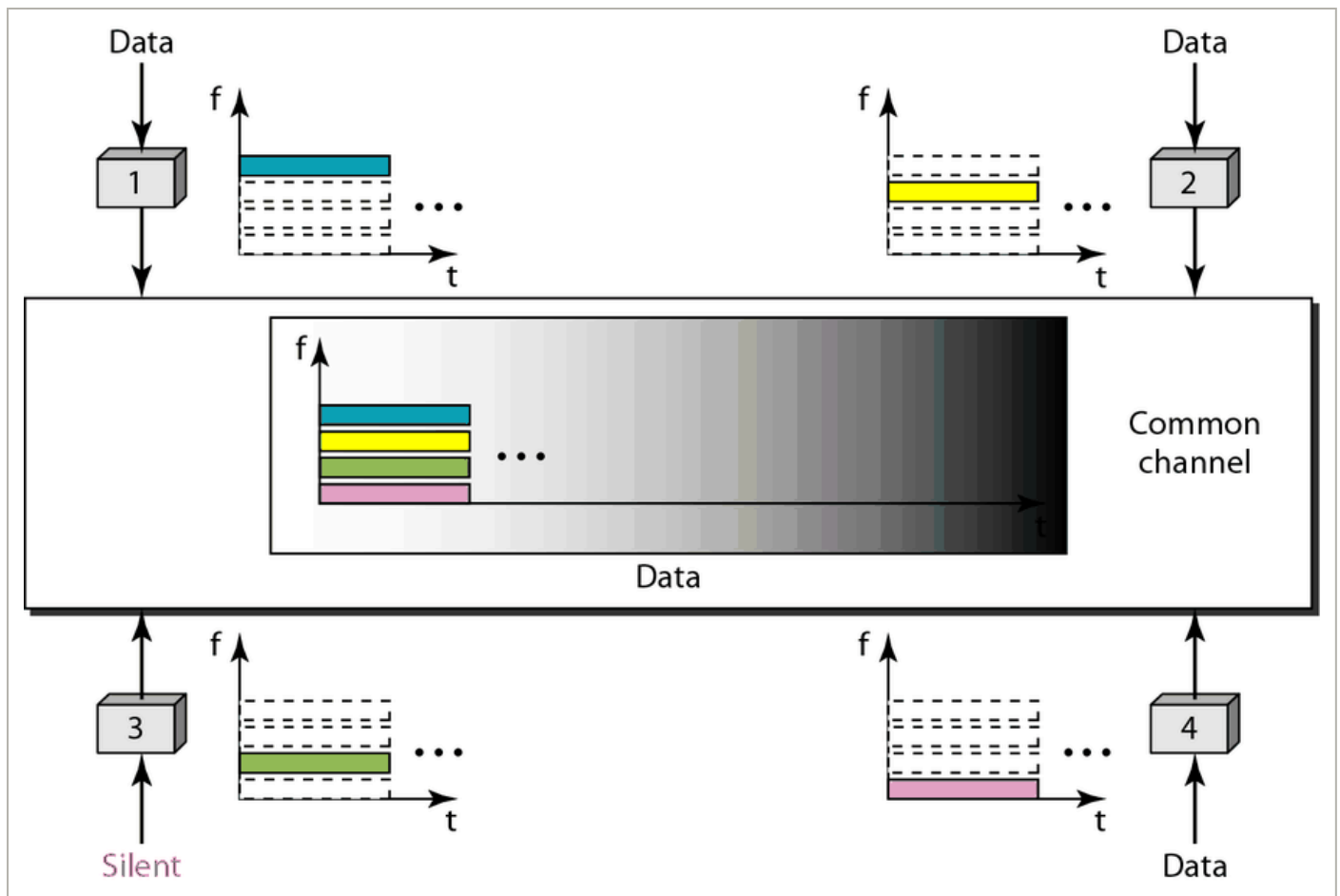
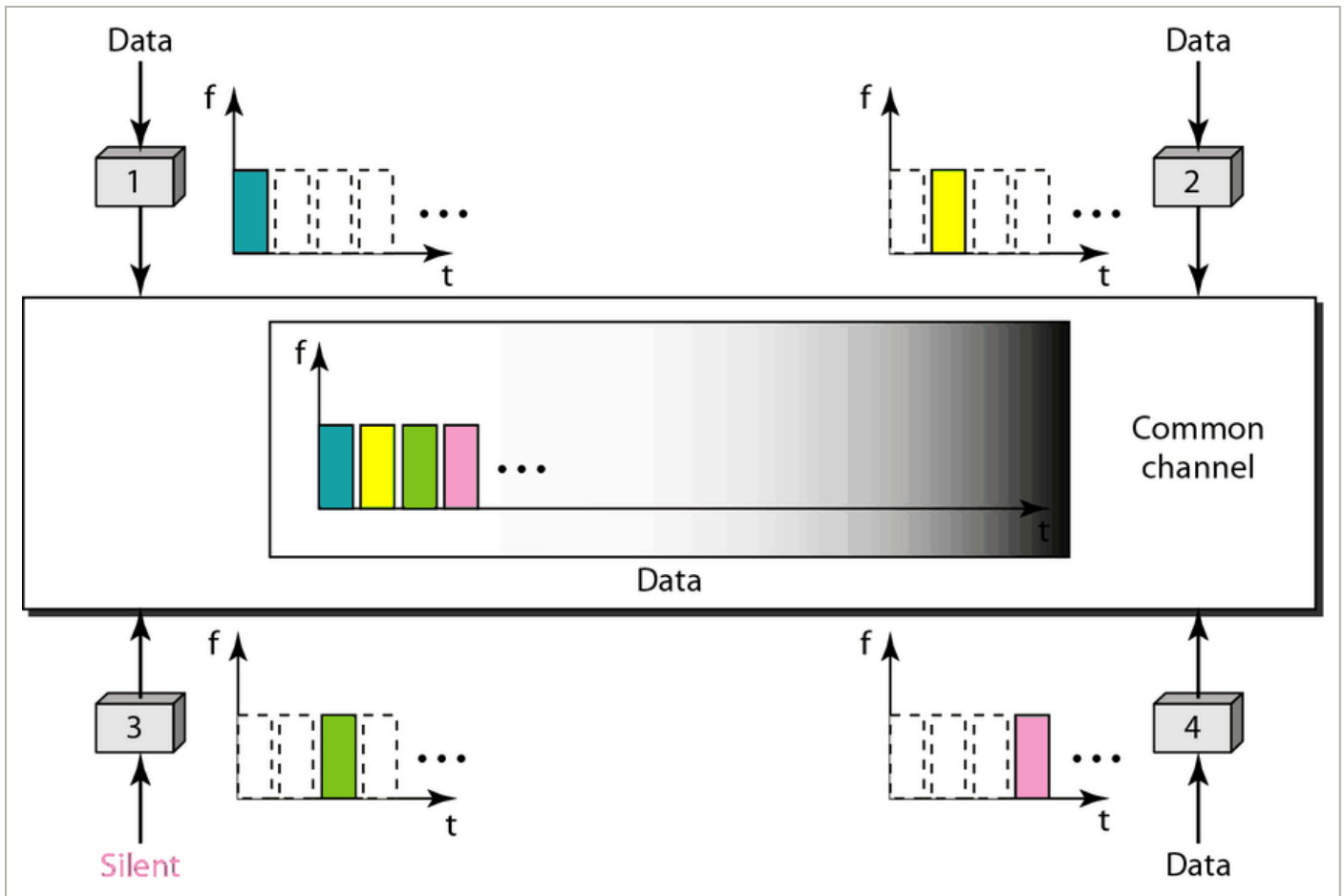


Fig 6.9 The same shared channel, divided three ways. The grid axes are the point: FDMA cuts it horizontally, TDMA cuts it vertically, and CDMA does not cut it at all — every station fills the whole rectangle and mathematics separates them afterwards.



Slide 204 Frequency-division multiple access (FDMA) — bands separated by guard bands.



Slide 206 Time-division multiple access (TDMA) — one channel shared in time.

FDMA — Frequency-Division Multiple Access

The available bandwidth is **divided into frequency bands**. Each station is allocated a band to send its data — that band is reserved for that station **at all times**.

REMEMBER

In FDMA, the available bandwidth of the common channel is divided into bands that are **separated by guard bands**. The guard bands are small unused strips of spectrum that stop one station's signal spilling into its neighbour's band.

FDMA is not the same as FDM. FDM is a *physical layer* technique that combines several signals into one; FDMA is a *data link layer* access method that tells stations how to use the bands. FDMA uses FDM.

TDMA — Time-Division Multiple Access

The stations **share the bandwidth in time**. Each station is allocated a **time slot** during which it may send.

REMEMBER

In TDMA, the bandwidth is just **one channel that is time-shared** between different stations.

The main practical difficulty is **synchronisation**. Every station must know exactly when its slot begins, so **guard times** are inserted between slots to absorb small timing errors and propagation delay differences. As with FDMA, TDMA is the access method and TDM is the underlying physical-layer technique.

CDMA — Code-Division Multiple Access

CDMA is completely different, and it is the part of this module worth the most careful reading.

THE CORE IDEA

In CDMA, **one channel carries all transmissions simultaneously**. There is only one frequency band and there are no time slots. Every station transmits at the same time over the whole bandwidth, and they are separated by **code**.

PLAIN WORDS

Imagine a room with four pairs of people talking at once, each pair in a different language. All the sound waves mix in the air. But if you only understand Tamil, your brain filters out everything else and you hear only the Tamil conversation. In CDMA the “language” is a numeric code, and the “filtering” is multiplication.

Chip sequences

Each station is assigned a unique code, a sequence of numbers called **chips**. For a four-station network:

CHIP SEQUENCES FOR FOUR STATIONS

$$c_1 = [+1 + 1 + 1 + 1]$$

$$c_2 = [+1 - 1 + 1 - 1]$$

$$c_3 = [+1 + 1 - 1 - 1]$$

$$c_4 = [+1 - 1 - 1 + 1]$$

These sequences are **orthogonal**: the inner product of any sequence with itself is N (here 4), and with any *other* sequence is 0. That single mathematical property is what makes CDMA work.

Walsh tables — how the chips are generated

Chip sequences come from a **Walsh table**, built by a recursive rule.

THE TWO BASIC RULES FOR GENERATING A WALSH TABLE

$$W_1 = [+1]$$

$$W_{2N} = \begin{bmatrix} W_N & W_N \end{bmatrix}$$

$$\begin{bmatrix} W_N & W_N \end{bmatrix} \leftarrow \text{the bottom-right block is the}$$

complement of W_N (every sign flipped)

The number of sequences in a Walsh table needs to be $N = 2^m$. So tables exist for 1, 2, 4, 8, 16, 32, 64, 128 ... stations, and nothing in between.

GENERATING W_1 , W_2 AND W_4

$$W_1 = \begin{bmatrix} +1 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix}$$

$$W_4 = \begin{bmatrix} +1 & +1 & | & +1 & +1 \\ +1 & -1 & | & +1 & -1 \\ \hline +1 & +1 & | & -1 & -1 \\ +1 & -1 & | & -1 & +1 \end{bmatrix}$$

W_2 complement of W_2

EXAMPLE 7.6 – FROM THE SLIDES

Find the chips for a network with (a) two stations, (b) four stations.

We use the rows of W_2 and W_4 .

(a) For a two-station network, take the two rows of W_2 .

(b) For a four-station network, take the four rows of W_4 .

(a) $[+1 \ +1]$ and $[+1 \ -1]$

(b) $[+1 \ +1 \ +1 \ +1]$, $[+1 \ -1 \ +1 \ -1]$,
 $[+1 \ +1 \ -1 \ -1]$, and $[+1 \ -1 \ -1 \ +1]$

EXAMPLE 7.7 – FROM THE SLIDES

What is the number of sequences if we have 90 stations in our network?

The number of sequences must be a power of two: $N = 2^m$.

Try $m = 6$: $2^6 = 64$. Not enough — $64 < 90$.

Try $m = 7$: $2^7 = 128$. That is enough, since $128 \geq 90$.

So we choose $m = 7$ and $N = 128$. We can then use 90 of the sequences as the chips, and the remaining 38 are spare.

$m = 7$, $N = 2^7 = 128$ sequences. Use 90 of them; 38 are left unused.

The chip sequence length is also 128, so each data bit becomes 128 chips.

Encoding: how four stations share one channel

The rules for turning data into chips:

- To send bit **1**, a station sends **+1 × its chip sequence**.
- To send bit **0**, it sends **−1 × its chip sequence**.
- If it is **silent**, it sends **0 × its chip sequence** — that is, all zeros.

All four signals then add together on the channel:

DATA ON THE CHANNEL

$$D = (d_1 \cdot c_1) + (d_2 \cdot c_2) + (d_3 \cdot c_3) + (d_4 \cdot c_4)$$

where d_i is +1, −1 or 0 according to what station i is sending, and c_i is its chip sequence.

EXAMPLE 6.2 – FULL CDMA ENCODING, WORKED

In a four-station CDMA network: station 1 sends bit 0, station 2 sends bit 0, station 3 is silent, and station 4 sends bit 1. Find the composite signal on the channel.

Step 1 — convert each station's data to a value d .

$$d_1 = -1 \text{ (bit 0)} \cdot d_2 = -1 \text{ (bit 0)} \cdot d_3 = 0 \text{ (silent)} \cdot d_4 = +1 \text{ (bit 1)}$$

Step 2 — multiply each d by its chip sequence.

$$d_1 \cdot c_1 = -1 \times [+1 +1 +1 +1] = [-1 -1 -1 -1]$$

$$d_2 \cdot c_2 = -1 \times [+1 -1 +1 -1] = [-1 +1 -1 +1]$$

$$d_3 \cdot c_3 = 0 \times [+1 +1 -1 -1] = [0 0 0 0]$$

$$d_4 \cdot c_4 = +1 \times [+1 -1 -1 +1] = [+1 -1 -1 +1]$$

Step 3 — add the four rows, column by column.

$$\text{Chip 1: } -1 - 1 + 0 + 1 = -1$$

$$\text{Chip 2: } -1 + 1 + 0 - 1 = -1$$

$$\text{Chip 3: } -1 - 1 + 0 - 1 = -3$$

$$\text{Chip 4: } -1 + 1 + 0 + 1 = +1$$

Data on the channel: $D = [-1 -1 -3 +1]$

This single sequence of four numbers is all that travels. It contains all four stations' contributions added together.

Decoding: getting one station's data back out

THE DECODING RULE

1. Multiply the data on the channel by the sender's chip code (element by element).
2. Add up the results.
3. Divide by N, the number of stations.

Data on the channel

-1	-1	-3	+1
----	----	----	----

×

Station 2's chip code c_2

+1	-1	+1	-1
----	----	----	----

Inner product result

-1	+1	-3	-1
----	----	----	----

$(-1 \times +1, -1 \times -1, -3 \times +1, +1 \times -1)$

Sum the values

$$-1 + 1 - 3 - 1 =$$

-4

Divide by $N = 4$

$$-4 / 4 =$$

-1

→ BIT 0

$(-1 \text{ means bit 0, } +1 \text{ means bit 1, } 0 \text{ means silent})$

Why the other three stations vanish:

The chip codes are **orthogonal** — $c_2 \cdot c_2 = 4$, but $c_2 \cdot c_1 = c_2 \cdot c_3 = c_2 \cdot c_4 = 0$.

So multiplying by c_2 keeps station 2's contribution intact and cancels everyone else's to exactly zero.

Fig 6.10 Decoding station 2's bit out of the composite signal. The mechanism is orthogonality: $c_2 \cdot c_2 = 4$ while $c_2 \cdot c_j = 0$ for every other j , so the multiplication acts as a perfect filter.

Data bit 0 → -1

Data bit 1 → +1

Silence → 0

Slide 211 Data representation in CDMA: bit 1 → +1, bit 0 → -1, silent → 0.

$$W_1 = \begin{bmatrix} +1 \end{bmatrix}$$

$$W_{2N} = \begin{bmatrix} W_N & W_N \\ W_N & \overline{W_N} \end{bmatrix}$$

a. Two basic rules

$$W_1 = \begin{bmatrix} +1 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix}$$

$$W_4 = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

b. Generation of W_1 , W_2 , and W_4

Slide 215 General rule and examples of creating Walsh tables.

EXAMPLE 7.8 – THE PROOF, FROM THE SLIDES

Prove that a receiving station can get the data sent by a specific sender if it multiplies the entire data on the channel by the sender's chip code and then divides it by the number of stations.

Let us prove this for the first station, using the four-station example. The data on the channel is

$$D = (d_1 \cdot c_1 + d_2 \cdot c_2 + d_3 \cdot c_3 + d_4 \cdot c_4)$$

The receiver that wants station 1's data multiplies D by c_1 :

$$D \cdot c_1 = d_1(c_1 \cdot c_1) + d_2(c_2 \cdot c_1) + d_3(c_3 \cdot c_1) + d_4(c_4 \cdot c_1)$$

Now apply the orthogonality property. The inner product of a chip sequence with *itself* is N , and with any *other* sequence is 0:

$$c_1 \cdot c_1 = N = 4, \quad c_2 \cdot c_1 = 0, \quad c_3 \cdot c_1 = 0, \quad c_4 \cdot c_1 = 0$$

Substituting:

$$D \cdot c_1 = d_1 \times 4 + d_2 \times 0 + d_3 \times 0 + d_4 \times 0 = 4 d_1$$

When we divide the result by N , we get d_1 .

$(D \cdot c_1) / N = 4d_1 / 4 = d_1$ — exactly station 1's data, with every other station cancelled out.

The proof works identically for any station, which is why CDMA scales to hundreds of simultaneous users on one frequency.

FDMA VS TDMA VS CDMA			
FEATURE	FDMA	TDMA	CDMA
What is divided	Frequency — into bands	Time — into slots	Nothing — separation is by code
A station gets	One band, all the time	All the bandwidth, in its slot	All the bandwidth, all the time
Separator used	Guard bands	Guard times	None — orthogonal chip codes
Synchronisation	Not needed	Critical — slots must line up	Needed for chip alignment
Number of users	Limited by available bands	Limited by number of slots	$N = 2^m$, from the Walsh table size
Underlying technique	FDM (physical layer)	TDM (physical layer)	Spread spectrum
Idle-user waste	Band sits unused	Slot sits unused	None — a silent station just contributes zeros
Used in	Analogue cellular (1G), radio	2G GSM	3G cellular, GPS

Module 6 at a glance

- The data link layer splits into **LLC** (flow and error control) and **MAC** (who gets the channel).
- Performance: **PDR, throughput, latency** (propagation + transmission + processing + queuing) and **jitter**. Bandwidth is what the link could do; throughput is what you actually get.
- **Random access**: pure ALOHA $S = G e^{-2G}$ peaking at 18.4%, slotted ALOHA $S = G e^{-G}$ peaking at 36.8%. CSMA listens first; CSMA/CD detects collisions and needs $T_{\text{frame}} \geq 2T_{\text{prop}}$; CSMA/CA avoids them, because a wireless station cannot hear a collision while transmitting.
- **Controlled access**: reservation, polling, token passing. **Channelization**: FDMA shares frequency, TDMA shares time, CDMA shares both using orthogonal Walsh codes.

CHECK YOURSELF — MODULE 6

Q Name the two sublayers of the data link layer and say what each does.

LLC (Logical Link Control) — the upper sublayer, responsible for framing, flow control and error control.
MAC (Media Access Control) — the lower sublayer, responsible for access control: deciding which station may transmit.

Q Write the formulas for PDR, throughput and jitter.

PDR = (packets received by destination / packets sent by source) × 100%
Throughput = total data successfully received / total time taken for transmission
Jitter = $|D(i+1) - D(i)|$, where $D(i)$ is the delay of packet i

Q Name the four components of latency, and say which one depends on frame size and which on distance.

Propagation, transmission, processing and queuing delay. **Transmission delay** depends on frame size (frame size ÷ bandwidth). **Propagation delay** depends on distance (distance ÷ propagation speed).

Q State the vulnerable time and maximum throughput for pure and slotted ALOHA, and explain why one is exactly double the other.

Pure: vulnerable time = $2T_{fr}$, $S = Ge^{-2G}$, $S_{max} = 0.184$ at $G = 0.5$.

Slotted: vulnerable time = T_{fr} , $S = Ge^{-G}$, $S_{max} = 0.368$ at $G = 1$.

Slotting forces frames to start only at slot boundaries, which removes partial overlaps and so **halves the vulnerable time** — and halving the exposure exactly doubles the maximum throughput.

Q Stations are 900 km apart, signals travel at 3×10^8 m/s. Find T_p and the possible back-off times for $K = 2$.

$$T_p = (900 \times 10^3) / (3 \times 10^8) = 3 \text{ ms.}$$

For $K = 2$, $R \in \{0, 1, 2, 3\}$, so $T_B = R \times T_p \in \{0, 3, 6, 9\} \text{ ms.}$

Q A pure ALOHA network sends 500-bit frames at 500 kbps and generates 800 frames/s. Find the throughput.

$T_{fr} = 500 \text{ bits} / 500 \text{ kbps} = 1 \text{ ms.}$ 800 frames/s = 0.8 frames per ms, so $G = 0.8$.

$$S = Ge^{-2G} = 0.8 \times e^{-1.6} = 0.8 \times 0.2019 = \mathbf{0.1615} \text{ (16.15\%).}$$

Throughput = $800 \times 0.1615 \approx 129$ frames/s.

Q What is the vulnerable time in CSMA, and why is it not zero even though every station listens first?

Vulnerable time = T_p , the propagation time. It is not zero because a signal takes time to travel. During that time another station can sense the medium, find it *apparently* idle, and start sending — the first station's signal simply has not reached it yet.

Q Which persistence method has the highest collision risk, which wastes the most channel time, and why?

1-persistent has the highest collision risk, because every waiting station transmits at exactly the same instant the channel goes idle. **Non-persistent** wastes the most channel time, because after finding the channel busy a station waits a random period — so the channel can sit idle while stations are still counting down.

Q A CSMA/CD network runs at 100 Mbps with a maximum propagation time of 20 μ s. Find the minimum frame size.

$$T_{fr} = 2T_p = 2 \times 20 = 40 \mu s.$$

$$\text{Minimum frame size} = \text{bandwidth} \times T_{fr} = (100 \times 10^6) \times (40 \times 10^{-6}) = 4000 \text{ bits} = 500 \text{ bytes}.$$

Q Why can't CSMA/CD be used on a wireless network? Name all three reasons.

1. Detecting a collision requires **sending and receiving at the same time**, which doubles the cost and complexity of the radio. 2. Because of **fading**, a station may not hear a collision happening at the other end. 3. The **hidden station problem**: two stations can both reach the receiver while being out of range of each other, so neither can sense the other at all.

Q Name the three collision-avoidance strategies of CSMA/CA. What happens to the contention-window timer if the channel goes busy mid-countdown?

Interframe space (IFS), contention window (random slots, binary exponential size), and **acknowledgments**.

If the station finds the channel busy, it does **not restart** the contention-window timer — it **stops** it and **restarts from where it left off** when the channel becomes idle. This keeps the credit a station has already earned by waiting.

Q Besides staggering stations, what second purpose does the IFS serve?

It can be used to **define the priority** of a station or a frame. A shorter IFS means the station waits less before contending, so it effectively gets in first — higher priority.

Q In polling, what is the difference between a select frame and a poll frame?

Select: the primary has data *for* a secondary, so it sends a select frame first to tell that station to get ready to receive. **Poll:** the primary asks a secondary whether *it* has anything to send; the secondary replies with data, or with a NAK meaning nothing to send.

Q In token passing, does the logical ring have to be a physical ring?

No. The **logical** ring defines each station's predecessor and successor, and it can be built on any physical topology — physical ring, dual ring, bus ring, or star ring.

Q Build W_8 from W_4 using the generation rule. (Just describe the structure.)

W_8 is a 2×2 arrangement of 4×4 blocks: **top-left** = W_4 , **top-right** = W_4 , **bottom-left** = W_4 , **bottom-right** = **the complement of W_4** (every sign flipped). That gives 8 orthogonal sequences, each 8 chips long.

Q A network has 200 stations. What Walsh table size do you need, and how long is each chip sequence?

$N = 2^m$ must be at least 200. $2^7 = 128$ (too small), $2^8 = 256$ (enough). So **$m = 8$, $N = 256$** . Each chip sequence is **256 chips long**, and 56 sequences go unused.

Q In the four-station example, station 1 sends 1, station 2 is silent, station 3 sends 0, station 4 sends 1. Find the channel data, then decode station 3's bit.

Encode. $d_1 = +1$, $d_2 = 0$, $d_3 = -1$, $d_4 = +1$.

$$d_1 c_1 = [+1 \ +1 \ +1 \ +1]$$

$$d_2 c_2 = [0 \ 0 \ 0 \ 0]$$

$$d_3 c_3 = -1 \times [+1 \ +1 \ -1 \ -1] = [-1 \ -1 \ +1 \ +1]$$

$$d_4 c_4 = [+1 \ -1 \ -1 \ +1]$$

Sum column by column: **$D = [+1, -1, +1, +3]$**

Decode for station 3. Multiply by $c_3 = [+1 \ +1 \ -1 \ -1]$:

$[+1, -1, -1, -3]$, sum = -4, divide by $N = 4 \rightarrow -1 = \text{bit } 0 \checkmark$ correct.

Q What property of chip sequences makes CDMA decoding work, and what does “silent” contribute to the channel?

Orthogonality: the inner product of a chip sequence with itself is N , and with any other sequence is 0 . So multiplying the composite signal by one station's code preserves that station's data and cancels every other station's to exactly zero.

A **silent** station sends $0 \times$ its chip sequence — all zeros — so it contributes nothing at all and wastes no capacity.

MODULE 07

HDLC — High-Level Data Link Control

Everything from Module 5 — bit stuffing, sliding windows, piggybacking, Go-Back-N — assembled into one real, standardised protocol. HDLC is the ancestor of PPP, Frame Relay and X.25, so learning it once explains a whole family.

Lecture slides 221–246 · completes Chapter 4

7.1

What HDLC is

High-Level Data Link Control (HDLC) is a **bit-oriented synchronous data link layer protocol** developed by the **International Organization for Standardization (ISO)**. It is one of the most fundamental protocols in data communication and networking, and it forms the basis for many other network protocols.

Key features

Bit-oriented

HDLC treats data as a **continuous stream of bits** rather than as discrete characters. This lets it transmit **any** type of binary data efficiently — text, an image, a compiled program, anything.

Synchronous

Sender and receiver are **synchronised in time**: data is transmitted at a constant rate and both ends share the same clock signal.

Error detection

HDLC provides mechanisms for detecting and possibly correcting errors, using a **cyclic redundancy check (CRC)** to ensure data integrity.

BIT-ORIENTED MEANS BIT STUFFING

Because HDLC is bit-oriented and its flag is `01111110`, everything in §5.1 about bit stuffing applies directly — HDLC inserts a 0 after every five consecutive 1s in the data. Go back and re-read that section if it is hazy; it is the same mechanism, in a real protocol.

7.2

Frame structure

HDLC encapsulates data into **frames**, which are the basic units of communication. Each frame consists of five fields.

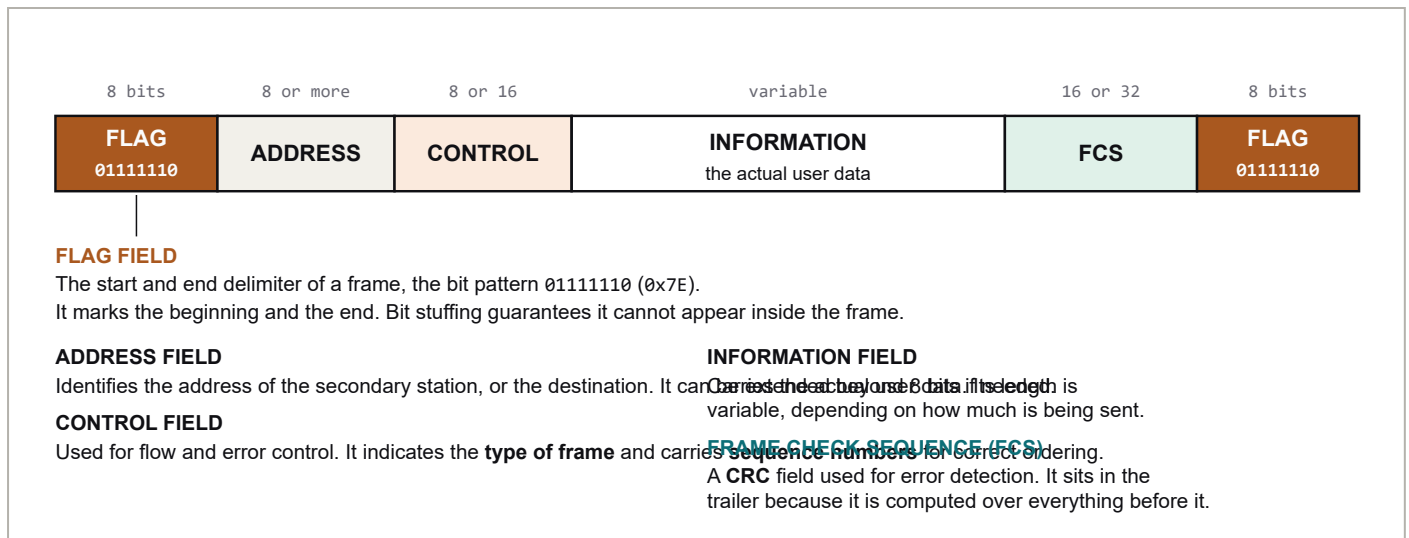
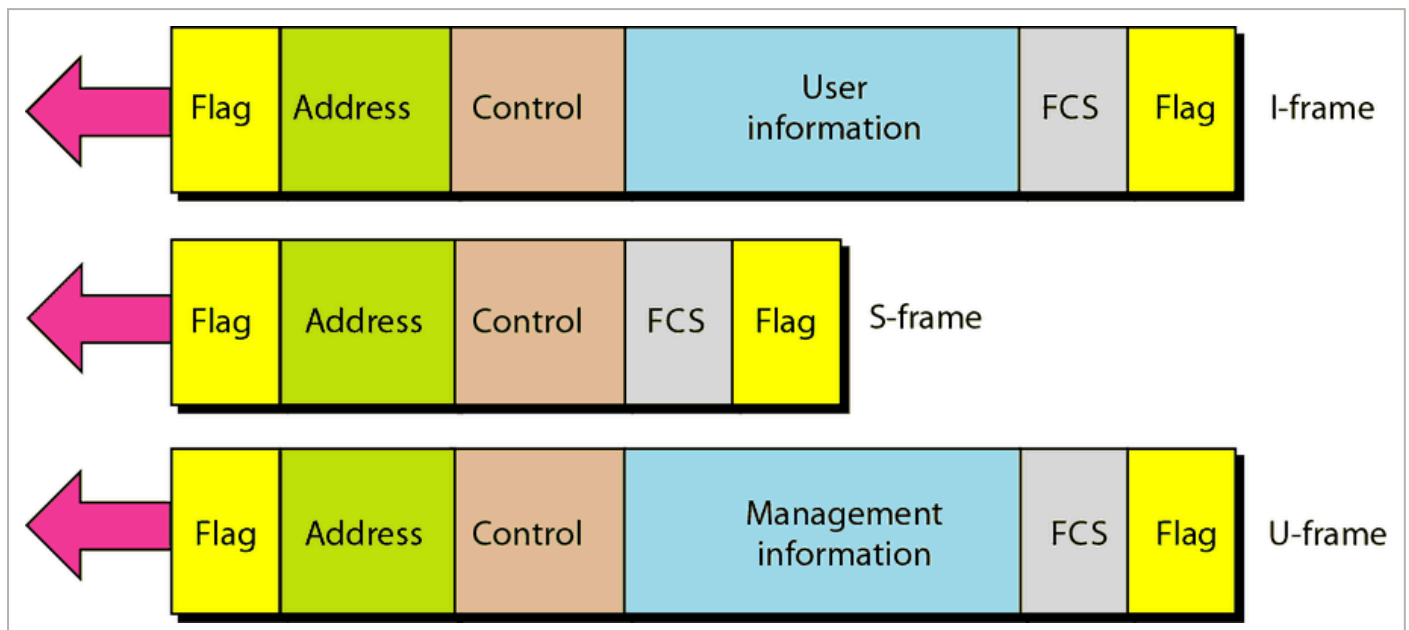


Fig 7.1 The HDLC frame. Note the two ends: the **flags are identical**, so a single flag can serve as the closing flag of one frame and the opening flag of the next when frames run back to back.



Slide 225 HDLC frames as drawn in the lecture deck.

7.3

The three frame types

HDLC uses **three types of frame**, each serving a different purpose. Which type a frame is, is decided by the **first one or two bits of the control field**.

THE THREE HDLC FRAME TYPES

TYPE	FULL NAME	CARRIES USER DATA?	PURPOSE
I-frame	Information frame	Yes	Transmits user data, and can carry both data and control information . Its control field includes a sequence number that helps ensure proper data sequencing and error correction.
S-frame	Supervisory frame	No	Controls the flow of data and handles error recovery. Carries control information only: acknowledgment of received frames, requests for retransmission, and flow control commands .
U-frame	Unnumbered frame	No	Used for network management tasks: establishing or terminating a connection, resetting the link, and exchanging control information — all without sequence numbering .

The control field formats

I-FRAME — the first bit is 0



N(S) = sequence number of the data *in this frame*

N(R) = the frame I expect *next* from you — the piggybacked ACK

One frame, two sequence numbers — that is piggybacking.

S-FRAME — the first two bits are 1 0



No **N(S)**, because an S-frame carries no data of its own — only an acknowledgment or a flow-control instruction.

U-FRAME — the first two bits are 1 1



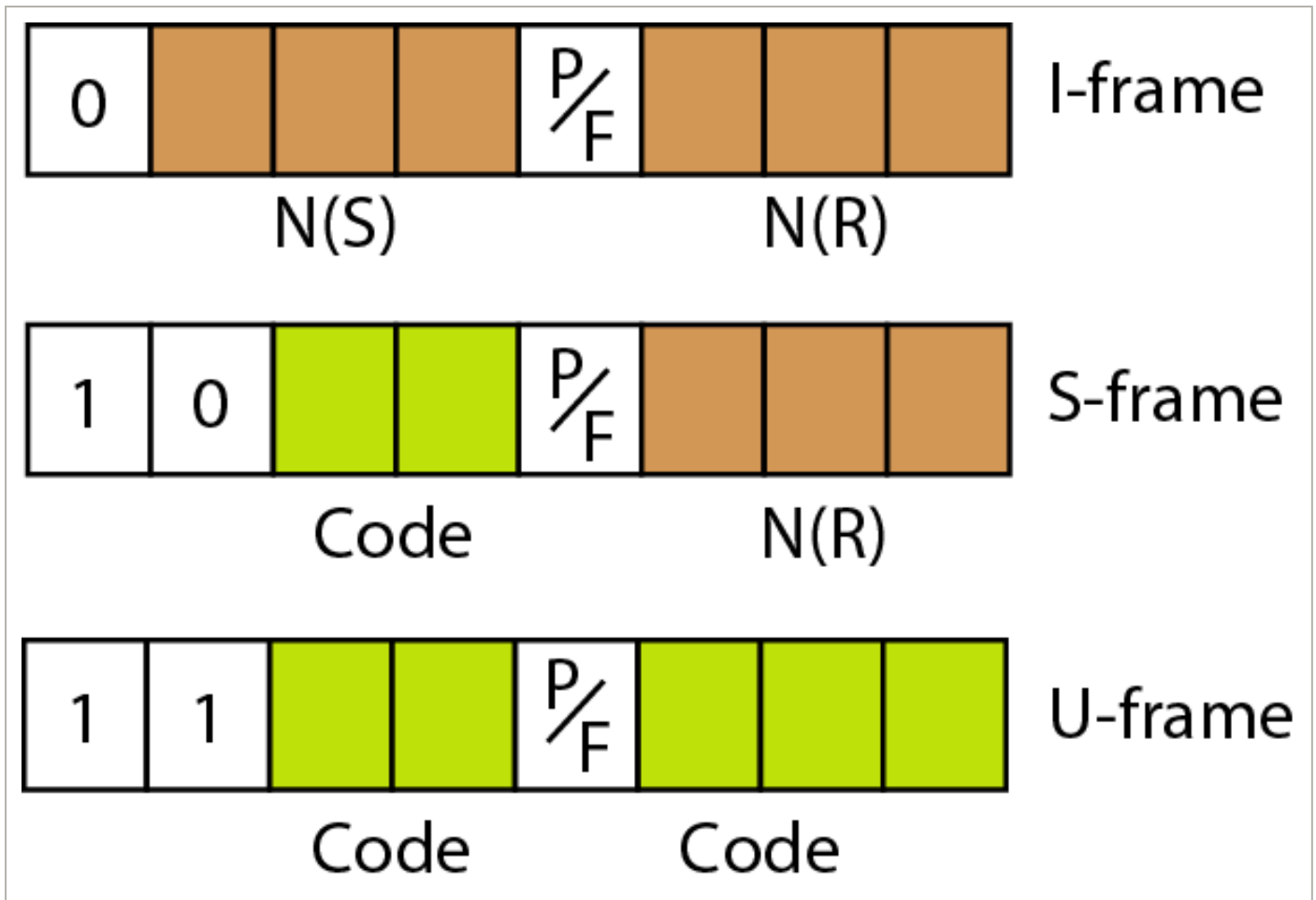
No sequence numbers at all — hence *unnumbered*.

The 2-bit and 3-bit code fields join to make a **5-bit** command.

The P/F bit — Poll / Final

One single bit with two meanings depending on who sets it. Set by the **primary** it means **P (poll)**: “you may now send.” Set by the **secondary** it means **F (final)**: “I have finished sending.”

Fig 7.2 The three control-field formats. Reading the leading bits is how a receiver identifies the frame type: **0** → I-frame, **10** → S-frame, **11** → U-frame. Only the I-frame has an **N(S)**, because only the I-frame carries data.



Slide 238 Control field format for the three frame types.

S-frame codes — the four supervisory functions

THE 2-BIT S-FRAME CODE FIELD			
CODE	NAME	MEANING	USED BY
00	RR — Receive Ready	A plain ACK . “I am ready to receive; frames up to $N(R)-1$ are accepted.”	Both protocols
01	REJ — Reject	A NAK . “Resend frame $N(R)$ and everything after it.”	Go-Back-N
10	RNR — Receive Not Ready	An ACK <i>plus</i> a flow-control instruction: “I got your frames, but stop sending for now — my buffer is full.”	Both protocols
11	SREJ — Selective Reject	A NAK for one specific frame only. “Resend just frame $N(R)$.”	Selective Repeat

MODULE 5 SHOWS UP HERE

REJ is the NAK of Go-Back-N — it asks for a frame *and everything after it*. **SREJ** is the NAK of Selective Repeat — it asks for *one* frame. The two codes exist because HDLC supports both protocols, and this is the field where that choice becomes visible.

U-frame codes — the management commands

U-FRAME CONTROL COMMANDS AND RESPONSES			
CODE	COMMAND	RESPONSE	MEANING
00 001	SNRM	—	Set normal response mode
11 011	SNRME	—	Set normal response mode, extended
11 100	SABM	DM	Set asynchronous balanced mode / disconnect mode
11 110	SABME	—	Set asynchronous balanced mode, extended
00 000	UI	UI	Unnumbered information
00 110	—	UA	Unnumbered acknowledgment
00 010	DISC	RD	Disconnect / request disconnect
10 000	SIM	RIM	Set initialization mode / request information mode
00 100	UP	—	Unnumbered poll
11 001	RSET	—	Reset
11 101	XID	XID	Exchange ID
10 001	FRMR	FRMR	Frame reject

THE FOUR YOU MUST KNOW

Do not try to memorise all twelve. The four that appear in every exam question are: **SABM** (please open a connection), **UA** (yes, connection open / acknowledged), **DISC** (please close the connection), and **DM** (I am in disconnect mode). Those four run the whole connection life cycle.

7.4**Operation modes**

HDLC supports different operational modes, depending on the nature of the communication. Three exist; two are used in practice.

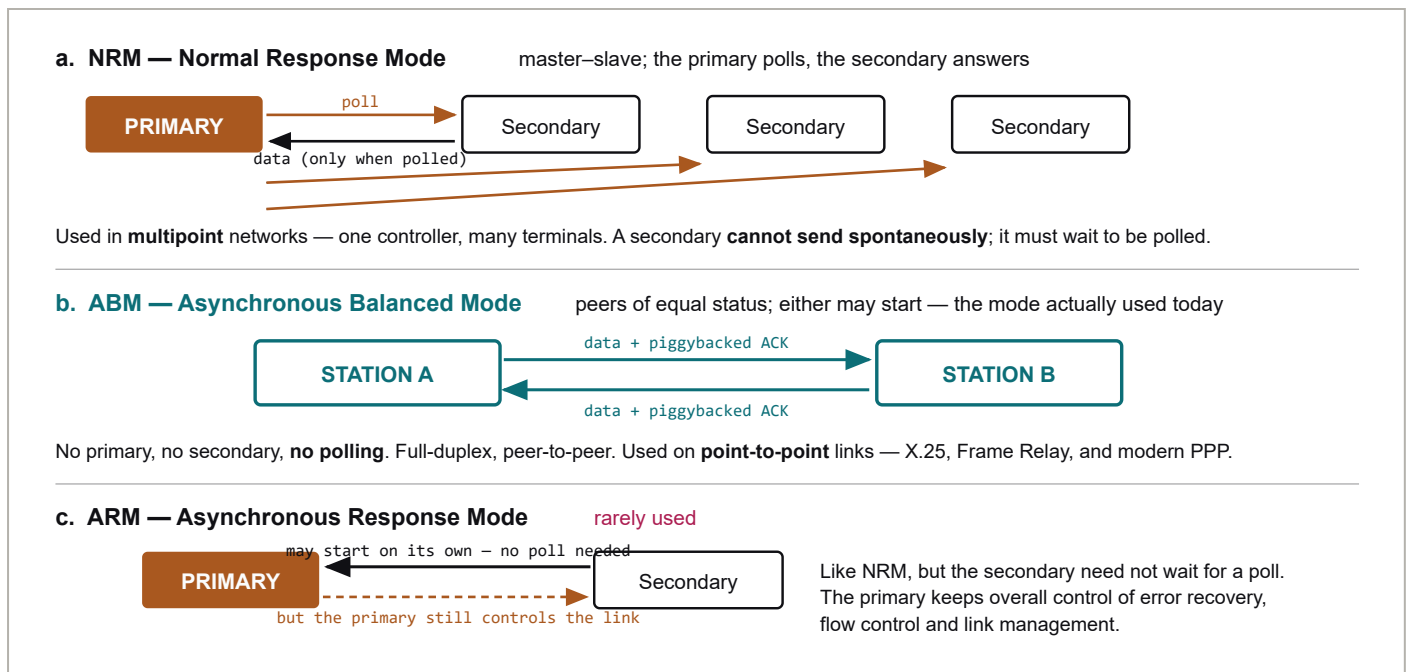


Fig 7.3 The three modes, drawn by *who is allowed to start talking*. NRM: only the primary. ABM: either peer. ARM: the secondary may start, but the primary still owns the link. That one question separates all three.

Normal Response Mode (NRM)

NRM is used where a central device (the **primary station**) controls communication with other devices (the **secondary stations**). Communication is unidirectional from primary to secondary, and the secondary can send data only when explicitly allowed — **polled** — by the primary.

Characteristics:

- **Master–slave relationship.** The primary is the master, the secondaries are slaves. The primary initiates all communication.
- **Polling mechanism.** The secondary cannot send data spontaneously. It must wait for the primary to poll or request data; when polled, it responds by sending data.
- **Flow control.** Since the primary controls all communication, it can manage the flow of data and ensure orderly transmission.
- **Error control.** The primary is responsible for handling any errors and retransmitting data if needed.

Use case: **multipoint networks** where multiple secondary devices are connected to a single primary. For example, legacy IBM mainframe networks (Synchronous Data Link Control, or SDLC) often used NRM for managing communication with terminals.

Advantages: simplified communication for centralised systems; the primary has full control over the network.

Disadvantage: secondary stations are passive and must wait for permission to send, which can lead to inefficient use of network resources.

Asynchronous Balanced Mode (ABM)

ABM is the **most flexible and widely used mode** of HDLC. Both devices on the link have **equal status**, meaning either can initiate communication. ABM allows **full-duplex, peer-to-peer** communication with no master–slave relationship.

Characteristics:

- **Balanced relationship.** Both stations (peers) can initiate data transmission independently, with **no polling mechanism** at all.
- **Full-duplex.** Both stations can send and receive simultaneously, which is more efficient.
- **Error and flow control.** *Both* stations are responsible for error detection, acknowledgment and retransmission. This makes the link more robust and decentralised.
- **Simple peer communication.** With no primary/secondary distinction, peer-to-peer communication is simpler, which suits modern bidirectional links.

Use case: point-to-point links where two devices exchange data without a central controller. Widely used in WAN protocols like **X.25**, **Frame Relay**, and modern **PPP (Point-to-Point Protocol)**, all of which evolved from HDLC.

Advantages: flexibility — any station can initiate communication, giving higher efficiency; supports full-duplex, maximising link utilisation.

Disadvantage: requires more complex error and flow control than NRM or ARM, because both stations are involved in managing the link.

Asynchronous Response Mode (ARM)

ARM also follows a master–slave model, similar to NRM, but with one key difference: the **secondary stations can initiate communication with the primary without being explicitly polled**. The primary still has control over the link and can manage data flow.

Characteristics:

- **Master–slave relationship.** Like NRM, the primary has overall control, but secondaries are allowed more flexibility in initiating communication.
- **Unsolicited responses.** Secondaries do not need to wait for a poll to send data; they can start whenever they need to, though the primary can still manage or restrict their responses.
- **Control of communication.** Although secondaries can initiate, the primary remains responsible for overall control — error recovery, flow control and link management.

Use case: legacy networks. ARM was not widely adopted, but it can be used where secondaries occasionally need to send data without waiting for permission, while the network design still requires a central controller.

Advantage: allows secondaries to initiate, increasing efficiency compared with NRM.

Disadvantage: rarely used in modern networks, and can be less efficient than ABM because of the continued master–slave relationship.

THE THREE MODES COMPARED			
FEATURE	NRM	ABM	ARM
Relationship	Master–slave	Balanced peers	Master–slave
Who may initiate	Primary only	Either station	Either, but primary controls
Polling needed?	Yes	No	No
Duplex	Effectively half-duplex flow	Full-duplex	Full-duplex possible
Who does error control	Primary	Both stations	Primary
Typical topology	Multipoint	Point-to-point	Multipoint
Used in	SDLC, IBM mainframe terminals	X.25, Frame Relay, PPP	Legacy only — rarely used
U-frame to enter it	SNRM	SABM	SARM

7.5

Worked exchanges

Connection establishment and release

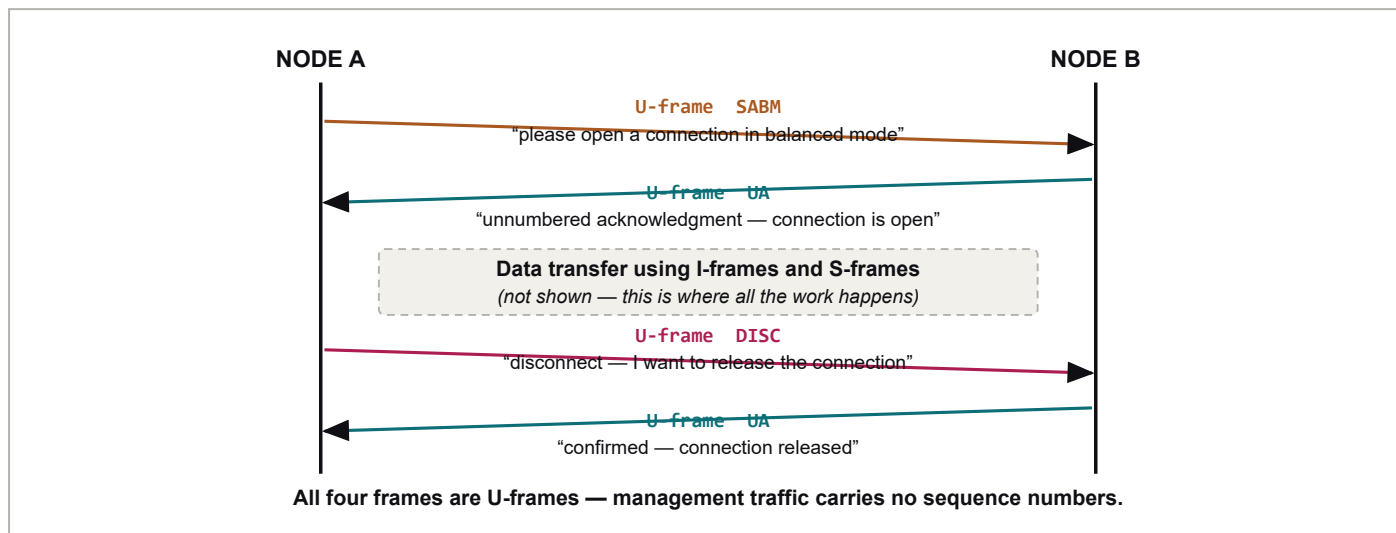


Fig 7.4 The connection life cycle. Node A asks for a connection with a **SABM** frame; node B gives a positive response with a **UA** frame. After these two exchanges, data can be transferred. After data transfer, node A sends a **DISC** frame to release the connection; it is confirmed by node B responding with a **UA**.

Piggybacking, with and without an error

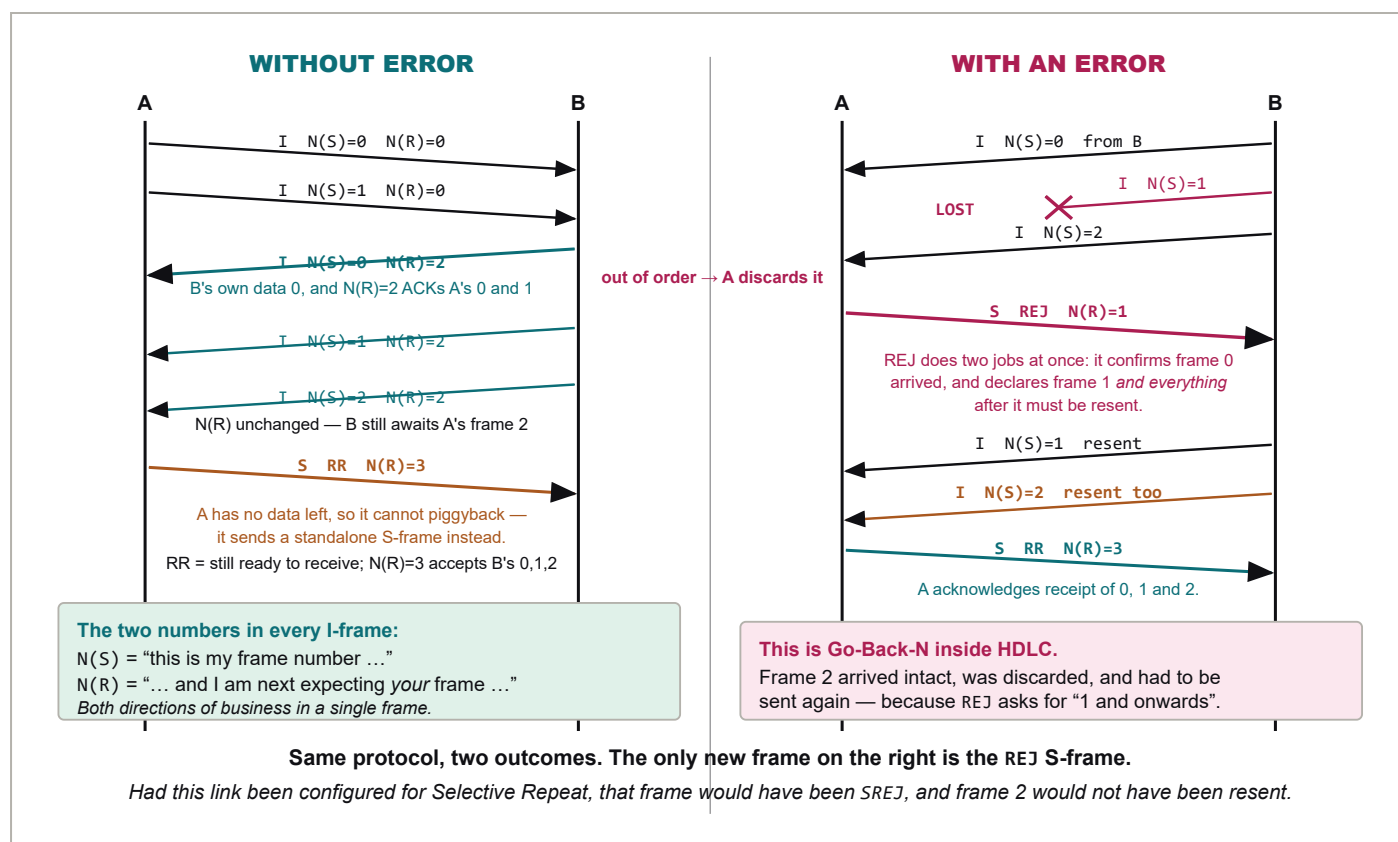


Fig 7.5 Piggybacking in HDLC, error-free and with a lost frame. Trace the N(R) values on the left: B sends three I-frames all carrying N(R) = 2, because its expectation of A never changes during that run.

WORKED EXCHANGE – PIGGYBACKING WITHOUT ERROR, IN WORDS

Follow the left-hand side of Fig 7.5 step by step.

1. Node A begins the exchange of information with an I-frame numbered 0, followed by another I-frame numbered 1.
2. Node B piggybacks its acknowledgment of both frames onto an I-frame of its own. Node B's first I-frame is also numbered 0 in its N(S) field, and contains a 2 in its N(R) field — acknowledging receipt of A's frames 0 and 1, and indicating that it expects frame 2 to arrive next.
3. Node B transmits its second and third I-frames (numbered 1 and 2) before accepting further frames from node A. Its N(R) information has not changed: B's frames 1 and 2 both indicate that node B is still expecting A's frame 2 to arrive next.
4. Node A has now sent all its data. Therefore it cannot piggyback an acknowledgment onto an I-frame, and sends an S-frame instead. The RR code indicates that A is still ready to receive. The number 3 in the N(R) field tells B that frames 0, 1 and 2 have all been accepted, and that A is now expecting frame number 3.

The key mechanism: an I-frame's N(R) is "free" acknowledgment space. When you have no data to send, that space is not available — so you must fall back to a separate S-frame (RR).

WORKED EXCHANGE – PIGGYBACKING WITH AN ERROR, IN WORDS

Follow the right-hand side of Fig 7.5 step by step.

1. Node B sends three data frames (0, 1 and 2), but **frame 1 is lost**.
2. When node A receives frame 2, it **discards it** (out of order) and sends a **REJ frame for frame 1**.
3. Note that the protocol being used is **Go-Back-N**, with the special use of a **REJ frame as a NAK frame**. The NAK frame does two things here: it **confirms the receipt of frame 0**, and it **declares that frame 1 and any following frames must be resent**.
4. Node B, after receiving the REJ frame, **resends frames 1 and 2**.
5. Node A acknowledges the receipt by sending an **RR frame (ACK) with acknowledgment number 3**.

Frames retransmitted: 2 (frames 1 and 2), for 1 lost frame. Classic Go-Back-N waste – frame 2 was fine and was thrown away.

Note the double duty of REJ: a single S-frame both acknowledges everything up to $N(R)-1$ and requests $N(R)$ onward.

7.6

Error handling, applications, and a note on wireless

Error handling in HDLC

HDLC employs two mechanisms for error detection and correction:

- **Cyclic Redundancy Check (CRC).** Each frame includes a CRC code in the **FCS** field, calculated from the frame's content. On receiving the frame, the receiver **recalculates the CRC and compares it with the one in the FCS field**. If the two match, the frame is considered error-free.
- **Acknowledgments and retransmissions.** HDLC uses acknowledgments (in **S-frames**) to confirm successful receipt of frames. If a frame is not acknowledged, it is assumed to be lost or corrupted, and it is retransmitted.

Applications

HDLC is widely used in communication systems:

- **WAN protocols.** HDLC is used in point-to-point communication links in WAN protocols like **X.25, Frame Relay, and ISDN**.
- **PPP (Point-to-Point Protocol).** PPP, used in many internet connections, is **based on HDLC**. However, PPP includes additional features like **authentication** and **network layer protocol multiplexing**.
- **Embedded systems.** HDLC is used for communication between devices in embedded systems, because of its robustness and simplicity.

NOTE ON WIRELESS

HDLC is **not limited to wired communication** — it can also be used in wireless environments. It was initially designed for point-to-point and multipoint connections over synchronous serial links, which are often wired (leased lines, X.25 networks, Frame Relay), but its principles and mechanisms have been **adapted for use in various communication systems, including wireless networks.**

Module 7 at a glance

- **HDLC** is a bit-oriented protocol for point-to-point and multipoint links, and the ancestor of PPP and Frame Relay.
- Frame: **Flag · Address · Control · Information · FCS · Flag**, with 01111110 as the flag and bit stuffing inside.
- Three frame types, read from the first bits of the control field: **I-frames** carry data, **S-frames** carry acknowledgements and flow control, **U-frames** manage the link.
- Three modes: **NRM** (one primary, several secondaries), **ABM** (two equal stations, the common one), and ARM.

CHECK YOURSELF — MODULE 7

Q Who developed HDLC, and what does “bit-oriented synchronous” mean?

Developed by the **ISO**. **Bit-oriented** means it treats data as a continuous stream of bits rather than discrete characters, so it can carry any binary data. **Synchronous** means sender and receiver are synchronised in time, transmitting at a constant rate with a shared clock signal.

Q Name the five fields of an HDLC frame in order, and say what the FCS holds.

Flag, Address, Control, Information, FCS, Flag. The **FCS** (Frame Check Sequence) holds a **CRC** used for error detection, computed over the rest of the frame.

Q What is the HDLC flag pattern, and what mechanism stops it appearing inside the data?

The flag is 01111110 (hex 0x7E). **Bit stuffing** stops it appearing inside the frame: a 0 is inserted after every five consecutive 1s, so a run of six 1s can never occur in the data.

Q How does a receiver tell an I-frame from an S-frame from a U-frame?

By the leading bits of the **control field**: **0** → I-frame, **10** → S-frame, **11** → U-frame.

Q Which frame type has an N(S) field, and why do the other two not?

Only the **I-frame** has N(S), because only the I-frame **carries user data** that needs a sequence number. S-frames carry only control information, and U-frames are *unnumbered* by definition.

Q In an I-frame, what do N(S) and N(R) mean? What is this arrangement called?

N(S) = the sequence number of the data in *this* frame. **N(R)** = the frame this station expects to receive *next* from the other side — that is, an acknowledgment. Carrying both in one frame is **piggybacking**.

Q Give the four S-frame codes with their meanings, and say which one belongs to Go-Back-N and which to Selective Repeat.

00 RR (Receive Ready) — ACK. **01 REJ** (Reject) — NAK asking for N(R) and onward: **Go-Back-N**. **10 RNR** (Receive Not Ready) — ACK plus “stop sending, buffer full”. **11 SREJ** (Selective Reject) — NAK for one frame only: **Selective Repeat**.

Q What does the P/F bit mean, and how does its meaning depend on who set it?

It is the **Poll/Final** bit. Set by the **primary** it is **P (poll)**: “you may now send.” Set by the **secondary** it is **F (final)**: “this is my last frame.”

Q Name the three HDLC operation modes. Which is most widely used, and which is rarely used?

NRM (Normal Response Mode), **ABM** (Asynchronous Balanced Mode), **ARM** (Asynchronous Response Mode). **ABM is the most flexible and widely used; ARM is rarely used.**

Q What is the single difference between NRM and ARM?

In **NRM** a secondary can send only when **polled** by the primary. In **ARM** the secondary **can initiate communication without being polled** — though the primary still keeps overall control of the link, error recovery and flow control.

Q Why does ABM need more complex error and flow control than NRM?

Because in ABM **both stations** are responsible for error detection, acknowledgment and retransmission — control is decentralised. In NRM the primary alone handles all of it, so each secondary can be simple.

Q Write the four-frame sequence to open and then close an HDLC connection in balanced mode.

SABM ($A \rightarrow B$: open a connection in balanced mode) $\rightarrow$ **UA** ($B \rightarrow A$: acknowledged, open) $\rightarrow$ *data transfer* $\rightarrow$ **DISC** ($A \rightarrow B$: disconnect) $\rightarrow$ **UA** ($B \rightarrow A$: confirmed). All four are **U-frames**.

Q A station has received data but has nothing of its own to send. How does it acknowledge?

It cannot piggyback, because piggybacking needs an I-frame to ride on. So it sends a **standalone S-frame with the RR code** and the appropriate $N(R)$ — meaning “still ready to receive, and I have accepted everything up to $N(R)-1$.”

Q Node B sends frames 0, 1, 2 and frame 1 is lost. What does A send, and how many frames does B resend?

A discards frame 2 as out of order and sends **REJ with $N(R) = 1$** . That single frame both confirms frame 0 and demands frame 1 onward. B therefore resends **two frames — 1 and 2**. A then sends **RR with $N(R) = 3$** .

Had SREJ been used instead, only frame 1 would have been resent.

Q Name three protocols built on or derived from HDLC.

X.25, Frame Relay, ISDN use HDLC on their point-to-point links, and **PPP** is based on HDLC — PPP adds **authentication and network-layer protocol multiplexing**. HDLC is also used in **embedded systems**.

MODULE 08

Connecting Devices and MAC Addressing

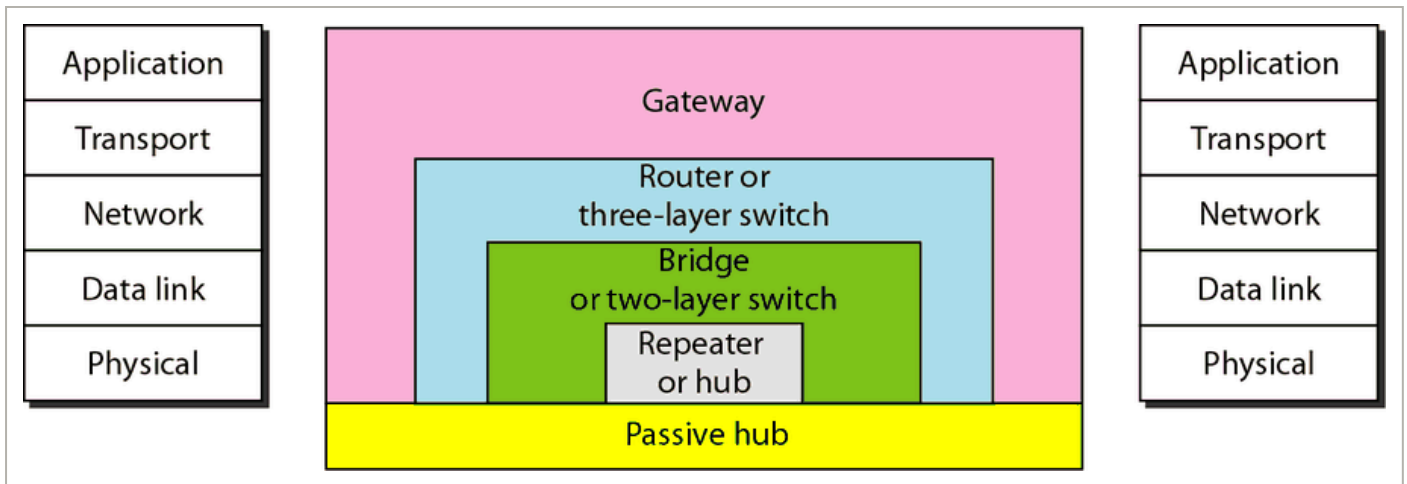
Repeaters, hubs, bridges, switches, routers and gateways — six devices that all “join networks together”, but at six different layers, which is what actually separates them. Then the learning bridge, the loop problem it creates, and how MAC addresses are classified.

Lecture slides 247–292 · Chapters 5 and 6 · mid-sem topics 14, 15, 16

8.1

Five categories of connecting device

Connecting devices divide into five categories **based on the layer at which they operate**. This single fact decides everything else about a device — what it can read, what it can filter, and what it can decide.



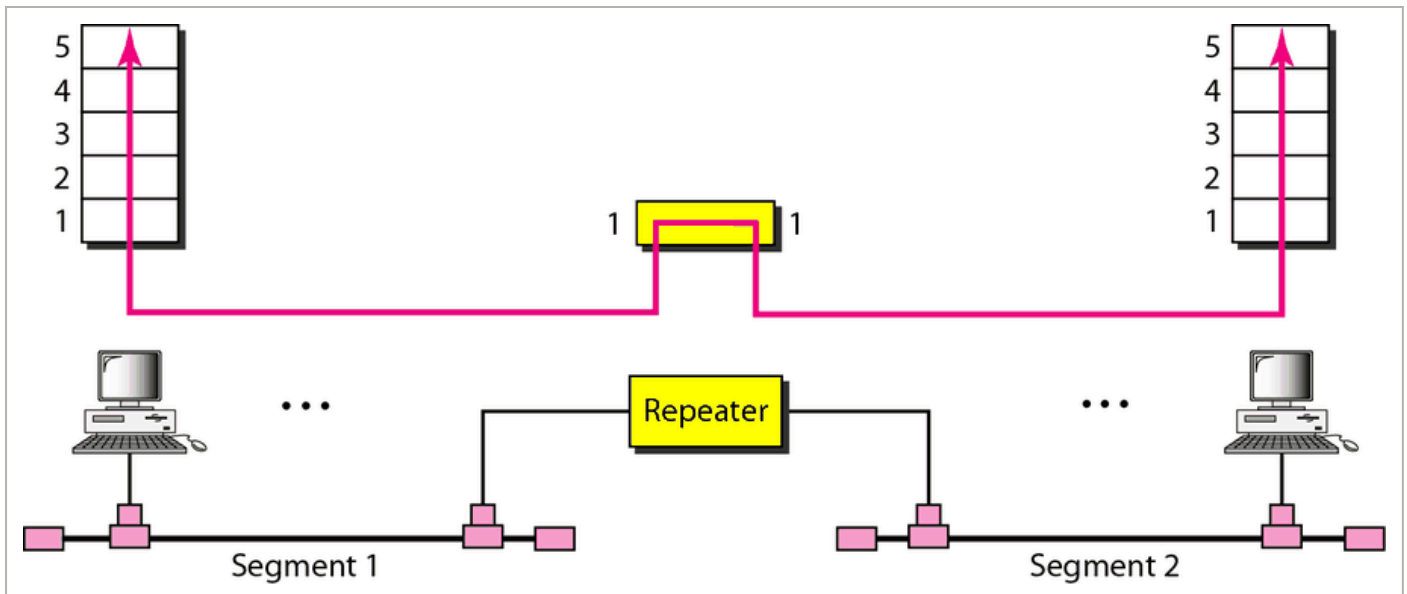
PPT Fig 5.1 The five categories, drawn against the protocol stack. Read the height of each box: the taller it reaches, the more of the frame that device can open and understand — and the more expensive and slower it is.

WHAT EACH DEVICE CAN SEE			
DEVICE	LAYER	WORKS ON	CAN IT FILTER?
Passive hub	—	Just a connector; a junction point for wires	No
Repeater / active hub	1 — Physical	Bits, as electrical signals	No — forwards everything
Bridge / two-layer switch	2 — Data link	Frames and MAC addresses	Yes — by MAC address
Router / three-layer switch	3 — Network	Packets and IP addresses	Yes — by IP address, and it routes
Gateway	1-5 — all	Everything, up to the application	Yes — it can even translate between protocols

8.2

Repeaters and hubs

A **repeater** is a device that operates **only in the physical layer**. Signals carrying information can travel a fixed distance before attenuation endangers the integrity of the data. A repeater is installed on a link before the signal becomes too weak or corrupted.



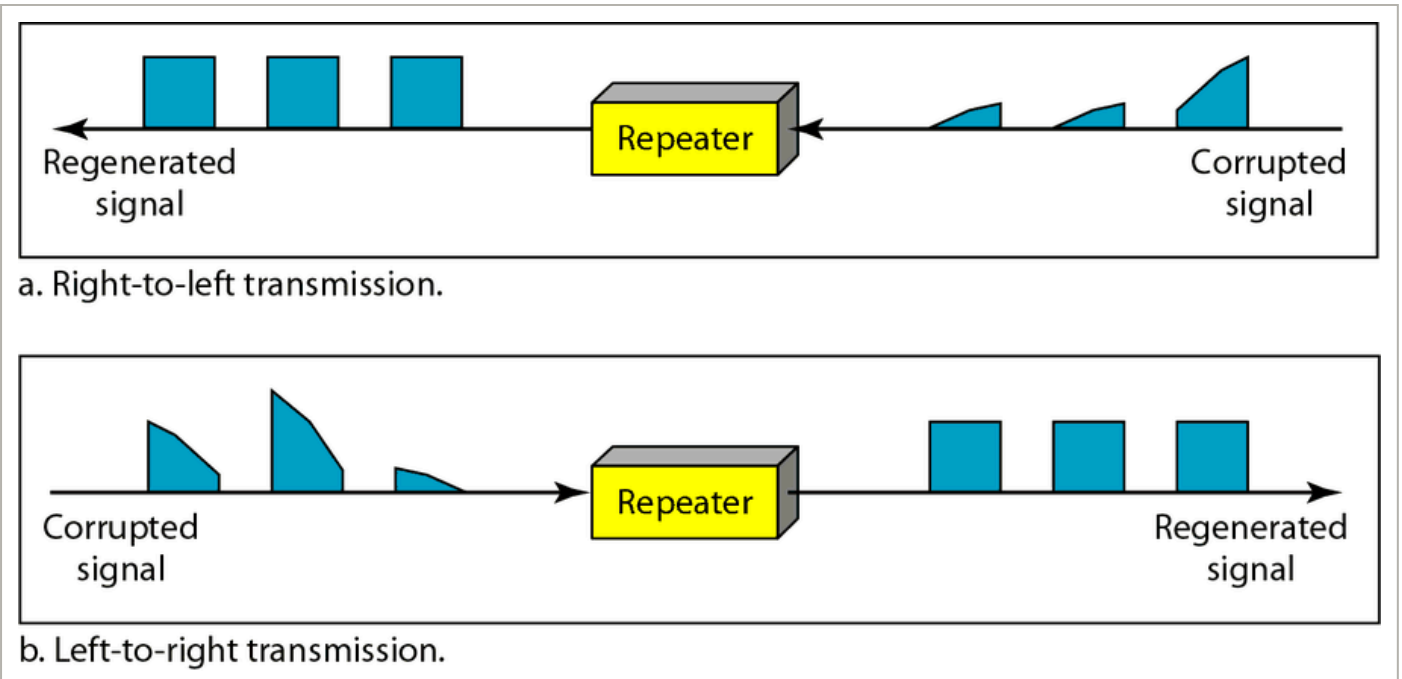
PPT Fig 5.2 A repeater connecting two segments of a LAN. It extends the physical reach of the network without adding any intelligence.

THREE FACTS ABOUT REPEATERS — ALL EXAMINABLE

- A repeater **connects segments** of a LAN. It does not connect two different LANs.
- A repeater **forwards every frame**; it has **no filtering capability**.
- A repeater is a **regenerator, not an amplifier**.

Regenerator, not amplifier — why the distinction matters

An **amplifier** would make the incoming signal bigger — and it would make the *noise* bigger along with it, because it cannot tell them apart. A **regenerator** reads the weakened signal, decides for each bit whether it was a 0 or a 1, and then **creates a brand-new, clean copy** at full strength. The noise is discarded entirely.



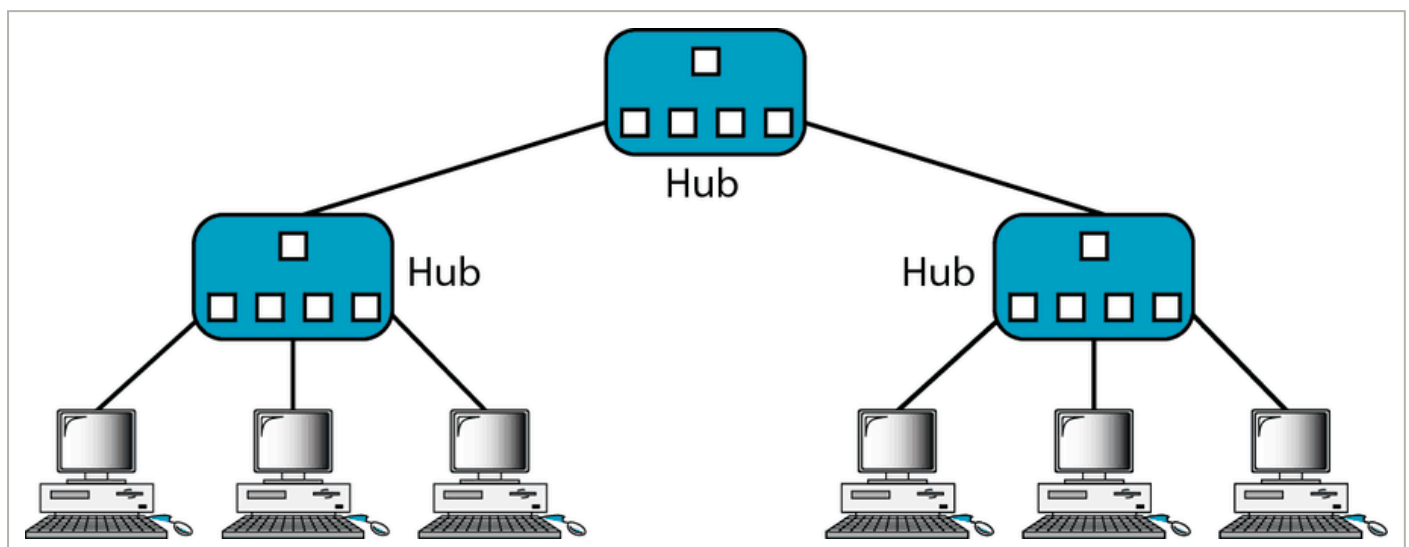
PPT Fig 5.3 The function of a repeater. Compare the two waveforms — the outgoing signal is not a louder version of the incoming one, it is a *fresh* one, rebuilt from the bits that were read.

PLAIN WORDS

An amplifier is like shouting a whisper you half-heard — louder, but still garbled. A regenerator is like *understanding* the whisper and then saying the sentence again clearly. That is why repeaters can be chained without noise piling up.

Hubs

A **hub** is essentially a **multiport repeater**. It is normally used to create connections between stations in a physical star topology, and hubs can be arranged in a hierarchy.



PPT Fig 5.4 A hierarchy of hubs. Because a hub repeats every signal out of every port, all these stations remain in **one collision domain** — adding hubs extends the network but does not reduce contention.

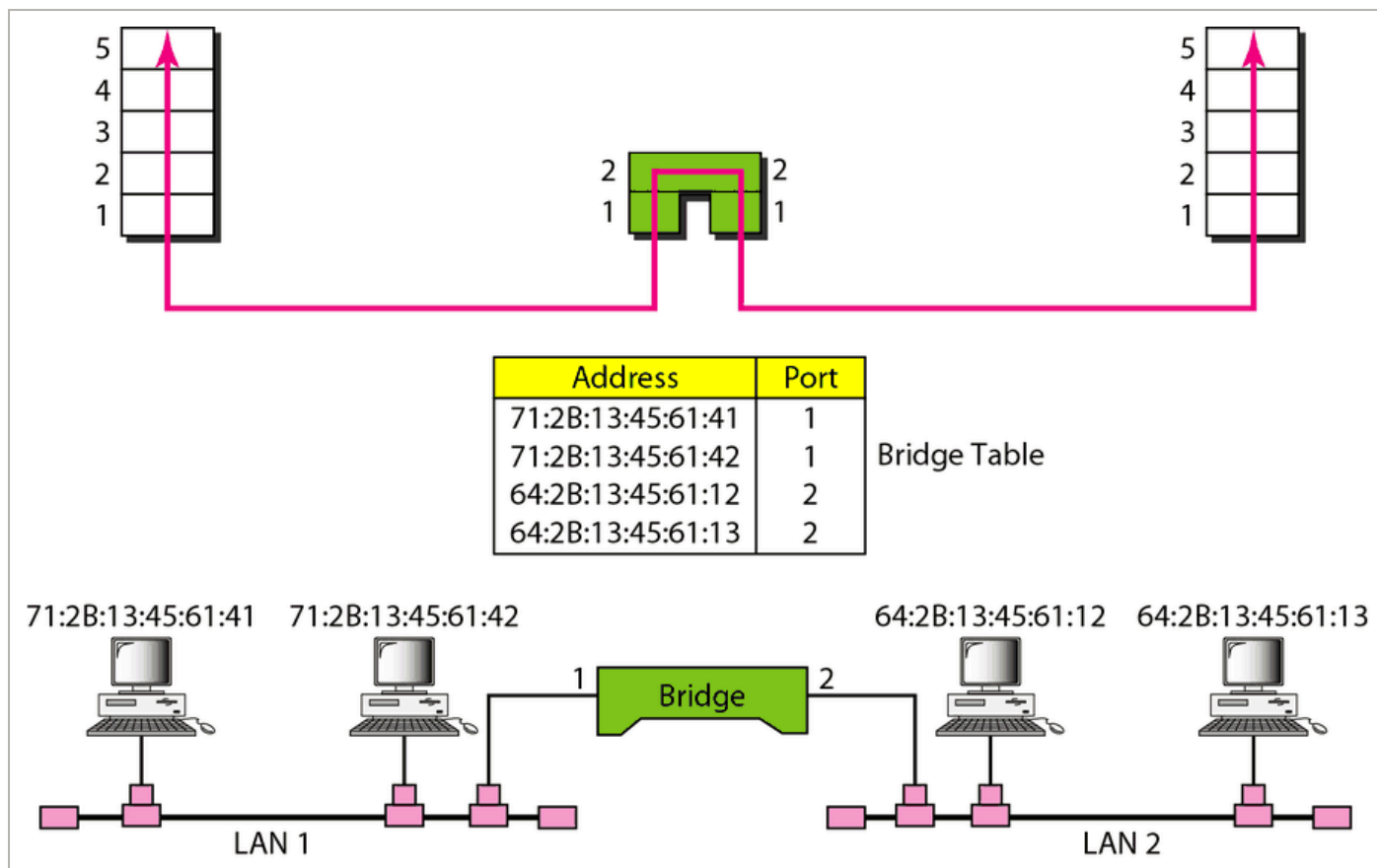
8.3

Bridges

A **bridge** operates in **both the physical and the data link layer**. As a physical-layer device it regenerates the signal it receives. As a data-link-layer device, it can **check the physical (MAC) addresses** contained in the frame.

TWO KEY FACTS

- A bridge has a **table used in filtering decisions**.
- A bridge **does not change the physical (MAC) addresses** in a frame.



PPT Fig 5.5 A bridge connecting two LANs. The table is the whole point: it lets the bridge answer “is the destination on the same side as the sender?” — and if so, **drop the frame** instead of forwarding it, keeping local traffic local.

This filtering is what makes a bridge better than a repeater. A repeater blindly copies everything to the other side; a bridge only forwards a frame when the destination is genuinely on the other side. That splits the network into **separate collision domains** and cuts unnecessary traffic.

8.4

The learning bridge

Early bridges had their tables typed in by hand by a network administrator — tedious, and wrong the moment a machine moved. A **learning bridge** (or transparent bridge) builds its own table **automatically, by watching traffic**.

For every frame that arrives:

1. Look at the frame's **SOURCE** address, and the **port it arrived on**.

Record that pair in the table. (This is how it learns.)

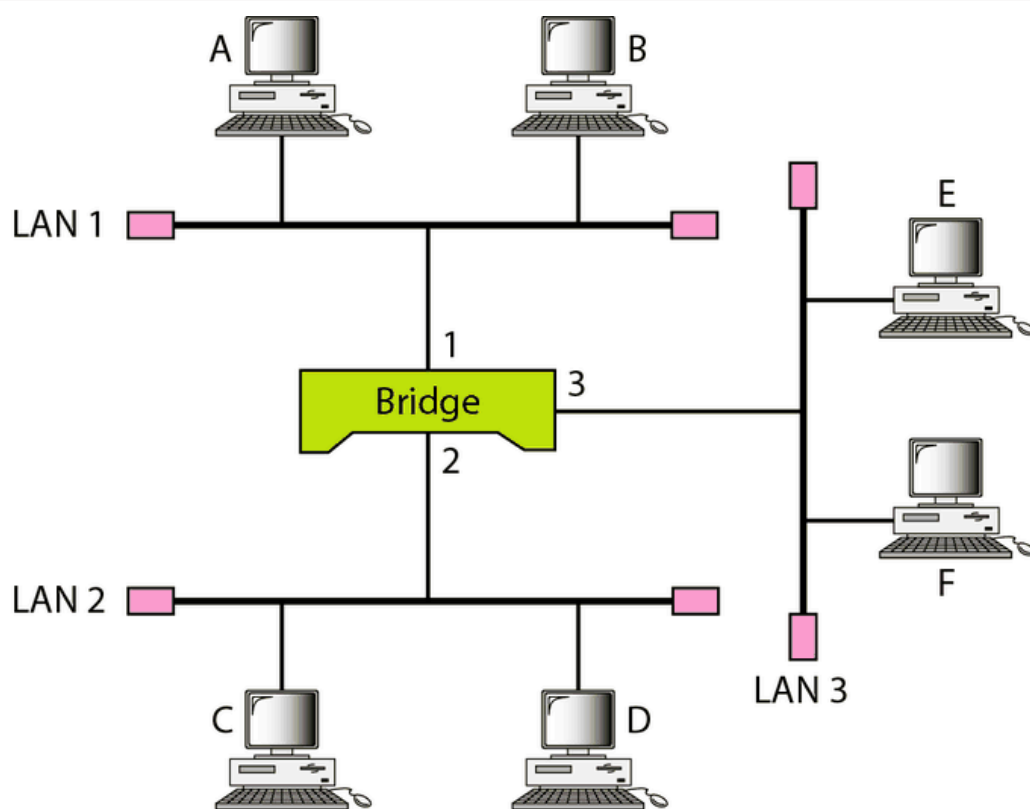
2. Look at the frame's **DESTINATION** address.

- In the table? → forward out that one port only

(or drop it, if that port is the one it came in on)

- Not in table? → **flood** it out of every port except the one it arrived on

The bridge learns from the **source** address but makes its decision on the **destination** address. Mixing those two up is the classic exam mistake.



Address	Port

a. Original

Address	Port
A	1

b. After A sends a frame to D

Address	Port
A	1
E	3

c. After E sends a frame to A

Address	Port
A	1
E	3
B	1

d. After B sends a frame to C

PPT Fig 5.6 A learning bridge and the process of learning. Follow the four tables left to right: the bridge starts knowing nothing and learns exactly one new entry per newly-heard sender.

WALKTHROUGH – HOW THE TABLE ABOVE FILLS UP

The bridge has three ports: port 1 to LAN 1 (stations A, B), port 2 to LAN 2 (stations C, D), port 3 to LAN 3 (stations E, F). Trace the four stages.

- a. **Original.** The table is **empty**. The bridge knows nothing, so every frame it sees will have to be flooded.
- b. **A sends a frame to D.** The frame arrives on port 1. The bridge records the *source*: **A is on port 1**. The destination D is not in the table, so the frame is **flooded** out of ports 2 and 3.
- c. **E sends a frame to A.** The frame arrives on port 3, so the bridge records **E is on port 3**. This time the destination A *is* known — port 1 — so the frame is sent out of **port 1 only**. Nothing is wasted on LAN 2.
- d. **B sends a frame to C.** Arrives on port 1, so the bridge records **B is on port 1**. Destination C is still unknown, so this frame is flooded again.

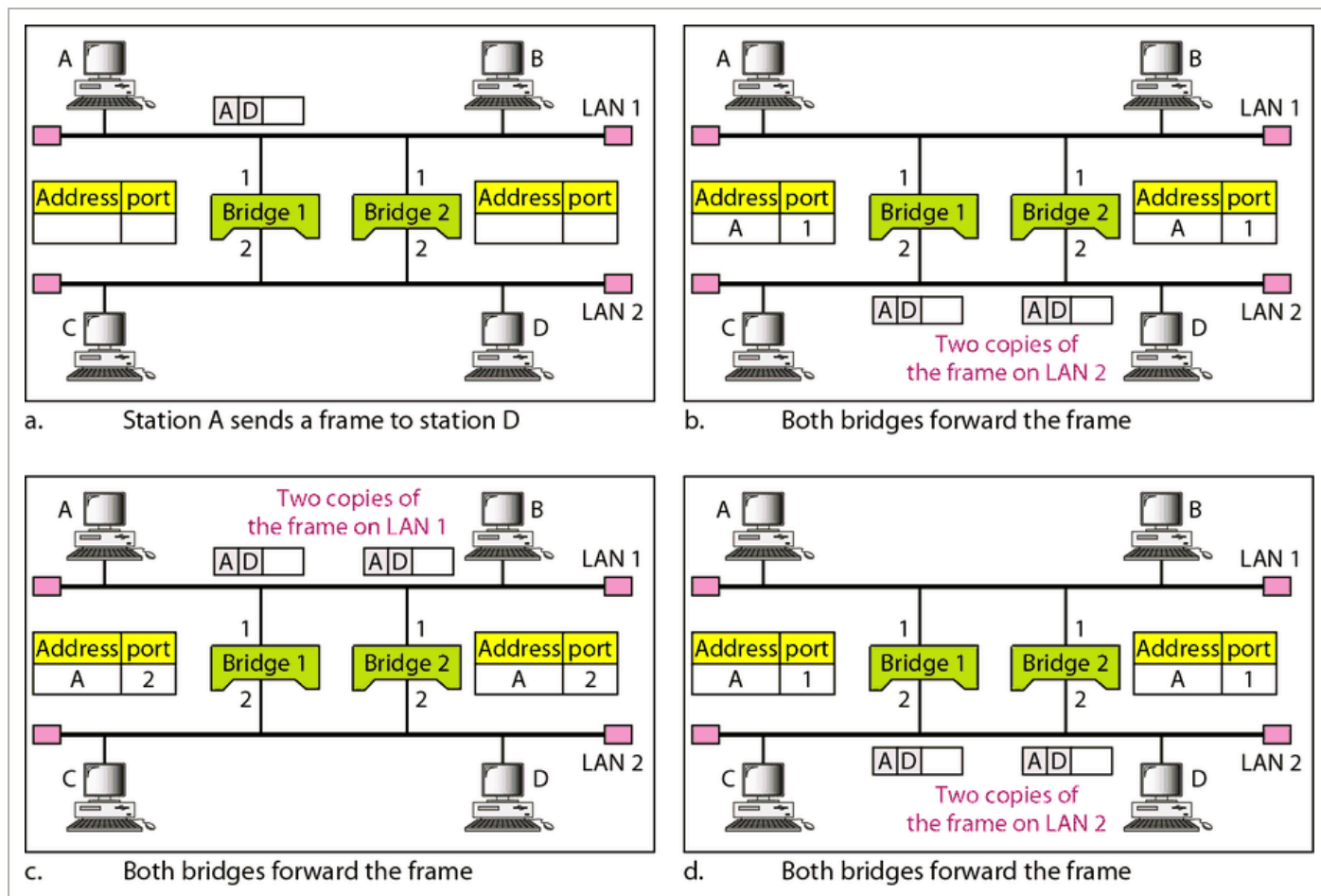
Final table: A → 1, E → 3, B → 1

Notice the bridge has still never heard from C, D or F, so it has no entry for them — it only learns about a station when that station **sends** something.

8.5

The loop problem, and the spanning tree solution

Administrators often install **two bridges between the same pair of LANs**, so that the network keeps working if one bridge fails. That redundancy is sensible — and it creates a serious problem, because learning bridges flood frames they do not recognise.



PPT Fig 5.7 The loop problem in a learning bridge. The frame is never consumed and never expires — each pass multiplies the copies, and each pass also *corrupts the address tables*, because A keeps appearing to arrive on a different port.

WALKTHROUGH — HOW THE LOOP BUILDS

Two LANs are joined by Bridge 1 and Bridge 2 in parallel. Station A on LAN 1 sends a frame to station D, whose address neither bridge knows yet.

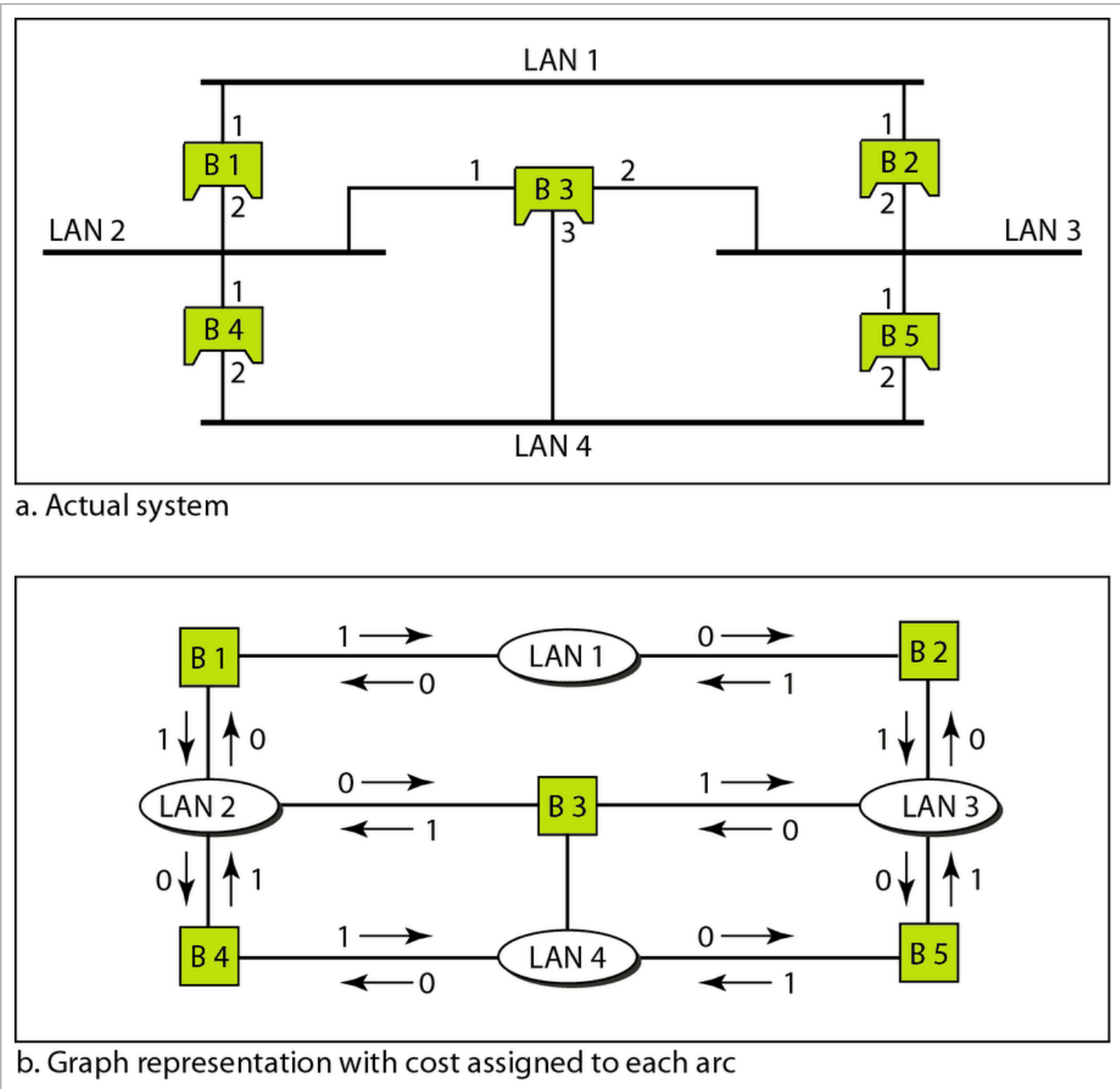
- Station A sends the frame onto LAN 1. Both bridges hear it. Both record **A on port 1**.
- Neither bridge knows where D is, so **both flood the frame onto LAN 2**. There are now **two copies** of the same frame on LAN 2.
- Each bridge now hears the *other* bridge's copy arriving from LAN 2. It is still addressed to D, still unknown — so each bridge forwards it back up onto LAN 1. There are now **two copies on LAN 1**. Worse, each bridge has just seen a frame from A arrive on **port 2**, so both tables now wrongly record **A on port 2**.
- The copies on LAN 1 are forwarded down to LAN 2 again, and the cycle repeats — forever, with the number of copies growing at every pass.

Result: an infinite loop. Frames multiply until they saturate both LANs, and the address tables oscillate so the bridges can never learn anything correctly.

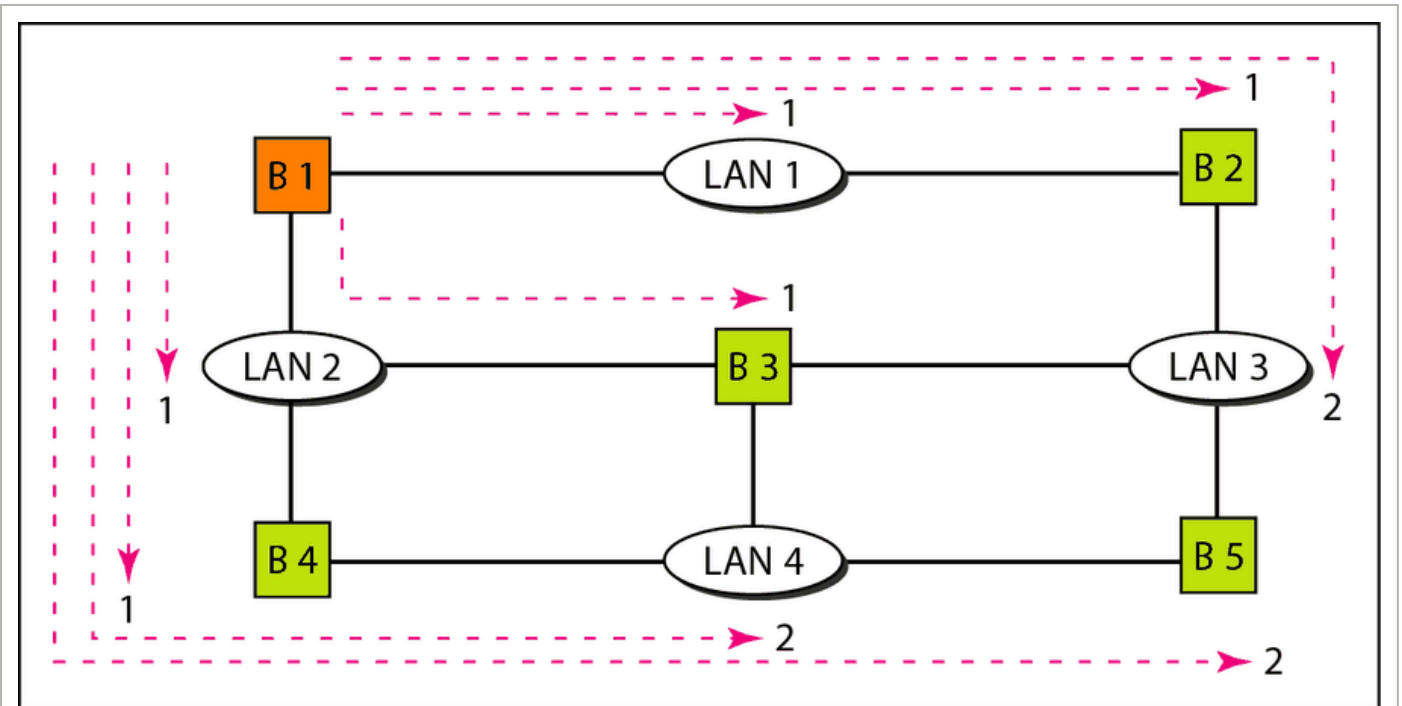
Why nothing stops it: unlike an IP packet, a data link frame has **no hop count or time-to-live field** — nothing counts down, so nothing ever discards it.

The fix: the spanning tree algorithm

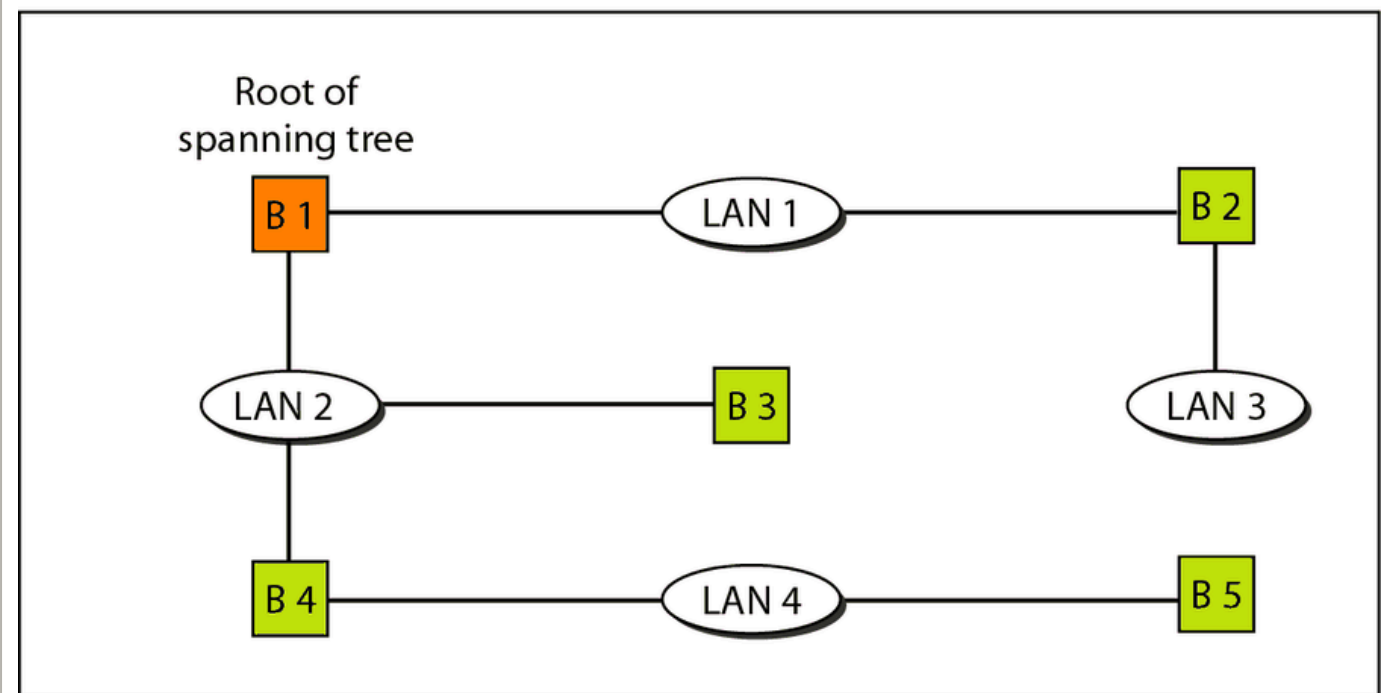
The solution is to leave the redundant physical link in place but **stop using it**, so that between any two LANs there is exactly **one active path** — no loop. The bridges work this out among themselves using the **spanning tree algorithm**.



PPT Fig 5.8 Step 1 — redraw the network as a graph. Every LAN and every bridge becomes a node; every attachment becomes an edge. The looping problem is now visibly a cycle in a graph.

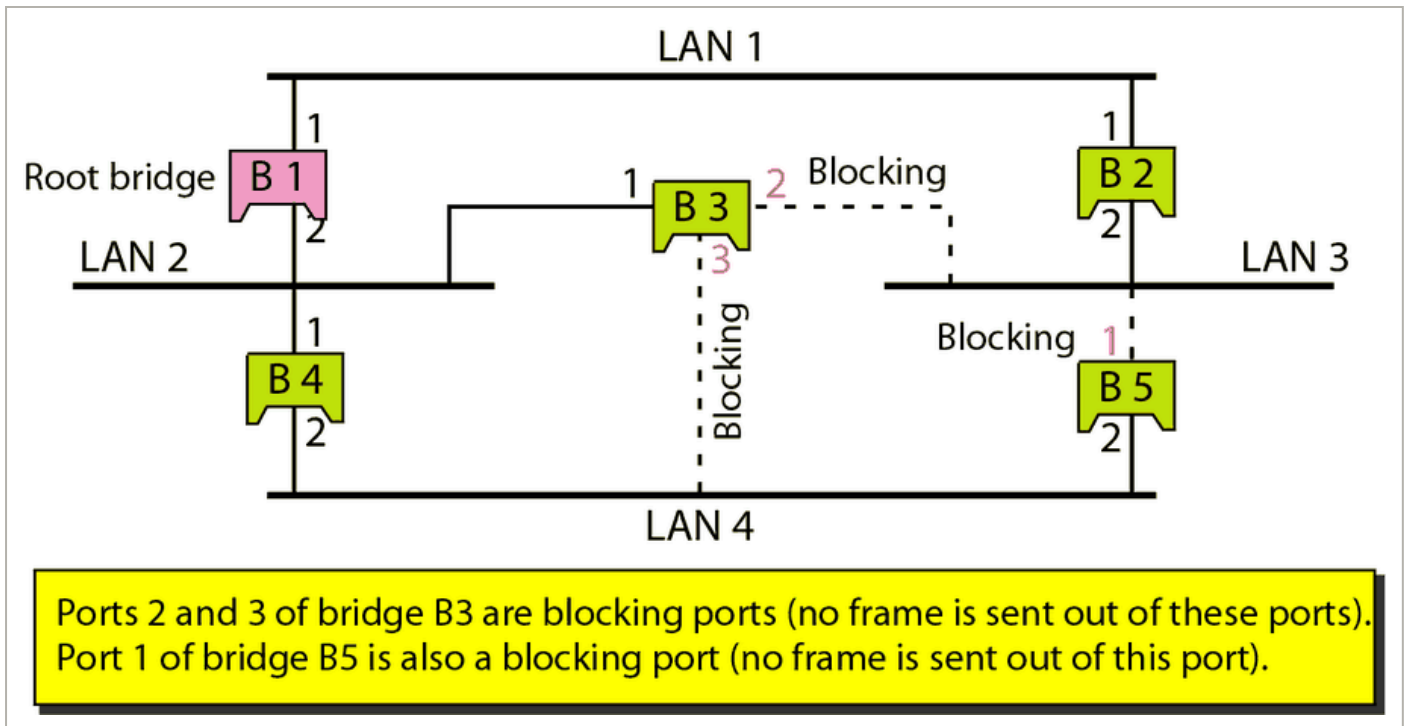


a. Shortest paths



b. Spanning tree

PPT Fig 5.9 Step 2 — elect a **root bridge** (the one with the smallest ID), find the shortest path from the root to every LAN, and keep only those edges. The result is a *tree*, and a tree by definition has no cycles.



PPT Fig 5.10 Step 3 — every port on the tree becomes a **forwarding port**; every port that would close a loop becomes a **blocking port**. The cable stays physically connected, ready to be switched back on if the active path fails.

- 1 **Elect a root bridge** — the bridge with the smallest ID becomes the root of the tree.
- 2 **Find the shortest path** from the root bridge to every LAN, using the cost of each link.
- 3 **Mark the ports.** Ports lying on those shortest paths become **forwarding ports**; all other ports become **blocking ports**.
- 4 Frames travel only through forwarding ports, so **exactly one path** exists between any two LANs and the loop is gone.

WHY REDUNDANCY IS STILL WORTH HAVING

A blocking port is not disconnected — it is only switched off. If the active path breaks, the algorithm runs again and a previously blocked port is promoted to forwarding. You get the safety of a redundant link *without* the loop.

8.6

Switches, routers and gateways

Two-layer switch

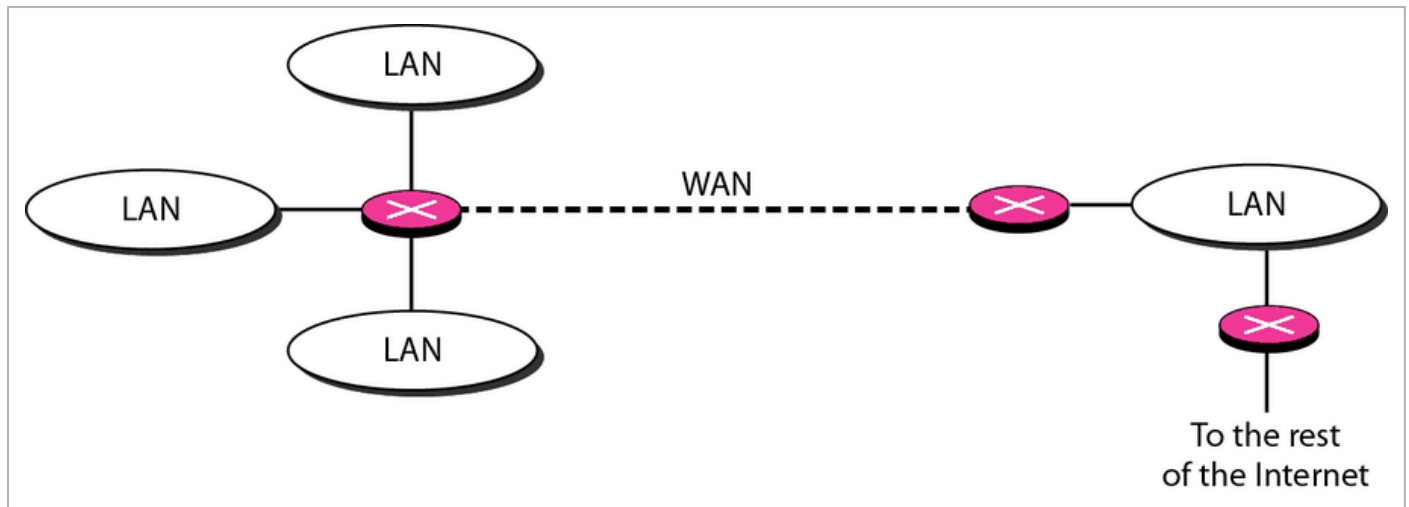
A **bridge with many ports**, able to allocate a fraction of its bandwidth to each. It does the same job as a bridge — filtering on MAC addresses — only faster and on more ports.

Three-layer switch

A **router**, but with the routing done in hardware so it is much faster, and optimised for LAN use.

A **router** is a **three-layer device** that routes packets based on their **logical (IP) addresses**. It connects LANs and WANs together on the internet. Two things separate a router from a bridge:

- A router has a **physical and logical address for every one of its interfaces** (as you saw in §2.5).
- A router **changes the physical addresses** of a packet as it passes through — it strips the old frame and builds a new one for the next hop. A bridge never does this.



PPT Fig 5.11 Routers connecting independent LANs and WANs. This is the picture behind Fig 2.6 in Module 2 — each router boundary is where the MAC addresses get rewritten while the IP addresses stay the same.

A **gateway** is normally a computer that operates in **all five layers**. It takes an application message, reads it, and reinterprets it for a different protocol family — so it can connect two networks that do not speak the same protocols at all.

Hub vs Switch vs Router

COMPARISON OF HUB, SWITCH AND ROUTER			
#	HUB	SWITCH	ROUTER
1	Physical layer device — layer 1	Data link layer device — layer 2	Network layer device — layer 3
2	Works on the basis of broadcasting	Works on the basis of the MAC address	Works on the basis of the IP address
3	A multiport repeater : a signal introduced at any port appears at all the other ports	Receives a message from any connected device and transmits it only to the device it is meant for	Reads the header of an incoming packet and forwards it to the port it is intended for, so determining the route. It can also filter and encapsulate
4	Not intelligent ; may include an amplifier or repeater	Intelligent — it inspects the address and passes the message selectively	Most sophisticated — reads IP addresses, directs packets to another network, and builds address tables to help make routing decisions
5	At least a single network is required to connect	At least a single network is required to connect	Needs at least two networks to connect
6	Cheapest	More expensive than a hub	Most expensive

8.7

MAC address classification

From §2.5 you know a physical address is **48 bits**, **written as 12 hexadecimal digits** with a colon between each byte. Now we look at how those addresses are **classified** — and it comes down to a single bit.

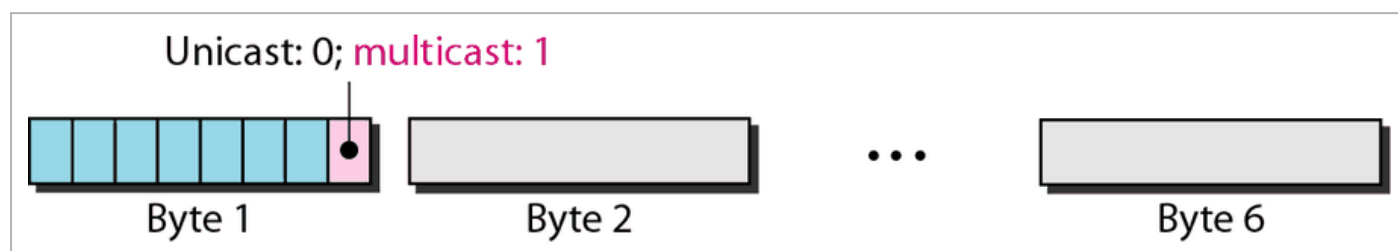


PPT Fig 6.6 An Ethernet address in hexadecimal notation — 6 bytes, 12 hex digits.

How a MAC address is split by purpose

THE TWO HALVES OF EVERY MAC ADDRESS	
A4:BB:6D	: 2E:08:EE
first 3 bytes	last 3 bytes
OUI	NIC / interface ID
who made it	which card it is
(assigned to	(chosen by that
the manufacturer	manufacturer, unique
by the IEEE)	within their range)
This split is why MAC addresses are globally unique without any central registry of individual cards — the IEEE only has to hand out one OUI per manufacturer.	

The three types: unicast, multicast, broadcast



PPT Fig 6.7 Unicast and multicast addresses. Everything hangs on the **least significant bit of the first byte** — one single bit out of forty-eight.

THE RULE

The **least significant bit of the first byte** defines the type of address. If the bit is **0**, the address is **unicast**; otherwise it is **multicast**.

The **broadcast** destination address is a **special case of the multicast address** in which **all bits are 1s** — that is, **FF:FF:FF:FF:FF:FF**.

THE THREE MAC ADDRESS TYPES			
TYPE	GOES TO	TEST	EXAMPLE
Unicast	One specific interface	LSB of first byte = 0 → second hex digit is even	4A:30:10:21:10:1A
Multicast	A group of interfaces	LSB of first byte = 1 → second hex digit is odd	47:20:1B:2E:08:EE
Broadcast	Every interface on the LAN	All 48 bits are 1	FF:FF:FF:FF:FF:FF

THE EXAM SHORTCUT

You never need to convert the whole address. Look only at the **second hexadecimal digit from the left**. If it is **even**, the address is unicast. If it is **odd**, it is multicast. If every digit is F, it is broadcast.

Why the second digit? Because the first byte is two hex digits, and the *second* of those two carries the least significant bit.

EXAMPLE 6.1 – FROM THE SLIDES

Define the type of each of the following destination addresses:

- a. 4A:30:10:21:10:1A b. 47:20:1B:2E:08:EE c. FF:FF:FF:FF:FF:FF

To find the type of the address, we look at the **second hexadecimal digit from the left**. If it is even the address is unicast; if it is odd the address is multicast; if all digits are Fs the address is broadcast.

- a. The second digit is **A**. In binary A is **1010**, whose last bit is 0 — even. This is a **unicast** address.
 b. The second digit is **7**. In binary 7 is **0111**, whose last bit is 1 — odd. This is a **multicast** address.
 c. All the digits are Fs. This is a **broadcast** address.

a. Unicast · b. Multicast · c. Broadcast

EXAMPLE 6.2 – FROM THE SLIDES

Show how the address 47:20:1B:2E:08:EE is sent out on the line.

The address is sent **left to right, byte by byte**; but within each byte it is sent **right to left, bit by bit**.

Take the first byte, 47. In binary that is 0100 0111. Sent least-significant-bit first, it goes onto the wire as 1110 0010.

The same reversal is then applied to each following byte in turn: 20, then 1B, then 2E, then 08, then EE.

Bytes go left to right; bits within each byte go right to left.

Hex	47	20	1B	2E	08	EE
Sent	11100010	00000100	11011000	01110100	00010000	01110111

This is why the **least significant** bit of the first byte is the one that reaches the receiver **first** — a station can tell unicast from multicast from the very first bit it sees, before the rest of the address has even arrived.

THAT LAST POINT IS WORTH PAUSING ON

The unicast/multicast bit is not in an odd place by accident. Because bits go out least-significant-first, that bit is **the very first bit of the whole address** on the wire. A network card can therefore start deciding “is this for me?” immediately, instead of buffering all 48 bits first.

Module 8 at a glance

- Five categories of connecting device, one per layer: **repeater** and **hub** (physical), **bridge** and **switch** (data link), **router** (network), **gateway** (all layers).
- A hub floods; a bridge **filters**. A **learning bridge** fills its table from the *source* address of every frame it sees, and entries expire so a moved station is re-learned.
- Two bridges joined by more than one path make a **loop**, and a broadcast then circulates forever. The cure is the **spanning tree**, built from BPDUs, which blocks ports until exactly one path remains.
- A **MAC address** is 48 bits. All ones is **broadcast**; otherwise bit 0 of the first octet gives **0 = unicast**, **1 = multicast**, and bit 1 gives universal or locally administered.

CHECK YOURSELF — MODULE 8

Q What single property separates a repeater, a bridge, a router and a gateway from each other?

The **layer they operate at**. Repeater = physical (layer 1), bridge/two-layer switch = data link (layer 2), router/three-layer switch = network (layer 3), gateway = all five layers.

Q Is a repeater an amplifier? Explain the difference and why it matters.

No — a repeater is a regenerator, not an amplifier. An amplifier makes the signal bigger *and the noise with it*, because it cannot tell them apart. A regenerator reads the weakened signal, decides each bit, and creates a **fresh clean copy** at full strength, discarding the noise. That is why repeaters can be chained without noise accumulating.

Q Does a repeater filter frames? Does a bridge? Does a bridge change MAC addresses?

A repeater **forwards every frame — it has no filtering capability**. A bridge **does filter**, using a table of addresses and ports. A bridge **does not change** the physical (MAC) addresses in a frame — a *router* does that.

Q In a learning bridge, which address does it learn from, and which does it act on?

It **learns** from the frame's **source** address (recording it against the port the frame arrived on), and it **decides** using the frame's **destination** address. If the destination is unknown, it **floods** the frame out of every port except the one it came in on.

Q A bridge's table is empty. Station C sends a frame to station A. What does the bridge learn, and what does it do with the frame?

It **learns C**, recording C against the port the frame arrived on. It does **not** learn A. Since destination A is not in the table, it **floods** the frame out of all other ports.

Q Why does having two bridges between the same two LANs cause a loop?

Because a frame with an unknown destination is **flooded**. Both bridges flood A's frame onto LAN 2, making two copies; each then hears the other's copy and floods it back onto LAN 1; and so on forever. Copies multiply at every pass, and the address tables are corrupted too, because A keeps appearing to arrive on a different port.

Q Why doesn't the looping frame simply expire, the way a looping IP packet does?

Because a data link layer frame has **no hop count or time-to-live field**. An IP packet has a TTL that is decremented at every router and eventually hits zero; a frame has nothing that counts down, so nothing ever discards it.

Q Outline the spanning tree algorithm in four steps.

1. **Elect a root bridge** — the one with the smallest ID. 2. Find the **shortest path from the root to every LAN**. 3. Mark ports on those paths as **forwarding** and all others as **blocking**. 4. Traffic then uses forwarding ports only, so exactly **one path** exists between any two LANs and the loop is gone.

Q If the redundant link is blocked anyway, what was the point of installing it?

A blocking port is **switched off, not disconnected**. If the active path fails, the algorithm runs again and a previously blocked port is promoted to forwarding. You keep the **fault tolerance** of the redundant link without paying the price of a loop.

Q Give two differences between a bridge and a router.

A bridge works at **layer 2 on MAC addresses**; a router works at **layer 3 on IP addresses**. A bridge **does not change** the physical addresses in a frame; a router **rewrites them at every hop**. Also, a router has an address pair for *each* of its interfaces and needs at least two networks to connect.

Q Which bit of a MAC address decides unicast vs multicast, and what is the quick test?

The **least significant bit of the first byte**. 0 = unicast, 1 = multicast. The quick test: look at the **second hexadecimal digit from the left** — **even** → **unicast**, **odd** → **multicast**. All Fs → broadcast.

Q Classify: (a) B4:22:1C:09:AA:01 (b) 33:33:00:00:00:01 (c) FF:FF:FF:FF:FF:FF

- (a) Second digit is **4** — even → **unicast**.
- (b) Second digit is **3** — odd → **multicast**.
- (c) All digits **F** → **broadcast**.

Q What are the two halves of a MAC address called?

The first 3 bytes are the **OUI** (Organizationally Unique Identifier), which the IEEE assigns to the manufacturer. The last 3 bytes are the **NIC / interface ID**, which the manufacturer assigns uniquely within its own range.

Q How is a MAC address transmitted on the line?

Left to right byte by byte, but right to left (least significant bit first) within each byte. This puts the unicast/multicast bit first on the wire, so a card can start deciding whether the frame is for it before the whole address has arrived.

REVISION KIT · A

Formula sheet

Every formula in these six modules, on one page. If you can reproduce this sheet from memory, the numerical part of the paper is handled.

MODULE 1 — TOPOLOGY

Mesh : duplex links = $\frac{n(n-1)}{2}$ ports per device = $n - 1$

Star : links = n ports per device = 1

Ring : links = n ports per device = 2

MODULE 2 — ADDRESS SIZES

Physical (MAC) address = **48 bits** = 6 bytes = 12 hex digits
 IPv4 logical address = **32 bits**
 Port address = **16 bits** (0 - 65535)

Rule: physical addresses **change hop to hop**;
 logical and port addresses **stay the same**.

MODULE 5 — FLOW AND ERROR CONTROL

Transmission time $T_t = \text{frame size} / \text{bandwidth}$

Propagation time $T_p = \text{distance} / \text{propagation speed}$

Ratio $a = T_p / T_t$

Bandwidth-delay product = **bandwidth × round-trip delay**

Utilisation = (bits sent per round trip) / (bandwidth-delay product)

Efficiency: Stop-and-Wait $\eta = 1 / (1 + 2a)$

GBN / Sel. Rep. $\eta = N / (1 + 2a)$, capped at 1

Window sizes:

Stop-and-Wait ARQ send = 1, receive = 1

Go-Back-N ARQ send $\leq 2^m - 1$, receive = 1

Selective Repeat send = receive $\leq 2^{m-1}$

MODULE 6 — PERFORMANCE PARAMETERS

$$\text{PDR} = \left(\frac{\text{packets received}}{\text{packets sent}} \right) \times 100\%$$

$$\text{Throughput} = \frac{\text{data successfully received}}{\text{total time}}$$

Latency = propagation + transmission + processing + queuing

$$\text{Jitter} = |D(i+1) - D(i)|$$

MODULE 6 — RANDOM ACCESS

$$\text{Frame transmission time } T_{fr} = \frac{\text{frame size}}{\text{bandwidth}}$$

$$\text{Vulnerable time : Pure ALOHA} = 2T_{fr}$$

$$\text{Slotted ALOHA} = T_{fr}$$

$$\text{CSMA} = T_p$$

$$\text{Throughput : Pure ALOHA } S = Ge^{-2G}, S_{\max} = 0.184 \text{ at } G = 0.5$$

$$\text{Slotted ALOHA } S = Ge^{-G}, S_{\max} = 0.368 \text{ at } G = 1$$

$$\text{Back-off : } T_B = R \times T_p, R \text{ random from } \{0, 1, \dots, 2^K - 1\}$$

K capped at 10; give up after $K_{\max} = 15$ attempts

$$\text{CSMA/CD : } T_{fr} \geq 2T_p$$

$$\text{minimum frame size} = \text{bandwidth} \times 2T_p$$

(Standard Ethernet: 512 bits = 64 bytes)

MODULE 6 — CDMA

Walsh table: $W_1 = [+1]$

$$W_{2N} = \begin{bmatrix} W_N & W_N \\ W_N & W_N \end{bmatrix} \leftarrow \text{bottom-right is the complement}$$

Number of sequences $N = 2^m$ (chip length is also N)

Data value: bit 1 $\rightarrow +1$ bit 0 $\rightarrow -1$ silent $\rightarrow 0$

$$\text{Encode: } D = d_1c_1 + d_2c_2 + d_3c_3 + d_4c_4$$

$$\text{Decode: } d_i = (D \cdot c_i) / N$$

$$\text{Orthogonality: } c_i \cdot c_i = N \quad c_i \cdot c_j = 0 \text{ for } i \neq j$$

REVISION KIT · B

Numerical drill

Ten fresh problems in the exact style of the lecture examples, fully solved. Cover the answers, work them out on paper, then check.

Q D1. A fully connected mesh network needs 45 links. How many devices does it have, and how many ports does each need?

$n(n-1)/2 = 45$, so $n(n-1) = 90$. Try $n = 10$: $10 \times 9 = 90$ ✓

10 devices, each needing $n - 1 = 9$ ports.

Q D2. Byte-stuff this data: ESC FLAG ESC ESC FLAG . How many bytes go on the wire?

Rule: put an ESC in front of every FLAG and every ESC.

Input, byte by byte: ESC → ESC ESC ; FLAG → ESC FLAG ; ESC → ESC ESC ; ESC → ESC ESC ; FLAG → ESC FLAG .

Stuffed: ESC ESC · ESC FLAG · ESC ESC · ESC ESC · ESC FLAG

5 bytes in → **10 bytes out**. Worst case for byte stuffing: the data doubles in size.

Q D3. Bit-stuff 1111111111 (ten 1s). Then de-stuff to verify.

Count 1s; insert a 0 at every fifth, then reset the count.

11111 → stuff 0 → 11111 → stuff 0

Stuffed: 11111 0 11111 0 = 111110111110 (12 bits)

De-stuff: after five 1s the next bit is 0 → delete. Twice. Recovers 1111111111 ✓

Note the stuffed 0 at the very end is still required — the rule fires on the count, not on what follows.

Q D4. A 10 Mbps link is 200 km long; signals travel at 2×10^8 m/s. Frames are 5000 bits. Find the efficiency of Stop-and-Wait ARQ, and the window size needed for 100% efficiency.

$$T_t = 5000 / (10 \times 10^6) = 0.5 \text{ ms}$$

$$T_p = (200 \times 10^3) / (2 \times 10^8) = 1 \text{ ms}$$

$$\alpha = T_p / T_t = 1 / 0.5 = 2$$

$$\eta = 1 / (1 + 2\alpha) = 1 / (1 + 4) = 0.2 = 20\%$$

For $\eta = 1$ we need $N \geq 1 + 2\alpha = 5$. So a send window of **5 frames** fills the pipe.

With Go-Back-N that needs $2^m - 1 \geq 5$, so $2^m \geq 6 \rightarrow m = 3$.

Q D5. Go-Back-N uses $m = 4$. What are the maximum send and receive window sizes? If Selective Repeat used the same m , what would they be?

$m = 4$, so $2^m = 16$.

Go-Back-N: send window $\leq 2^m - 1 = 15$; receive window = 1.

Selective Repeat: send = receive $\leq 2^{m-1} = 2^3 = 8$.

Selective Repeat gets a smaller send window for the same m — the price of needing an unambiguous receive window too.

Q D6. Stations on an ALOHA network are 450 km apart, signals travel at 3×10^8 m/s. Find T_p , and list the possible back-off times for $K = 1, 2$ and 3.

$$T_p = (450 \times 10^3) / (3 \times 10^8) = 1.5 \text{ ms}$$

$$K = 1: R \in \{0,1\} \rightarrow T_B \in \{0, 1.5\} \text{ ms}$$

$$K = 2: R \in \{0,1,2,3\} \rightarrow T_B \in \{0, 1.5, 3, 4.5\} \text{ ms}$$

$$K = 3: R \in \{0 \dots 7\} \rightarrow T_B \in \{0, 1.5, 3, 4.5, 6, 7.5, 9, 10.5\} \text{ ms}$$

Q D7. A slotted ALOHA network sends 1000-bit frames at 1 Mbps and generates 600 frames/s. Find G , S , and the throughput in frames/s. Then find the offered load that would maximise throughput.

$$T_{fr} = 1000 / (1 \times 10^6) = 1 \text{ ms. So } 600 \text{ frames/s} = 0.6 \text{ frames per ms} \rightarrow G = 0.6.$$

$$S = Ge^{-G} = 0.6 \times e^{-0.6} = 0.6 \times 0.5488 = 0.329 \text{ (32.9\%)}$$

$$\text{Throughput} = 600 \times 0.329 \approx 198 \text{ frames/s}$$

Maximum for slotted ALOHA is at $G = 1$, i.e. **1000 frames/s offered**, giving $S = 0.368 \rightarrow 368 \text{ frames/s}$.

Q D8. A CSMA/CD LAN runs at 1 Gbps. Its minimum frame size must stay at 512 bits. What is the maximum allowed propagation time, and what does that imply about cable length (at 2×10^8 m/s)?

$$\text{Minimum frame size} = \text{bandwidth} \times 2T_p$$

$$512 = (1 \times 10^9) \times 2T_p \rightarrow 2T_p = 512 / 10^9 = 0.512 \mu\text{s} \rightarrow T_p = 0.256 \mu\text{s}$$

$$\text{Length} = T_p \times \text{speed} = (0.256 \times 10^{-6}) \times (2 \times 10^8) = 51.2 \text{ m}$$

The insight: at 1 Gbps a 512-bit frame flies past so fast that the cable can only be about 51 m. This is exactly why Gigabit Ethernet had to introduce *carrier extension* — padding short frames — to keep usable cable lengths.

Q D9. A CDMA network has 4 stations with the standard W_4 codes. Station 1 sends 1, station 2 sends 1, station 3 sends 0, station 4 is silent. Find the channel data, then decode station 2's bit.

Encode. $d_1 = +1, d_2 = +1, d_3 = -1, d_4 = 0$.

$$d_1 c_1 = +1 \times [+1 \ +1 \ +1 \ +1] = [+1 \ +1 \ +1 \ +1]$$

$$d_2 c_2 = +1 \times [+1 \ -1 \ +1 \ -1] = [+1 \ -1 \ +1 \ -1]$$

$$d_3 c_3 = -1 \times [+1 \ +1 \ -1 \ -1] = [-1 \ -1 \ +1 \ +1]$$

$$d_4 c_4 = 0 \times [+1 \ -1 \ -1 \ +1] = [0 \ 0 \ 0 \ 0]$$

Column sums: $1+1-1+0 = +1$; $1-1-1+0 = -1$; $1+1+1+0 = +3$; $1-1+1+0 = +1$

$$D = [+1, -1, +3, +1]$$

Decode for station 2 using $c_2 = [+1 \ -1 \ +1 \ -1]$:

$[+1, +1, +3, -1]$, sum = +4, divide by $N = 4 \rightarrow +1 = \text{bit } 1 \checkmark$

Q D10. A network needs to support 500 simultaneous CDMA users. Find m , N , and the chip sequence length. If the raw data rate per user is 10 kbps, what chip rate does the channel need?

$$N = 2^m \geq 500. \ 2^8 = 256 \text{ (too small); } 2^9 = 512 \text{ (enough).}$$

$m = 9, N = 512$, chip sequence length = **512 chips**. 12 sequences spare.

Each data bit becomes 512 chips, so the channel chip rate = $10 \text{ kbps} \times 512 = \mathbf{5.12 \text{ Mchips/s}}$.

The trade-off CDMA makes: it buys the ability to have everyone talk at once by spending a great deal of bandwidth — $512\times$ the raw data rate here.

REVISION KIT · C

Glossary

Every term that mattered, in plain words. Use this when a question uses a word you half-remember.

ACK / NAK

A small control frame saying “I got that one” / “that one was broken, send it again.”

ARQ — Automatic Repeat Request

The whole idea of error control at the data link layer: when something goes wrong, just send it again.

Attenuation

The weakening of a signal as it travels. Higher attenuation means repeaters must be placed closer together.

Bandwidth-delay product

Bandwidth $\times$ round-trip delay. The number of bits that can be “in flight” on a link at one moment — the size of the pipe.

Bit stuffing

Inserting an extra 0 after five consecutive 1s, so the flag pattern can never occur inside the data.

Byte stuffing

Inserting an extra ESC byte before any FLAG or ESC byte in the data, for the same reason.

Chip sequence

A station’s unique code in CDMA, a row of a Walsh table. Its orthogonality is what separates the users.

Cladding

The layer of glass around a fiber’s core, with a *lower* refractive index, so light is totally internally reflected back into the core.

Collision

Two or more stations transmitting at once on a shared medium, destroying *both* frames.

Crosstalk

One pair of wires’ signal leaking into a neighbouring pair in the same cable.

Cumulative acknowledgment

An ACK that confirms everything up to a point, not just one frame. ACK 3 covers frames 0, 1 and 2 — so a lost ACK 2 does no harm.

De facto / de jure standard

“By fact” — standard because everyone uses it. “By law” — formally approved by a recognised body.

Differential signalling

Sending the same signal on two wires with opposite polarity, so the receiver takes the difference and identical noise cancels out.

Encapsulation

Each layer wrapping what it got from above inside its own header (and, at the data link layer, a trailer).

FCS — Frame Check Sequence

The CRC field in an HDLC trailer. The receiver recalculates it and compares.

Flag

The bit pattern `01111110` that marks the start and end of an HDLC frame.

Flow control

Limiting how much a sender may send before waiting, so a fast sender cannot flood a slow receiver.

Framing

Cutting the bit stream into distinguishable units, each with its own header and trailer.

Guard band / guard time

An unused strip of frequency (FDMA) or an unused gap of time (TDMA) that stops neighbours interfering.

Hop

One link, from one node to the next. “Hop-to-hop” is one link; “end-to-end” is the whole journey.

Interframe space (IFS)

In CSMA/CA, the wait after finding the channel idle, in case a distant station’s signal has not arrived yet. A shorter IFS gives higher priority.

Jitter

The wobble in packet delay: $|D(i+1) - D(i)|$. High jitter makes voice and video choppy.

Modal dispersion

A light pulse spreading out because its different rays take paths of different length and so arrive at different times.

Multipoint (multidrop)

More than two devices sharing one link, so capacity is shared. The reason Module 6 exists.

N(S) and N(R)

In an HDLC I-frame: the sequence number of *this* frame, and the number of the frame expected *next* from the other side (the piggybacked ACK).

Offered load G

The average number of frames the whole system generates during one frame transmission time. The x-axis of the ALOHA throughput curve.

Orthogonality

The property $c_i \cdot c_j = N$ and $c_i \cdot c_j = 0$. The entire mathematical basis of CDMA.

P/F bit

Poll/Final. Set by a primary it means “you may send”; set by a secondary it means “this is my last frame.”

PDR — Packet Delivery Ratio

The percentage of sent packets that actually arrive. A reliability measure.

Piggybacking

Carrying an acknowledgment inside a data frame going the other way, instead of sending a separate ACK frame.

Protocol

The rules governing data communication. Three elements: syntax (format), semantics (meaning), timing (when and how fast).

Repeater

A device that regenerates a weakened signal. Each station in a ring topology contains one.

Sliding window

The mechanism that lets several frames be outstanding at once, instead of one at a time. S_f , S_n and S_{size} at the sender; R_n at the receiver.

Throughput

The rate of *successful* data delivery. Always less than bandwidth, often much less.

Token

A special frame that circulates a logical ring; only the station holding it may transmit.

Topology

The arrangement of links and nodes — the shape of the connections, not the physical placement of the machines.

Total internal reflection

Light going from denser to less dense medium at more than the critical angle is reflected *entirely* back. How fiber optics works.

Vulnerable time

The window during which a collision is possible. $2T_{fr}$ for pure ALOHA, T_{fr} for slotted, T_p for CSMA.

Walsh table

The recursive construction that generates orthogonal chip sequences. Size must be $N = 2^m$.

REVISION KIT · D

What comes next

So you know where you stand, and what is still ahead.

These notes end where the lecture deck reaches **slide 246**. Slide 247 opens Chapter 5 — *Connecting LANs, Backbone Networks, and Virtual LANs* — which is outside the scope you asked for. Here is what is still to come, and how it connects to what you have just learnt.

THE REST OF THE SYLLABUS, AND WHAT IT BUILDS ON		
COMING UP	SYLLABUS UNIT	BUILDS DIRECTLY ON
Backbone networks (bus and star backbone, remote LANs) and Virtual LANs (VLANs).	Data Link	Module 8 — the connecting devices themselves are already covered there.
Wired LANs: IEEE 802.3 Ethernet — frame format, Standard, Fast, Gigabit Ethernet.	Data Link	§6.4 — Ethernet is CSMA/CD, and its 64-byte minimum frame is Example 7.5.
Wireless LANs: IEEE 802.11 — architecture, MAC sublayer, frame format.	Data Link	§6.4 — 802.11 is CSMA/CA, with the IFS and contention window of Fig 6.7.
Network layer: IPv4 classful and classless addressing, subnetting, supernetting, special and private addresses, NAT, IPv6.	Network	§2.5 — the logical address that “usually remains the same” is what gets designed here.
Routing: static vs dynamic, RIP (distance vector), OSPF (link state), BGP (path vector).	Network	§2.3 — the network layer’s second duty after logical addressing.
IPv4 datagram format; ARP, RARP, ICMP, IGMP.	Network	§2.5 — ARP is literally the mechanism that maps a logical address to a physical one.
Transport layer: UDP, TCP, SCTP — three-way handshake, congestion control, flow control.	Transport	Module 5 — TCP does the same sliding-window job, but end to end instead of hop to hop.
Application layer: TELNET, FTP, HTTP, HTTPS, SNMP, DNS, DHCP, BOOTP, SMTP, POP3, IMAP.	Application	§2.4 — these are the protocols that absorbed OSI’s session and presentation layers.
Switching techniques: circuit switching, packet switching, virtual circuit switching.	Overview	Module 1 — listed in your first syllabus unit but not yet in the slides.

ONE LAST THING WORTH NOTICING

Almost every item in that table is **a topic you already have the machinery for**. Ethernet is CSMA/CD. Wi-Fi is CSMA/CA. TCP is a sliding window. PPP is HDLC with authentication bolted on. The first six modules were not an introduction to the subject — they were the subject. What follows is mostly the same ideas, applied.